"The Stakers' Roulette"

The Stakers' Roulette

> "不是谁力气大谁记账，而是谁押得多、谁押得久，谁获得记账权。但押得多也意味着错得多时，输得越多。"

故事

云州城的账本过去由一群矿工维护。他们通过比赛算数学题来决定谁有权在账本上写下新的一笔。谁先算出答案，谁获得记账权和奖励。

这个制度的问题是：比赛算题消耗了大量的资源和电力。云州城的北边甚至专门建了一座煤山，只为供给矿工们的计算竞赛。

新制度：押注轮盘

云州城的智者提议了一种新制度：不再比赛算题，而是比赛"押注"。

具体规则是这样的：

1. 任何人都可以成为"记账人"，但前提是你必须在城中的公共金库里锁定一笔保证金。锁定的金额越大、锁定的时间越长，你获得的"权重"就越高。

2. 每过一分钟，轮盘转动一次。轮盘上每个扇区代表一个记账人。扇区的大小取决于这个记账人的权重——押得多的人，扇区大；押得少的人，扇区小。

3. 轮盘停止时，指针指向谁，谁就获得这一分钟的记账权——在账本上写下新的交易，并获得手续费奖励。

4. 但记账不是一个人说了算。被选中的记账人写下新页后，其他记账人需要"确认"——检查这一页的内容是否正确。如果大多数人确认无误，这一页就被正式接受。

押注的风险

押注轮盘有一个严格的惩罚机制：

如果一个记账人写下了错误的内容——比如故意遗漏某笔交易、或者写入了虚假的交易——其他记账人会拒绝确认。如果这个记账人的错误被证实，他锁定的保证金会被部分或全部没收。

这叫"削减"（Slashing）。押得越多的人，获得记账权的概率越大，赚钱的机会越多。但一旦犯错，损失的也越多。

一位老记账人说："在旧制度下，矿工浪费的是电。在新制度下，记账人冒险的是钱。电浪费了可以再生，钱被罚没了就是真没了。所以新制度下的人更谨慎。"

排队与退出

押注轮盘还有一个特点：你不能随时取回保证金。

当你决定不再做记账人时，你需要"申请退出"。从申请到真正拿回钱，有一

段"冷却期"——在这段时间里，你仍然可能被选中记账，但如果你在这段时间里犯了错，保证金仍然会被罚。

冷却期的存在是为了防止一种攻击：攻击者押了一大笔钱，做了一次恶意记账，然后立刻取钱跑路。冷却期让系统有时间发现并惩罚这种攻击。

轮盘的公平性

有人质疑：押得多的人扇区大，岂不是富人永远占优？

智者的回答是："是的。但旧制度下，买得起更多矿机的人也有优势。任何制度下，资源多的人都有优势。押注轮盘至少把资源消耗从'浪费电'变成了'锁定资本'。而且，小押注者可以联合起来，把资金池化，共同分享记账权和奖励。"

事实上，云州城很快出现了"押注池"——几十个小额押注者把钱汇集在一起，由专业的管理人运行记账节点。奖励按出资比例分配。这让小额参与者也能分享共识的收益。

这则寓言在说什么

这则寓言对应的是 **Proof of Stake**（**PoS**，权益证明） 共识机制——以太坊（以及大多数现代区块链）使用的共识算法。验证者通过质押 ETH 来参与区块的提议和确认，质押越多，被选中提议区块的概率越大。作恶会导致质押资金被削减（slashed）。

关键映射

故事元素	技术概念	说明
矿工算题	Proof of Work (PoW)	比特币和早期以太坊使用的共识机制，通过算力竞争出块权
公共金库锁保证金	Staking / 质押	验证者将 ETH 存入 Beacon Chain 合约，作为参与共识的保证金
权重	有效余额 (Effective Balance)	验证者的权重由其质押金额决定，上限 32 ETH 每个验证者
轮盘转动	RANDAO / 随机信标	每个 slot（12 秒）通过可验证随机函数选出一个区块提议者
扇区大小	被选中的概率	质押越多，成为提议者的概率越高
其他记账人确认	Attestation / 验证委员会	每个 slot 有一个验证者委员会，负责投票确认区块的有效性
大多数人确认	委员会共识 / 超级多数	如果 2/3 的委员会成员投票确认，区块被接受
错误被证实	可罚没行为 (Slashable Offense)	双重投票、 surround 投票等恶意行为可被证明并惩罚
削减保证金	Slashing	作恶验证者的质押资金被销毁（最多 1 ETH 立即罚没，剩余部分在 36 天内逐步罚没）
冷却期	Exit Queue / 提款周期	验证者退出时需要等待，且仍需参与共识直到完全退出。防止"作恶后立刻提款"
押注池	Staking Pool / Lido / Rocket Pool	小额持有者通过池化参与质押，获得流动性代币（如 stETH）作为凭证

为什么是这个故事？

PoS 是区块链从能源密集型向资本密集型转变的核心。理解"为什么质押能代替算力"以及"削减机制如何防止作恶"是理解现代区块链的关键。

用"押注轮盘"来隐喻 PoS，有几个直觉优势：

1. 概率出块的可视化：轮盘 = 随机选择提议者，扇区大小 = 质押权重
2. 资本替代能源：从烧煤到锁定资金——对应了 PoS 的经济设计
3. 削减的威慑：押得多赚得多但错得多罚得多——对应了 PoS 的激励相容性
4. 押注池：小额参与者的联合——对应了流动性质押协议（Lido 等）

延伸阅读

- wiki-web3 概念原文：权益证明 PoS 共识机制
- Ethereum PoS FAQ
- Consensys: Ethereum 2.0 Economics
- Slashing Conditions in Ethereum PoS

"The Brand of the Faithless"

The Brand of the Faithless

> "信任是用钱买来的，背叛是用钱偿还的。"

故事

云州城的押注轮盘制度运转良好，但智者们很快发现了一个问题：如果记账人作恶的成本只是"失去未来的收益"，那么某些情况下作恶仍然是有利可图的。

比如，一个记账人在另一个城市同时参与了一个竞争性的账本系统。如果他在这边破坏共识，导致云州城的账本混乱，他可以从那边获得巨额回报。即使他在这边损失了保证金，那边的收益可能更多。

这叫"外部激励冲突"——作恶的收益可能超过被惩罚的损失。

烙印制度的引入

为了解决这个问题，智者们引入了"烙印制度"。

烙印制度的核心是：作恶不仅仅损失金钱，还会留下永久的标记。

具体规则：

1. 轻度烙印：如果一个记账人因为网络故障或操作失误，短暂地离线了，没有及时确认区块。他的保证金会被扣除一小部分作为惩罚，但不会留下烙印。

2. 中度烙印：如果一个记账人在同一轮中被发现投了两次票——比如同时支持两个互相冲突的区块。这是"双重投票"，他的保证金会被扣除较大部分，并且留下一个"黄烙印"。黄烙印不会永久禁止他参与，但会让他在一段时间内的权重降低。

3. 重度烙印：如果一个记账人被发现" surround 投票"——这是一种更复杂的攻击，通过在不同时间点投互相矛盾的票来破坏共识。他的保证金会被几乎全部没收（最高可达全部质押），并且留下一个"红烙印"。红烙印意味着他几乎永远不能再参与记账。

最关键的是：烙印是公开的、永久的、不可撤销的。任何人都可以查询一个记账人是否有烙印，以及烙印的类型和原因。

烙印的威慑

烙印制度的引入彻底改变了押注轮盘的激励结构。

在旧制度下，一个记账人考虑的是："如果我作恶，我会损失多少保证金？"

在新制度下，他需要考虑："如果我作恶，我会损失保证金，还会被永远标记为失信者。即使我在另一个城市有收益，我的名声在这里已经毁了。"

对于专业的记账机构来说，名声就是一切。一个被烙印的机构，即使换了名字、换了保证金，烙印记录仍然可以被追溯。客户不会把资金委托给一个有过红烙印历史的机构。

一位智者说："金钱惩罚解决的是'经济激励'问题。烙印解决的是'身份和声誉'问题。两者结合，才能让制度真正不可攻破。"

误伤与申诉

烙印制度也有其阴暗面。

有一个记账人，他的节点因为数据中心的一场火灾而离线了三天。按照规则，他因为"持续离线"被扣除了部分保证金。他申诉说："这是不可抗力，不是我的错。"

申诉委员会审查后，认定火灾确实属于不可抗力，退还了他的保证金。但烙印记录上仍然留下了一条注释："曾因不可抗力离线三天。"这条注释不会影响他的权重，但任何人查询他的历史时都能看到。

这位记账人不满："既然你们认定我无错，为什么不删除这条记录？"

委员会的回答是："我们不能删除记录，因为记录是写入石碑的。但我们可以添加注释，说明真相。历史不可更改，但历史可以被解释。"

这则寓言在说什么

这则寓言对应的是 **Slashing**（削减）机制——PoS 共识中对作恶验证者的经济惩罚。Slashing 不仅仅罚没资金，还会降低验证者的有效余额、影响其声誉，并在链上留下永久记录。

关键映射

| 故事元素 | 技术概念 | 说明 |

|---------|---------|------|
外部激励冲突	外部攻击动机	攻击者可能从链外获得比质押更多的收益（如做空、竞争对手贿赂）
轻度烙印	Inactivity Leak / 离线惩罚	验证者长时间不参与共识，其有效余额缓慢流失，但不会留下 slashable 记录
中度烙印	Slashing: Double Vote	在同一 slot 对两个不同区块投票。惩罚 1 ETH 立即罚没，剩余部分在 36 天内逐步罚没
重度烙印	Slashing: Surround Vote	surround 投票（一种更复杂的双签）。惩罚更严厉，可能罚没全部有效余额
黄烙印 / 红烙印	惩罚级别	不同严重程度的可罚没行为对应不同的罚没比例
烙印公开永久	Slashing 记录的不可篡改性	罚没事件永久记录在链上，任何人可以查询
专业机构的声誉	验证者声誉风险	对于机构验证者，被罚没会严重损害其商业信誉
申诉与注释	不可抗力与规则刚性	链上规则自动执行，无法因为"非恶意"而回滚。但可以在链下解释
石碑不可更改	区块链的不可篡改性	一旦发生的事实在链上确认，无法删除，只能补充信息

为什么是这个故事？

Slashing 是 PoS 安全性的核心支柱，但很多人只理解"罚钱"，不理解"声誉"和"永久性"的维度。

用"烙印"来隐喻 Slashing，有几个直觉优势：

1. 经济惩罚 + 声誉惩罚：金钱损失 + 永久标记 = 完整的威慑体系
2. 公开的耻辱：烙印公开可查 = 链上罚没记录的透明性
3. 规则的刚性：不可抗力仍然留下记录 = 区块链自动执行、不可申诉的特性

延伸阅读

- wiki-web3 概念原文：权益证明 PoS 共识机制
- Ethereum Slashing Specifications
- Slashing Risks for Validators
- Penalties and Rewards in Ethereum PoS

"The Irrevocable Seal"

The Irrevocable Seal

> "在文件上盖章只是开始。等到章上的墨迹渗入纸纤维，才算真正生效。"

故事

云州城的文书系统在盖章后还有一个传统：等待。

即时章与最终章

旧制度下，文书盖章后立即生效。但有一个问题：如果后来发现章是伪造的，或者盖章的人后来被证实没有权限，已经生效的文书怎么办？

答案是：撤回。但撤回已经执行的文书极其困难——比如已经完成的交易、已经转移的财产，如何撤销？

新制度引入了两种章：

即时章：盖章后文书立即可以被引用、被阅读。但它有一个隐藏的属性——它可能被"替换"。如果在后续的检查中发现问题，即时章可以被一个更正后的文书替换。

最终章：文书被盖了足够多的章（来自不同官员的确认），并且经过了足够长的时间，它就变成了"不可撤销"的。最终章一旦被盖，文书永远生效，没有任何力量可以修改或撤回。

从即时到最终

文书从即时章到最终章的过程是这样的：

1. 第一个官员盖章（提议）。文书进入"即时"状态。
2. 接下来的一段时间里，其他官员陆续检查文书，如果同意，就盖上自己的确认章。
3. 当确认章的数量达到阈值（比如总官员数的 2/3），文书就进入"最终"状态。

关键在于第三步的"一段时间"。在这段时间里，有可能出现两个互相矛盾的文书都被部分官员确认的情况。比如：

- 文书 A 被 40% 的官员确认。
- 文书 B（与 A 矛盾）被 35% 的官员确认。

如果系统现在停止，A 和 B 都没有达到 2/3 的阈值，所以都不会最终生效。官员们会继续确认，直到其中一个达到阈值。

但如果有一个恶意官员集团，他们同时确认了 A 和 B（双重确认），试图让系统混乱怎么办？

答案是：双重确认的官员会被发现并惩罚——他们的确认资格会被取消，之前他们的所有确认都失效。这样，系统最终会收敛到一个唯一的最终文书。

概率 **vs** 确定性

云州城的旧账本（基于算力竞争）有一个不同的最终性模型：

在旧制度下，文书盖章后，它"很可能"是正确的，但不是"绝对"正确。理论上，如果有人拥有超过一半的算力，他们可以重写历史——替换已经被盖章的文书。但随着文书被越来越多的后续文书覆盖，重写的成本呈指数级增长。

在新制度下（基于押注共识），最终性是确定性的：一旦文书被足够多的官员确认并达到最终状态，它就绝对不可更改。不需要概率估算，不需要假设"没有人拥有超过一半的算力"。

用数字来说：
- 旧制度：一笔交易在 6 个区块后"几乎不可能"被回滚。但"几乎不可能"不是"绝对不可能"。
- 新制度：一笔交易在大约 12 分钟后（2 个 epochs）达到最终状态。一旦最终，绝对不可回滚。

回滚的幽灵

即使在新制度下，在最终章盖下之前，仍然可能发生"回滚"。

如果一个官员的节点出现了软件 bug，或者网络分区导致他看到了一个与其他官员不同的世界，他可能会确认一个"错误的"文书。在这种情况下，系统会短暂地出现分叉——两个版本的账本同时存在。

但这种分叉是暂时的。当网络恢复、官员们重新同步后，他们会根据规则选择"权重更大"的版本继续，抛弃另一个版本。在分叉期间被确认的文书，如果最终没有被选中，就会失效。

这就是为什么"即时章"不等于"最终章"。在即时章盖下后、最终章盖下前，一切都还有变数。

多年后，云州城的商人们形成了一个习惯：小额交易信任即时章，大额交易等待

最终章。一位老账房先生说："即时章让你跑得更快，最终章让你睡得更安稳。聪明的交易者知道什么时候该求快，什么时候该求稳。"

这则寓言在说什么

这则寓言对应的是 区块最终性（**Block Finality**）——区块链中交易被确认后不可回滚的属性。PoW（如比特币）提供的是"概率最终性"，而 PoS（如以太坊）通过 Casper FFG 提供"确定性最终性"。

关键映射

故事元素	技术概念	说明
即时章	Block Inclusion / 区块确认	交易被打包进区块，可以被引用，但尚未最终
最终章	Finalized Block	通过 Casper FFG 等最终性小工具，区块被足够多的验证者确认后不可回滚
官员确认	Validator Attestation	验证者对区块的有效性进行投票确认
2/3 阈值	超级多数（Supermajority）	以太坊 PoS 中，需要 2/3 的验证者投票确认，区块才能达到最终性
双重确认	Double Vote / 可罚没行为	验证者对矛盾的两个区块都投票，会被 slash
惩罚取消资格	Slashing	双重投票的验证者会被惩罚，其投票失效
收敛到唯一文书	共识的最终性	系统保证最终会收敛到一个唯一的、一致的最终状态
旧制度概率最终性	PoW Probabilistic Finality	比特币等。交易被更多区块覆盖后，回滚的概率指数级降低，但理论上永远不达到 100%
新制度确定性最终性	PoS Deterministic Finality	以太坊 PoS。2 个

epochs（约 12.8 分钟）后，区块绝对不可回滚 |
6 个区块后	Bitcoin 6 confirmations	比特币中，6 个区块确认后认为交易"足够安全"（约 1 小时）
12 分钟后	Ethereum PoS Finality	以太坊 PoS 中，约 12.8 分钟（2 epochs）后交易最终化
分叉与回滚	Reorg / Fork	在最终性达到前，可能出现临时分叉，较短的链会被抛弃
网络分区	Network Partition	网络分裂导致部分验证者看到不同的状态，可能产生分叉
小额信任即时章	日常使用接受确认即可	普通转账在被打包进区块后就可认为有效
大额等待最终章	大额交易等待最终性	交易所存款、跨链桥等通常等待多个确认或最终性

为什么是这个故事？

最终性是区块链安全的核心保证，但"概率最终性"和"确定性最终性"的区别对很多用户来说很模糊。

用"即时章 vs 最终章"来隐喻最终性，有几个直觉优势：
1. 两阶段确认：即时章（确认）→ 最终章（最终化）= 交易的两个阶段
2. 概率 **vs** 确定：旧制度"几乎不可能"回滚 vs 新制度"绝对不可能"回滚
3. 分叉的临时性：官员看到不同世界 = 网络分区导致的临时分叉
4. 实际应用：小额求快、大额求稳 = 不同场景对最终性的不同需求

延伸阅读

- wiki-web3 概念原文：权益证明 PoS 共识机制
- Ethereum Finality
- Casper FFG Paper

- Probabilistic vs Deterministic Finality
- Bitcoin Confirmations

"The Proof of the Family Tree"

The Proof of the Family Tree

> "不需要展开整本族谱，只需一片叶子和一条路径，就能证明你是这个家族的人。"

故事

云州城的户籍系统保存着全城十万户人家的族谱。族谱不是一本书，而是一棵巨大的家族树——从始祖开始，每一代分叉，最终到达每一个活着的人。

传统上，如果有人要证明"张三属于这个家族"，官府需要展开整棵家族树，从始祖一路查到张三。这在只有几百人的时候还行，但在十万人时，每次证明都需要翻阅数千页羊皮纸。

一位叫梅克的数学家发明了一种新的证明方法。

家族树的重新组织

梅克把家族树改造成了"哈希家族树"。

规则是这样的：
1. 每个人有一个"姓名哈希"——把他的名字通过一个数学公式变成一个独一无二

的数字指纹。
2. 每对父母的姓名哈希相加，再通过同样的公式，产生"家庭哈希"。
3. 每对祖父母的家庭哈希相加，产生"宗族哈希"。
4. 一层一层向上，最终产生一个"树根哈希"——代表整棵家族树的唯一指纹。

这棵树的奇妙之处在于：树根哈希包含了整棵树的所有信息。如果任何一个人的名字被改动，从他本人到树根路径上的所有哈希都会改变，树根哈希也会改变。

梅克证明

现在，张三要证明自己是家族的一员，他不需要展开整棵树。他只需要提供两样东西：

1. 自己的名字
2. 一条"路径"——从他本人到树根，路径上每一层兄弟姐妹的哈希值

验证官拿到这两样东西后，从张三的名字开始，依次和路径上的兄弟姐妹哈希相加，逐层向上计算。如果最终算出来的树根哈希和官府记录的树根哈希一致，证明就通过了。

如果张三撒谎，说自己是家族的人但实际上不是，他要么：
- 提供一个假名字——但验证时算不出正确的树根哈希。
- 伪造路径上的某个兄弟姐妹哈希——但哈希是数学计算出来的，无法伪造。

最关键的是：验证官不需要知道家族里其他九万九千九百九十九人是谁。他只需要张三提供的那一小段路径——通常只有几十个哈希值——就能完成验证。

Patricia 压缩

梅克后来又做了一个优化：他发现家族树里有很多"独生子女"——某个家庭只有一个孩子，这个孩子又只有一个孩子，这样连续十几代都是单传。

在传统家族树里，这十几代都要单独占一层，浪费了很多空间。梅克的优化是：把连续的单传压缩成一层。比如"张三 → 张四 → 张五 → 张六"（都是独子），压缩成"张三 → 张六"，中间三代被跳过。

这大大减少了树的深度，也减少了证明时需要提供的路径长度。

家族树的应用

哈希家族树在云州城有了广泛应用：

- 户籍验证：证明某个人是否属于云州城。
- 财产登记：证明某块土地确实登记在某人名下，不需要查阅全部土地记录。
- 交易记录：证明某笔交易确实发生过，不需要下载全部账本。

在交易记录的应用中，账房先生把每一笔交易看作家族树的一片叶子。树根哈希每分钟更新一次，公布在城门口。任何人要证明"某笔交易确实发生在这一分钟内"，只需要提供该交易的路径——几十个字节——就能让任何路人验证。

这则寓言在说什么

这则寓言对应的是 **Merkle Patricia Trie（MPT）**——以太坊用于存储账户状态、交易、收据等数据的核心数据结构。MPT 结合了 Merkle Tree（可验证性）和 Patricia Trie（压缩效率），允许用极小的"证明"来验证某个键值是否存在于巨大的状态树中。

关键映射

故事元素	技术概念	说明
家族树	Merkle Patricia Trie	以太坊的状态树，存储所有账户状态、合约代码、存储等
姓名哈希	Keccak256 哈希	对键（如账户地址）进行哈希，确定其在树中的位置
家庭哈希 / 宗族哈希	内部节点的哈希	每个节点的哈希 = 其子节点哈希的组合，层层向上
树根哈希	State Root / Merkle Root	代表整棵树状态的 32 字节哈希，存储在区块头中
梅克证明	Merkle Proof	证明某个叶子存在于树中，只需要提供从叶子到根路径上的兄弟节点哈希
不需要展开整棵树	轻客户端验证	轻节点不需要下载全部状态，只需要区块头和 Merkle Proof 就能验证某个状态
独生子女压缩	Patricia Trie 压缩	如果路径上没有分支（即只有一个子节点），Patricia Trie 会压缩路径，减少树深
财产登记	账户状态证明	可以证明某个地址的余额、nonce、codeHash、storageRoot
交易记录	Transaction Receipt Proof	可以证明某笔交易确实被包含在某个区块中
账本	Blockchain State	整个以太坊世界状态的集合

为什么是这个故事？

Merkle Tree 是区块链可扩展性的基石——没有它，轻客户端就不可能存在。但"树"和"证明"的概念对非技术读者来说非常抽象。

用"家族树"来隐喻 MPT，有几个直觉优势：
1. 路径证明的可视化：从叶子到根的路径 = 从后代到祖先的血缘链

2. 压缩的直觉：连续单传压缩 = Patricia Trie 的路径压缩
3. 轻客户端的价值：不需要知道全部族人 = 轻节点不需要下载全部状态

延伸阅读

- wiki-web3 概念原文：Merkle Patricia Trie Explained
- Ethereum Wiki: Patricia Tree
- Merkle Trees and SPV
- EVM Deep Dives: State Trie

"The Fingerprint Seal"

The Fingerprint Seal

> "你的印章不能被别人复制，但任何人都能验证它确实是你盖的。"

故事

云州城的公文需要盖章才能生效。传统的章是物理的——玉石雕刻，每人一枚。但物理章有三个问题：可以被偷、可以被复制、可以被强迫使用。

数学家们发明了一种"指纹印章"系统，彻底改变了云州城的认证方式。

指纹印章的原理

指纹印章不是实体。它是一套数学规则：

1. 每个人在出生时（或创建身份时），通过一种特殊的数学仪式，生成一对密钥：
- 私钥：一个只有本人知道的秘密数字。这个数字从不离开持有者的脑海（或密码箱）。
- 公钥：从私钥通过数学公式推导出来的公开数字。任何人都可以知道公钥，但从公钥反推私钥在数学上是不可能的。

2. 当需要盖章时，持有者把"文件内容"和"私钥"一起输入一个数学公式。公式输出一个独特的"签名"——一串看起来像乱码的数字。

3. 验证时，任何人都可以用"文件内容"+"签名"+"公钥"这三个公开的信息，输入另一个数学公式。如果公式输出"真"，就证明签名确实是这个公钥对应的私钥生成的。

关键之处：
- 不可伪造：没有私钥，无法生成有效的签名。而私钥只有持有者知道。
- 不可抵赖：一旦签名被验证通过，持有者无法否认"这不是我签的"——因为只有他的私钥能生成这个签名。
- 公开可验证：验证签名只需要公钥（公开的），不需要私钥（秘密的）。

椭圆曲线的魔法

指纹印章的数学基础是"椭圆曲线"。

椭圆曲线是一种特殊的数学曲线，它有一个奇妙的性质：在曲线上定义一种"加法"运算，使得从任何一个起点出发，反复做"自加"运算，会产生一个看似随机、实际上完全确定的点序列。

私钥就是"自加的次数"。公钥就是"自加之后的终点"。

从终点反推自加次数？在数学上被认为是不可行的——就像给你一个钟表上的终点位置，让你反推指针转了多少圈。理论上可能，但实际上需要的计算量超出了宇宙的寿命。

指纹印章的脆弱性

指纹印章系统有一个微妙的漏洞：如果持有者用同一个私钥对两份不同的文件盖了章，而且两份文件的某些部分有特定的关系，攻击者就可能通过这两份签名反

推出私钥。

这个漏洞被发现后，云州城的数学家增加了一个规则：每次盖章时，必须先生成一个一次性的"随机盐"，把盐和文件一起输入签名公式。这样即使文件内容相同，两次盖章产生的签名也完全不同——因为盐值不同。

这个改进彻底消除了漏洞。

指纹印章的日常

在云州城，指纹印章已经渗透到每个角落：

- 商人转账时，用指纹印章证明"这笔钱确实是我授权的"。
- 官员发布命令时，用指纹印章证明"这道命令确实出自我"。
- 契约签订时，双方都用指纹印章，确保事后无法抵赖。

没有人再使用物理印章了。一位老官员说："物理印章证明的是'谁拥有这枚玉'。指纹印章证明的是'谁掌握这个秘密'。前者可以被偷，后者不能。"

这则寓言在说什么

这则寓言对应的是 **ECDSA（Elliptic Curve Digital Signature Algorithm）**——以太坊（及大多数区块链）使用的数字签名算法。它允许用户用私钥对交易签名，任何人可以用公钥验证签名的有效性，而无法从公钥反推出私钥。

关键映射

故事元素	技术概念	说明
私钥	Private Key	256 位随机数，是账户的唯一控制凭证。必须保密
公钥	Public Key	从私钥通过椭圆曲线乘法（secp256k1）推导出来。可以公开，无法反推私钥
数学仪式	Key Generation	`PublicKey = PrivateKey * G`，其中 G 是椭圆曲线的基点
签名公式	ECDSA Sign	对消息哈希进行签名，生成 (r, s) 两个值
验证公式	ECDSA Verify	用公钥、消息、签名验证签名的有效性
不可伪造	没有私钥无法签名	签名生成的计算需要私钥参与，从公钥推导私钥是 ECDLP 难题，计算不可行
不可抵赖	Non-repudiation	有效的签名证明私钥持有者确实授权了该消息
椭圆曲线	secp256k1	以太坊使用的椭圆曲线。其离散对数问题的困难性保证了安全性
自加运算	Scalar multiplication	`k * G` = G 自加 k 次。这是椭圆曲线的核心运算
反推次数的困难	Elliptic Curve Discrete Logarithm Problem (ECDLP)	已知 Q = k*G，求 k。在经典计算机上没有已知的多项式时间算法
随机盐	Nonce (k)	ECDSA 签名需要一个随机数 k。如果 k 泄露或用相同的 k 签两条不同消息，私钥可被恢复
一次性盐	RFC 6979 deterministic nonce	使用消息和私钥的哈希来生成确定性 nonce，避免随机数生成器的弱点
转账授权	Transaction Signing	以太坊交易的 `v, r, s` 就是 ECDSA 签名

为什么是这个故事？

数字签名是区块链安全的第一道防线，但"公钥、私钥、椭圆曲线"的组合对新手

来说极其抽象。

用"指纹印章"来隐喻 ECDSA，有几个直觉优势：
1. 私钥的秘密性：只有持有者知道私钥 = 只有持有者能盖章
2. 公钥的公开性：任何人可以验证 = 任何人可以核对印章
3. 数学保证：从公钥反推私钥不可能 = 印章无法被伪造
4. 随机盐的重要性：重复使用相同的随机数会导致私钥泄露——这是一个真实且严重的漏洞（如 PS3 的 ECDSA 漏洞）

延伸阅读

- wiki-web3 概念原文：椭圆曲线加密与数字签名
- Ethereum ECDSA Recovery
- ECDSA: The Digital Signature Algorithm of a Better Internet
- secp256k1 Explained

"The Shredder"

The Shredder

> "无论放进去的是一页纸还是一本书，出来的总是同样大小的碎屑。你无法把碎屑拼回原样，但同样的输入永远产生同样的碎屑。"

故事

云州城的档案馆里有一台神奇的机器：碎纸机。

这台碎纸机的规矩非常特别：

第一，任何大小的输入，输出总是固定大小。

你放进去一页纸，出来一百片碎屑。你放进去一千页书，出来的也是一百片碎屑——不多不少。输入可以无限大，输出永远固定。

第二，同样的输入，永远产生同样的输出。

如果你把同一页纸放进去两次，两次出来的碎屑图案完全一样。这看起来是废话，但在云州城的应用场景中极其重要：它意味着任何人都可以独立验证"这份文件确实被这台机器处理过"。

第三，输出看起来是完全随机的。

你只改变输入的一个字，出来的碎屑图案会完全不同——和原来的输出没有任何相似之处。你无法从输出的图案反推输入的内容。

第四，不可能找到两个不同的输入产生相同的输出。

理论上存在这种可能（因为输入无限而输出有限），但在实践中，找到这样的两个输入需要的尝试次数超出了宇宙的寿命。云州城的数学家把这叫"抗碰撞性"。

碎纸机的应用

碎纸机在云州城有无数用途：

密码验证：
云州城的客栈不再存储客人的真实密码。客人第一次入住时，说出密码，客栈用碎纸机处理密码，只保存碎屑图案。下次客人来，再说一遍密码，客栈用碎纸机处理，对比图案。如果一致，就放行。

好处是：即使客栈的账本被偷，小偷得到的只是一堆碎屑图案，无法反推出原始密码。

文件完整性：
商人签订契约后，双方各自用碎纸机处理契约全文，保存碎屑图案。日后如果任何一方声称"契约被篡改了"，另一方可以把契约重新放入碎纸机，对比图案。如果图案变了，就说明契约确实被改过。

寻址：
云州城的图书馆用碎纸机来给书编号。不是按顺序编号，而是把书的内容放入碎纸机，用输出的碎屑图案作为书的编号。这样每本书的编号是独一无二的，而且与书的内容直接绑定。如果书的内容被改动，编号也会变。

碎纸机的局限

碎纸机不是万能的。

如果输入的"空间"很小——比如密码只有四个数字——攻击者可以尝试所有 10000 种组合，找到产生目标碎屑图案的那个输入。这叫"暴力破解"。

所以云州城的密码要求至少有十二个字符。可能的组合数变得如此之大，暴力破解在宇宙寿命内都不可能完成。

另一个局限是：碎纸机无法区分"相似的输入"。你把"同意"和"同意。"（多了一个句号）分别放入碎纸机，出来的碎屑图案完全不同——尽管对人类来说这两个词意思几乎一样。

这则寓言在说什么

这则寓言对应的是 密码学哈希函数（如 Keccak-256，以太坊使用的哈希算法）。哈希函数将任意长度的输入映射为固定长度的输出（如 32 字节），具有确定性、抗碰撞性、单向性等特性。

关键映射

故事元素	技术概念	说明
碎纸机	哈希函数 (Keccak-256)	将任意输入映射为固定长度输出的确定性函数
任何大小输入，固定大小输出	固定输出长度	Keccak-256 无论输入多大，总是输出 32 字节
同样输入，同样输出	确定性 (Deterministic)	相同的输入永远产生相同的

哈希值，这是验证的基础 |
改一个字，输出完全不同	雪崩效应 (Avalanche Effect)	输入的微小变化导致输出的巨大变化
无法反推输入	单向性 / 原像抗性 (Preimage Resistance)	从哈希值反推输入在计算上不可行
抗碰撞性	碰撞抗性 (Collision Resistance)	找到两个不同输入产生相同哈希值在计算上不可行
密码验证	密码哈希存储	不存储明文密码，只存储哈希值。验证时重新计算哈希对比
文件完整性	数据完整性校验	文件的哈希值作为指纹，任何改动都会导致哈希变化
图书馆寻址	内容寻址 (Content Addressing)	IPFS、Swarm 等使用内容的哈希值作为地址
四个数字密码被暴力破解	弱输入空间	如果输入空间太小（如短密码），哈希的单向性无法提供有效保护
十二个字符要求	足够大的输入空间	确保暴力破解在计算上不可行

为什么是这个故事？

哈希函数是密码学的基石，也是区块链中无处不在的工具——从地址生成到交易哈希到 Merkle Tree。但它的抽象性让很多人难以理解"为什么同样的输入永远产生同样的输出，同时看起来又完全随机"。

用"碎纸机"来隐喻哈希函数，有几个直觉优势：
1. 固定输出：无论书多厚，碎屑数量固定 = 哈希长度固定
2. 确定性：同一页纸碎两次，结果一样 = 哈希的确定性
3. 单向性：碎屑拼不回原纸 = 哈希的单向性
4. 完整性校验：对比碎屑图案 = 对比哈希值验证数据完整性

延伸阅读

- wiki-web3 概念原文：哈希函数与默克尔树
- Keccak Team: The Keccak Reference
- NIST: SHA-3 Standard
- Ethereum: Accounts and Keys

"The Cave of Zero Knowledge"

The Cave of Zero Knowledge

> "我可以向你证明我知道密码，而不告诉你密码是什么。"

故事

云州城外有一座神奇的洞穴，入口分成左右两条通道，在洞穴深处交汇于一道魔法门。这道门需要密码才能打开。

有一天，一个探险家声称自己知道门的密码。他的朋友不相信。探险家说："我可以证明给你看，但我不会告诉你密码是什么。"

洞穴证明

探险家和朋友设计了一个验证游戏：

1. 朋友站在洞穴外，看不到里面的情况。
2. 探险家走进洞穴，随机选择左通道或右通道进入。
3. 朋友在外面喊一声："从左通道出来！"或者"从右通道出来！"
4. 探险家必须从朋友指定的通道走出来。

如果探险家不知道密码，他只能原路返回——从进去的通道出来。朋友有 50%

的概率恰好喊他进去的那条通道，他能成功。但有 50% 的概率朋友喊的是另一条通道，他没有密码打不开魔法门，无法从另一条通道出来，证明失败。

但如果探险家知道密码呢？他可以打开魔法门，从任意一条通道出来。所以无论朋友喊哪条通道，他都能成功。

概率的力量

一次游戏不能证明什么——即使不知道密码，探险家也有 50% 的概率蒙对。

但如果重复这个游戏一百次呢？如果探险家每次都能从朋友指定的通道出来，而朋友每次随机选择左右，那么连续一百次蒙对的概率是 `(1/2)^100`——这比宇宙中所有原子的数量还小。

朋友不需要知道密码是什么。他只需要观察"探险家是否每次都能从指定的通道出来"，就能以极高的置信度相信：探险家确实知道密码。

而且，在整个过程中，朋友没有获得任何关于密码本身的信息。他不知道密码是一个字还是一百个字，不知道密码是数字还是文字。他只验证了"探险家知道密码"这个事实，而知识本身是零泄露的。

从洞穴到数学

数学家们后来把这个洞穴证明抽象成了一般的数学框架：

- 证明者（探险家）：掌握某个秘密知识的人。
- 验证者（朋友）：想要确认证明者确实掌握知识，但不想知道这个知识本身的人。
- 挑战：验证者提出一个需要知识才能回答的问题。
- 响应：证明者回答问题。
- 重复：通过多次独立的挑战-响应，置信度指数级增长。

在现代密码学中，这个过程被数学化了。证明者不需要真的走进洞穴——他只需要执行一系列数学运算，生成一个"证明字符串"。验证者用另一个数学公式检查这个字符串。如果检查通过，验证者就相信证明者确实掌握了秘密知识。

整个证明过程的核心是：验证者从证明中获得的信息量为零—— hence "零知识"。

零知识证明的应用

隐私交易：
我要证明"我的账户余额足够支付这笔转账"，但我不想透露我的具体余额是多少，也不想透露我转给了谁。零知识证明让我可以证明"这个声明为真"，而不泄露任何附加信息。

扩容：
我要执行一千笔复杂的交易。不是在主链上一笔一笔执行（这很慢），而是先在链下执行，然后生成一个零知识证明："这千笔交易都被正确执行了，最终状态是 X。"主链只需要验证这个证明（很快），就能确认千笔交易的正确性，而不需要知道每笔交易的细节。

这就是 ZK-Rollup 的核心原理。

多年后，洞穴成了云州城的著名景点。导游对每个游客说："这个洞穴证明了数学上最不可思议的一件事——你可以证明你知道一个秘密，而整个证明过程中，你关于这个秘密的信息量是零。"

这则寓言在说什么

这则寓言对应的是 零知识证明（**Zero-Knowledge Proof, ZKP**），特别是 **ZK-SNARKs**——一种允许证明者向验证者证明某个声明为真，而不泄露任何超出声明本身的信息的密码学技术。

关键映射

故事元素	技术概念	说明
洞穴的魔法门	秘密知识 / Witness	证明者掌握的秘密（如私钥、解谜答案、中间计算结果）
密码	Witness / Secret Input	零知识证明中的私密输入，不对外泄露
左右通道	挑战-响应的两个分支	验证者随机选择挑战，证明者必须根据挑战做出正确响应
朋友随机喊左右	验证者的随机挑战	验证者生成随机数作为挑战，确保证明者无法预先伪造
一百次游戏	重复挑战降低错误概率	ZKP 通过多次独立挑战，将作弊概率降至可忽略
不知道密码也能蒙对一次	完备性与可靠性	完备性：诚实的证明者总能通过验证。可靠性：不知道知识的证明者几乎无法通过
朋友不知道密码内容	零知识性 (Zero-Knowledge)	验证者从交互中获得的信息量为零，无法推导出秘密
数学化的洞穴证明	ZK-SNARK / ZK-STARK	非交互式零知识证明，将交互过程压缩为单次证明生成和验证
证明字符串	Proof	由证明者生成的一串数据，验证者用算法验证其有效性
隐私交易	Zcash / Aztec / Tornado Cash	使用 ZKP 隐藏交易金额、发送方和接收方，同时确保交易有效性
千笔交易的证明	ZK-Rollup	在链下执行大量交易，生成一个 ZKP 证明其正确性，主链只验证证明

| 链下执行链上验证 | 计算外包 + 正确性证明 | ZK-Rollup 的核心：把计算移到链下，把验证留在链上 |

为什么是这个故事？

零知识证明是密码学中最反直觉的概念之一——"证明你知道 X，但不泄露任何关于 X 的信息"听起来像魔法。

"洞穴证明"（Cave Proof，由 Quisquater 等人提出）是解释零知识证明最经典的直觉模型。用这个故事来隐喻 ZK-SNARKs，有几个优势：
1. 零知识的可视化：朋友验证了探险家知道密码，但确实不知道密码是什么
2. 概率置信度：多次重复挑战 = ZKP 的安全保证
3. 现代应用的联系：从洞穴到 ZK-Rollup 的过渡展示了零知识证明从理论到实践的演进

延伸阅读

- wiki-web3 概念原文：ZK-SNARKs 简介
- Zero-Knowledge Proofs: An Illustrated Primer
- zk-SNARKs: Under the Hood
- ZK-Rollup FAQ
- The Cave of Zero Knowledge (Original Paper)

"The Automated Market"

The Automated Market

> "不需要掌柜的市集，靠一根永远平衡的天平运转。"

故事

云州城有两大特产：东边产丝绸，西边产茶叶。两地的商人自古以来就需要交易，但一直有个麻烦——谁来决定"一匹丝绸换多少斤茶叶"？

传统的做法是靠"市侩"——一种专门的中间人，他们每天观察供需，报出一个双方都能接受的价格。但市侩有个致命的毛病：他们会撒谎。当丝绸供过于求时，市侩告诉茶商"丝绸很便宜"，同时告诉丝商"丝绸很抢手"，从中赚取差价。

云州城的新任知府决定废除市侩制度。他在城中心建了一座"自动市集"。市集里没有掌柜，没有报价员，只有一座巨大的天平——左边托盘放丝绸，右边托盘放茶叶。

自动市集的运行规则极其简单，刻在石板上，一字不改：

"左边托盘里的丝绸数量 × 右边托盘里的茶叶数量 = 一个固定的常数。"

这个常数是天平在建造时设定的——由最初注入市集的丝绸和茶叶数量决定。如果最初注入了 100 匹丝绸和 1000 斤茶叶，常数就是 100,000。

规则生效的第一天，一个丝商带着 10 匹丝绸来换茶叶。他把丝绸放到左边托盘上——现在左边有 110 匹。根据规则，右边必须调整，使得 110 × 右边 = 100,000。所以右边的茶叶必须变成 100,000 ÷ 110 ≈ 909 斤。

右边托盘原来有 1000 斤，现在变成 909 斤——少了 91 斤。这 91 斤茶叶就是丝商用 10 匹丝绸换到的。

丝商很满意。他不需要和市侩讨价还价，规则是公开的，结果是确定的。他放下 10 匹丝绸，取走 91 斤茶叶，整个过程不需要任何人的许可。

但这里有一个微妙之处：丝商换到的 91 斤茶叶，不是按"10 匹丝绸值 91 斤茶叶"的固定汇率计算的。如果他只带 1 匹丝绸来换，左边变成 101 匹，右边变成 100,000 ÷ 101 ≈ 990 斤——他只换到 10 斤茶叶。如果他带 50 匹丝绸来换，左边变成 150 匹，右边变成 100,000 ÷ 150 ≈ 667 斤——他换到 333 斤茶叶，但付出了 50 匹丝绸。

换的越多，单匹丝绸能换到的茶叶就越少。这就是"价格影响"——大额交易会显著改变天平的平衡。

自动市集的另一个重要参与者是"注池人"。注池人不交易，他们只是往天平两边同时注入等值的丝绸和茶叶。作为回报，他们获得一种叫"池票"的凭证——代表他们在市集中所占的份额。

每当有交易发生，交易商会支付一笔极小的"滑点费"（比如 0.3%），这笔费用不会进入任何人的口袋，而是直接留在天平里——让左边和右边的总量都略微增加。

这意味着：注池人的份额虽然比例没变，但对应的绝对数量在缓慢增长。他们什么都不做，只是在收钱。

有个聪明的商人发现了这一点。他计算出：如果自动市集的交易量足够大，滑点

费的累积速度会超过"无常损失"（因为价格波动导致的相对损失）。于是他把全部身家都投入了注池，成了最大的注池人。

三年后，他成了云州城最富有的人之一。他没有做过一笔交易，他的财富来自于无数交易者的涓滴积累。

但自动市集也有它的脆弱之处。

有一年，东边的桑树遭遇了虫害，丝绸产量骤减。市场上的丝绸价格飙升，但自动市集的天平反应迟钝——它仍然按照原有的比例运转，直到有人带着大量茶叶来换丝绸，把左边的丝绸几乎换空。

当天平左边的丝绸只剩下 1 匹时，右边还有 100,000 斤茶叶。此时如果有人想用 1 匹丝绸换茶叶，他能换到多少？100,000 ÷ (1+1) = 50,000 斤。但这意味着市集里的茶叶几乎被抽空。

更大的问题是：当一边的资产几乎被掏空时，市集就失去了意义——没有人还能在这里交易。这叫"流动性枯竭"。

知府后来做了两个改进：第一，鼓励更多的注池人参与，让天平的基数足够大，单笔交易不会显著影响比例。第二，引入了一个"价格预言"——当外部市场的丝绸价格与天平的比例严重偏离时，自动市集会给出套利空间，吸引套利者来恢复平衡。

多年后，自动市集已经从云州城的一座城市设施，变成了整个帝国的标准基础设施。各地的自动市集互相连通，形成了一个庞大的交易网络。没有市侩，没有掌柜，只有石板上刻着的那个简单公式：

左边 × 右边 = 常数

一位游历的数学家在自动市集前驻足良久，最后说了一句被刻在石板背面的

话："这不是市集，这是数学在统治商业。"

这则寓言在说什么

这则寓言对应的是 **Uniswap V2** 的自动做市商（**AMM**）机制。Uniswap V2 使用恒定乘积公式 `x × y = k` 来决定交易价格，其中 `x` 和 `y` 是流动性池中两种代币的数量，`k` 是常数。

关键映射

故事元素	技术概念	说明
自动市集	Uniswap V2 Pair	两种代币组成的流动性池，无订单簿，无做市商
天平	恒定乘积公式 `x × y = k`	交易后两种代币数量的乘积必须保持恒定
丝商换茶叶	Swap 交易	用户向池子注入一种代币，取出另一种代币。池子按照 `x × y = k` 自动调整价格
换得越多单价越低	价格影响 / Slippage	大额交易会导致价格显著偏离市场均价。交易规模相对于池子深度越大，价格影响越大
注池人	流动性提供者（LP）	向池子同时注入两种代币，获得 LP Token 作为份额凭证
滑点费	交易手续费（0.3%）	每笔交易收取的手续费自动重新注入池子，增加 `k` 值，LP 按比例分享收益
池票	LP Token（ERC-20）	代表流动性份额的可转让代币
无常损失	Impermanent Loss	当外部市场价格变化时，LP 的资产组合价值低于单纯持有的价值，因为 AMM 的再平衡机制会"买高卖低"
虫害导致的价格失衡	外部价格与池子价格偏离	当外部市场价格大幅变动时，套利者会在池子中套利，直到池子价格与外部市场一致

| 流动性枯竭 | 单边资产被掏空 | 如果池子一侧资产耗尽，交易将无法进行。需要足够的流动性和激励机制来维持双边余额 |

为什么是这个故事？

AMM 是 DeFi 最核心的创新之一，但它的机制对新手来说非常反直觉——"没有买方卖方，价格从哪里来？"

用"天平"来隐喻 `x × y = k` 极其直观：
- 往一边加东西，另一边必须按比例减少
- 加的越多，取出的比例越不划算
- 天平的总价值（k）会因为手续费而缓慢增长

故事中的"注池人"也对应了 LP 的角色——他们承担无常损失的风险，换取手续费收益。而"市侩的废除"则隐喻了 AMM 的核心价值：去掉中间人，用不可更改的数学规则来定价。

延伸阅读

- wiki-web3 概念原文：Uniswap V2 之书
- Uniswap V2 Whitepaper
- Uniswap V2 Core Concepts
- Impermanent Loss Explained

"The Anchored Vessel"

The Anchored Vessel

> "一艘船不靠码头，而是靠一根链子把自己锁在水底。链子越短，船越稳；但浪太大时，链子会断。"

故事

银海国的商人做跨国贸易时，最头疼的不是风浪，而是汇率。今天丝绸值一百两银子，明天可能因为邻国打仗涨到一百二十两，后天又跌到八十两。价格的不确定性让长期契约几乎不可能签订。

国师提议发行一种"锚定币"——一种价值永远等于一百斤大米的代币。无论银价如何波动，一枚锚定币永远能兑换一百斤大米。商人可以用锚定币签订长期契约，不用担心银价变动。

但问题是：谁来保证锚定？

第一代锚定船：全额抵押

国师的第一种方案很简单：每发行一枚锚定币，国库就实际储备一百斤大米。任何人随时可以用一枚锚定币来国库兑换一百斤大米。

这被称为"全额抵押"模式。锚定币的信用完全来自于国库里的真实大米。只要国库不空，锚定币就稳如磐石。

但全额抵押有个致命的成本：国库里的百万斤大米，绝大部分时间只是躺在仓库里，没有任何收益。这些大米本可以借给农民种地、借给商人周转——现在它们被"锁死"了，只为支撑锚定币的信用。

国库的官员算了笔账：为了发行一百万枚锚定币，国库需要锁定一百万斤大米，年损失的机会成本约十万两银子。这笔代价最终由谁来承担？要么是纳税人补贴，要么是锚定币的使用者支付高额手续费。

第二代锚定船：部分抵押

第二代设计师想出了一个聪明的办法：不需要全额抵押，只需要"足够的"抵押。

具体做法是：国库只储备五十斤大米来支撑一枚锚定币。如果所有持币人同时来兑换，国库当然不够——但这种情况几乎不可能发生。日常情况下，只有一小部分人会来兑换，五十斤大米绰绰有余。

剩下的五十斤大米可以拿出去投资、放贷、生息。投资收益的一部分用来补贴锚定币的持有者，让锚定币不仅稳定，还有收益。

但部分抵押有个隐含的赌注：它赌的是"不会所有人都同时来兑换"。一旦市场恐慌，所有人同时涌向国库，五十斤大米瞬间被掏空，剩下的持币人手中的锚定币就变成了废纸。

这就是"脱锚"——锚定币的价值跌破了一百斤大米的承诺。

第三代锚定船：算法调节

第三代设计师走得更远。他们说："既然全额抵押太贵，部分抵押太危险，那我

们干脆不要抵押了。"

他们设计了一种"算法锚定船"。这艘船没有国库，没有储备，只有一套自动调节的规则：

- 当锚定币的市场价格高于一百斤大米时，系统自动增发锚定币，卖给市场，把价格压回去。增发的收益被系统保留。
- 当锚定币的市场价格低于一百斤大米时，系统用保留的收益来回购并销毁锚定币，把价格拉回去。

理论上，这套系统可以自我平衡。不需要任何抵押，纯粹靠算法来维持价格。

但现实中的风浪远比理论复杂。有一年，市场上突然流传谣言："银海国要打仗了。"锚定币的持有者开始恐慌抛售，价格从一百斤大米暴跌到五十斤。按照规则，系统应该启动回购——但系统里保留的收益远远不够买回市场上抛售的全部锚定币。

算法锚定船沉了。没有任何抵押品可以支撑它，它就像一艘没有压舱物的船，一阵大风就能掀翻。

第四代锚定船：混合锚定

第四代设计师吸取了前三代的教训。他们设计了一种"混合锚定"系统：

- 核心层：每一枚锚定币都有实打实的超额抵押（比如一百二十斤大米支撑一百枚锚定币）。这是安全的底线。
- 缓冲层：一部分抵押品可以投入低风险收益协议，产生的收益分配给持币人。这是效率的提升。
- 调节层：当市场价格轻微偏离时，算法自动微调供给；当偏离严重时，系统启动紧急抵押清算，确保核心层的足额兑付。

更关键的是，第四代锚定船的抵押品不是单一的大米，而是多样化的资产组合：大米、丝绸、茶叶，甚至其他国家的国债。这样即使某一种资产暴跌，整体抵押

率仍然安全。

多年后，银海国的锚定币成了整个大陆最稳定的货币。没有人知道它的背后是全额抵押还是部分抵押还是算法调节——因为第四代的设计让这三者融为一体。

一位老船长在港口看着来来往往的锚定币商船，对学徒说："你知道锚定币最厉害的地方是什么吗？不是它不会贬值，而是当它真的面临风浪时，你甚至不会感觉到颠簸。因为那艘船在设计时，已经假设了最坏的海况。"

这则寓言在说什么

这则寓言对应的是现代稳定币的设计演变，从全额抵押（USDC）、超额抵押（DAI）、算法稳定币（UST）到混合型设计（如 FX 2.0）。稳定币的核心挑战是在稳定性、资本效率和去中心化之间取得平衡——这三者构成了著名的"稳定币不可能三角"。

关键映射

故事元素	技术概念	说明
锚定币	稳定币（Stablecoin）	价值锚定于法币（通常是 USD）或其他资产的加密货币
全额抵押	法币抵押稳定币（USDC、USDT）	每一枚稳定币都有等值的法币或短期国债作为储备。最安全，但资金利用率低
部分抵押	部分储备模型	历史上传统银行的做法，也是一些算法稳定币的早期尝试

算法调节	纯算法稳定币（UST、AMPL）	无抵押，完全靠智能合约调节供给来维持价格。资本效率最高，但抗风险能力最弱
UST 的崩溃	死亡螺旋（Death Spiral）	当市场恐慌抛售导致价格跌破锚定，系统回购能力不足，信心崩溃导致进一步抛售，恶性循环
混合锚定	混合型稳定币 / FX 2.0	结合超额抵押、收益生成和算法调节的多层安全设计
多样化抵押	多资产抵押（DAI、FX 2.0）	使用多种加密资产作为抵押，降低单一资产暴跌的风险
超额抵押	Over-collateralization	如 DAI 要求抵押率 150%+，即每发行 100 DAI 需要锁定至少 150 美元的 ETH
紧急清算	Liquidation Mechanism	当抵押品价值下跌导致抵押率不足时，系统自动拍卖抵押品，保护系统偿付能力

为什么是这个故事？

稳定币是 DeFi 的基石，但 2022 年 UST/Luna 的崩溃让数百亿美元蒸发，证明了算法稳定币在极端压力下的脆弱性。

用"锚定船"的演变来隐喻稳定币设计的三代失败和第四代融合，有几个好处：
1. 历史脉络清晰：从全额抵押到算法调节再到混合设计，对应了真实稳定币的发展历程
2. 不可能三角直觉化：安全性（链子牢）、资本效率（船轻快）、去中心化（无船长），三者不可兼得
3. 死亡螺旋可视化：算法船的沉没过程被具象化为"一阵大风掀翻没有压舱物的船"

延伸阅读

- wiki-web3 概念原文：现代稳定币 FX 2.0 是如何构建的
- [MakerDAO: How DAI Works]

- Terra/Luna Collapse Analysis
- FX 2.0 Stablecoin Design

"The Pawnshop and the Vault"

The Pawnshop and the Vault

> "借钱不难，难的是让借出去的钱能自己回家。"

故事

云州城有两家老字号的借贷机构。一家叫"老周当铺"，一家叫"新丰保险库"。它们的业务看起来一样——都让人拿东西抵押来借钱——但运作方式天差地别。

老周当铺的规矩

老周当铺的做法很简单：你把值一百两银子的东西押在他那儿，他借给你六十两。三个月后，你要还六十六两（本金加利息）。如果到期还不上，东西就归老周，他拿去卖了抵债。

这套规矩运转了三十年，没出过什么大乱子。但有几个显而易见的痛点：

第一，老周只收金银珠宝和房产地契。如果你有一船即将到港的香料，价值很高，但货还没上岸，老周不收——"看不见的东西我不认"。

第二，老周的额度是有限的。他一个人能调动的银子就那么多，当十个人同时来借钱时，后面的人只能排队等。

第三，也是最麻烦的：如果你押的是房产，三个月到期时房价跌了，你的房产只值五十两了，但老周借给你的是六十两。这时候老周就会陷入两难：把房产卖了只能收回五十两，亏十两；不卖房，继续等你还钱，但房价可能继续跌。

新丰保险库的创新

新丰保险库的掌柜姓林，是个学过算术的年轻人。他看了老周当铺的痛点，设计了一套全新的规矩：

第一，万物皆可押。

林掌柜说："我不看东西是什么，我只看它值多少。"他雇了一批估价师，每天给城中各种资产定价：大米多少、丝绸多少、香料多少、甚至未来某个商队的预期收益多少。只要估价师给出了价格，这东西就可以抵押。

第二，池子而不是柜子。

老周当铺的钱是老周自己的。林掌柜的钱来自"注池人"——任何人都可以把钱放进保险库的公共池子里，供借款人借用。作为回报，注池人按比例分享借款人支付的利息。

这意味着：保险库的借贷能力不再受限于掌柜一个人的财力，而是取决于整个社区愿意投入多少资金。

第三，实时清算。

这是林掌柜最 radical 的发明。在老周当铺，只有当三个月到期时，才会检查抵押品是否还值六十两。在保险库，这个检查是实时的——每一天、每一个时辰，系统都在自动重新评估抵押品的价值。

如果抵押的香料价格下跌，导致抵押品价值从一百两跌到七十两，而借款额是六十两——安全。但如果继续跌到五十两，系统就会立即触发"清算"：保险库自动

把香料卖掉，收回五十两，亏损的十两从借款人预先缴纳的"保证金"里扣。

借款人不爽，但保险库的注池人是安全的——他们的钱不会被坏账侵蚀。

第四，闪电借贷。

林掌柜还推出了一项前无古人的业务："闪电借贷"。

规矩是这样的：你可以借到保险库里任意数量的钱，但有一个条件——你必须在同一笔交易里把钱还回来，外加一笔极小的手续费。

"同一笔交易里？"所有人都懵了。借钱和还钱之间，总得有时间差吧？

林掌柜解释："不是给你时间去投资。闪电借贷是给那些需要'瞬间大量资金来做一件事'的人用的。比如，有人在两个市场之间发现了价格差——甲市场丝绸卖一百两，乙市场卖一百一十两。但他自己没有一百两的本金。他可以借闪电贷，用一百两在甲市场买丝绸，立刻在乙市场卖一百一十两，还掉一百两借款和一两手续费，净赚九两——全部在一笔交易里完成。"

如果交易最后没有赚到钱，或者借款人没有按时还钱怎么办？

"那就整笔交易回滚，"林掌柜说，"就像什么都没发生过。保险库的钱一分都不会少。"

新丰保险库的生意很快就超过了老周当铺。不是因为林掌柜更慷慨，而是因为他的设计让风险在系统中流动，而不是堆积在某一个人身上。

老周当铺的风险是老周一个人承担。新丰保险库的风险由注池人、借款人、清算人三方分担。注池人提供资金并赚取利息，但承担的是"市场系统性风险"（如果整个市场暴跌，所有人一起亏）。借款人享受随时借贷的便利，但承担的是"被实时清算"的风险。清算人则是在市场动荡时赚取佣金的"鬣狗"——他们时刻盯着哪些抵押品濒临跌破阈值，然后在触发瞬间执行清算，赚取清算奖励。

这三方形成了一个动态的生态系统。没有人是纯粹的赢家或输家，每个人都在用自己的方式管理风险。

这则寓言在说什么

这则寓言对应的是 **Aave** 借贷协议——一个去中心化的、非托管的流动性协议。用户可以存入资产赚取利息，也可以抵押资产借出其他资产。

关键映射

故事元素	技术概念	说明
老周当铺	传统中心化借贷	需要信任第三方（老周），额度有限，清算周期长
新丰保险库	Aave 协议	去中心化、非托管、池化借贷
万物皆可押	多资产抵押支持	Aave 支持多种 ERC-20 代币作为抵押品，每种资产有独立的 Loan-to-Value (LTV) 比率和清算阈值
公共池子	流动性池（Liquidity Pool）	存款人（lenders）提供资金到池中，借款人从池中借出。利率由算法根据资金利用率动态调整
注池人	存款人 / aToken 持有者	存入资产后获得 aToken（如 aDAI），aToken 余额自动增长以反映累计利息
实时清算	自动清算机制	Aave 使用 Chainlink 预言机实时更新资产价格。当抵押品价值低于清算阈值时，任何人都可以触发清算
清算人	Liquidators	在清算触发时偿还借款人的债务，以折扣价获得抵押品，赚取清算奖励
保证金	超额抵押（Over-collateralization）	借款人必须存入价值高于借款额的抵押品。例如借 100 DAI 可能需要锁定 150 美元的 ETH
闪电借贷	Flash Loan	Aave 首创的无抵押、零风险借贷。借款和还款必须

在同一笔交易的原子操作内完成。如果最终无法归还，整笔交易回退 |
| 利率算法 | Interest Rate Model | Aave 根据资金利用率（借出/存入比率）动态调整利率。利用率高时借款利率上升，鼓励还款和新存款 |

为什么是这个故事？

传统金融的借贷是"中心化、低频、低透明"的，而 Aave 的颠覆性在于把它变成了"去中心化、高频、全透明"的自动协议。

用"当铺 vs 保险库"的对比来呈现这个转变，有三个直觉优势：
1. 抵押品范围的扩展：从"只收金银珠宝"到"万物皆可押"，对应了 DeFi 的可组合性
2. 风险的分摊：老周一个人承担 vs 系统多方分担，对应了去中心化金融的风险社会化
3. 闪电借贷的奇妙："同一笔交易里借钱还钱"这种在传统金融中不可能存在的机制，被具象化为一种"瞬间套利工具"

延伸阅读

- wiki-web3 概念原文：Aave
- Aave Docs: Protocol Overview
- Aave V3 Whitepaper
- Flash Loans Explained

"The Messengers in the Alley"

The Messengers in the Alley

> "在所有人都能读信的系统里，读得最快的人制定规则。"

故事

云州城的邮政系统有一个奇特的设计：所有的信件在送达之前，都会先被挂在城门口的公告板上，供全城人免费阅读。邮差按照邮费高低来决定取信的顺序——谁付的邮费高，谁的信就先被取走、先被送达。

这个设计的初衷是透明。但很快，一群特殊的"信使"在城中的暗巷里形成了一个隐秘的江湖。

第一类信使：夹心贼

夹心贼不写信，他们只读别人的信。

当商人许真写了一封信给市集："我要用一百两银子买十匹丝绸。"这封信被挂在公告板上。夹心贼读到了。

他立刻做了三件事：
1. 写第一封信："我要用高价买光市集上所有便宜的丝绸。"（付加急邮费，确保

先于许真的信送达）
2. 等许真的信送达，许真只能以更高的价格买入丝绸。
3. 写第二封信："我要卖出刚才买入的全部丝绸。"（付加急邮费，确保紧接在许真的信之后送达）

结果：夹心贼低价买入、高价卖出，差价就是他的利润。许真什么也没做错——他只是按照市价买了丝绸——但他买的"市价"已经被夹心贼人为抬高了。

这叫"夹心攻击"——在受害者的交易前后各插入一笔交易，像三明治一样夹住受害者。

第二类信使：套利鹰

套利鹰同时在观察两个市集：东城市集和西城市集。

东城市集的丝绸卖一百两一匹，西城市集卖一百零五两一匹。正常情况下，这种价差会被商人自然抹平——有人从东城买入、西城卖出，直到价格一致。

但套利鹰比正常商人快得多。他不需要自己有丝绸库存。他只需要：
1. 从"闪电钱庄"借一笔钱（同一笔交易内必须归还）。
2. 用借来的钱在东城买入丝绸。
3. 立刻在西城卖出丝绸。
4. 归还借款和手续费。
5. 净赚差价。

全部在五秒钟内完成。正常商人还在算账的时候，套利鹰已经把钱装进口袋了。

第三类信使：清道夫

清道夫不追求利润，他们追求"确定性"。

保险库的规矩是：当抵押品价值跌破阈值时，任何人都可以执行清算，以折扣价获得抵押品。但清算需要精确的时机——如果太早，抵押品还没跌破阈值，清算无效；如果太晚，可能有人抢在你前面执行了清算。

清道夫的工作就是：实时监控所有保险库的抵押品，在阈值被触发的瞬间，用最高邮费发送清算信，确保自己的信第一个被邮差取走。

清道夫的收入不高——每笔清算只有几个百分点的折扣——但他们是零风险的。只要他们足够快，就永远不会亏。

暗巷的秩序

这三类信使的活动让云州城的邮政系统陷入了混乱。正常的商人发现：无论他们怎么出价，信总是比夹心贼慢一步。市场不再由供需决定，而由"谁能付更高的邮费"决定。

更糟的是，暗巷里的信使开始互相攻击。两个夹心贼同时盯上了同一封信，他们竞相提高邮费，最后利润被邮费吃掉了。套利鹰发现东城和西城的价差越来越窄——因为太多人在套利，机会转瞬即逝。

一位叫费洛的老邮差看不下去了。他做了一件事：在城外建了一个"私人投递站"。

私人投递站的规矩是：信使们不再把信挂到公共公告板上，而是直接把信交给费洛。费洛不看信的内容，只按照信使的报价排序，然后把排好序的信打包，一次性送给城门口的邮差。

这样做有两个好处：
1. 信的"内容"不再公开——夹心贼无法读取别人的信了。
2. 信使们不再需要在公告板上竞价——他们在费洛这里进行"密封拍卖"，出价最高的人排在最前面，但没人知道别人的出价。

费洛的私人投递站很快成了主流。夹心贼失业了——他们再也读不到别人的信。

套利鹰仍然存在，但他们只能从公开的市场信息中寻找机会，而不能利用 mempool 的透明性。清道夫也可以继续工作，但竞争变得更有序。

费洛从每笔交易中抽取一点点费用，作为自己的报酬。他没有发明任何新技术，他只是改变了信息的流动方式。

这则寓言在说什么

这则寓言对应的是 **MEV**（**Maximal Extractable Value**，最大可提取价值）和 **Flashbots** 等 MEV 缓解方案。MEV 是指区块生产者（或搜索者）通过控制交易排序所能提取的额外价值，包括抢跑、尾随、三明治攻击、清算套利等。

关键映射

故事元素	技术概念	说明
公共公告板	内存池（Mempool）	待处理交易在被打包进区块前是公开可见的，任何人都可以读取
邮费决定顺序	Gas Price / Priority Fee	矿工/验证者优先打包 gas 价格更高的交易
夹心贼	三明治攻击者（Sandwich Attacker）	在受害者的大额 swap 交易前后插入买卖交易，利用价格滑点套利
套利鹰	套利搜索者（Arbitrage Searcher）	利用不同 DEX 之间的价格差进行原子套利，通常使用闪电贷
闪电钱庄	闪电贷（Flash Loan）	同一笔交易内借入和归还的零抵押贷款。Aave、dYdX 等协议提供
清道夫	清算机器人（Liquidation Bot）	监控抵押品价格，在触发清算条件

时第一时间提交清算交易，赚取清算奖励 |
暗巷混乱	MEV 的负面外部性	MEV 提取导致普通用户面临更差的价格、更高的滑点，以及网络拥堵
费洛的私人投递站	Flashbots / MEV-Boost / 私有 mempool	通过私有交易池（如 Flashbots Protect）或 PBS（Proposer-Builder Separation）来减少公开的 gas 竞价战争
密封拍卖	MEV-Boost 的拍卖机制	搜索者向区块构建者提交密封的交易包（bundle），构建者选择最有利可图的组合
费洛的抽成	区块构建者的利润	构建者从 MEV 中提取价值，与验证者分享

为什么是这个故事？

MEV 是区块链中最黑暗、最复杂的话题之一。它不是一个单一的技术问题，而是一个"激励结构"问题——当交易顺序可以被影响时，影响交易顺序本身就成了利润来源。

用"暗巷中的信使"来隐喻 MEV 生态系统，有三个优势：
1. 可视化：夹心贼、套利鹰、清道夫是三种典型的 MEV 策略，对应了现实中的三明治攻击、DEX 套利和清算机器人
2. 系统性：MEV 不是一个 bug，而是一个 feature of the system。故事展示了"公开 mempool + gas 竞价"这个设计如何必然地催生 MEV
3. 解决方案的隐喻：Flashbots 的私有 mempool 和 PBS 被具象化为"私人投递站"，让普通用户理解了"为什么需要改变信息流动方式"

延伸阅读

- wiki-web3 概念原文：Mempool 监听与抢先交易实战
- Flashbots: MEV Overview
- MEV-Boost: PBS for Ethereum

- Ethereum.org: MEV
- EigenPhi: MEV Dashboard

"The Deep Well and the Shallow Pond"

The Deep Well and the Shallow Pond

> "不是水越多越好，而是水要在对的地方。"

故事

云州城的自动市集（Uniswap V2）运转良好，但注池人们逐渐发现了一个令人沮丧的事实：他们投入的资金，大部分时间在"闲置"。

在 V2 的自动市集里，注池人必须同时往天平两边注入等值的资产。这些资金分布在整个价格区间——从接近零到无限大。这意味着：无论市场价格在哪里，只有很小一部分资金在真正参与交易。绝大部分资金躺在价格区间的两端，像冬眠的熊一样一动不动。

一位叫陈深的数学家计算了一个惊人的数字：在 V2 市集中，大约只有 0.5% 的资金在任何一个给定的价格点附近真正"工作"。其余 99.5% 的资金虽然躺在池子里，但对交易毫无贡献——它们只是安静地等待一个可能永远不会到来的极端价格。

"这太浪费了，"陈深说，"如果我知道丝绸和茶叶的合理价格区间是一匹丝绸换

80-120 斤茶叶，为什么我要把钱分布在 0 到 ∞ 之间？"

陈深设计了一种新型市集："区间市集"。

在区间市集里，注池人不再被迫把资金分布在整个价格轴上。他们可以指定一个区间："我只在 80-120 斤茶叶/匹丝绸的范围内提供流动性。"

这就像一个注池人在市集里租了一个特定位置的摊位，而不是均匀地分布在整个广场上。当市场价格在他租的范围内时，他的资金参与交易，赚取手续费。当市场价格跑到他的范围之外时，他的资金自动"退出"——全部变成单一资产，不再参与交易，也不再赚取手续费。

这个设计带来了几个戏剧性的变化：

第一，资金效率提高了数十倍。

在 V2 市集中，一万两银子的投入可能只产生十两的日手续费。在区间市集中，如果注池人把资金集中在当前价格附近最活跃的区间，同样的资金可以产生数百两的日手续费——因为他的资金不再被稀释到永远不会触及的价格区间。

第二，摊位位置成了策略。

在 V2 市集中，所有注池人都是一样的——你投入多少，就按份额比例赚多少。在区间市集中，不同的摊位位置意味着不同的风险和回报。

保守的注池人选择一个很宽的区间（比如 50-200），这样市场价格很少会跑出他的范围。但他的资金利用率低，因为宽区间内只有一小部分资金在价格点附近真正工作。

激进的注池人选择一个很窄的区间（比如 95-105），把全部资金压缩在最活跃的价格点附近。他的资金利用率极高，手续费收入也是保守者的几十倍。但风险也极大——如果市场价格波动稍微大一点，跑出他的区间，他的全部资金就会变成单一资产，停止赚钱。

第三，市集出现了"深度图"。

V2 市集像一个均匀深度的游泳池——无论你从哪个位置跳下去，深度都一样。V3 区间市集像一个不规则的深井——有些位置深不见底，有些位置只是浅浅的水洼。

交易者喜欢深井：在深度大的地方，大额交易对价格的影响很小。在浅洼处，哪怕一笔小交易也会引起巨大的价格跳动。

聪明的注池人会观察市集的深度图，在"浅洼"处注入资金——因为那里的竞争少，手续费会被更少的人分享。但这也意味着他们在冒险：浅洼之所以浅，往往是因为那里的价格区间不被市场看好。

区间市集上线一年后，出现了一个令人意外的现象：市集整体的流动性并没有增加，但资金的分布变得极不均匀。

在价格 100 附近——这是历史最活跃的价格——深井的深度是 V2 时代的二十倍。交易者在这里可以享受极低的滑点。但在价格 200 附近——这是很少触及的极端价格——浅洼几乎干涸。如果有人突然需要在这个价格做一笔大额交易，他可能会发现：根本没有足够的流动性来支撑。

陈深对此的解释是："这不是 bug，这是 feature。V2 用均匀的方式隐藏了真实的供需关系。V3 把真实的供需关系暴露出来了。资金自然会流向最有利可图的地方，这是市场的选择。"

但批评者说："极端情况下的流动性枯竭是一个安全隐患。当市场恐慌时，所有人都想逃离，但浅洼里没有足够的水来缓冲冲击。"

这场争论至今没有定论。

这则寓言在说什么

这则寓言对应的是 **Uniswap V3** 的集中流动性（**Concentrated Liquidity**）机制。V3 最大的创新是允许流动性提供者（LP）自定义价格区间，将资金集中在他们认为最活跃的价格范围内，从而大幅提高资金效率。

关键映射

故事元素	技术概念	说明
V2 自动市集	Uniswap V2 AMM	流动性均匀分布在 `0, ∞` 价格区间，资金利用率低
99.5% 资金闲置	V2 的资金效率问题	在任何一个实际价格点，V2 池中只有极小比例的资金在起作用
区间市集	Uniswap V3 Concentrated Liquidity	LP 可以指定价格区间 `[lower, upper]`，只在区间内提供流动性
摊位位置	Tick Range / Position	V3 将价格轴划分为离散的 ticks，每个 position 占据一个 tick 范围
宽区间 50-200	保守 LP 策略	选择较宽的区间，降低无常损失风险，但资金效率也较低
窄区间 95-105	激进 LP 策略	资金高度集中，手续费收益最大化，但无常损失风险和价格跑出区间的风险极高
跑出区间	Out of Range	当价格超出 position 的范围时，该 position 的全部流动性变成单一资产，不再赚取手续费
深度图	Liquidity Distribution	V3 允许观察不同价格区间的流动性深度。主流价格区间极深，极端价格区间极浅
浅洼中的竞争少	非均匀分布的套利空间	在流动性薄弱的区间，小资金也能获得较高比例的手续费分成，但面临更大的无常损失
极端情况流动性枯竭	尾部风险	V3 的流动性集中在当前价格附近，极端行情下可能出现流动性真空，加剧价格波动

为什么是这个故事？

Uniswap V3 是 DeFi 中最精密的设计之一，但也是最难直觉理解的设计。"集中流动性"这个概念——把资金从无限区间压缩到有限区间——对很多初学者来说非常反直觉。

用"深井与浅池"来隐喻流动性分布，有几个直觉优势：
1. 空间感：把抽象的价格区间具象化为"摊位位置"，让读者理解"位置选择本身就是一种策略"
2. 风险回报：窄区间 = 深井 = 高收益高风险，宽区间 = 浅洼 = 低收益低风险
3. 系统性影响：V3 让流动性分布"真实化"，暴露了市场的真实结构——这不是 bug，而是市场效率的提升，但也带来了新的尾部风险

延伸阅读

- wiki-web3 概念原文：Uniswap V3 开发指南
- Uniswap V3 Whitepaper
- Uniswap V3 Core Concepts
- Charged Particles: V3 LP Strategies

"The Water Dividers"

收入拆分协议.md'
tags: [defi, revenue-sharing, splits, royalty, collaboration]

The Water Dividers

> "最难的不是赚钱，而是公平地分钱。"

故事

云州城的东边有一条运河，来自三个水源：山泉、雨水渠和地下水脉。三个水源的主人约定：运河的收益（灌溉费、船运费）按照水源贡献的比例分配——山泉占 50%，雨水渠占 30%，地下水脉占 20%。

听起来简单，但执行起来 nightmare。

老办法：人工分水

最开始，运河的终点建了一个分水闸。闸工老赵每天根据收益总额，手动计算每个水源应得的份额，然后分别送去。

今天收益一百两：老赵拿出账本，一百 × 50% = 五十两给山泉主人，三十两给

雨水渠主人，二十两给地下水脉主人。然后分别写三张银票，派人送去。

如果只有两个水源有贡献呢？比如今天没有下雨，雨水渠没有水流入运河。老赵还是得走一遍流程：计算、写银票、派人送——虽然雨水渠主人收到的是零两。

更麻烦的是，当有新的水源加入时。假设第四个水源——水库——也想接入运河。老赵需要：重新计算比例、改造分水闸、更新账本、通知所有相关方。整个过程需要三天。

而且，如果有人质疑："老赵，你算的五十两对吗？"老赵只能翻到账本的某一页，指着一行字说："你看，我记着呢。"但这无法证明他没有记错，也无法证明他没有偷偷改数字。

新办法：自动分水渠

有一天，一个年轻的工程师设计了一套"自动分水系统"。

系统的核心是一个"分水契约"——写在青铜板上，挂在运河边的公共祠堂里，任何人都可以阅读。契约的内容极其简单：

"凡流入运河之水，其产生的收益，按以下比例自动分配：山泉 50%，雨水渠 30%，地下水脉 20%。"

契约还附带了一个"自动分水渠"的装置：当收益流入运河终点时，装置会自动按照契约规定的比例，把收益分别导流至三个水源主人的账户。不需要老赵，不需要账本，不需要银票。

更令人惊讶的是：这个装置是"不可更改的"。一旦青铜板上的契约被刻好，任何人都不能单方面修改比例。如果三个水源主人协商一致要修改比例，他们必须一起重新铸造一块新的青铜板，把旧的换掉。

这意味着：老赵的"信任问题"被彻底解决了。没有人需要相信老赵不会算错账——因为老赵已经被移出了整个流程。

新问题的出现

自动分水系统运行了半年后，出现了一个新的需求。

山泉主人是一个家族，家族内部有五个兄弟，需要把山泉的 50% 份额再按兄弟五人平分。老办法是老赵手动再分一次。但自动分水系统不支持"嵌套分配"——它只能按照第一层契约分配，无法自动处理第二层。

年轻工程师升级了系统：允许"子契约"。

现在，分水系统变成了两层：
- 第一层：运河总收益 → 按主契约分给三个水源
- 第二层：山泉的份额 → 按子契约分给五兄弟
- 第二层还可以继续：老大的份额 → 按孙契约分给他的妻子和孩子

每一层契约都是独立的青铜板，都有自己的自动分水装置。收益像水一样，从源头层层分流，直到到达最终的接收者。

而且，任何一层契约都可以被"分叉"——如果五兄弟中的老三决定自立门户，他可以把自己的份额从家族子契约中提取出来，创建自己的独立子契约，不再参与家族分配。这个过程不需要其他四兄弟的同意，也不需要老赵的批准。

自动分水系统的意外应用

自动分水系统最初是为运河设计的，但很快被人发现了更广泛的应用场景。

- 乐坊分成：一个乐曲由词作者、曲作者、演奏者共同创作。他们创建一个三向分水契约：词 30%，曲 40%，演奏 30%。每当有人付费听这首乐曲，收益自动按此比例分配。
- 客栈合伙：一家客栈由十个人合伙出资。他们创建一个十向分水契约，按照出

资比例分配利润。任何人随时可以把他的份额卖给其他人——只需要把分水契约中自己的地址改成新主人的地址。
- 慈善募捐：一个慈善项目创建了双向分水契约：90% 给受助者，10% 给平台运营者。捐赠者可以看到契约内容，确认自己的钱不会被截留。

在这些场景中，自动分水系统的核心价值不是"计算"——计算本身很简单——而是"信任"。参与各方不需要信任任何中间人，只需要信任那块公开的青铜板。

多年后，云州城的分水系统已经连接了数千个收益流。有些收益流只有两层分配，有些有十几层。有些契约是永久不变的，有些每季度根据业绩自动调整比例。

老赵没有失业。他转型成了"契约顾问"——帮助人们设计最合适的分水结构。毕竟，自动分水系统能完美地执行任何契约，但它不能替人决定"什么样的契约是公平的"。

老赵常说："机器解决了'怎么分'的问题。但'分多少'的问题，永远需要人来回答。"

这则寓言在说什么

这则寓言对应的是 **0xSplits**——一个链上的收入拆分协议，允许创作者、合作者、团队成员等按照预设比例自动分配收益。它的核心价值在于：去信任化、不可篡改、可组合。

关键映射

故事元素	技术概念	说明
运河与三个水源	多方的协作收益	任何需要按固定比例分配收益的场景
老赵的人工分水	传统链下收益分配	依赖第三方（如平台、会计）手动计算和转账，存在信任成本和延迟
青铜板上的分水契约	0xSplits 的智能合约	链上不可更改的分配规则，代码即法律
自动分水装置	自动执行的分账逻辑	当收益流入时，合约自动按照预设比例分配给各接收方
不可更改的契约	不可篡改性 / Immutability	一旦部署，分配比例不能单方面修改，防止中途变卦
嵌套子契约	Splits 的可组合性	一个 Split 的接收方可以是另一个 Split 的地址，形成多层分配结构
老三自立门户	份额的可转让性	任何人可以把自己在 Split 中的份额提取出来或转让给别人
乐坊分成	版税/版权分配	NFT 创作者、音乐人可以设置永久版税，二级市场交易收益自动分配
客栈合伙	投资/合伙收益分配	投资者、团队成员按约定比例自动分润
慈善募捐	透明资金流向	捐赠者可以验证资金去向，确保不被挪用

为什么是这个故事？

收入拆分是一个看起来简单、实际极复杂的问题。链下解决方案要么依赖信任第三方，要么执行成本高、透明度低。

用"分水渠"来隐喻 0xSplits 有几个直觉优势：
1. 流动性：收益像水一样流动，分配是自然的物理过程
2. 可组合性：子契约嵌套对应了 Splits 的递归结构
3. 信任替代：青铜板替代了老赵，解决了"谁来监督监督者"的问题

延伸阅读

- wiki-web3 概念原文：分析 0xSplits — 收入拆分协议
- 0xSplits Official Docs
- Mirror: Splits for Creators

"The Programmable Bazaar"

The Programmable Bazaar

> "市集不仅卖东西，还允许你在每个关键节点插入自己的规矩。"

故事

云州城的自动市集（Uniswap V2/V3）已经运转了很多年。它的规则是固定的：买卖双方按照恒定乘积公式交易，手续费按比例分配给注池人。没有人可以改变这些规则。

但有一天，市集的建造者宣布了一个革命性的升级："可编程市集"。

市集的核心变革

在旧市集里，每种商品组合（丝绸-茶叶、大米-银子）都需要建一个独立的市集。云州城里有几千种商品，意味着有几千个独立的市集，每个都有自己的天平、自己的注池、自己的账本。

可编程市集把所有这些合并成了一个"超级市集"。所有商品组合共享同一个天平、同一个账本。不同的是：每个商品组合可以安装自己的"插件"。

插件可以在以下关键时刻介入：

1. 交易前：当有人想要交易时，插件可以先检查条件——"这个交易者是否在黑名单上？""今天的交易量是否已经超过限额？"

2. 价格计算时：插件可以修改价格的计算方式——"如果单笔交易超过一千两，手续费从 0.3% 提高到 1%。"

3. 交易后：插件可以触发后续操作——"完成交易后，把手续费的 10% 自动捐赠给慈善基金。"

4. 注池人加入时：插件可以设定准入条件——"只有持有特定徽章的人才能成为注池人。"

插件的创造力

可编程市集的插件系统催生了很多前所未有的创新。

动态手续费：
一个插件让手续费根据市场波动自动调整。当市场平静时，手续费很低，吸引交易。当市场剧烈波动时，手续费自动提高，保护注池人免受无常损失。

限价单：
另一个插件实现了"限价单"功能。交易者可以设定："当丝绸价格跌到 80 两/匹时，自动买入一百匹。"在旧市集里，这需要不断手动检查价格。在新市集里，插件自动监控并执行。

时间加权平均价：
第三个插件在每次交易时记录价格，维护一个"过去一小时的平均价"。其他合约可以引用这个平均价，而不需要自己计算——这大大节省了 gas。

自定义曲线：
最激进的插件完全抛弃了恒定乘积公式。一个插件实现了"稳定币曲线"——当两种资产的价格接近 1:1 时，交易几乎没有滑点。这让稳定币之间的兑换变得极其

高效。

插件的风险

但插件也引入了新的风险。

一个恶意的插件可以在交易前偷偷修改价格，让交易者以极不利的价格成交。或者，插件可以在交易后把手续费转到插件作者自己的口袋，而不是给注池人。

市集的建造者意识到了这个问题。他们设计了一个规则：

- 插件的代码是公开的，任何人都可以审计。
- 但安装插件需要"市集管理员"的批准。管理员不审查插件的"创意"，只审查插件是否"安全"——即不会偷钱、不会破坏市集的基本功能。
- 一旦插件被批准安装，它的行为就是确定性的、不可更改的。管理员也不能事后修改插件。

这个设计平衡了"创造力"和"安全性"：任何人都可以提出插件，但只有经过安全审查的插件才能被安装。而且一旦安装，插件就不再受任何人的控制——它按照代码自动执行。

超级市集的账本革新

可编程市集的另一个重大变化是"瞬时记账"。

在旧市集里，每笔交易都会立即改变账本——天平两边的数量实时更新。这意味着每笔交易都需要写一次账本，消耗一次存储费用。

在可编程市集里，引入了一个"临时账本"。交易者可以在短时间内进行多笔交易，所有这些交易只在临时账本上记录。只有当交易者"结算"时，临时账本的内容才被一次性写入正式账本。

这意味着：一个套利者可以在一次结算中完成十笔交易，但只需要支付一次存储费用。这极大地提高了高频交易的效率。

多年后，可编程市集成了云州城最繁荣的商业中心。一位老商人说："旧市集告诉你能做什么。新市集只告诉你不不能做什么——在这两条线之间，是你的创造空间。"

这则寓言在说什么

这则寓言对应的是 **Uniswap V4** 的 **Hooks** 机制——V4 最大的创新。V4 将每个交易池（pool）从独立的合约转变为"Singleton"架构下的一个状态条目，并允许在每个 pool 上安装自定义的 Hooks 合约，在交易生命周期的关键点（如 beforeSwap、afterSwap、beforeModifyPosition 等）执行自定义逻辑。

关键映射

故事元素	技术概念	说明
旧市集几千个独立天平	Uniswap V3 的 Pool 架构	每个 pool 是一个独立合约，部署成本高
超级市集共享天平	Uniswap V4 Singleton 架构	所有 pools 共享一个合约，通过状态存储区分，部署成本大幅降低
插件	Hooks 合约	自定义合约，在交易生命周期的特定钩子点被调用
交易前检查	beforeSwap hook	在 swap 执行前调用，可以实施 KYC、限

额、黑名单等 |
价格计算时	Dynamic fee hook	在 swap 过程中可以动态调整手续费
交易后触发	afterSwap hook	在 swap 完成后调用，可以触发捐赠、奖励分配等
注池人加入时	before/afterModifyPosition hook	在流动性添加/移除时调用，可以实施准入控制
动态手续费	Dynamic fee tiers	根据波动率、交易量等动态调整手续费
限价单	Limit order hook	在价格达到阈值时自动执行交易
时间加权平均价	TWAP hook	在每次 swap 时更新累计价格，供外部引用
自定义曲线	Custom curves	Hooks 可以实现非恒定乘积的 AMM 曲线（如 Curve 的稳定币公式）
插件审计	Hooks 的安全审查	Uniswap V4 的 hooks 由 pool 创建者选择，但用户需要自行审计 hook 的安全性
瞬时记账	Flash Accounting	V4 引入"瞬时结算"——多个操作可以先在临时状态中累积，最后一次性结算，节省 gas
临时账本	Transient Storage (EIP-1153)	V4 使用 EIP-1153 的 transient storage 来实现瞬时记账，操作结束后自动清零

为什么是这个故事？

Uniswap V4 是 DeFi 协议设计的一个范式转变——从"固定功能的白盒"到"可编程的黑盒"。Hooks 让 AMM 从一个单一的交易机制变成了可定制的平台。

用"可编程市集"来隐喻 V4 Hooks，有几个直觉优势：
1. 插件化：市集的关键节点插入自定义规则 = Hooks 在交易生命周期中的介入点
2. **Singleton** 架构：共享的超级市集 = V4 将所有 pool 合并到一个合约中
3. 瞬时记账：临时账本一次性结算 = Flash Accounting 节省 gas
4. 创造力与安全性的平衡：公开代码 + 管理员审查 = Hooks 的开放性与风险

延伸阅读

- wiki-web3 概念原文：Uniswap V4 Hooks 开发详解
- Uniswap V4 Whitepaper
- Uniswap V4 Core Concepts
- EIP-1153: Transient Storage
- Uniswap V4 Hooks Directory

"The Stakers' Roulette"

The Stakers' Roulette

> It is not about who is strongest that keeps the ledger, but who stakes the most and longest that earns the right. But staking more also means losing more when wrong.

The Story

Yunzhou City's ledger was once maintained by a group of miners. They competed by solving mathematical puzzles to determine who had the right to write new entries. Whoever solved the puzzle first earned the right and the reward.

This system's problem was: the puzzle-solving competition consumed vast resources and electricity. Yunzhou City's north even had a dedicated coal mountain solely to supply the miners' computational race.

New System: The Stakers' Roulette

Yunzhou City's sages proposed a new system: instead of competing through puzzles, they would compete through "staking."

The specific rules were:

1. Anyone could become a "ledger keeper," but the prerequisite was that you must lock a security deposit in the city's public treasury. The larger the amount locked and the longer the lock time, the higher "weight" you received.

2. Every minute, the roulette spun once. Each sector on the roulette represented a ledger keeper. Sector size depended on the keeper's weight—those who staked more had larger sectors; those who staked less had smaller sectors.

3. When the roulette stopped, whoever the pointer pointed to earned that minute's ledger right—writing new transactions onto the ledger and receiving fee rewards.

4. But ledger keeping was not one person's decision alone. The selected keeper wrote the new page, and other keepers needed to "confirm"—checking whether the page's content was correct. If most people confirmed it was correct, the page was officially accepted.

Staking Risks

The stakers' roulette had a strict penalty mechanism:

If a keeper wrote incorrect content—such as deliberately omitting a transaction, or writing a false transaction—other keepers would refuse confirmation. If the keeper's error was proven, part or all of their locked security deposit would be confiscated.

This was called "slashing." Those who staked more had greater probability of earning ledger rights, more opportunities to earn

money. But once they made mistakes, their losses were also greater.

An old keeper said: "Under the old system, miners wasted electricity. Under the new system, keepers risk money. Wasted electricity can be regenerated; confiscated money is truly gone. So people under the new system are more cautious."

Queuing and Exiting

The stakers' roulette also had a feature: you could not withdraw your security deposit at any time.

When you decided to stop being a keeper, you needed to "apply to exit." From application to actually receiving your money, there was a "cooling period"—during which you might still be selected to keep the ledger, but if you made mistakes during this time, your security deposit would still be fined.

The cooling period existed to prevent one type of attack: an attacker stakes a large sum, makes one malicious ledger entry, then immediately withdraws and runs. The cooling period gives the system time to discover and punish such attacks.

Roulette Fairness

Someone questioned: Those who stake more have larger sectors; doesn't this mean the rich always have the advantage?

The sage answered: "Yes. But under the old system, those who could afford more mining machines also had advantages. Under any system, those with more resources have advantages. The stakers'

roulette at least changes resource consumption from 'wasting electricity' to 'locking capital.' Moreover, small stakers can unite, pooling their funds to share ledger rights and rewards."

Indeed, Yunzhou City quickly saw "staking pools" emerge—dozens of small stakers pooling their money, managed by professional operators running ledger nodes. Rewards were distributed according to contribution. This let small participants also share in consensus benefits.

What This Fable Is About

This fable illustrates **Proof of Stake (PoS)** consensus mechanism—the algorithm used by Ethereum (and most modern blockchains). Validators participate in block proposal and confirmation by staking ETH; the more they stake, the higher their probability of being selected to propose a block. Misbehavior causes their staked funds to be slashed.

Key Mapping

Story Element	Technical Concept	Explanation
Miners solving puzzles	Proof of Work (PoW)	Bitcoin and early Ethereum consensus mechanism; competing for block rights through computational power
Public treasury lock	Staking / Delegation	Validators deposit ETH

		into Beacon Chain contract as security for consensus participation
	Weight	Effective Balance	Validator weight determined by staked amount; cap of 32 ETH per validator
	Roulette spin	RANDAO / Random Beacon	Each slot (12 seconds) selects a block proposer through a verifiable random function
	Sector size	Selection probability	More staked = higher probability of becoming proposer
	Other keepers confirm	Attestation / Validator Committee	Each slot has a validator committee responsible for voting to confirm block validity
	Most people confirm	Committee consensus / Supermajority	If 2/3 of committee members vote to confirm, the block is accepted
	Error proven	Slashable offense	Double voting, surround voting, and other malicious behaviors can be proven and punished
	Confiscate deposit	Slashing	Misbehaving validators' staked funds are destroyed (up to 1 ETH immediate slashing; remaining portion gradually slashed over 36 days)
	Cooling period	Exit Queue / Withdrawal period	Validators must wait to exit and still participate in consensus until fully exited; prevents "misbehave then immediately withdraw"
	Staking pool	Staking Pool / Lido / Rocket Pool	Small holders participate in staking through pooling, receiving liquid staking tokens (e.g., stETH) as vouchers

Why This Story?

PoS is the core of blockchain's shift from energy-intensive to capital-intensive consensus. Understanding "why staking can replace computational power" and "how slashing prevents misbehavior" is key to understanding modern blockchain.

The "stakers' roulette" metaphor for PoS offers several intuitive

advantages:

1. **Probabilistic block visualization**: Roulette = random proposer selection; sector size = staking weight
2. **Capital replacing energy**: From burning coal to locking capital—corresponding to PoS's economic design
3. **Slashing deterrence**: Stake more earn more but wrong more lose more—corresponding to PoS incentive compatibility
4. **Staking pools**: Small participants' unions—corresponding to liquid staking protocols (Lido, etc.)

Further Reading

- wiki-web3: Proof of Stake Consensus Mechanism
- Ethereum PoS FAQ
- Consensys: Ethereum 2.0 Economics
- Slashing Conditions in Ethereum PoS

"The Brand of the Faithless"

The Brand of the Faithless

> Trust is bought with money; betrayal is repaid with money.

The Story

Yunzhou City's stakers' roulette operated well, but the sages soon discovered a problem: if the cost of a keeper's misbehavior was only "losing future earnings," then in certain situations misbehavior remained profitable.

For example, a keeper simultaneously participated in a competing ledger system in another city. If he disrupted consensus here, causing Yunzhou City's ledger chaos, he could earn huge returns from the other side. Even if he lost his security deposit here, the other side's earnings might exceed the loss.

This was called "external incentive conflict"—the gains from misbehavior might exceed the losses from punishment.

Introduction of the Branding System

To solve this problem, the sages introduced the "branding system."

The core of the branding system: misbehavior not only cost money but also left permanent marks.

Specific rules:

1. **Light brand**: If a keeper went offline briefly due to network failure or operational error, and failed to confirm blocks in time. A small portion of his security deposit would be deducted as punishment, but no brand was left.

2. **Medium brand**: If a keeper was discovered voting twice in the same round—such as simultaneously supporting two conflicting blocks. This was "double voting." A larger portion of his security deposit would be deducted, and a "yellow brand" would be left. The yellow brand did not permanently ban participation, but reduced the keeper's weight for a period.

3. **Heavy brand**: If a keeper was discovered doing "surround voting"—a more complex attack where contradictory votes were cast at different times to disrupt consensus. Nearly all of his security deposit would be confiscated (up to the entire stake), and a "red brand" would be left. The red brand meant he could almost never participate in ledger keeping again.

Most critically: brands were public, permanent, and irreversible. Anyone could query whether a keeper had brands, and what type and reason.

Deterrence of Brands

The branding system completely changed the incentive structure of

the stakers' roulette.

Under the old system, a keeper considered: "If I misbehave, how much security deposit will I lose?"

Under the new system, he had to consider: "If I misbehave, I will lose my security deposit and be permanently marked as faithless. Even if I have gains in another city, my reputation here is ruined."

For professional ledger institutions, reputation was everything. A branded institution, even if it changed names or security deposits, could still have its brand history traced. Clients would not delegate funds to an institution with red brand history.

A sage said: "Monetary punishment solves the 'economic incentive' problem. Brands solve the 'identity and reputation' problem. Combined, they make the system truly unbreakable."

False Accidents and Appeals

The branding system also had its dark side.

One keeper's node went offline for three days due to a data center fire. According to rules, he was deducted part of his security deposit for "continuous inactivity." He appealed: "This was force majeure, not my fault."

The appeal committee reviewed and determined the fire was indeed force majeure, refunding his security deposit. But the brand record still carried an annotation: "Once offline for three days due to force majeure." This annotation did not affect his weight, but anyone querying his history could see it.

The keeper was dissatisfied: "Since you determined I was not at fault, why not delete this record?"

The committee answered: "We cannot delete records, because records are carved in stone. But we can add annotations explaining the truth. History cannot be changed, but history can be interpreted."

What This Fable Is About

This fable illustrates **Slashing**—the economic punishment for misbehaving validators in PoS consensus. Slashing not only confiscates funds but also reduces validators' effective balance, affects their reputation, and leaves permanent records on-chain.

Key Mapping

Story Element	Technical Concept	Explanation
External incentive conflict	External attack motivation	Attackers may earn more from off-chain than they lose from staking (e.g., shorting, competitor bribery)
Light brand	Inactivity Leak / offline penalty	Validators not participating in consensus for extended periods slowly bleed effective balance but do not receive slashable records
Medium brand	Slashing: Double Vote	Voting for two different blocks in the same slot. 1 ETH immediate slash; remaining portion

gradually slashed over 36 days |
Heavy brand	Slashing: Surround Vote	Surround voting (more complex double signing). More severe punishment; may slash entire effective balance
Yellow / red brand	Punishment levels	Different severity slashable behaviors correspond to different slashing proportions
Public and permanent brands	Slashing record immutability	Slashing events are permanently recorded on-chain; anyone can query
Professional institution reputation	Validator reputation risk	For institutional validators, being slashed severely damages commercial credibility
Appeals and annotations	Force majeure and rule rigidity	On-chain rules execute automatically; cannot be rolled back for "non-malicious" reasons. But off-chain explanations are possible
Carved in stone	Blockchain immutability	Once confirmed on-chain, facts cannot be deleted; only information can be supplemented

Why This Story?

Slashing is the core pillar of PoS security, but many people only understand "losing money," not the dimensions of "reputation" and "permanence."

The "brand" metaphor for Slashing offers several intuitive advantages:

1. **Economic + reputation punishment**: Money loss + permanent mark = complete deterrence system
2. **Public shame**: Brands publicly queryable = transparent on-chain slashing records
3. **Rule rigidity**: Force majeure still leaves records = blockchain's

automatic execution and non-appealability

Further Reading

- wiki-web3: Proof of Stake Consensus Mechanism
- Ethereum Slashing Specifications
- Slashing Risks for Validators
- Penalties and Rewards in Ethereum PoS

"The Irrevocable Seal"

The Irrevocable Seal

> Stamping a document is only the beginning. Only when the ink seeps into the paper fibers does it truly take effect.

The Story

Yunzhou City's document system had a tradition after stamping: waiting.

Instant Seal vs. Final Seal

Under the old system, documents took effect immediately after stamping. But there was a problem: if the seal was later discovered to be forged, or if the stamper was proven unauthorized, what happened to already-executed documents?

The answer: withdrawal. But withdrawing already-executed documents was extremely difficult—for example, completed transactions, transferred property—how to reverse?

The new system introduced two types of seals:

Instant Seal: After stamping, the document could immediately be referenced and read. But it had a hidden property—it could be "replaced." If subsequent checks discovered problems, the instant seal could be replaced by a corrected document.

Final Seal: After a document received enough seals (confirmations from different officials), and enough time had passed, it became "irrevocable." Once the final seal was applied, the document was permanently effective; no power could modify or withdraw it.

From Instant to Final

The process from instant seal to final seal was:

1. The first official stamped (proposal). The document entered "instant" status.
2. Over the following period, other officials reviewed the document; if they agreed, they stamped their confirmation.
3. When the number of confirmation seals reached the threshold (say, two-thirds of total officials), the document entered "final" status.

The key was the "period" in step two. During this period, it was possible for two contradictory documents to both receive partial official confirmations. For example:

- Document A received 40% of officials' confirmation.
- Document B (contradicting A) received 35% of officials' confirmation.

If the system stopped now, neither A nor B reached the two-thirds threshold, so neither would finally take effect. Officials would

continue confirming until one reached the threshold.

But what if a malicious official group simultaneously confirmed both A and B (double confirmation), trying to confuse the system?

The answer: double-confirming officials would be discovered and punished—their confirmation qualifications would be revoked, and all their previous confirmations would become invalid. Thus, the system would eventually converge to a single final document.

Probability vs. Determinism

Yunzhou City's old ledger (based on computational competition) had a different finality model:

Under the old system, after a document was stamped, it was "likely" correct, but not "absolutely" correct. Theoretically, if someone controlled over half the computational power, they could rewrite history—replacing already-stamped documents. But as documents were covered by more and more subsequent documents, the cost of rewriting grew exponentially.

Under the new system (based on staking consensus), finality was deterministic: once a document received enough official confirmations and reached final status, it was absolutely unchangeable. No probability estimation was needed; no assumption that "no one controls over half the power" was required.

In numbers:
- Old system: A transaction was "almost impossible" to roll back after six blocks. But "almost impossible" is not "absolutely impossible."
- New system: A transaction reached final status after about twelve minutes (two epochs). Once final, absolutely irreversible.

The Ghost of Rollback

Even under the new system, before the final seal was applied, "rollback" could still occur.

If an official's node had a software bug, or network partitioning caused him to see a different world from other officials, he might confirm an "erroneous" document. In such cases, the system would briefly fork—two versions of the ledger coexisting temporarily.

But this fork was temporary. When the network recovered and officials resynchronized, they would select the "heavier" version to continue according to rules, discarding the other. Documents confirmed during the fork that were ultimately not selected would become invalid.

This is why "instant seal" does not equal "final seal." After the instant seal is applied but before the final seal, everything is still variable.

Years later, Yunzhou City's merchants formed a habit: small transactions trusted instant seals; large transactions waited for final seals. An old accountant said: "Instant seals let you run faster; final seals let you sleep more soundly. Smart traders know when to seek speed and when to seek stability."

What This Fable Is About

This fable illustrates **Block Finality**—the property that transactions, once confirmed, cannot be rolled back. PoW (like Bitcoin) provides "probabilistic finality," while PoS (like Ethereum) provides "deterministic finality" through Casper FFG.

Key Mapping

Story Element	Technical Concept	Explanation
Instant seal	Block inclusion / block confirmation	Transaction is packaged into a block; can be referenced but not yet final
Final seal	Finalized block	Through Casper FFG and other finality gadgets, blocks confirmed by enough validators are irreversible
Official confirmation	Validator attestation	Validators vote to confirm block validity
Two-thirds threshold	Supermajority	In Ethereum PoS, 2/3 of validators must vote to confirm for a block to reach finality
Double confirmation	Double vote / slashable behavior	Validators voting for two contradictory blocks are slashed
Punish, revoke qualification	Slashing	Double-voting validators are punished; their votes become invalid
Converge to single document	Consensus finality	The system guarantees eventual convergence to a single, consistent final state
Old system probabilistic finality	PoW probabilistic finality	Bitcoin, etc. More blocks covering a transaction exponentially reduce rollback probability, but theoretically never reach 100%
New system deterministic finality	PoS deterministic finality	Ethereum PoS. After two epochs (~12.8 minutes), blocks are

absolutely irreversible |
Six blocks later	Bitcoin 6 confirmations	In Bitcoin, six block confirmations are considered "sufficiently secure" (~1 hour)
Twelve minutes later	Ethereum PoS finality	In Ethereum PoS, transactions finalize after ~12.8 minutes (2 epochs)
Fork and rollback	Reorg / fork	Before finality is reached, temporary forks may occur; shorter chains are abandoned
Network partition	Network partition	Network splits cause some validators to see different states, potentially creating forks
Small trust instant seal	Daily use accepting confirmation	Ordinary transfers are considered valid after being packaged into blocks
Large wait for final seal	Large transactions waiting for finality	Exchange deposits, cross-chain bridges typically wait for multiple confirmations or finality

Why This Story?

Finality is the core guarantee of blockchain security, but the difference between "probabilistic finality" and "deterministic finality" is vague for many users.

The "instant seal vs. final seal" metaphor for finality offers several intuitive advantages:

1. **Two-phase confirmation**: Instant seal (confirmation) → final seal (finalization) = two stages of transactions
2. **Probability vs. determinism**: Old system "almost impossible" to roll back vs. new system "absolutely impossible"
3. **Temporary nature of forks**: Officials seeing different worlds = temporary forks caused by network partitioning
4. **Practical application**: Small seeks speed, large seeks stability = different scenarios' different finality needs

Further Reading

- wiki-web3: Proof of Stake Consensus Mechanism
- Ethereum Finality
- Casper FFG Paper
- Probabilistic vs Deterministic Finality
- Bitcoin Confirmations

"The Proof of the Family Tree"

The Proof of the Family Tree

> You do not need to unfold the entire genealogy; only a leaf and a path are needed to prove you belong to this family.

The Story

Yunzhou City's household registration system preserved genealogies for all one hundred thousand families in the city. The genealogy was not a book but a giant family tree—starting from the founding ancestor, each generation branching out, eventually reaching every living person.

Traditionally, if someone wanted to prove "Zhang San belongs to this family," the authorities needed to unfold the entire family tree, tracing from the founding ancestor all the way to Zhang San. This was manageable with hundreds of people, but with one hundred thousand, each proof required flipping through thousands of pages of parchment.

A mathematician named Meike invented a new proof method.

Reorganization of the Family Tree

Meike transformed the family tree into a "hash family tree."

The rules were:
1. Every person had a "name hash"—their name run through a mathematical formula to produce a unique digital fingerprint.
2. Each pair of parents' name hashes were added together and run through the same formula to produce a "family hash."
3. Each pair of grandparents' family hashes were added to produce a "clan hash."
4. Layer by layer upward, eventually producing a "tree root hash"—a unique fingerprint representing the entire family tree.

The tree's marvelous property was: the tree root hash contained all information of the entire tree. If any person's name was altered, all hashes from that person to the root path would change, and the tree root hash would change.

Meike's Proof

Now, to prove he was a family member, Zhang San did not need to unfold the entire tree. He only needed to provide two things:

1. **His own name**
2. **A "path"**—from himself to the root, the hash values of siblings at each layer

The verifier took these two items, started from Zhang San's name, and successively added the sibling hashes from the path, calculating upward layer by layer. If the final calculated tree root hash matched the authorities' recorded tree root hash, the proof passed.

If Zhang San lied, claiming to be a family member when he was not, he would either:
- Provide a fake name—but verification would not produce the correct tree root hash.
- Forge a sibling hash on the path—but hashes were mathematically computed and could not be forged.

Most critically: the verifier did not need to know who the other ninety-nine thousand nine hundred ninety-nine family members were. He only needed the small path Zhang San provided—usually only dozens of hash values—to complete verification.

Patricia Compression

Meike later made an optimization: he discovered the family tree had many "only children"—a family with one child, who had one child, with single inheritance continuing for over a dozen generations.

In the traditional family tree, each of these dozen generations occupied a separate layer, wasting much space. Meike's optimization was: compress consecutive single inheritance into one layer. For example, "Zhang San → Zhang Si → Zhang Wu → Zhang Liu" (all only sons) compressed into "Zhang San → Zhang Liu," skipping the middle three generations.

This greatly reduced tree depth and the path length required for proofs.

Applications of the Family Tree

The hash family tree found wide application in Yunzhou City:

- **Household verification**: proving whether someone belonged to Yunzhou City.
- **Property registration**: proving a piece of land was indeed registered under someone's name, without needing to review all land records.
- **Transaction records**: proving a transaction had indeed occurred, without downloading the entire ledger.

In transaction record applications, the accountant treated each transaction as a leaf of the family tree. The tree root hash updated every minute, posted at the city gate. Anyone wanting to prove "a transaction indeed occurred this minute" only needed to provide the transaction's path—dozens of bytes—to let any passerby verify.

What This Fable Is About

This fable illustrates **Merkle Patricia Trie (MPT)**—the core data structure Ethereum uses to store account states, transactions, receipts, and other data. MPT combines Merkle Tree (verifiability) and Patricia Trie (compression efficiency), allowing extremely small "proofs" to verify whether a key-value pair exists in a massive state tree.

Key Mapping

| Story Element | Technical Concept | Explanation |

|---|---|---|
Family tree	Merkle Patricia Trie	Ethereum's state tree storing all account states, contract code, storage, etc.
Name hash	Keccak256 hash	Keys (e.g., account addresses) are hashed to determine their positions in the tree
Family hash / clan hash	Internal node hashes	Each node's hash = combination of its child node hashes; layered upward
Tree root hash	State Root / Merkle Root	A 32-byte hash representing the entire tree state; stored in block headers
Meike's proof	Merkle Proof	Proving a leaf exists in the tree only requires providing sibling node hashes on the path from leaf to root
No need to unfold entire tree	Light client verification	Light nodes do not need to download all states; only block headers and Merkle proofs are needed to verify a specific state
Single child compression	Patricia Trie compression	If there are no branches on a path (only one child node), Patricia Trie compresses the path, reducing tree depth
Property registration	Account state proof	Can prove an address's balance, nonce, codeHash, storageRoot
Transaction records	Transaction receipt proof	Can prove a transaction was indeed included in a specific block
Ledger	Blockchain state	The entire collection of Ethereum world states

Why This Story?

Merkle Tree is the cornerstone of blockchain scalability—without it, light clients would be impossible. But the concepts of "tree" and "proof" are very abstract for non-technical readers.

The "family tree" metaphor for MPT offers several intuitive advantages:

1. **Path proof visualization**: Leaf to root path = descendant-to-ancestor bloodline chain
2. **Compression intuition**: Consecutive single inheritance compression = Patricia Trie path compression
3. **Light client value**: Not needing to know all family members = light nodes not needing to download all states

Further Reading

- wiki-web3: Merkle Patricia Trie Explained
- Ethereum Wiki: Patricia Tree
- Merkle Trees and SPV
- EVM Deep Dives: State Trie

"The Fingerprint Seal"

The Fingerprint Seal

> Your seal cannot be copied by others, but anyone can verify it was indeed stamped by you.

The Story

Yunzhou City's official documents required stamps to be valid. Traditional seals were physical—jade carvings, one per person. But physical seals had three problems: they could be stolen, copied, or coerced.

Mathematicians invented a "fingerprint seal" system that completely changed Yunzhou City's authentication methods.

Fingerprint Seal Principle

The fingerprint seal was not physical. It was a set of mathematical rules:

1. Everyone, at birth (or identity creation), underwent a special mathematical ritual to generate a pair of keys:

- **Private key**: a secret number known only to the individual. This number never left the holder's mind (or safebox).
- **Public key**: derived from the private key through a mathematical formula. Anyone could know the public key, but deriving the private key from the public key was mathematically impossible.

2. When a seal was needed, the holder input the "document content" and "private key" together into a mathematical formula. The formula output a unique "signature"—a string of numbers that looked like gibberish.

3. During verification, anyone could use "document content" + "signature" + "public key"—these three pieces of public information—input into another mathematical formula. If the formula output "true," it proved the signature was indeed generated by the private key corresponding to this public key.

Key points:
- **Unforgeable**: Without the private key, no valid signature could be generated. And only the holder knew the private key.
- **Non-repudiable**: Once a signature was verified, the holder could not deny "this was not me"—because only his private key could produce this signature.
- **Publicly verifiable**: Verifying a signature only required the public key (public); the private key (secret) was not needed.

Elliptic Curve Magic

The mathematical foundation of the fingerprint seal was the "elliptic curve."

An elliptic curve was a special mathematical curve with a wonderful property: defining an "addition" operation on the curve, such that

starting from any point and repeatedly performing "self-addition" produced a seemingly random but actually completely deterministic sequence of points.

The private key was the "number of self-additions." The public key was the "endpoint after self-addition."

Deriving the number of self-additions from the endpoint? Mathematically considered infeasible—like giving you a clock's hand position and asking you to reverse-calculate how many turns the hand made. Theoretically possible, but practically requiring more computation than the universe's lifetime.

Fingerprint Seal Vulnerability

The fingerprint seal system had a subtle vulnerability: if a holder used the same private key to stamp two different documents, and certain parts of those documents had specific relationships, an attacker might reverse-engineer the private key from these two signatures.

After this vulnerability was discovered, Yunzhou City's mathematicians added a rule: every time a seal was stamped, a one-time "random salt" must first be generated, and the salt and document were input together into the signature formula. Thus, even if document contents were identical, two stampings produced completely different signatures—because the salts differed.

This improvement completely eliminated the vulnerability.

Daily Life of the Fingerprint Seal

In Yunzhou City, fingerprint seals had permeated every corner:

- Merchants transferring funds used fingerprint seals to prove "this money was indeed authorized by me."
- Officials issuing orders used fingerprint seals to prove "this order indeed came from me."
- Contract signings used fingerprint seals by both parties, ensuring neither could deny it later.

No one used physical seals anymore. An old official said: "Physical seals prove 'who owns this jade.' Fingerprint seals prove 'who masters this secret.' The former can be stolen; the latter cannot."

What This Fable Is About

This fable illustrates **ECDSA (Elliptic Curve Digital Signature Algorithm)**—the digital signature algorithm used by Ethereum (and most blockchains). It allows users to sign transactions with private keys; anyone can verify signature validity with public keys, while deriving private keys from public keys is infeasible.

Key Mapping

Story Element	Technical Concept	Explanation
Private key	Private Key	A 256-bit random number; the account's

sole control credential. Must be kept secret |
Public key	Public Key	Derived from private key through elliptic curve multiplication (secp256k1). Can be public; cannot reverse-engineer private key
Mathematical ritual	Key Generation	`PublicKey = PrivateKey * G`, where G is the elliptic curve base point
Signature formula	ECDSA Sign	Signs message hash to generate (r, s) two values
Verification formula	ECDSA Verify	Uses public key, message, and signature to verify signature validity
Unforgeable	No private key = no signature	Signature generation requires private key participation; deriving private key from public key is ECDLP hard problem; computationally infeasible
Non-repudiable	Non-repudiation	Valid signature proves the private key holder indeed authorized the message
Elliptic curve	secp256k1	The elliptic curve used by Ethereum. The difficulty of its discrete logarithm problem guarantees security
Self-addition	Scalar multiplication	`k * G` = G self-added k times. Core operation of elliptic curves
Reverse-calculation difficulty	Elliptic Curve Discrete Logarithm Problem (ECDLP)	Known Q = k*G, find k. No known polynomial-time algorithm on classical computers
Random salt	Nonce (k)	ECDSA signatures require a random number k. If k leaks or the same k signs two different messages, the private key can be recovered
One-time salt	RFC 6979 deterministic nonce	Uses hash of message and private key to generate deterministic nonce, avoiding random number generator weaknesses
Transfer authorization	Transaction Signing	Ethereum transactions' `v, r, s` are ECDSA signatures

Why This Story?

Digital signatures are the first line of defense for blockchain security, but the combination of "public key, private key, elliptic curve" is extremely abstract for newcomers.

The "fingerprint seal" metaphor for ECDSA offers several intuitive advantages:

1. **Private key secrecy**: Only the holder knows the private key = only the holder can stamp
2. **Public key publicity**: Anyone can verify = anyone can check the seal
3. **Mathematical guarantee**: Deriving private key from public key impossible = seal cannot be forged
4. **Random salt importance**: Reusing the same random number leads to private key leakage—a real and serious vulnerability (e.g., PS3's ECDSA vulnerability)

Further Reading

- wiki-web3: Elliptic Curve Encryption and Digital Signatures
- Ethereum ECDSA Recovery
- ECDSA: The Digital Signature Algorithm of a Better Internet
- secp256k1 Explained

"The Shredder"

The Shredder

> Whether you put in one page or a book, the output is always the same size of shreds. You cannot reconstruct the original from the shreds, but the same input always produces the same shreds.

The Story

In Yunzhou City's archives, there was a miraculous machine: the shredder.

The shredder's rules were quite special:

First: any size of input, output is always fixed size.

You put in one page, out came one hundred shreds. You put in a thousand-page book, out came one hundred shreds—no more, no less. Input could be infinitely large; output was always fixed.

Second: same input always produces same output.

If you put the same page through twice, the shred patterns were exactly the same. This seemed obvious, but in Yunzhou City's application scenarios, it was extremely important: it meant anyone

could independently verify "this document was indeed processed by this machine."

Third: output appears completely random.

You changed one character in the input, and the shred pattern became completely different—bearing no resemblance to the original output. You could not infer input content from the output pattern.

Fourth: impossible to find two different inputs producing the same output.

Theoretically possible (because input is infinite while output is finite), but in practice, finding such two inputs required more attempts than the universe's lifetime. Yunzhou City's mathematicians called this "collision resistance."

Shredder Applications

The shredder had countless uses in Yunzhou City:

Password verification:
Yunzhou City's inns no longer stored guests' real passwords. When a guest first checked in, they spoke their password; the inn shredded the password, only saving the shred pattern. Next time the guest came, they spoke it again; the inn shredded it and compared patterns. If they matched, entry was granted.

The benefit: even if the inn's ledger was stolen, thieves only got a pile of shred patterns, unable to reverse-engineer original passwords.

Document integrity:
After merchants signed contracts, both parties shredded the full contract text, saving shred patterns. Later, if either party claimed "the contract was altered," the other could resubmit the contract to the shredder and compare patterns. If the pattern changed, the contract had indeed been modified.

Addressing:
Yunzhou City's library used shredders to number books. Not sequential numbering, but putting book contents into the shredder and using the output shred pattern as the book's number. Thus each book's number was unique and directly bound to its content. If book content was altered, the number would also change.

Shredder Limitations

The shredder was not omnipotent.

If the input "space" was small—like a password of only four digits—attackers could try all ten thousand combinations, finding the input that produced the target shred pattern. This was called "brute force."

So Yunzhou City required passwords to be at least twelve characters. Possible combinations became so large that brute force was impossible within the universe's lifetime.

Another limitation: the shredder could not distinguish "similar inputs." You put "agree" and "agree." (with an extra period) into the shredder separately, and the shred patterns were completely different—though to humans these two words meant almost the same thing.

What This Fable Is About

This fable illustrates **cryptographic hash functions** (like Keccak-256, the hashing algorithm used by Ethereum). Hash functions map inputs of any length to fixed-length outputs (e.g., 32 bytes), with properties of determinism, collision resistance, and one-wayness.

Key Mapping

Story Element	Technical Concept	Explanation
Shredder	Hash function (Keccak-256)	Maps any input to a fixed-length output through a deterministic function
Any size input, fixed size output	Fixed output length	Keccak-256 always outputs 32 bytes regardless of input size
Same input, same output	Deterministic	Same input always produces same hash value; this is the foundation of verification
Change one character, output completely different	Avalanche Effect	Tiny input changes cause huge output changes
Cannot reverse-engineer input	One-wayness / Preimage Resistance	Deriving input from hash value is computationally infeasible
Collision resistance	Collision Resistance	Finding two different inputs producing the same hash value is computationally infeasible
Password verification	Password hash storage	Do not store plaintext passwords; only store hash values. Verify by recalculating

and comparing hashes |
Document integrity	Data integrity check	File hash value serves as fingerprint; any modification causes hash change
Library addressing	Content Addressing	IPFS, Swarm, etc. use content hash values as addresses
Four-digit password brute force	Weak input space	If input space is too small (e.g., short password), hash one-wayness cannot provide effective protection
Twelve-character requirement	Sufficiently large input space	Ensures brute force is computationally infeasible

Why This Story?

Hash functions are the cornerstone of cryptography and ubiquitous tools in blockchain—from address generation to transaction hashes to Merkle Trees. But their abstract nature makes it hard for many to understand "why the same input always produces the same output while looking completely random."

The "shredder" metaphor for hash functions offers several intuitive advantages:

1. **Fixed output**: Whether the book is thick or thin, shred count is fixed = hash length is fixed
2. **Determinism**: Same page shredded twice produces same result = hash determinism
3. **One-wayness**: Shreds cannot be reassembled into original paper = hash one-wayness
4. **Integrity check**: Compare shred patterns = compare hash values to verify data integrity

Further Reading

- wiki-web3: Hash Functions and Merkle Trees
- Keccak Team: The Keccak Reference
- NIST: SHA-3 Standard
- Ethereum: Accounts and Keys

"The Cave of Zero Knowledge"

The Cave of Zero Knowledge

> I can prove to you that I know the password, without telling you what the password is.

The Story

Outside Yunzhou City stood a magical cave. The entrance split into left and right passages, meeting at a magic door deep inside the cave. This door required a password to open.

One day, an explorer claimed to know the door's password. His friend did not believe him. The explorer said: "I can prove it to you, but I won't tell you what the password is."

The Cave Proof

The explorer and his friend designed a verification game:

1. The friend stood outside the cave, unable to see inside.
2. The explorer entered the cave, randomly choosing the left or right passage.

3. The friend outside shouted: "Come out from the left passage!" or "Come out from the right passage!"
4. The explorer had to exit from the passage his friend specified.

If the explorer did not know the password, he could only return the way he entered. The friend had a 50% chance of calling out the passage he entered, and he could succeed. But there was a 50% chance the friend called the other passage; without the password to open the magic door, he could not exit from the other side, and proof failed.

But if the explorer knew the password? He could open the magic door and exit from either passage. So no matter which passage the friend called, he could succeed.

The Power of Probability

One game could not prove anything—even without knowing the password, the explorer had a 50% chance of guessing correctly.

But what if the game was repeated one hundred times? If the explorer successfully exited from the specified passage every time, and the friend randomly chose left or right each time, then the probability of guessing correctly one hundred consecutive times was `(1/2)^100`—smaller than the number of all atoms in the universe.

The friend did not need to know the password. He only needed to observe "whether the explorer could exit from the specified passage every time" to believe with extremely high confidence: the explorer indeed knew the password.

And throughout the process, the friend gained no information about the password itself. He did not know whether the password was one

character or one hundred characters, whether it was numbers or text. He only verified "the explorer knows the password"—and the knowledge itself was zero-leakage.

From Cave to Mathematics

Mathematicians later abstracted this cave proof into a general mathematical framework:

- **Prover** (explorer): someone who masters a secret piece of knowledge.
- **Verifier** (friend): someone who wants to confirm the prover indeed masters the knowledge, without wanting to know the knowledge itself.
- **Challenge**: the verifier poses a question requiring knowledge to answer.
- **Response**: the prover answers the question.
- **Repetition**: through multiple independent challenge-response cycles, confidence increases exponentially.

In modern cryptography, this process has been mathematized. The prover does not actually need to walk into a cave—he only needs to execute a series of mathematical operations, generating a "proof string." The verifier checks this string with another mathematical formula. If the check passes, the verifier believes the prover indeed masters the secret knowledge.

The core of the entire proof process: the information the verifier gains from the proof is zero—hence "zero knowledge."

Applications of Zero-Knowledge Proofs

Private transactions:
I want to prove "my account balance is sufficient to pay for this transfer," but I don't want to reveal my specific balance, nor whom I'm transferring to. Zero-knowledge proof lets me prove "this statement is true" without leaking any additional information.

Scaling:
I want to execute one thousand complex transactions. Instead of executing them one by one on the main chain (which is slow), execute them off-chain first, then generate a zero-knowledge proof: "These thousand transactions were correctly executed; the final state is X." The main chain only needs to verify this proof (quickly), confirming the correctness of a thousand transactions without knowing each transaction's details.

This is the core principle of ZK-Rollup.

Years later, the cave became a famous scenic spot in Yunzhou City. Every tour guide told visitors: "This cave proves the most incredible thing in mathematics—you can prove you know a secret, and during the entire proof process, your information about this secret is zero."

What This Fable Is About

This fable illustrates **Zero-Knowledge Proofs (ZKP)**, particularly **ZK-SNARKs**—a cryptographic technology that allows a prover to prove to a verifier that a statement is true without revealing any

information beyond the statement itself.

Key Mapping

Story Element	Technical Concept	Explanation
Cave's magic door	Secret knowledge / Witness	The secret the prover masters (e.g., private key, puzzle answer, intermediate computation result)
Password	Witness / Secret Input	Private input in zero-knowledge proofs; not disclosed externally
Left and right passages	Two branches of challenge-response	The verifier randomly selects a challenge; the prover must respond correctly based on the challenge
Friend randomly calls left or right	Verifier's random challenge	The verifier generates a random number as challenge, ensuring the prover cannot pre-fabricate
One hundred games	Repeated challenges reduce error probability	ZKP reduces cheating probability to negligible through multiple independent challenges
Can guess once without password	Completeness and reliability	Completeness: honest prover can always pass verification. Reliability: prover without knowledge almost cannot pass
Friend does not know password content	Zero-knowledge (Zero-Knowledge)	Information gained by verifier from interaction is zero; secret cannot be derived
Mathematical cave proof	ZK-SNARK / ZK-STARK	Non-interactive zero-knowledge proofs compress the interaction process into single proof generation and verification
Proof string	Proof	Data generated by the prover; the verifier verifies its validity through an algorithm
Private transactions	Zcash / Aztec / Tornado Cash	Use ZKP to

hide transaction amounts, senders, and recipients while ensuring transaction validity |
| Thousand-transaction proof | ZK-Rollup | Execute large numbers of transactions off-chain, generate ZKP proving correctness; main chain only verifies proof |
| Off-chain execution, on-chain verification | Computation outsourcing + correctness proof | ZK-Rollup core: move computation off-chain, leave verification on-chain |

Why This Story?

Zero-knowledge proofs are one of cryptography's most counterintuitive concepts—"proving you know X without leaking any information about X" sounds like magic.

The "cave proof" (proposed by Quisquater et al.) is the most classic intuitive model for explaining zero-knowledge proofs. Using this story to metaphorize ZK-SNARKs offers several advantages:

1. **Visualization of zero-knowledge**: The friend verified the explorer knew the password, but indeed did not know what the password was
2. **Probability confidence**: Multiple repeated challenges = ZKP's security guarantee
3. **Connection to modern applications**: From cave to ZK-Rollup, showing zero-knowledge proofs' evolution from theory to practice

Further Reading

- wiki-web3: ZK-SNARKs Introduction
- Zero-Knowledge Proofs: An Illustrated Primer

- zk-SNARKs: Under the Hood
- ZK-Rollup FAQ
- The Cave of Zero Knowledge (Original Paper)

"The Automated Market"

The Automated Market

> A marketplace with no shopkeeper, run by a balance scale that never tips.

The Story

Yunzhou City had two great products: silk from the east and tea from the west. Merchants had always traded between them, but there was a persistent problem: who decided how many pounds of tea one bolt of silk was worth?

Traditionally, this was the job of the "market brokers"—specialized middlemen who observed supply and demand daily and quoted a price both sides would accept. But brokers had a fatal flaw: they lied. When silk was oversupplied, they told tea merchants "silk is cheap" while telling silk merchants "silk is scarce," pocketing the spread.

The new magistrate of Yunzhou abolished the broker system. He built an "Automated Market" in the city center. There were no shopkeepers, no price quoters—only a giant balance scale, with silk on the left tray and tea on the right.

The market's operating rules were carved in stone and never changed:

"The amount of silk on the left tray multiplied by the amount of tea on the right tray equals a fixed constant."

This constant was set when the market was built—based on the initial amounts of silk and tea deposited. If one hundred bolts of silk and one thousand pounds of tea were initially deposited, the constant was 100,000.

On the first day of operation, a silk merchant arrived with ten bolts of silk to exchange for tea. He placed the silk on the left tray—now there were one hundred ten bolts. According to the rule, the right tray had to adjust so that 110 × right = 100,000. So the tea had to become 100,000 ÷ 110 ≈ 909 pounds.

The right tray had held one thousand pounds; now it held nine hundred nine—a reduction of ninety-one pounds. Those ninety-one pounds were what the silk merchant received for his ten bolts.

The merchant was satisfied. He did not need to haggle with a broker; the rules were public, the outcome deterministic. He deposited ten bolts of silk and took ninety-one pounds of tea, all without anyone's permission.

But there was a subtlety: the ninety-one pounds were not calculated at a fixed exchange rate of "ten bolts equals ninety-one pounds." If he had brought only one bolt, the left tray would have one hundred one bolts, and the right would become 100,000 ÷ 101 ≈ 990 pounds—he would receive only ten pounds of tea. If he brought fifty bolts, the left tray would have one hundred fifty, and the right would become 100,000 ÷ 150 ≈ 667 pounds—he would receive three

hundred thirty-three pounds of tea, but at the cost of fifty bolts.

The more he exchanged, the less tea each bolt was worth. This is "price impact"—large trades significantly shift the balance of the scale.

Another important participant was the "pool depositor." Depositors did not trade; they simply deposited equal values of silk and tea into both trays simultaneously. In return, they received a token called a "pool receipt"—representing their share of the market.

Whenever a trade occurred, the trader paid a tiny "slippage fee" (say, 0.3%). This fee did not go into anyone's pocket; it stayed directly on the scale—slightly increasing the total amounts on both sides.

This meant that while depositors' percentage shares remained the same, the absolute quantities they represented slowly grew. They earned money without doing anything.

A clever merchant discovered this. He calculated that if the Automated Market's trading volume was high enough, the accumulated slippage fees would exceed "impermanent loss" (the relative loss caused by price fluctuations). He invested his entire fortune into the pool and became one of the largest depositors.

Three years later, he was one of the wealthiest men in Yunzhou City. He had never made a single trade; his wealth came from the accumulated drip of countless traders.

But the Automated Market had its vulnerabilities.

One year, a pest infestation destroyed mulberry trees in the east, and silk production plummeted. Silk prices soared in external markets, but the Automated Market's scale reacted slowly—it continued operating at its original ratio until someone traded massive amounts of tea for silk, nearly emptying the left tray.

When only one bolt of silk remained on the left tray, one thousand pounds of tea remained on the right. At this point, if someone wanted to trade one bolt for tea, how much would they receive? 100,000 ÷ (1+1) = 50,000 pounds. But this would nearly empty the market's tea supply.

The bigger problem: when one side's assets were nearly exhausted, the market lost its purpose—no one could trade there anymore. This is called "liquidity exhaustion."

The magistrate later made two improvements: first, encourage more depositors to participate, making the scale's base large enough that individual trades would not significantly impact the ratio. Second, introduce a "price oracle"—when external market prices diverged significantly from the scale's ratio, arbitrage opportunities would attract traders to restore balance.

Years later, the Automated Market had evolved from a city facility into standard infrastructure across the entire empire. Markets across the land were interconnected, forming a vast trading network. No brokers, no shopkeepers—just the simple formula carved in stone:

Left × Right = Constant

A traveling mathematician stood before the Automated Market for a

long time and finally said something that was carved on the back of the stone: "This is not a market. This is mathematics ruling commerce."

What This Fable Is About

This fable illustrates the **Uniswap V2 Automated Market Maker (AMM)**. Uniswap V2 uses the constant product formula `x × y = k` to determine trade prices, where `x` and `y` are the quantities of the two tokens in the liquidity pool, and `k` is a constant.

Key Mapping

Story Element	Technical Concept	Explanation
Automated Market	Uniswap V2 Pair	A liquidity pool of two tokens with no order book and no market maker
Balance scale	Constant product formula `x × y = k`	The trade automatically adjusts prices so that the product of the two token quantities remains constant
Silk merchant exchanging tea	Swap transaction	User deposits one token into the pool and withdraws the other. The pool automatically adjusts prices according to `x × y = k`
More exchanged, less per unit	Price impact / Slippage	Large trades relative to pool depth cause significant price deviation from market average
Pool depositor	Liquidity Provider (LP)	Deposits both tokens

simultaneously into the pool and receives LP tokens as share certificates |
Slippage fee	Trading fee (0.3%)	Automatically reinvested into the pool, increasing `k`. LPs earn proportional to their share
Pool receipt	LP Token (ERC-20)	A transferable token representing a share of liquidity
Impermanent loss	Impermanent Loss	When external market prices change, the LP's portfolio is worth less than simply holding the assets because the AMM's rebalancing mechanism "buys high and sells low"
Pest disaster causing imbalance	External price deviation from pool price	Arbitrageurs trade against the pool until its price aligns with external markets
Liquidity exhaustion	Single-sided asset depletion	If one side of the pool is drained, trading halts. Sufficient liquidity and incentives are needed to maintain two-sided balances

Why This Story?

The AMM is one of DeFi's core innovations, but its mechanics are deeply counterintuitive for newcomers—"Where does the price come from if there are no buyers and sellers?"

The "balance scale" metaphor makes `x × y = k` viscerally intuitive:
- Add to one side, the other must decrease proportionally
- The more you add, the less favorable the exchange rate becomes
- The total value (`k`) slowly grows from fees

The pool depositor also maps to the LP's role—they bear the risk of impermanent loss in exchange for fee revenue. And the "abolition of brokers" metaphorizes the AMM's core value: removing middlemen and replacing them with immutable mathematical rules.

Further Reading

- wiki-web3: Uniswap V2 Book
- Uniswap V2 Whitepaper
- Uniswap V2 Core Concepts
- Impermanent Loss Explained

"The Anchored Vessel"

The Anchored Vessel

> A ship that does not dock at the pier, but anchors itself to the seabed with a chain. The shorter the chain, the steadier the ship—but when the waves grow too high, the chain may snap.

The Story

The merchants of Silver Sea Kingdom faced a chronic problem in cross-border trade: exchange rate volatility. Today, one bolt of silk was worth one hundred taels of silver; tomorrow, it might be one hundred twenty due to a war in a neighboring kingdom, and eighty the day after. This uncertainty made long-term contracts nearly impossible.

The Royal Advisor proposed issuing an "anchor coin"—a token whose value was permanently fixed at one hundred pounds of rice. Regardless of silver price fluctuations, one anchor coin would always be redeemable for one hundred pounds of rice. Merchants could use anchor coins to enter long-term contracts without worrying about silver price changes.

But the question was: who guarantees the anchor?

First Generation: Full Collateral

The Advisor's first scheme was simple: for every anchor coin issued, the Royal Treasury would hold one hundred pounds of actual rice. Anyone could redeem an anchor coin at the Treasury for one hundred pounds of rice at any time.

This was called the "full collateral" model. The anchor coin's credibility came entirely from the real rice in the Treasury. As long as the Treasury was not empty, the anchor coin was rock-solid.

But full collateral had a fatal cost: millions of pounds of rice in the Treasury sat idle most of the time, generating no returns. This rice could have been lent to farmers for planting or to merchants for trade—but now it was "locked up," solely to support the anchor coin's credibility.

The Treasury officials calculated: to issue one million anchor coins, the Treasury had to lock up one million pounds of rice, losing opportunity costs of approximately one hundred thousand taels of silver annually. Who would bear this cost? Either taxpayers subsidized it, or anchor coin users paid high fees.

Second Generation: Partial Collateral

Second-generation designers came up with a clever idea: you don't need full collateral, just "enough" collateral.

The approach: the Treasury would hold only fifty pounds of rice to back each anchor coin. If all coin holders came to redeem simultaneously, the Treasury would not have enough—but this was

practically impossible. Under normal circumstances, only a small fraction of people would redeem, and fifty pounds was more than sufficient.

The remaining fifty pounds could be invested, lent, and generate yield. A portion of the investment returns would subsidize anchor coin holders, giving them not just stability but also income.

But partial collateral contained an implicit bet: it gambled that "not everyone will redeem at the same time." Once market panic struck and everyone rushed to the Treasury simultaneously, fifty pounds per coin would be exhausted instantly. The remaining holders would find their anchor coins had become worthless paper.

This is called "depegging"—the anchor coin's value falling below its promised one hundred pounds of rice.

Third Generation: Algorithmic Adjustment

Third-generation designers went further. They said: "Since full collateral is too expensive and partial collateral too dangerous, let's eliminate collateral entirely."

They designed an "algorithmic anchor vessel." This vessel had no Treasury, no reserves—only a set of automatic adjustment rules:

- When the anchor coin's market price exceeded one hundred pounds of rice, the system automatically minted new anchor coins and sold them into the market, pushing the price back down. The proceeds from issuance were retained by the system.
- When the market price fell below one hundred pounds of rice, the system used retained proceeds to buy back and burn anchor coins, pulling the price back up.

In theory, this system could self-balance. No collateral was needed; pure algorithms maintained the price.

But real-world waves proved far more complex than theory. One year, rumors spread: "Silver Sea Kingdom is going to war." Anchor coin holders panicked and sold, driving the price from one hundred to fifty pounds. According to the rules, the system should have initiated buybacks—but the retained proceeds were nowhere near enough to buy back all the anchor coins being dumped.

The algorithmic anchor vessel sank. With no collateral to support it, it was like a ship without ballast—one strong wind could capsize it.

Fourth Generation: Hybrid Anchor

Fourth-generation designers learned from the failures of the first three generations. They designed a "hybrid anchor" system:

- **Core layer**: Every anchor coin was backed by solid over-collateralization (e.g., one hundred twenty pounds of rice backing one hundred anchor coins). This was the safety floor.
- **Buffer layer**: A portion of collateral could be deployed in low-risk yield protocols, with returns distributed to holders. This was efficiency enhancement.
- **Adjustment layer**: When market prices deviated slightly, algorithms automatically fine-tuned supply; when deviations became severe, the system triggered emergency collateral liquidation to ensure core-layer solvency.

More critically, fourth-generation anchor vessel collateral was not single rice, but a diversified portfolio: rice, silk, tea, even government bonds of other nations. This way, even if one asset

collapsed, the overall collateral ratio remained safe.

Years later, the Silver Sea Kingdom's anchor coin had become the most stable currency on the continent. No one knew whether it was fully collateralized, partially collateralized, or algorithmically adjusted—because the fourth-generation design had merged all three into one.

An old captain stood at the harbor watching anchor coin merchant vessels come and go. He said to his apprentice: "Do you know what's most impressive about the anchor coin? It's not that it doesn't depreciate—it's that when it truly faces a storm, you don't even feel the rocking. Because that vessel was designed assuming the worst sea conditions from the very beginning."

What This Fable Is About

This fable illustrates the **evolution of stablecoin design**, from full collateral (USDC), over-collateralization (DAI), algorithmic stablecoins (UST), to hybrid designs (like FX 2.0). The core challenge of stablecoins is balancing the "stablecoin impossible triangle": **stability, capital efficiency, and decentralization**.

Key Mapping

Story Element	Technical Concept	Explanation
Anchor coin	Stablecoin	A cryptocurrency pegged to a fiat currency (usually USD) or other asset
Full collateral	Fiat-backed stablecoin (USDC, USDT)	Each stablecoin backed 1:1 by fiat or short-term Treasuries. Safest, but capital inefficient
Partial collateral	Fractional reserve model	Historically used by traditional banks; also tried by some algorithmic stablecoins
Algorithmic adjustment	Pure algorithmic stablecoin (UST, AMPL)	No collateral; supply adjusted by smart contracts to maintain price. Highest capital efficiency, but weakest resilience
UST collapse	Death Spiral	Market panic selling drives price below peg; system lacks sufficient reserves to buy back supply; confidence collapses, triggering further selling
Hybrid anchor	Hybrid stablecoin / FX 2.0	Combines over-collateralization, yield generation, and algorithmic adjustment in a multi-layer safety design
Diversified collateral	Multi-asset collateral (DAI, FX 2.0)	Using multiple crypto assets as collateral reduces risk of single-asset crashes
Over-collateralization	Over-collateralization	E.g., DAI requires 150%+ collateral ratio—100 DAI issued requires at least $150 of ETH locked
Emergency liquidation	Liquidation Mechanism	When collateral value falls below required ratio, the system automatically auctions collateral to protect solvency

Why This Story?

Stablecoins are the foundation of DeFi, but the 2022 UST/Luna collapse wiped out hundreds of billions of dollars, proving the

fragility of algorithmic stablecoins under extreme stress.

The "anchored vessel" metaphor maps the evolution through generations of failure and fourth-generation fusion, offering several advantages:

1. **Historical clarity**: From full collateral to algorithmic adjustment to hybrid design, mirroring the real development of stablecoins
2. **The impossible triangle**: Safety (strong chain), capital efficiency (light ship), decentralization (no captain)—three incompatible goals
3. **Death spiral visualization**: The algorithmic vessel's sinking is concretely depicted as "a strong wind capsizing a ship without ballast"

Further Reading

- wiki-web3: How Modern Stablecoin FX 2.0 Is Built
- MakerDAO: How DAI Works
- Terra/Luna Collapse Analysis
- FX 2.0 Stablecoin Design

"The Pawnshop and the Vault"

The Pawnshop and the Vault

> Lending money is easy. What's hard is making sure the lent money finds its own way home.

The Story

Yunzhou City had two venerable lending institutions. One was "Old Zhou's Pawnshop." The other was "Xinfeng Vault." Their businesses looked the same—both accepted collateral for loans—but their operations were worlds apart.

Old Zhou's Rules

Old Zhou's approach was simple: you deposited something worth one hundred taels of silver, and he lent you sixty. After three months, you repaid sixty-six (principal plus interest). If you defaulted, the collateral became Old Zhou's, and he sold it to recover his money.

This system had operated for thirty years without major incident. But there were obvious pain points:

First, Old Zhou only accepted gold, silver, jewelry, and property deeds. If you had a cargo ship of spices arriving next week—highly valuable, but not yet in port—Old Zhou would not accept it. "I don't recognize what I can't see."

Second, Old Zhou's capacity was limited. He could only mobilize so much silver himself. When ten people came to borrow simultaneously, the rest had to wait in line.

Third, and most troublesome: if you collateralized property and the property value dropped at the three-month mark, your property was now worth only fifty taels, but Old Zhou had lent you sixty. Old Zhou faced a dilemma: sell the property and lose ten taels, or wait for you to repay while the property value might drop further.

Xinfeng Vault's Innovations

Manager Lin of Xinfeng Vault was a young man trained in mathematics. He saw Old Zhou's pain points and designed an entirely new system:

First: Everything Is Collateral.

Lin said: "I don't care what the thing is; I only care what it's worth." He employed a team of appraisers who priced everything in the city daily: rice, silk, spices, even the expected returns of merchant fleets. As long as an appraiser gave a price, it could be collateralized.

Second: Pools, Not Coffers.

Old Zhou's money was Old Zhou's own. Lin's money came from "pool depositors"—anyone could deposit money into the Vault's public pool for borrowers to use. In return, depositors shared the interest paid

by borrowers proportionally.

This meant the Vault's lending capacity was no longer limited by the manager's personal wealth, but by how much capital the community was willing to contribute.

Third: Real-Time Liquidation.

This was Lin's most radical innovation. At Old Zhou's, collateral was only checked at the three-month maturity. At the Vault, this check was continuous—every day, every hour, the system automatically reappraised collateral values.

If collateralized spice prices dropped, reducing collateral value from one hundred to seventy taels while the loan was sixty—safe. But if prices continued falling to fifty, the system immediately triggered "liquidation": the Vault automatically sold the spice, recovered fifty taels, and the ten-tael shortfall was covered by the borrower's pre-deposited "margin."

Borrowers were unhappy, but Vault depositors were safe—their money was never eroded by bad debt.

Fourth: Lightning Loans.

Lin also introduced a service unprecedented in history: "lightning loans."

The rules: you could borrow any amount from the Vault, but with one condition—you had to repay principal plus a tiny fee within the same hour.

"Within the same hour?" Everyone was baffled. There had to be time between borrowing and repayment for investment, right?

Lin explained: "This isn't for investment time. Lightning loans are for those who need 'instant large capital to do one thing.' For example, someone discovers a price discrepancy between two markets—Market A sells silk for one hundred taels, Market B for one hundred ten. But he doesn't have one hundred taels of capital. He borrows a lightning loan, buys silk at Market A with borrowed funds, immediately sells at Market B for one hundred ten, repays the one hundred plus one-tael fee, and keeps nine taels profit—all within one transaction."

What if the trade didn't profit, or the borrower didn't repay on time?

"Then the entire transaction rolls back," Lin said. "As if nothing ever happened. The Vault's money doesn't decrease by a single tael."

Xinfeng Vault's business quickly surpassed Old Zhou's. Not because Lin was more generous, but because his design allowed risk to flow through the system rather than accumulating on any single person.

Old Zhou's risk was borne by Old Zhou alone. Xinfeng Vault's risk was shared among depositors, borrowers, and liquidators. Depositors provided capital and earned interest, bearing "systemic market risk" (if the entire market crashed, everyone lost together). Borrowers enjoyed anytime lending convenience, bearing the risk of "real-time liquidation." Liquidators were the "hyenas" who earned commissions during market turmoil—they constantly monitored which collateral was nearing its threshold, executing liquidation the instant it was triggered.

These three parties formed a dynamic ecosystem. No one was a pure winner or loser; everyone managed risk in their own way.

What This Fable Is About

This fable illustrates the **Aave lending protocol**—a decentralized, non-custodial liquidity protocol where users can deposit assets to earn interest or collateralize assets to borrow other assets.

Key Mapping

Story Element	Technical Concept	Explanation
Old Zhou's Pawnshop	Traditional centralized lending	Requires trusting a third party; limited capacity; long liquidation cycles
Xinfeng Vault	Aave Protocol	Decentralized, non-custodial, pooled lending
Everything is collateral	Multi-asset collateral support	Aave supports multiple ERC-20 tokens as collateral, each with independent Loan-to-Value (LTV) ratios and liquidation thresholds
Public pool	Liquidity Pool	Lenders provide capital to the pool; borrowers draw from the pool. Rates adjust algorithmically based on utilization
Pool depositors	Depositors / aToken holders	After depositing assets, users receive aTokens (e.g., aDAI) whose balances automatically increase to reflect accumulated interest
Real-time liquidation	Automatic liquidation mechanism	Aave uses Chainlink oracles to update asset prices in real time. When collateral value falls below the liquidation threshold, anyone can trigger liquidation
Liquidators	Liquidators	At liquidation trigger, liquidators repay

borrower debt at a discount to receive collateral, earning liquidation rewards |
Margin	Over-collateralization	Borrowers must deposit collateral worth more than the loan amount. E.g., borrowing 100 DAI may require locking $150 of ETH
Lightning loan	Flash Loan	Aave pioneered uncollateralized, zero-risk loans. Borrow and repayment must occur within a single atomic transaction. If final repayment fails, the entire transaction reverts
Interest algorithm	Interest Rate Model	Aave dynamically adjusts rates based on utilization ratio (borrowed/deposited). High utilization increases borrowing rates, encouraging repayment and new deposits

Why This Story?

Traditional finance lending is "centralized, low-frequency, low-transparency," while Aave's disruption is turning it into "decentralized, high-frequency, fully transparent" automated protocol.

The "pawnshop vs. vault" comparison offers three intuitive advantages:
1. **Expanded collateral range**: From "only gold and silver" to "everything collateralizable"—corresponding to DeFi's composability
2. **Risk distribution**: Old Zhou bears it alone vs. the system shares it—corresponding to decentralized finance's risk socialization
3. **The wonder of lightning loans**: "Borrowing and repaying within the same transaction"—a mechanism impossible in traditional finance, concretely depicted as an instant arbitrage tool

Further Reading

- wiki-web3: Aave
- Aave Docs: Protocol Overview
- Aave V3 Whitepaper
- Flash Loans Explained

"The Messengers in the Alley"

The Messengers in the Alley

> In a system where everyone can read the mail, the fastest reader writes the rules.

The Story

Yunzhou City's postal system had a peculiar design: all letters were hung on a public bulletin board at the city gate before delivery. Couriers decided pickup order based on postage fees—whoever paid the highest fee got their letter picked up and delivered first.

The design's intention was transparency. But soon, a special class of "messengers" formed a secret society in the city's alleyways.

The Sandwich Thief

The sandwich thief did not write letters; he only read others' mail.

When merchant Xu Zhen wrote a letter to the marketplace: "I wish to buy ten bolts of silk for one hundred taels of silver," the letter was posted on the bulletin board. The sandwich thief read it.

He immediately did three things:
1. Sent a first letter: "Buy all cheap silk at the marketplace." (Paid express postage to ensure delivery before Xu Zhen's letter.)
2. Waited for Xu Zhen's letter to be delivered, forcing Xu to buy at the inflated price.
3. Sent a second letter: "Sell all the silk I just bought." (Paid express postage to ensure delivery immediately after Xu Zhen's letter.)

Result: the thief bought low, sold high, and pocketed the spread. Xu Zhen had done nothing wrong—he simply bought silk at market price—but the "market price" had been artificially inflated by the thief.

This is a "sandwich attack"—inserting two transactions around the victim's transaction, like bread around meat.

The Arbitrage Hawk

The arbitrage hawk simultaneously watched two marketplaces: East Market and West Market.

East Market sold silk for one hundred taels per bolt; West Market for one hundred five. Normally, such price differences would be naturally arbitraged away by merchants buying low and selling high.

But the hawk was far faster than ordinary merchants. He did not need his own silk inventory. He simply:
1. Borrowed from the "Lightning Money House" (same-transaction loan, must be repaid within the hour).
2. Bought silk at East Market with borrowed funds.
3. Immediately sold it at West Market.
4. Repaid the loan and its fee.

5. Kept the profit.

All within five minutes. By the time ordinary merchants finished their calculations, the hawk had already pocketed the money.

The Scavenger

The scavenger did not seek profit; he sought certainty.

The money house's rule was: when collateral value fell below a threshold, anyone could execute liquidation, acquiring collateral at a discount. But liquidation required precise timing—too early, and the threshold had not been reached; too late, and someone else might execute first.

The scavenger's job was to monitor all collateral in all money houses. The instant a threshold was triggered, he sent a liquidation letter with the highest postage fee, ensuring his letter was delivered first.

The scavenger's income was modest—only a few percentage points of discount per liquidation—but he was risk-free. As long as he was fast enough, he never lost.

Order in the Alley

The activities of these three messenger classes threw Yunzhou City's postal system into chaos. Ordinary merchants discovered that no matter what they bid, their letters were always slower than the sandwich thieves. The market was no longer driven by supply and demand, but by "who could pay higher postage fees."

Worse, the alleyway messengers began attacking each other. Two sandwich thieves targeting the same letter would competitively raise postage until profits were eaten by fees. Arbitrage hawks found East-West price spreads narrowing—too many were arbitraging, and opportunities vanished in seconds.

An old courier named Feiluo could no longer stand it. He did one thing: built a "private delivery station" outside the city.

The private station's rule was: messengers no longer posted letters on the public bulletin board; they handed letters directly to Feiluo. Feiluo did not read letter contents; he only sorted them by the messenger's bid, then packaged the sorted letters and delivered them to the city gate courier in one batch.

This offered two benefits:
1. Letter "contents" were no longer public—sandwich thieves could no longer read others' mail.
2. Messengers no longer needed to compete on the bulletin board; they conducted "sealed auctions" at Feiluo's station, where the highest bidder went first, but no one knew others' bids.

Feiluo's private station quickly became the mainstream. Sandwich thieves were unemployed—they could no longer read others' mail. Arbitrage hawks still existed, but they could only exploit publicly available market information, not mempool transparency. Scavengers could continue working, but competition became more orderly.

Feiluo took a tiny fee from each transaction as his compensation. He invented no new technology; he only changed how information flowed.

What This Fable Is About

This fable illustrates **MEV (Maximal Extractable Value)** and **Flashbots**—solutions that mitigate the harmful effects of MEV extraction. MEV refers to the additional value that block producers (or searchers) can extract by controlling transaction ordering, including front-running, back-running, sandwich attacks, and liquidation arbitrage.

Key Mapping

Story Element	Technical Concept	Explanation
Public bulletin board	Mempool	Pending transactions are publicly visible before being included in a block; anyone can read them
Postage determines order	Gas Price / Priority Fee	Miners/validators prioritize transactions with higher gas prices
Sandwich thief	Sandwich attacker	Wraps a victim's large swap between buy and sell transactions, exploiting price slippage
Arbitrage hawk	Arbitrage searcher	Exploits price differences between DEXes, typically using flash loans
Lightning Money House	Flash loan	Uncollateralized loan borrowed and repaid within a single atomic transaction. Provided by Aave, dYdX, etc.
Scavenger	Liquidation bot	Monitors collateral prices and submits liquidation transactions immediately when conditions are triggered, earning liquidation rewards
Alley chaos	Negative externalities of MEV	MEV extraction

causes ordinary users to face worse prices, higher slippage, and network congestion |
Feiluo's private station	Flashbots / MEV-Boost / private mempool	Reduces open gas-price auction wars through private transaction pools (e.g., Flashbots Protect) or PBS (Proposer-Builder Separation)
Sealed auction	MEV-Boost auction mechanism	Searchers submit sealed transaction bundles to block builders, who select the most profitable combination
Feiluo's cut	Builder profit	Builders extract value from MEV and share it with validators

Why This Story?

MEV is one of the darkest and most complex topics in blockchain. It is not a single technical problem but an incentive-structure problem—when transaction order can be influenced, influencing order itself becomes a profit source.

The "messengers in the alley" metaphor makes the MEV ecosystem tangible: sandwich thieves, arbitrage hawks, and scavengers are the three typical MEV strategies. The story also shows how Flashbots' private mempool and PBS—represented by Feiluo's private delivery station—change information flow to protect ordinary users.

Further Reading

- wiki-web3: Mempool Monitoring and Front-running with Flashbots
- Flashbots: MEV Overview
- MEV-Boost: PBS for Ethereum

- Ethereum.org: MEV
- EigenPhi: MEV Dashboard

"The Deep Well and the Shallow Pond"

The Deep Well and the Shallow Pond

> It is not about having more water, but about having water in the right places.

The Story

Yunzhou City's Automated Market (Uniswap V2) operated well, but depositors gradually discovered a frustrating fact: most of their funds were "idle" most of the time.

In V2's Automated Market, depositors had to deposit equal values of both assets into both sides of the scale. These funds were distributed across the entire price range—from near zero to infinity. This meant that wherever the market price was, only a tiny fraction of funds was actually participating in trades. The vast majority lay at the extremes of the price range, like hibernating bears, doing nothing.

A mathematician named Chen Shen calculated a startling number: in V2 markets, approximately only 0.5% of funds at any given price

point were actually "working." The remaining 99.5% sat in the pool but contributed nothing to trading—they simply waited for extreme prices that might never come.

"This is wasteful," Chen Shen said. "If I know the reasonable price range for silk and tea is one bolt for eighty to one hundred twenty pounds of tea, why should my money be distributed from zero to infinity?"

Chen Shen designed a new market: the "Range Market."

In the Range Market, depositors were no longer forced to distribute funds across the entire price axis. They could specify a range: "I will only provide liquidity within the eighty to one hundred twenty pounds of tea per bolt of silk range."

This was like a depositor renting a booth at a specific location in the market, rather than being evenly distributed across the entire square. When the market price was within their rented range, their funds participated in trading and earned fees. When the market price moved outside their range, their funds automatically "exited"—becoming entirely single-asset, no longer participating in trading or earning fees.

This design brought several dramatic changes:

First, capital efficiency increased tenfold.

In V2 markets, ten thousand taels of capital might generate only ten taels of daily fees. In Range Markets, if depositors concentrated funds in the most active price range near the current price, the same capital could generate hundreds of taels daily—because their funds were no longer diluted across never-touched price ranges.

Second, booth location became strategy.

In V2 markets, all depositors were identical—you deposited a certain amount, and earned proportionally. In Range Markets, different booth locations meant different risks and returns.

Conservative depositors chose a wide range (say, fifty to two hundred), so market prices rarely moved outside their range. But their capital utilization was low, because only a small portion of funds in a wide range worked near the current price.

Aggressive depositors chose a narrow range (say, ninety-five to one hundred five), compressing all funds into the most active price point. Their capital utilization was extremely high, fee income potentially dozens of times that of conservatives. But risk was equally high—if market prices fluctuated even slightly and moved outside their range, all their funds became single-asset and stopped earning.

Third, the market developed a "depth map."

V2 markets were like uniformly deep swimming pools—wherever you jumped in, the depth was the same. V3 Range Markets were like irregular deep wells—some locations were bottomless, others were shallow puddles.

Traders loved deep wells: where depth was great, large trades had minimal price impact. In shallow puddles, even small trades caused huge price jumps.

Clever depositors observed the market's depth map and injected funds into "shallow puddles"—because competition was less there, fees were shared among fewer people. But this also meant they were taking risks: shallow puddles were shallow precisely because those price ranges were not favored by the market.

One year after the Range Market launched, an unexpected phenomenon emerged: overall market liquidity had not increased, but fund distribution became extremely uneven.

Near price one hundred—the historically most active price—well depth was twenty times that of V2-era levels. Traders enjoyed minimal slippage here. But near price two hundred—a rarely touched extreme—puddles nearly dried up. If someone suddenly needed to make a large trade at that price, they might find there was simply insufficient liquidity to support it.

Chen Shen's explanation was: "This is not a bug; it is a feature. V2 used uniform distribution to hide true supply and demand. V3 exposes true supply and demand. Capital naturally flows to the most profitable places; this is the market's choice."

But critics said: "Liquidity exhaustion in extreme situations is a safety hazard. When markets panic and everyone wants to flee, there is not enough water in shallow puddles to cushion the shock."

This debate remains unresolved.

What This Fable Is About

This fable illustrates **Uniswap V3's Concentrated Liquidity** mechanism. V3's biggest innovation is allowing liquidity providers (LPs) to customize price ranges, concentrating funds in the price

ranges they believe are most active, thereby dramatically improving capital efficiency.

Key Mapping

Story Element	Technical Concept	Explanation
V2 Automated Market	Uniswap V2 AMM	Liquidity uniformly distributed across [0, ∞) price range; low capital utilization
99.5% idle funds	V2 capital efficiency problem	At any actual price point, only a tiny fraction of V2 pool funds actively participate
Range Market	Uniswap V3 Concentrated Liquidity	LPs can specify price range [lower, upper]; liquidity only provided within that range
Booth location	Tick Range / Position	V3 divides the price axis into discrete ticks; each position occupies a tick range
Wide range fifty to two hundred	Conservative LP strategy	Choosing wider ranges reduces impermanent loss risk but also lowers capital efficiency
Narrow range ninety-five to one hundred five	Aggressive LP strategy	Highly concentrated capital maximizes fee income, but risk of price moving out of range and impermanent loss are extremely high
Out of range	Out of Range	When price exceeds a position's range, all liquidity becomes single-asset, no longer earning fees
Depth map	Liquidity Distribution	V3 allows observing liquidity depth at different price ranges. Mainstream ranges are extremely deep; extreme ranges are extremely shallow
Less competition in shallow puddles	Non-uniform distribution arbitrage space	In liquidity-thin ranges, small capital can earn higher fee proportions, but faces greater impermanent loss
Extreme liquidity exhaustion	Tail risk	V3 concentrates liquidity

near current prices; extreme market conditions may create liquidity vacuums, amplifying price volatility |

Why This Story?

Uniswap V3 is one of DeFi's most sophisticated designs, but also one of the hardest to intuitively understand. The concept of "concentrating liquidity from infinite ranges to finite ranges" is deeply counterintuitive for many beginners.

The "deep well and shallow pond" metaphor offers several intuitive advantages:

1. **Spatial sense**: Abstract price ranges are concretized as "booth locations," letting readers understand that "location choice itself is a strategy"
2. **Risk-return**: Narrow range = deep well = high return high risk; wide range = shallow pond = low return low risk
3. **Systemic impact**: V3 "authenticates" liquidity distribution, exposing market structure—this is not a bug but market efficiency improvement, though it brings new tail risks

Further Reading

- wiki-web3: Uniswap V3 Development Guide
- Uniswap V3 Whitepaper
- Uniswap V3 Core Concepts
- Charged Particles: V3 LP Strategies

"The Water Dividers"

收入拆分协议.md"
tags: [defi, revenue-sharing, splits, royalty, collaboration]

The Water Dividers

> The hardest part is not earning money, but dividing it fairly.

The Story

East of Yunzhou City ran a canal fed by three sources: a mountain spring, a rain channel, and an underground aquifer. The three source owners agreed: canal revenue (irrigation fees, boat tolls) would be distributed according to each source's contribution—spring fifty percent, rain channel thirty percent, underground aquifer twenty percent.

It sounded simple, but execution was a nightmare.

Old Method: Manual Division

Initially, the canal's terminus had a water-dividing gate. Gatekeeper

Zhao manually calculated each source's share daily, wrote three silver drafts, and sent them out.

If total revenue was one hundred taels, Zhao would open his ledger, calculate one hundred × fifty percent = fifty taels for the spring owner, thirty for the rain channel owner, twenty for the aquifer owner. Then write three drafts and send them separately.

If only two sources contributed today? Say it didn't rain, and the rain channel contributed nothing. Zhao still had to go through the entire process: calculate, write drafts, send—though the rain channel owner received zero.

More troublesome: when a new source joined. Suppose a fourth source—a reservoir—wanted to connect to the canal. Zhao had to: recalculate percentages, rebuild the dividing gate, update the ledger, notify all parties. The entire process took three days.

And if someone questioned: "Zhao, is my fifty taels correct?" Zhao could only flip to a certain page and point: "See, I wrote it down." But this proved he hadn't miscalculated, nor that he hadn't secretly altered numbers.

New Method: Automatic Water Dividers

One day, a young engineer designed an "automatic water division system."

The system's core was a "division contract"—carved on a bronze plate, hung in the public temple by the canal, readable by anyone. The contract was extremely simple:

"All water entering the canal, and the revenue it generates,

shall be automatically distributed as follows: spring fifty percent, rain channel thirty percent, underground aquifer twenty percent."

The contract was accompanied by an "automatic water divider" device: when revenue reached the canal terminus, the device automatically channeled it according to contract-specified proportions into the three source owners' accounts. No Zhao needed, no ledger, no silver drafts.

More surprisingly: this device was "immutable." Once the contract on the bronze plate was carved, no one could unilaterally modify the percentages. If all three source owners agreed to change percentages, they had to cast a new bronze plate together and replace the old one.

This meant: Zhao's "trust problem" was completely solved. No one needed to trust Zhao not to miscalculate—because Zhao had been removed from the entire process.

New Problems Emerge

The automatic water division system operated for half a year before a new requirement emerged.

The spring owner was a family with five brothers who needed to split the spring's fifty percent share equally among themselves. Under the old method, Zhao manually divided again. But the automatic system did not support "nested distribution"—it could only distribute according to the first-layer contract, not automatically handle second layers.

The young engineer upgraded the system: "sub-contracts" were

allowed.

Now, the water division system became two layers:
- Layer one: Total canal revenue → distributed to three sources according to main contract
- Layer two: Spring's share → distributed to five brothers according to sub-contract
- Layer two could continue: Eldest brother's share → distributed to his wife and children according to grandchild-contract

Each layer's contract was an independent bronze plate with its own automatic water divider. Revenue flowed like water, layer by layer, until reaching final recipients.

Moreover, any layer's contract could be "forked"—if the third brother decided to go independent, he could extract his share from the family sub-contract and create his own independent sub-contract, no longer participating in family distribution. This process required no consent from the other four brothers, nor Zhao's approval.

Unexpected Applications of Automatic Water Division

The automatic water division system was originally designed for canals, but people quickly discovered broader applications:

- **Music workshop royalties**: A composition jointly created by lyricist, composer, and performer. They created a three-way division contract: lyrics thirty percent, composition forty percent, performance thirty percent. Whenever someone paid to listen, revenue automatically distributed according to these proportions.
- **Inn partnerships**: An inn jointly funded by ten partners. They created a ten-way division contract, distributing profits according to

investment shares. Anyone could sell their share to others anytime—simply changing their address in the division contract to the new owner's.
- **Charitable donations**: A charity project created a two-way division contract: ninety percent to beneficiaries, ten percent to platform operations. Donors could see the contract content and confirm their money would not be intercepted.

In these scenarios, the automatic water division system's core value was not "calculation"—calculation itself was simple—but "trust." Participating parties did not need to trust any intermediary; they only needed to trust that publicly visible bronze plate.

Years later, Yunzhou City's water division system had connected thousands of revenue streams. Some had only two layers of distribution; others had over a dozen. Some contracts were permanently immutable; others automatically adjusted proportions quarterly based on performance.

Zhao did not lose his job. He transformed into a "contract consultant"—helping people design the most suitable division structures. After all, the automatic water division system could perfectly execute any contract, but it could not decide for people "what a fair contract looks like."

Zhao often said: "Machines solved the 'how to divide' problem. But the 'how much to divide' problem will always require humans to answer."

What This Fable Is About

This fable illustrates **0xSplits**—an on-chain revenue splitting protocol that allows creators, collaborators, team members, and others to automatically distribute earnings according to preset proportions. Its core values are: trustlessness, immutability, and composability.

Key Mapping

Story Element	Technical Concept	Explanation
Canal and three sources	Multi-party collaborative revenue	Any scenario requiring fixed-proportion revenue distribution
Zhao's manual division	Traditional off-chain revenue distribution	Relies on third parties (platforms, accountants) for manual calculation and transfer; has trust costs and delays
Bronze plate division contract	0xSplits smart contract	On-chain immutable distribution rules; code is law
Automatic water divider	Automatic splitting logic	When revenue arrives, the contract automatically distributes according to preset proportions to each recipient
Immutable contract	Immutability	Once deployed, distribution proportions cannot be unilaterally modified; prevents mid-stream changes
Nested sub-contracts	Splits composability	A Split's recipient can be another Split's address, forming multi-layer distribution structures
Third brother going independent	Share transferability	Anyone can extract or transfer their share in a Split to someone else

Music workshop royalties	Royalties / copyright distribution	NFT creators, musicians can set permanent royalties; secondary market trading revenue automatically distributes
Inn partnerships	Investment / partnership revenue distribution	Investors, team members automatically share profits according to agreed proportions
Charitable donations	Transparent fund flows	Donors can verify fund destinations, ensuring no misappropriation

Why This Story?

Revenue splitting looks simple but is actually extremely complex in practice. Off-chain solutions either rely on trusted third parties or have high execution costs and low transparency.

The "water dividers" metaphor for 0xSplits offers several intuitive advantages:
1. **Liquidity**: Revenue flows like water; distribution is a natural physical process
2. **Composability**: Sub-contract nesting corresponds to Splits' recursive structure
3. **Trust replacement**: The bronze plate replaces Zhao, solving the "who watches the watchers" problem

Further Reading

- wiki-web3: Analysis of 0xSplits — Revenue Splitting Protocol
- 0xSplits Official Docs
- Mirror: Splits for Creators

"The Programmable Bazaar"

The Programmable Bazaar

> The marketplace not only sells goods, but allows you to insert your own rules at every critical juncture.

The Story

Yunzhou City's Automated Market (Uniswap V2/V3) had operated for many years. Its rules were fixed: buyers and sellers traded according to the constant product formula; fees were distributed proportionally to depositors. No one could change these rules.

But one day, the market's builder announced a revolutionary upgrade: the "Programmable Bazaar."

Core Transformation of the Bazaar

In the old market, each product pair (silk-tea, rice-silver) required its own independent marketplace. With thousands of products in Yunzhou, there were thousands of independent markets, each with its own scale, own depositors, own ledger.

The Programmable Bazaar merged all of these into a "Super Bazaar." All product pairs shared the same scale, the same ledger. The difference: each product pair could install its own "plugins."

Plugins could intervene at the following critical moments:

1. **Before trading**: When someone wanted to trade, the plugin could first check conditions—"Is this trader on a blacklist?" "Has today's trading volume exceeded the limit?"

2. **During price calculation**: The plugin could modify the price calculation method—"If a single trade exceeds one thousand taels, raise the fee from 0.3% to 1%."

3. **After trading**: The plugin could trigger follow-up actions—"After completing the trade, automatically donate ten percent of fees to the charity fund."

4. **When depositors join**: The plugin could set准入 conditions—"Only those holding a specific badge can become depositors."

Plugin Creativity

The Programmable Bazaar's plugin system spawned unprecedented innovations.

Dynamic Fees:
One plugin made fees automatically adjust based on market volatility. When markets were calm, fees were low, attracting trades. When markets fluctuated violently, fees automatically increased, protecting depositors from impermanent loss.

Limit Orders:

Another plugin implemented "limit order" functionality. Traders could set: "When silk price drops to eighty taels per bolt, automatically buy one hundred bolts." In the old market, this required constant manual price checking. In the new market, plugins automatically monitored and executed.

Time-Weighted Average Price:
A third plugin recorded prices at each trade, maintaining a "past hour's average price." Other contracts could reference this average without calculating it themselves—greatly saving gas.

Custom Curves:
The most radical plugin completely abandoned the constant product formula. One plugin implemented a "stablecoin curve"—when two assets' prices were near 1:1, trades had almost no slippage. This made stablecoin-to-stablecoin swaps extremely efficient.

Plugin Risks

But plugins also introduced new risks.

A malicious plugin could secretly modify prices before trading, causing traders to execute at extremely unfavorable prices. Or, a plugin could redirect fees to its own creator's pocket instead of depositors.

The bazaar's builder recognized this problem. He designed a rule:

- Plugin code was public; anyone could audit it.
- But installing a plugin required "bazaar administrator" approval. The administrator did not review plugin "creativity," only whether plugins were "safe"—i.e., would not steal money or destroy the bazaar's basic functions.

- Once a plugin was approved and installed, its behavior was deterministic and immutable. The administrator could not retroactively modify the plugin.

This design balanced "creativity" and "safety": anyone could propose plugins, but only safety-reviewed plugins could be installed. And once installed, plugins were beyond anyone's control—they executed automatically according to code.

Super Bazaar's Ledger Revolution

The Programmable Bazaar's other major change was "flash accounting."

In the old market, every trade immediately changed the ledger—the quantities on both sides of the scale updated in real time. This meant every trade required writing to the ledger once, consuming storage fees.

In the Programmable Bazaar, a "temporary ledger" was introduced. Traders could execute multiple trades in a short time; all these trades were only recorded on the temporary ledger. Only when the trader "settled" did the temporary ledger's contents get written to the formal ledger all at once.

This meant: an arbitrageur could complete ten trades in one settlement, but only pay storage fees once. This dramatically improved high-frequency trading efficiency.

Years later, the Programmable Bazaar became Yunzhou City's most prosperous commercial center. An old merchant said: "The old

market told you what you could do. The new market only tells you what you cannot do—between those two lines is your creative space."

What This Fable Is About

This fable illustrates **Uniswap V4's Hooks mechanism**—V4's biggest innovation. V4 transforms each trading pool from an independent contract into a state entry under a "Singleton" architecture, and allows custom Hook contracts to be installed on each pool, executing custom logic at key points in the trade lifecycle (beforeSwap, afterSwap, beforeModifyPosition, etc.).

Key Mapping

Story Element	Technical Concept	Explanation
Old market thousands of independent scales	Uniswap V3 Pool architecture	Each pool is an independent contract; deployment costs are high
Super Bazaar sharing one scale	Uniswap V4 Singleton architecture	All pools share one contract; differentiated by state storage; deployment costs drastically reduced
Plugin	Hooks contract	Custom contracts called at specific hook points in the trade lifecycle
Pre-trade check	beforeSwap hook	Called before swap execution; can implement KYC, limits, blacklists, etc.

Price calculation	Dynamic fee hook	Can dynamically adjust fees during the swap process
Post-trade trigger	afterSwap hook	Called after swap completion; can trigger donations, reward distributions, etc.
Depositor joining	before/afterModifyPosition hook	Called when liquidity is added/removed; can implement access control
Dynamic fees	Dynamic fee tiers	Fees adjust dynamically based on volatility, trading volume, etc.
Limit orders	Limit order hook	Automatically executes trades when price thresholds are reached
Time-weighted average price	TWAP hook	Updates cumulative prices at each swap for external reference
Custom curves	Custom curves	Hooks can implement non-constant-product AMM curves (e.g., Curve's stablecoin formula)
Plugin audit	Hooks security review	Uniswap V4 hooks are chosen by pool creators; users must independently audit hook safety
Flash accounting	Flash Accounting	V4 introduces "flash settlement"—multiple operations can accumulate in temporary state before final settlement, saving gas
Temporary ledger	Transient Storage (EIP-1153)	V4 uses EIP-1153 transient storage for flash accounting; cleared after operation completion

Why This Story?

Uniswap V4 represents a paradigm shift in DeFi protocol design—from "fixed-function white box" to "programmable black box." Hooks transform AMMs from a single trading mechanism into a customizable platform.

The "Programmable Bazaar" metaphor for V4 Hooks offers several

intuitive advantages:

1. **Plugin architecture**: Inserting custom rules at key market nodes = Hooks' intervention points in the trade lifecycle
2. **Singleton architecture**: Shared super bazaar = V4 merging all pools into one contract
3. **Flash accounting**: Temporary ledger with one-time settlement = Flash Accounting saving gas
4. **Creativity vs. safety balance**: Public code + administrator review = Hooks' openness and risk

Further Reading

- wiki-web3: Uniswap V4 Hooks Development Guide
- Uniswap V4 Whitepaper
- Uniswap V4 Core Concepts
- EIP-1153: Transient Storage
- Uniswap V4 Hooks Directory

"The Numbered Lockers"

The Numbered Lockers

> You think secrets locked in a cabinet are safe, but the cabinet itself stands in the middle of the hall.

The Story

In Yunzhou City's administrative hall stood a vast wall lined with countless numbered lockers—from number 0 to infinity. Each locker held exactly thirty-two characters, no more, no less.

Anyone could walk up to this wall, open any locker, and read its contents. No locks, no passwords. The numbered lockers' design philosophy was: transparency.

Locker Rules

The hall administrator established strict storage and retrieval rules:

1. Each locker had a fixed size: thirty-two characters. If an item was smaller than thirty-two characters, multiple items could be packed into the same locker. This was called "packing."

2. If an item exceeded thirty-two characters, it had to occupy consecutive lockers.
3. The administrator placed items in order: locker 0, locker 1, locker 2...
4. Once an item was placed in a locker, it could not be moved. Unless you actively cleared the locker, its contents remained there forever.

A shrewd merchant stored his trade secrets in the lockers—his cost prices, his client lists, his secret codes. He thought locking them in cabinets made them safe.

But he forgot: the lockers had no doors. Anyone who knew the number could open and read them.

One day, a competitor walked into the hall, opened locker 3—there was the merchant's cost price. Opened locker 4—the client list. Opened locker 5—the secret code.

The merchant collapsed: "This is my privacy!"

The administrator replied coldly: "The hall's rules are transparency. You should have known this when you put them in."

The Art of Packing

Clever storers learned "packing."

The locker size was thirty-two characters. If you had two sixteen-character items, you could pack them into one locker—left sixteen, right sixteen. This way you occupied only one locker, not two.

But packing had a subtle limitation: items had to be arranged from largest to smallest. If you placed an eight-character item first, then

tried to fit a twenty-four-character item—the remaining space was insufficient. But if you placed the twenty-four-character item first, then the eight-character one, it fit perfectly.

An experienced storage consultant made a fortune from this. His job was to help clients optimize packing order, allowing the same number of items to occupy the fewest lockers. Each locker saved meant one less storage tax paid.

Dynamic Lockers

Some storers needed to store quantities that were not fixed—like a constantly growing list. The administrator designed "dynamic lockers" for them:

- The list's contents were not placed directly in numbered lockers, but in another place called the "hall warehouse."
- The numbered locker only held a pointer—an address telling you where to find the actual list.
- When the list grew, the warehouse automatically expanded. The pointer in the numbered locker never changed.

This design's cost was: every time you read the list, you had to first read the pointer in the numbered locker, then go to the warehouse to find the actual contents. Slower than direct storage, but more flexible.

The Trap of Clearing

A common misconception: take an item out of a locker, and the locker is empty.

No. In the world of numbered lockers, "clearing" meant setting the locker's contents to zero. But the locker itself remained, its number remained, it still occupied wall space, and you still had to pay storage tax for it.

The only exception: if you created a locker, then zeroed it, the administrator would refund part of your storage tax as a reward for "releasing space."

Years later, a traveler from a foreign land stood before the wall of numbered lockers and said something that was carved into the wall: "This is not storage. This is nakedness."

What This Fable Is About

This fable illustrates the **EVM Storage Layout and Slot mechanism**. EVM storage is a massive key-value mapping where each slot is 32 bytes (256 bits). Solidity packs data into slots in declaration order, and all stored data is visible to everyone on-chain.

Key Mapping

Story Element	Technical Concept	Explanation

Wall of numbered lockers	EVM Storage	Each contract has a storage space of 2^256 slots; each slot is 32 bytes
Thirty-two characters	32 bytes / 256 bits	The fixed size of a storage slot
No locks, transparent	On-chain data is publicly readable	Any data in `storage` can be read by anyone via `eth_getStorageAt`
Trade secrets read	Storage is not private	Many beginners mistakenly believe `private` variables are secret; they are only restricted at the Solidity level, fully transparent on-chain
Packing	Storage Packing	Solidity attempts to pack multiple small variables (e.g., multiple `uint128`) into one slot to save gas
Packing order limitation	Packing rules	Variables are placed in slots in declaration order. Poor ordering (e.g., `uint8` before `uint256`) causes unnecessary slot waste
Dynamic lockers	Dynamic arrays and mappings	`mapping` and dynamic `array` use keccak256 hashing to calculate slot positions; data is stored in "hash space" rather than sequential slots
Pointer	Hash value in slot	A mapping's slot does not store actual data, only a seed for calculating offsets
Clearing	`delete` / zeroing	Setting a slot to zero refunds some gas (15,000 gas refund), but the slot still exists in state
Storage tax	SSTORE gas cost	First write to non-zero costs 20,000 gas; modification costs 5,000 gas; zeroing refunds 15,000 gas

Why This Story?

"On-chain data is public" is one of the most counterintuitive facts in Web3. Many developers mistakenly believe `private` variables are secure, leading to serious vulnerabilities (like the famous Parity wallet bug).

The "numbered lockers" metaphor for storage offers several intuitive advantages:

1. **Visualization of transparency**: Lockers have no doors; anyone can open them—corresponding to the complete publicity of on-chain data
2. **Packing concretized**: Packing data into fixed-size slots is like packing items into fixed-size lockers
3. **Gas cost intuition**: Creating lockers (SSTORE) is expensive, reading is cheap, clearing gives refunds—corresponding to EVM's gas pricing logic

Further Reading

- wiki-web3: Solidity Private Data On-Chain Transparency and Slot Storage
- EVM Deep Dives: Storage Layout
- Solidity Docs: Storage Layout

"The Craftsman's Workbench"

The Craftsman's Workbench

> The larger the workbench, the more expensive the work. But after each job, the table automatically clears itself.

The Story

In the workshops of Yunzhou City, every craftsman had a workbench. The workbench's rules were quite peculiar:

First: the workbench is temporary.

When a craftsman began work, the bench was empty. He laid out tools, parts, half-finished pieces. When the job was done, he removed the finished product—and the bench automatically cleared itself, leaving nothing behind. The next craftsman faced a brand-new empty table.

This meant: nothing on the workbench could stay overnight. Today's drawings, today's markings, would not exist tomorrow.

Second: the workbench could expand.

The bench started small, barely holding a few tools. If a craftsman

needed more space, he could pull the table wider. But pulling the table had a cost—every extra inch required paying an "expansion tax."

And the expansion tax was not linear. Stretching from three feet to four was cheap. But stretching from one hundred feet to one hundred one was shockingly expensive. Because workshop floor space was a scarce resource; the more you occupied, the higher the marginal cost.

Third: the workbench was linear.

A craftsman could not place items at arbitrary positions. He had to arrange them from left to right: first placing things at the far left, then progressively to the right. If he wanted to skip the empty middle and place something on the right—he could, but he had to first "claim" the middle empty space as his own, which also required paying tax.

Fourth: the workbench was public.

When a craftsman needed help from a master in another workshop, he could not invite the master to his own workshop—he had to "copy" his workbench and send it to the other workshop. The master worked on this copied table and sent the results back. The original workshop's workbench was unaffected.

A clever young craftsman discovered these properties and designed an extremely efficient workflow:

1. Before starting each job, mark the "current used length" at the table corner—like a ship's waterline.
2. During work, only expand the table within necessary limits.

3. After finishing, clear everything before the waterline, returning the table to its initial state.

This workflow cost him only one-third the expansion tax of other craftsmen.

But one trap many fell into: some thought "the bench auto-clears anyway," and left things on it "thinking they might use them later." Next time they started work, they found the bench was indeed empty—those things were gone. They had to recalculate and rearrange, wasting more time.

The old craftsman's advice was: "Only put 'things needed for this job' on the workbench. If you want to save something, carve it into stone (storage), not leave it on the table (memory)."

What This Fable Is About

This fable illustrates the **EVM Memory mechanism**. Memory is EVM's temporary data area during execution, byte-addressable, linearly expandable, and reset after each message call.

Key Mapping

Story Element	Technical Concept	Explanation
Workbench	EVM Memory	Temporary data area during contract

execution for storing dynamically sized data |
Temporariness	Memory lifecycle	Data in memory exists only during the current message call. Automatically cleared after call ends
Expandable	Memory expansion	Memory expands dynamically. Writing beyond current size triggers automatic expansion, consuming gas
Expansion tax	Memory expansion gas cost	Memory expansion cost = `memory_size_word^2 / 512 + 3 * memory_size_word`. Quadratic cost; larger expansion is more expensive
Linear arrangement	Memory linear address space	Memory starts at address 0 and is written sequentially. MSTORE/MLOAD operate in 32-byte word units
Waterline	Free Memory Pointer	The Solidity compiler maintains a pointer at memory's start, recording the end of currently used memory to avoid redundant calculation
Calling external help	External contract call (CALL)	Calling an external contract creates a new execution context with independent memory. Original memory is unaffected upon return
Copying workbench	CALL memory passing	When calling an external contract, input data must be copied to memory for the callee. Return data also requires memory space
Carving into stone	Storage vs Memory	Storage is persistent but expensive; memory is temporary but cheap. Data destination depends on whether persistence is needed
Thinking they'd use it later	Memory data is not persistent	If you cache data in memory expecting to use it next call—impossible. Each call starts with fresh memory

Why This Story?

Storage and Memory are among the first core concepts Solidity

developers must understand, but the abstraction of "temporary vs persistent" is vague for beginners.

The "workbench" metaphor for memory offers several intuitive advantages:

1. **Temporariness**: After work, the table auto-clears—corresponding to memory's lifecycle
2. **Expansion cost**: Pulling the table wider is increasingly expensive—corresponding to memory expansion's quadratic gas cost
3. **External calls**: Copying the table to another workshop—corresponding to CALL's memory isolation and passing costs

Further Reading

- wiki-web3: EVM Memory Mechanism Deep Dive
- EVM Deep Dives: Memory
- Ethereum Yellow Paper: Memory

"The Prophet's Mold"

The Prophet's Mold

> Before the object is cast, you already know exactly where it will appear.

The Story

In Yunzhou City, blacksmiths had a tradition: whenever someone commissioned a blade, the smith forged it and engraved a serial number on the hilt. The number was random—depending on which blade number was reached that day. Buyers only knew which blade was theirs after it was forged.

This tradition had operated for three hundred years until a young smith named Shen Suan proposed a bold idea.

Old Method: Random Numbers

The old method's problems were obvious:

1. Buyers could not know the blade's exact serial number in advance. If a buyer needed to assemble the blade with other

components, he had to wait until the blade was forged before starting assembly.
2. If the smithy relocated or the smith changed, the serial number sequence might be interrupted, causing conflicts.
3. Buyers could not "reserve" a number. If they wanted to ensure their blade was number 777, they could only pray that the day happened to reach that number.

New Method: The Prophet's Mold

Shen Suan designed a "prophet's mold." The mold's marvelous property was: before the blade was cast, you could accurately predict its final serial number.

The prophecy formula was carved on the mold itself:

Blade Number = Hash(Mold Number + Material Recipe + Salt)

Where:
- Mold number was fixed—each mold was born with a unique, unchangeable number.
- Material recipe was chosen by the buyer himself—different recipes produced different blades.
- Salt was an arbitrary number—the buyer could freely choose it to "adjust" the final number.

This meant: before the blade was cast, anyone could plug these three parameters into the formula and calculate the blade's number. No need to wait for forging, no need to trust the smith, no need for any prophecy.

Applications of the Prophet's Mold

This invention sparked a revolution in Yunzhou City.

First application: advance assembly.

A scabbard craftsman needed to make a custom scabbard for a commissioned blade. Under the old method, he had to wait for the blade to be forged and know its serial number before starting—because the scabbard had to match a specific blade.

Under the new method, the buyer knew the blade's number when placing the order. The scabbard craftsman could start making the matching scabbard while the blade was still in the furnace. When the blade was forged, the scabbard was also complete.

Second application: state channels.

Two merchants needed to establish a "joint account" in Yunzhou City to manage their transactions. Under the old method, they had to first establish the account, then start trading.

But under the new method, they could "prophesy" the account's number before establishing it, then sign a contract based on this prophecy: "When we finally establish the account, its number will be XYZ, and the contract automatically takes effect then."

This state of "the account does not yet exist, but can already be referenced" is called "counterfactual."

Third application: batch casting.

An army needed one thousand standard-issue blades. Under the old method, the smith forged them one by one, numbers random, unable

to pre-allocate.

Using the prophet's mold, the army could pre-calculate each blade's number before casting began, engraving numbers on scabbards in advance. After casting, blades and scabbards automatically matched.

The Art of Salt

The prophet's mold introduced an interesting variable: salt.

Salt itself had no meaning—its only purpose was to let buyers "adjust" the final number. If the first calculated number was already occupied, the buyer could change the salt, recalculate, and repeat until finding an unoccupied number.

This spawned "number hunters"—a profession dedicated to finding "pretty numbers." Some would pay a premium for a blade whose number contained eight 8s, or ended with specific digits. Number hunters used computers to continuously try different salts, searching for numbers that met buyers' requirements.

Years later, Shen Suan's prophet's mold had become standard equipment for all craftsmen in Yunzhou City. Someone asked Shen Suan: "What is the most powerful aspect of your mold?"

Shen Suan said: "It is not the prophecy. It is that it makes 'things that do not yet exist' become 'addressable.' Before the blade is cast, it is already an entity that can be referenced, trusted, and used. This changes our definition of 'existence.'"

What This Fable Is About

This fable illustrates **CREATE2**—a deterministic contract creation method in EVM. Unlike CREATE, CREATE2's contract address is determined solely by `deployer_address + salt + init_code_hash`, independent of the creation nonce. This means you can predict a contract's address precisely before deployment.

Key Mapping

Story Element	Technical Concept	Explanation
Traditional random numbers	CREATE (based on nonce)	Contract address = `keccak256(rlp([sender, nonce]))`. Each contract creation increments nonce; address is unpredictable
Prophet's mold	CREATE2	Contract address = `keccak256(0xff + sender + salt + keccak256(init_code))[12:]`. Completely deterministic; can be pre-computed
Mold number	Deployer address	CREATE2 calculation includes the deployer address, ensuring different deployers' address spaces do not conflict
Material recipe	Init code (initialization bytecode)	Contract creation bytecode determining contract logic. Different init code produces different addresses
Salt	Salt (bytes32)	A 32-byte value freely chosen by the deployer to generate different addresses with the same init code
Advance assembly	Pre-computed address application	Can know a contract's address before deployment, pre-configuring other

contracts to reference it |
Counterfactual	Counterfactual instantiation	Accounts or contracts not yet deployed on-chain can already be referenced and interacted with (e.g., state channels, Layer 2)
State channels	State Channels / Layer 2	Based on counterfactual addresses, contracts can be pre-signed off-chain; on-chain deployment only occurs in disputes
Number hunters	Vanity address mining	Brute-forcing salt values to find addresses with specific prefixes or patterns
Batch casting	Batch deployment + pre-calculation	Can pre-calculate large numbers of contract addresses for batch configuration and initialization

Why This Story?

CREATE2 is a seemingly simple but profoundly deep technology. It gives "not-yet-deployed contracts" addressability, which is the foundation for counterfactual states, state channels, batch deployment, and other advanced applications.

The "prophet's mold" metaphor for CREATE2 offers several intuitive advantages:

1. **Determinism visualized**: Knowing the number before casting—corresponding to CREATE2's address predictability
2. **Counterfactual concretized**: Scabbard work begins before blade forging—corresponding to counterfactual instantiation
3. **Salt flexibility**: Adjusting salt to find ideal numbers—corresponding to the salt parameter's role

Further Reading

- wiki-web3: CREATE2 Contract Creation and Address Prediction
- EIP-1014: Skinny CREATE2
- Counterfactual Instantiation
- Uniswap V3 Factory CREATE2 Usage

"The Soul in Borrowed Body"

The Soul in Borrowed Body

> Speak through another's body, but the words' consequences fall entirely on your own head.

The Story

Yunzhou City had an ancient secret art called "body borrowing." The practitioner could temporarily move his soul into another person's body, using that body to act. But body borrowing had an extremely dangerous property:

Everything you do in the borrowed body, the consequences fall on your original body.

Normal Visitation

Normally, if one person visited another, he stood in the other person's home speaking. The other person's furnishings, furniture, and property belonged to the host. The visitor merely borrowed a space to express himself.

This was "normal visitation"—the visitor had his own body, just speaking on the host's turf. The visitor said "I want water," drinking the host's water, interacting with the host's property.

Body Borrowing Is Different

Body borrowing was entirely different. When Jia moved his soul into Yi body:

- Jia used Yi's body, Yi's throat, Yi's limbs.
- But Jia's words, decisions, and consequences all fell on Jia's head.
- If Jia broke Bing's window using Yi's body, Bing would demand compensation from Jia, not Yi.
- If Jia signed a contract using Yi's body, the contract bound Jia, not Yi.

More critically: when Jia acted in Yi's body, he thought he was still in his own home. The environment he saw was actually Yi's home—Yi's furniture, Yi's property, Yi's diary. If Jia tried to "put something in his own safe," he was actually putting it in Yi's safe.

Body Borrowing's Ingenious Use: The Substitute Actor

Yunzhou City's theater troupe discovered a marvelous use for body borrowing: substitute actors.

The troupe had a lead actor named A-Long. A-Long had extraordinary performing talent, but he was advanced in age and could no longer perform on stage. The troupe found a young, vigorous substitute named A-Hu.

During performances, A-Long moved his soul into A-Hu's body. The audience saw A-Hu's body performing, but every movement, every

line was controlled by A-Long's soul. A-Hu was merely a human shell.

A-Long could continuously "upgrade" his performance skills—practicing new moves backstage, then having A-Hu's body execute them in the next performance through body borrowing. The audience always saw "A-Hu," but experienced A-Long's ever-evolving art.

If A-Long passed away, the troupe only needed to find a new substitute, and A-Long's soul—if backed up in advance—could continue performing through borrowing. The troupe did not need to rebuild the entire stage; they only needed to change substitutes.

Body Borrowing's Disaster: Soul Displacement

But body borrowing was also the most dangerous secret art in Yunzhou City's history.

One time, a practitioner made an error during borrowing: he tried to "put something in his own pocket," but forgot he was in a borrowed body. The item ended up in the substitute body's pocket—and this pocket's location happened to correspond to a hidden compartment storing deadly poison in the substitute's body.

Poison leaked; the substitute died instantly; the practitioner's soul was forever trapped in the dead body.

This accident was called "soul displacement"—when the practitioner thought he was operating "his own space," he was actually operating the substitute's space, and the two spaces had completely different layouts.

Body Borrowing Rules

After countless accidents, Yunzhou City's practitioners summarized body borrowing's safety code:

1. **The borrowed body's internal layout must exactly match the original body.** If the original body's "heart position" was on the left, the substitute's "heart position" must also be on the left. Otherwise, the soul would get lost.

2. **The borrowed body must not have its own soul.** If the substitute body already had a soul, two souls would fight for control, with disastrous results.

3. **During borrowing, the original body enters a dormant state.** The original body could not act simultaneously; otherwise, a paradox of "two me's doing different things at the same time" would arise.

4. **After borrowing ends, no secrets may be left in the borrowed body.** Because when the soul departed, it took memories with it, but physical traces left in the substitute body (like words written on paper) would be exposed to the next borrower.

What This Fable Is About

This fable illustrates **EVM's `delegatecall`**—a special low-level call method. Unlike normal `call`, `delegatecall` executes in the **target contract's code** but uses the **calling contract's storage, balance,**

and state.

Key Mapping

Story Element	Technical Concept	Explanation
Normal visitation	CALL	Executed in target contract's context: uses target's storage; msg.sender is the caller
Body borrowing	DELEGATECALL	Executed in target contract's code, but uses caller's storage; msg.sender and msg.value remain unchanged
Jia's soul in Yi's body	Caller uses proxy contract code to execute	Logic code comes from the called contract (implementation), but storage is the caller's (proxy)
Consequences fall on Jia	msg.sender remains unchanged	`delegatecall` does not change `msg.sender`; caller identity is preserved
Substitute actor	Proxy contract pattern	Proxy contract uses `delegatecall` to delegate execution to Implementation contract; proxy itself only stores state
A-Long upgrades moves	Upgradeable contract	Replacing Implementation contract address; proxy storage unchanged; logic upgraded
A-Long passes, new substitute found	Contract migration	Old implementation abandoned; new implementation connected to same proxy; state inherited
Soul displacement / poison compartment	Storage collision	If Implementation and Proxy storage layouts are inconsistent, `delegatecall` writes to wrong locations, potentially causing severe vulnerabilities (e.g., Parity multi-sig wallet vulnerability)
Layouts must match	Storage layout compatibility	Upgraded Implementations must maintain the same storage layout as old

versions; new variables can only be appended at the end |
| Substitute must not have soul | No state conflicts | Implementation contracts should not have their own important state; all state should be in the Proxy |
| No secrets after borrowing | Self-destruct risk | If an Implementation contract executes `selfdestruct`, it destroys the Proxy contract through `delegatecall` (one of delegatecall's dangers) |

Why This Story?

`delegatecall` is one of Solidity's most difficult concepts to understand and the source of many serious security vulnerabilities (e.g., Parity multi-sig wallet lockup, various proxy contract vulnerabilities).

The "body borrowing" metaphor for `delegatecall` offers several intuitive advantages:

1. **Context switching visualized**: In another's body, but consequences are your own—corresponding to `delegatecall`'s context preservation
2. **Proxy contract intuition**: Substitute actor = Proxy, A-Long = Implementation, audience always sees Proxy address
3. **Storage collision danger**: Soul displacement = catastrophic consequences of storage layout mismatch

Further Reading

- wiki-web3: Proxy Contracts and Upgradeability
- [EVM Deep Dives: Delegatecall]

- OpenZeppelin: Proxy Patterns
- Parity Multi-Sig Wallet Hack Analysis

"The New Grammar Manual"

The New Grammar Manual

> The old writing had no punctuation. The new manual divides sentences into chapters.

The Story

Yunzhou City's legal documents had a five-hundred-year tradition: all documents were continuous rolls of parchment, without paragraphs, without headings, without punctuation. Judges had to read from beginning to end to know whether a document was a contract, a lawsuit, or a will.

This tradition brought three problems.

First Problem: Unable to Quickly Identify

When a courier delivered a new document to the court, the judge could not tell at a glance what type of document it was. He had to unroll the parchment and read from the beginning until discovering keywords—like "hereby" or "witness"—before judging the document type.

If someone maliciously disguised a poison recipe as a contract (writing a contract-like opening, then suddenly turning into a poison recipe midway), the judge would only discover something was wrong halfway through. But by then it might be too late—certain actions had already been executed based on the misread first half.

Second Problem: Unable to Verify Integrity

Parchment could be worm-eaten, water-damaged, or cut. If a document's ending was missing, the judge could not easily discover this—because he did not know how long the original document should have been. He could only read to where it "suddenly ended," then guess: was this how it originally ended, or was the ending missing?

Some cunning lawyers would deliberately delete unfavorable final clauses from contracts, then present the truncated contracts in court. Judges could not distinguish whether this was originally a short contract or a truncated long contract.

Third Problem: Inefficient Execution

The court's executors (responsible for carrying out document contents) had to read the entire document line by line from the first line. They could not skip any paragraph, because they did not know which paragraph was relevant.

If a document's first half was background introduction ("Party A is a Yunzhou City resident, Party B is an Eastern Sea merchant..."), executors had to read all this background before reaching the truly

important execution clauses. This greatly wasted time.

The New Grammar Manual Revolution

A young judge named Zhang Jie proposed an entirely new document format: "chaptered documents."

Chaptered documents were no longer continuous rolls of parchment. They were explicitly divided into several parts:

1. **Type declaration** (first page): Explicitly stating "this is a contract" or "this is a lawsuit." The judge knew how to handle it the moment he received the document.
2. **Header information** (second page): Listing basic document information—author, date, version. Metadata could be obtained without unrolling the full text.
3. **Body** (from third page): The actual content, clearly divided into paragraphs, each with a heading.
4. **Verification page** (last page): A special text generated through mathematical operations on the full text. If any page of the document was altered, the verification page would not match, and the judge would immediately know the document had been tampered with.

More critically: before being sent to court, chaptered documents had to pass a "format reviewer's" inspection. The reviewer did not look at document content, only whether the format was correct—whether the type declaration came first, whether the verification page was complete, whether chapters had clear separators.

If format review failed, the document never reached the judge. This meant: all documents reaching judges had correct format, complete structure, and were not truncated.

Unexpected Benefits

After chaptered documents were introduced, judges discovered unexpected benefits:

1. **Execution became faster.** Executors no longer needed to read the entire document. If it was an "execution order," they could flip directly to the "execution clauses" chapter, skipping the preceding background.

2. **Errors were easier to find.** If a lawyer tried to insert an "automatic transfer" clause into a contract, the format reviewer would notice this clause was not among the allowed chapter types and reject the document outright.

3. **Version compatibility.** New-format documents could be executed in old courts (old courts could still read continuous text), but new courts could fully utilize chaptered advantages. This was a gradual upgrade, not a revolutionary break.

Years later, all legal documents in Yunzhou City adopted the chaptered format. Someone asked Judge Zhang Jie: "What is your greatest contribution?"

Zhang Jie said: "I did not invent any new content. I gave documents 'structure.' In the old format, content and structure were mixed, inseparable. In the new format, structure is visible, verifiable, and optimizable. Once structure is made explicit, many things naturally improve."

What This Fable Is About

This fable illustrates **EVM Object Format (EOF)**—a new bytecode container format being introduced to Ethereum. EOF transforms EVM bytecode from a "flat continuous byte sequence" into a "structured container" containing type declarations, header metadata, code sections, data sections, and introduces pre-deployment validation.

Key Mapping

Story Element	Technical Concept	Explanation
Continuous parchment roll	Traditional EVM bytecode	Flat byte sequence; no structure; EVM interprets byte by byte
Unable to quickly identify type	Cannot distinguish code/data	Traditional bytecode mixes code and data; JUMP targets require runtime resolution
Poison recipe disguised as contract	Malicious bytecode / code injection	Can embed executable code in data sections, or jump to arbitrary locations via JUMP to execute unintended logic
Unable to verify integrity	Cannot statically verify	Traditional bytecode cannot have its completeness and security verified without execution
Inefficient execution	JUMP parsing overhead	EVM needs to calculate JUMP targets at runtime; cannot pre-optimize execution paths

Chaptered document	EOF container format	Bytecode is organized as structured containers: magic number + version + header + code sections + data section
Type declaration	EOF header type information	Explicitly declares the contract's number of code segments, data segment size, and other metadata
Verification page	EOF built-in verification	Container format checksum/length fields ensuring bytecode has not been truncated or tampered with
Format reviewer	Pre-deployment EOF validation	EOF contracts must pass static validation before deployment, ensuring correct structure, legal JUMP targets, safe stack depth, etc.
Format failure means no court	Invalid EOF cannot be deployed	Bytecode not conforming to EOF format is rejected at deployment
Execution faster	EOF performance optimization	Because structure is known, EVM can precompile and optimize execution paths without runtime JUMP parsing
Gradual upgrade	EOF coexists with old bytecode	EOF is opt-in; old bytecode remains compatible; new contracts can choose to use EOF

Why This Story?

EOF is one of the longest and most complex improvements in the EVM ecosystem, involving changes to the underlying execution model. For those unfamiliar with EVM internals, "bytecode container format" sounds extremely abstract.

The "chaptered document" metaphor for EOF offers several intuitive advantages:

1. **Visualization of structure**: From "a roll of parchment" to "a document divided into chapters"—letting readers understand the

core value of a "container format"
2. **Safety improvement**: The format reviewer intercepts problems before documents reach judges—corresponding to EOF's pre-deployment validation
3. **Source of performance optimization**: Executors flipping directly to relevant chapters—corresponding to optimizations EOF can make by knowing code structure in advance

Further Reading

- wiki-web3: EVM Object Format EOF Introduction
- EIP-3540: EOF - EVM Object Format v1
- EIP-3670: EOF - Code Validation
- EIP-4200: EOF - Static relative jumps
- EVM Object Format (EOF) Overview

"The Message Tube"

The Message Tube

> A tube that can only be read, never written. Words passed to others that you yourself cannot change.

The Story

Yunzhou City's communication system had three ways to deliver messages.

First: Oral Messages

The oldest method was direct oral delivery. You walked up to someone and spoke. They heard what they heard.

Oral messages were simple and direct. The downside: if you wanted to tell someone a hundred names, you had to recite them one by one. What if they could not remember? You had to recite again.

And oral messages could not be "passed on"—you could not have a friend relay your oral message to a third person. Oral messages only worked between speaker and listener.

Second: Letters

You wrote a letter, put it in an envelope, and gave it to a courier. The courier delivered it to the recipient, who opened and read it.

Letters could carry complex content—a hundred names could be written on paper without omission. Letters could also be forwarded—the recipient could pass the original letter to someone else.

But letters had a cost: writing required paper and ink; delivery required courier payment. And once sent, the content could not be changed. If you wanted to modify, you had to write a new letter.

Third: Message Tubes

Yunzhou City's army used a special communication tool: message tubes.

A message tube was a sealed copper pipe containing a slip of paper. The paper's content was fixed when placed in the tube—anyone could open the tube and read the paper's content, but no one could modify it.

Message tubes had three unique properties:

First: read-only.

The recipient could only read the tube's content, not add, modify, or delete words. If the general's order was "advance ten miles," the paper in the tube always said these four characters. The recipient

could not change it to "advance twenty miles."

Second: transparently public.

The tube had no lock. Anyone passing by could open and read the content. In the army, this was not a problem—the orders themselves did not need secrecy, only needed to ensure they were not tampered with.

Third: single-use.

After reaching its destination, the tube completed its mission. The paper inside could be copied and archived, but the tube itself was not reused. Next time a message was sent, a new tube was used.

Message Tubes' Ingenious Use: Remote Command

Yunzhou City's general was stationed on the northern frontier; the emperor was in the capital. The emperor wanted to command the general to do something, but he could not personally travel to the frontier to deliver oral orders. He could write a letter, but letters could be intercepted or altered en route.

The message tube solved this problem: the emperor wrote the order on paper, placed it in the tube, and sealed it. The courier transported the tube, not a bare letter. Along the way, anyone could open the tube to read the order (transparent), but no one could modify the order without breaking the seal (tamper-proof).

After receiving the tube, the general opened it, read the order, and executed it. During execution, the general could cross-reference the tube's order to confirm he had not misunderstood.

Limitations of Message Tubes

But message tubes also had limitations.

If the emperor wanted the general to "flexibly decide based on frontline conditions," message tubes were insufficient—because tube orders were fixed; the general could not request real-time instructions from the emperor. He could only execute according to the received order after getting the tube.

If the order contained large amounts of data—like "here is a list of five hundred names; please distribute grain according to the list"—the tube could hold the list (because it was paper), but if the general needed to input these names one by one into a ledger, he had to manually copy them. The tube's data could not be directly "imported" into the ledger; it could only be read.

What This Fable Is About

This fable illustrates **EVM Calldata**—read-only data carried in transactions or message calls. Calldata is external call input data, stored in the transaction body, cannot be modified by the called contract, and can only be read via `msg.data` or function parameters.

Key Mapping

Story Element	Technical Concept	Explanation
Oral message	Internal call / local computation	Internal function calls within a contract; data passes on stack and memory without serialization
Letter	Memory / Storage	Writable data storage areas that can be modified and passed
Message tube	Calldata	Read-only data from external calls, transmitted with the transaction; called contracts can only read, not modify
Read-only	Calldata immutability	`calldata` location data cannot be modified. Solidity parameters marked `calldata` are read-only
Transparently public	Calldata publicity	Transaction data is public to the entire network; anyone can read via blockchain explorer
Single-use	Calldata lifecycle	Calldata exists only during the current message call; inaccessible after call ends
Sealed copper pipe	Transaction integrity	After a transaction is signed, calldata cannot be tampered with (otherwise signature verification fails)
Remote command	Cross-contract call	Contract A calls Contract B, passing function selector and parameters via calldata
Large data in tube	Calldata capacity	Calldata can carry large data (e.g., arrays, strings), but reading and parsing require gas
Manual copying	Calldata to Memory/Storage copy	If you need to modify calldata, you must first copy it to memory or storage
Cannot request real-time instructions	Calldata unidirectionality	In a single call, calldata is one-way. The callee cannot modify calldata and return it to the caller

Why This Story?

Calldata is one of EVM's three main data locations (storage, memory, calldata), but its "read-only" and "external" properties confuse many beginners.

The "message tube" metaphor for calldata offers several intuitive advantages:

1. **Read-only**: Tubes can only be read, not modified—corresponding to calldata immutability
2. **Externality**: Tubes are for remote communication—corresponding to calldata existing only in external calls
3. **Transparency**: Anyone can open and read—corresponding to the public nature of on-chain transactions

Further Reading

- wiki-web3: EVM Memory Mechanism Deep Dive
- Solidity Docs: Data Location
- Ethereum Yellow Paper: Message Call
- ABI Encoding Specification

"The Runestones"

The Runestones

> Each stone bears only one character, but arranged in the correct order, they can build a city.

The Story

In Yunzhou City's construction sites, no blueprints were used. Craftsmen used an ancient system: runestones.

Each runestone was a palm-sized rock with a symbol carved on it. There were few symbol types—only about two hundred. But these two hundred symbols, arranged in different orders, could express the construction method of any building.

How Runestones Worked

At the center of the construction site stood a large stone platform that could hold only thirty-two runestones at once. This was the platform's physical limit—no more could fit.

The craftsman's workflow was:

1. He retrieved runestones from the warehouse and placed them on the platform in order.
2. When the platform held enough runestones for a certain operation, an action was automatically triggered.
3. After the action completed, the runestones on the platform were replaced by the action's result.
4. He continued placing new runestones, triggering the next action.

For example:
- Operation "add": required two runestones on the platform (e.g., "3" and "5"). When triggered, both stones were removed and replaced by a new stone "8."
- Operation "duplicate": required one stone on the platform. When triggered, the stone was duplicated; now there were two identical stones on the platform.
- Operation "swap": required at least two stones on the platform. When triggered, the top two stones exchanged positions.

Translation of Runestones

High-level architects never dealt directly with runestones. They used a language called "blueprint language" to design buildings—"build a wall here, ten feet high, two feet wide."

Then, a specialized "translator" converted blueprint language into runestone sequences. This process was deterministic: the same blueprint always produced the same runestone sequence.

But translation was not one-to-one. A single line like "wall ten feet high" might be translated into dozens of runestone operations—calculating area, checking load-bearing, allocating materials. The translator also optimized: if the blueprint repeatedly said "wall ten

feet high," the translator would not translate it three times but cache the result for reuse.

Direct Runestone Manipulation

Some extremely precise buildings could not be accurately expressed in blueprint language. At these times, master-level craftsmen directly wrote runestone sequences.

Direct manipulation advantages:
- **Precise**: Every stone's position was under the craftsman's control; no translator ambiguity.
- **Efficient**: The translator added many checks for safety; direct manipulation could skip unnecessary checks, saving time and materials.
- **Access hidden functions**: Blueprint language only exposed common operations, but some special operations could only be used by directly manipulating runestones.

Risks were equally obvious:
- If a craftsman reversed the order of two stones, the entire building might collapse.
- If a craftsman forgot to clear the platform after a certain step, the platform would fill up, and subsequent operations could not proceed.
- If a craftsman used a "self-destruct" rune—a special stone that could make the entire construction site disappear—the consequences were irreversible.

Immutability of Runestones

Once a runestone sequence was carved and delivered to the

construction site, it could not be changed. If the architect discovered a design flaw, the only option was to carve a new runestone sequence and discard the old.

This meant: runestone writing required extreme caution. A small error—like confusing "add" with "subtract"—could produce completely wrong results, and could not be corrected on-site.

Years later, Yunzhou City's tallest tower was built with runestones. Its designer was a master who directly manipulated runestones. He said: "Blueprint language lets ordinary people build houses. But if you want to build a tower touching the clouds, you must understand the meaning of every single stone."

What This Fable Is About

This fable illustrates **EVM Opcodes and Bytecode**. EVM is a stack-based virtual machine; all smart contracts are ultimately compiled into bytecode—a sequence of opcode instructions. Solidity is a high-level language; the compiler translates it into EVM bytecode.

Key Mapping

Story Element	Technical Concept	Explanation

Runestones	EVM bytecode / opcodes	Each opcode is one byte (or more); EVM interprets and executes them one by one
About two hundred symbols	~150 valid opcodes	EVM defines about 150 opcodes covering arithmetic, logic, storage, calls, environment info, etc.
Platform holds thirty-two stones	EVM stack depth limit	EVM stack maximum depth is 1024 256-bit words. Exceeding causes stack overflow
Operation "add"	ADD opcode	Pops two values from stack top, adds them, pushes result back to stack top. All EVM computation occurs on the stack
Operation "duplicate"	DUP opcode	Duplicates a value from the stack to the stack top
Operation "swap"	SWAP opcode	Exchanges positions of top two stack values
Blueprint language	Solidity / Vyper and other high-level languages	Developers write contracts in high-level languages; compiler translates to EVM bytecode
Translator	Solidity compiler (solc)	Translates high-level language to EVM bytecode and optimizes
Translation optimization	Compiler optimization	Constant folding, dead code elimination, loop optimization, etc.
Direct runestone manipulation	Inline assembly (Yul / Inline Assembly)	Developers can directly write opcode sequences for more precise control and higher efficiency
Access hidden functions	Assembly accesses底层	Certain EVM features (e.g., fine-grained `create2` control, memory layout optimization) can only be accessed via assembly
Platform filled	Stack too deep error	Too many local variables in a Solidity function cause stack depth to exceed 16-slot compiler limit
Self-destruct rune	SELFDESTRUCT opcode	Destroys contract, sends balance to specified address; irreversible
Carved, cannot be changed	Deployed bytecode is immutable	After contract deployment, code cannot be modified (unless using

proxy contract pattern) |

Why This Story?

EVM bytecode and opcodes are smart contracts' "final form," but most developers only write Solidity and never directly look at bytecode. Understanding "what lies beneath the high-level language" is crucial for debugging, optimization, and security auditing.

The "runestones" metaphor for bytecode offers several intuitive advantages:
1. **Stack machine concretized**: Platform = stack; placing stones = push; operations = pop and compute
2. **Compilation process visualized**: Blueprint → translator → runestones = Solidity → solc → bytecode
3. **Value of assembly**: Direct runestone manipulation = inline assembly; precise but dangerous

Further Reading

- wiki-web3: Solidity Assembly and Memory Layout Optimization
- EVM Opcodes Reference
- Ethereum Yellow Paper
- Solidity Inline Assembly
- EVM Deep Dives by noxxx3xxon

"The Cooling Chamber"

The Cooling Chamber

> Decisions made in passion often need to sit in an ice room for a while.

The Story

Yunzhou City's merchant guild used to make decisions simply: the guild master convened everyone for a meeting, discussed, voted, decided on the spot, and executed immediately.

This method was fast, but it caused several serious accidents.

First Accident: Passionate Decision

One year, rumors suddenly spread that all spice ships from the Eastern Sea had sunk. The guild held an emergency meeting and, in panic, voted to "immediately sell all spice inventory at current prices."

The decision was executed immediately. The guild sold all inventory at low prices. Three days later, the news was proven false—the spice

ships had not sunk at all. Market spice prices instantly doubled. The guild lost half a year's profit due to panic selling.

Post-mortem, the vice president said: "If we had waited three more days for confirmation before deciding, we wouldn't have lost."

The master sighed: "But the atmosphere in the meeting room at the time left no room for calm thought."

Second Accident: Malicious Raid

Another year, an outsider merchant joined the guild. At an ordinary meeting, he suddenly proposed a motion: "Amend guild bylaws to allow the guild master alone to decide transactions over ten thousand taels."

The motion itself was absurd—it amounted to giving the guild's financial power to one person. But the outsider had bribed over a dozen small members in advance, who suddenly voted collectively in favor. Most members had not reacted before the vote was over.

Under the old rules, the motion took effect immediately. The guild nearly fell under one person's control. Fortunately, several old members took overnight action to block execution before the bylaws took effect—but this should never have happened.

Birth of the Cooling Chamber

After these accidents, the guild introduced the "cooling chamber" system.

The rules were:

1. Any major decision (involving amounts over one thousand taels, or amending bylaws), after passing the vote, would not take effect immediately.

2. The decision would be sent to the "cooling chamber"—a public, transparent waiting area. In the cooling chamber, the decision's content was visible to all; anyone could review, discuss, and raise objections.

3. The cooling period was fixed at seven days. During these seven days, if anyone discovered problems with the decision, they could initiate an "emergency veto." If more than one-third of members supported the veto, the decision would be canceled.

4. After seven days, if not vetoed, the decision automatically took effect.

Effects of the Cooling Chamber

After introducing the cooling chamber, the guild's decision quality significantly improved.

- Passionate decisions were filtered: seven days was enough for market emotions to settle and for rumors to be proven false or true.
- Raid attacks were blocked: even if malicious motions passed, the seven-day cooling period gave other members enough time to organize opposition.
- Errors were discovered in time: several times during the cooling period, hidden vulnerabilities were discovered in motions—seemingly harmless clauses that actually allowed massive power abuse. These vulnerabilities were discovered and fixed during the

cooling period.

But the cooling chamber also had costs: some truly urgent decisions were forced to wait seven days. One year, floods came; the guild needed emergency appropriations to repair dikes. But under the rules, the appropriation decision had to wait seven days. The dikes breached during the wait.

The guild later made adjustments: establishing an "emergency channel"—with agreement from over two-thirds of members, certain emergency decisions could skip the cooling chamber. But emergency channel use was permanently recorded and subject to post-hoc review.

Years later, the cooling chamber system was adopted by all major organizations in Yunzhou City. An old member said: "The cooling chamber is not delay; it is giving reason a chance. Most catastrophic decisions are made in moments of hot blood. The cooling chamber gives hot blood time to cool."

What This Fable Is About

This fable illustrates **Timelock governance mechanisms**—common safety measures in DeFi protocols. Major governance decisions (like modifying protocol parameters, upgrading contracts, transferring funds) must wait for a period (e.g., 2-7 days) after voting before execution, giving users time to react and exit.

Key Mapping

Story Element	Technical Concept	Explanation
On-the-spot decision and execution	Governance without timelock	Voting takes effect immediately; extremely high risk
Panic selling	Emotional governance attack	Decisions made during market panic are often wrong
Malicious raid	Governance flash loan attack / 51% voting raid	Attackers temporarily acquire massive voting power to pass malicious proposals
Cooling chamber	Timelock contract	After voting passes, proposals enter a waiting queue with delayed execution
Seven-day cooling period	Delay parameter	Common values are 2-7 days, sufficient for users to detect anomalies and exit
Emergency veto	Opposition / cancellation mechanism	Users can withdraw funds or initiate opposition during the delay period
Hidden vulnerabilities discovered	Community audit window	The delay period gives the community time to review proposal code and impact
Emergency channel	Multi-sig / emergency pause	In extreme cases, trusted multi-sigs can skip timelock to execute emergency actions
Permanent record	On-chain transparency	All operations are recorded on-chain; post-hoc auditability
Flood dike repair	Timelock vulnerability	True emergencies may miss optimal handling timing due to timelock

Why This Story?

Timelock is DeFi governance's last line of defense, but many protocols omit it (or set extremely short delays) due to inconvenience. The 2022 Beanstalk attack (governance flash loan) was a bloody lesson—attackers acquired voting power, passed proposals, and executed them in a single transaction, stealing $180 million.

The "cooling chamber" metaphor for timelock offers several intuitive advantages:

1. **Emotional filtering**: Hot blood cooling = preventing panic and FOMO-driven decisions
2. **Defense against raids**: Seven-day wait = giving community time to organize opposition
3. **Tradeoffs of emergency channels**: Skipping the cooling chamber = double-edged sword of multi-sig/emergency power

Further Reading

- wiki-web3: Timelock and Governance
- OpenZeppelin: Timelock Controller
- Beanstalk Governance Attack Analysis
- DeFi Governance Best Practices

"The Weighted Democracy"

The Weighted Democracy

> One person, one vote is the ideal. One coin, one vote is the reality. The problem with reality: the more money you have, the heavier your voice.

The Story

After abolishing the guild master's dictatorship, Yunzhou City's merchant guild tried a "one person, one vote" democratic system. Every member, regardless of contribution size, had one vote.

This system quickly exposed problems.

The Dilemma of One Person, One Vote

The guild had two major merchants, each contributing one hundred thousand taels, accounting for 80% of total guild funds. There were also two hundred small merchants, each contributing a few hundred taels, totaling only 20%.

In a vote on "whether to increase membership fees," two hundred

small merchants passed the "massively increase fees" proposal with a 200:2 overwhelming majority. The two major merchants' opposition was ineffective.

Result: the two major merchants exited the guild, taking 80% of funds. The guild instantly fell into financial crisis.

The vice president reflected: "One person, one vote protected the small but drove away the big. Without the big, the small have nothing to gain."

Introduction of Weighted Voting

The guild introduced a "weight voting" system: voting weight was proportional to contribution. You contributed how much silver, you had how much weight.

Under the new system, the two major merchants held 80% of weight; two hundred small merchants held 20%. Any major decision was dominated by the major merchants' opinions.

This system solved the "big players leaving" problem but brought new problems:

- Small merchants' opinions were almost ignored. Even if two hundred small merchants unanimously opposed, as long as the two major merchants favored it, the proposal passed.
- The guild became an organization where "two major merchants called the shots"; democracy existed in name only.
- Worse, external capital could buy weight to manipulate guild decisions.

Square Weight Experiment

A mathematician named Dela proposed a compromise: "square weights."

The rule: voting weight was not proportional to contribution, but proportional to the square root of contribution.

Example:
- Someone contributing 100 taels had weight = $\sqrt{100} = 10$
- Someone contributing 10,000 taels had weight = $\sqrt{10,000} = 100$

Note: contribution differed by 100 times, but weight only differed by 10 times. Those who contributed more still had advantages, but advantages were "compressed."

Under this system:
- Two major merchants (each contributing 100,000 taels) total weight = $2 \times \sqrt{100,000} \approx 632$
- Two hundred small merchants (averaging 100 taels) total weight = $200 \times \sqrt{100} = 2,000$

Small merchants' total weight actually exceeded major merchants! This meant small merchants, if united, could counterbalance major merchants. But major merchants were still much stronger than any single small merchant—their individual influence remained significant.

Delegation System

But square weights had a practical problem: not every small merchant had time and knowledge to study every proposal. Often,

they did not even know what they voted for.

The guild introduced a "delegation" system: small merchants could delegate their weight to a representative they trusted. The representative voted with pooled weight. Small merchants could withdraw delegation anytime and vote themselves.

This system made professional governance possible. Some people who made studying proposals their profession emerged—they were called "governance experts." Small merchants delegated weight to governance experts; experts voted with professional knowledge. In return, governance experts received compensation from the guild's governance rewards.

External Attacks

The weighted voting system's greatest enemy came from outside.

One year, an external consortium wanted to control the guild. They had no long-term intention of operating the guild—they only wanted to pass one malicious proposal through a single vote to empty the guild's treasury.

Under the old system, this required purchasing over 50% of weight—extremely costly.

But the external consortium discovered a vulnerability: they could borrow large amounts of weight, vote, transfer funds, then return the borrowed weight. The entire process could be completed within one day.

This was called a "flash weight attack." The guild nearly went bankrupt.

Afterward, the guild added a rule to voting: "Voting weight must be locked for a period (e.g., three days) before the vote; it cannot be borrowed or transferred during the voting period."

This simple modification completely eliminated the possibility of flash weight attacks.

Years later, Yunzhou City's weighted democracy continued to evolve. Some said it was not fair enough; others said it was the most realistic option. An old merchant summarized: "There is no perfect voting system. One person, one vote drives away capital; one coin, one vote becomes oligarchy. Square weights are a compromise, but they at least give 'united majorities' a chance to counter 'wealthy minorities.'"

What This Fable Is About

This fable illustrates **token voting governance**—the most common decision-making mechanism in DeFi protocols and DAOs. It explores the evolution from "one person, one vote" to "one coin, one vote" to "quadratic voting," and practical issues like delegation and flash loan attacks.

Key Mapping

Story Element	Technical Concept	Explanation
One person, one vote	Traditional democracy	Each address gets one vote; coin holdings not considered
Big merchants driven away	Capital flight risk	If large holders' interests cannot be protected, they exit the protocol, causing liquidity drain
Weight voting	Token-weighted voting	Voting weight proportional to coin holdings. Most common DAO governance mechanism
Oligarchy	Plutocracy	Wealthy holders control decisions; small holders lose influence
External capital buying weight	Governance token acquisition	Attackers purchase large amounts of governance tokens to pass malicious proposals
Square weights	Quadratic voting	Voting weight = √balance. Balances large holders' influence and small holders' collective power
Delegation	Vote delegation	Token holders delegate voting power to representatives (e.g., Compound's Governance Bravo)
Governance experts	Governance contributors / delegates	Professional governance participants who study proposals and vote on behalf of delegators
Flash weight attack	Flash loan governance attack	Attackers borrow funds to temporarily acquire voting power, pass malicious proposals
Weight lock before voting	Voting delay / vote locking	Requires voting power to be held before proposal creation; prevents flash loan attacks
Three-day lock	Common timelock parameters	E.g., Compound's voting delay is 1 block, but many protocols have extended to several days

Why This Story?

Token voting is the core of DAO governance but also one of the most criticized mechanisms—Vitalik Buterin and others have pointed out that "one coin, one vote" is essentially plutocracy.

The "weighted democracy" metaphor for token voting offers several intuitive advantages:

1. **Intuitive inequality**: Weight heaviness = voting weight; readers immediately understand "money = voice"
2. **Quadratic voting visualization**: Square root compression = giving majorities a chance to counter wealthy minorities
3. **Flash loan attack concretization**: Borrowing weight to vote then returning = flash loan governance attack

Further Reading

- wiki-web3: Token Voting Governance
- Vitalik: Moving Beyond Coin Voting Governance
- Quadratic Voting: How Mechanism Design Can Radicalize Democracy
- Beanstalk Flash Loan Governance Attack
- Compound Governance

"The Theater of Shadows"

The Theater of Shadows

> The puppets remain the same, but the puppeteer behind the screen can be replaced.

The Story

In Yunzhou City, there was a famous puppet theater. The audience saw only the puppets and the stage. Behind the screen, unseen by all, the puppeteers manipulated the strings.

One day, the master puppeteer fell ill. The show had to go on. So the theater devised a clever system: the stage, the puppets, and the script remained unchanged, but a new puppeteer stepped behind the screen and took control of the strings.

The audience noticed nothing. The puppets moved as they always had. The story unfolded as written. Only the puppeteer had changed.

This is the essence of the proxy contract: a single address that the world interacts with, while the logic behind it can be upgraded without changing the address.

What This Fable Is About

This fable illustrates **proxy contracts and upgradeability**—a design pattern that allows smart contract logic to be updated while preserving the contract's address and state.

Key Mapping

Story Element	Technical Concept	Explanation
Puppet theater stage	Proxy Contract Address	The fixed address that users interact with
Puppets	Contract State / Storage	Data persisted across upgrades
Puppeteer behind screen	Implementation Contract	The logic that controls how the contract behaves
Master puppeteer falls ill	Bug discovery / need for upgrade	The original implementation has a flaw or needs new features
New puppeteer steps in	Implementation upgrade	A new implementation contract is deployed and pointed to by the proxy
Audience notices nothing	Transparent upgrade	Users continue interacting with the same address; the upgrade is invisible to them

Why This Story?

Proxy patterns are fundamental to modern smart contract

development. The puppet theater metaphor makes the separation of "interface" (stage) and "logic" (puppeteer) intuitively clear.

Further Reading

- wiki-web3: Upgradeable Smart Contract Design
- OpenZeppelin: Proxy Patterns
- EIP-1967: Proxy Storage Slots

"The Keyless City"

The Keyless City

> A city where you need no key to enter. Your face, your voice, or simply your reputation is enough.

The Story

In Yunzhou City, every house had a lock, and every lock had a key. Lose the key, and you could never enter again. There were no locksmiths. There was no "forgot my password" button.

This system was simple and secure—if you had the key. But it was unforgiving. A merchant who lost his key in the river had to build a new house. A widow whose husband had died with the key could never access the family vault.

Then the Keyless City was built.

In the Keyless City, houses had no locks. Instead, each house had a "guardian spirit"—a small, intelligent entity that decided who could enter based on rules set by the owner.

One house might say: "Let in anyone who knows the secret rhyme." Another: "Let in anyone carrying the jade talisman."

Another: "Let in anyone approved by three of the ten council members."

The guardian spirit was not a physical lock. It was a piece of logic—rules encoded in language that anyone could understand.

What This Fable Is About

This fable illustrates **Account Abstraction (ERC-4337)**—a proposal that turns Ethereum accounts from simple key-controlled addresses into smart contract wallets with programmable access rules, social recovery, and custom transaction logic.

Key Mapping

Story Element	Technical Concept	Explanation
Traditional lock and key	EOA (Externally Owned Account)	Controlled by a single private key; lose the key, lose the account
Keyless City	Smart Contract Wallet	An account controlled by code rather than a single key
Guardian spirit	Smart contract logic	Programmable rules determining who can authorize transactions
Secret rhyme	Password / multi-factor	Authentication beyond just a private key
Jade talisman	Hardware token / NFT	Ownership of a specific token can grant access

| Council approval | Social recovery | Multiple guardians can collectively recover an account |
| No locksmiths | No centralized recovery | The system is self-sovereign; no third party can reset your access |

Why This Story?

Account abstraction is the most important UX improvement in Ethereum's roadmap. The Keyless City metaphor makes the shift from "key = identity" to "rules = identity" viscerally clear.

Further Reading

- wiki-web3: Account Abstraction ERC-4337 Explained
- ERC-4337: Account Abstraction via Entry Point Contract
- Vitalik: Soulbound Tokens and Account Abstraction

"The Factory of Echoes"

The Factory of Echoes

> A workshop that does not build new instruments, but creates perfect echoes of a single master instrument.

The Story

In Yunzhou City, a master instrument maker named Chen created the perfect scale. It was precise, beautiful, and expensive to build. Other merchants wanted similar scales, but they could not afford Chen's fees.

Chen had an idea. Instead of building each scale from scratch, he built a single "echo chamber." The chamber contained a perfect copy of his master scale. When a merchant wanted a scale, Chen did not build a new one—he simply opened a small window into the echo chamber.

Through the window, the merchant saw the master scale. The window itself was tiny and cost almost nothing to make. But through it, the merchant could use all the precision of the master scale.

The echo chamber had one rule: it never changed. All windows looked into the same chamber. If Chen improved the master scale,

every window automatically showed the improvement. If he discovered a flaw and fixed it, every window was fixed simultaneously.

The merchants loved it. They paid a fraction of the cost and received the same quality. Chen loved it too—he maintained only one instrument, not a thousand.

What This Fable Is About

This fable illustrates **EIP-1167 Minimal Proxy Contracts**—a pattern that allows deploying lightweight proxy contracts that delegate all calls to a single implementation contract, dramatically reducing deployment costs.

Key Mapping

Story Element	Technical Concept	Explanation
Master scale	Implementation Contract	The single contract containing all logic
Echo chamber	Proxy pattern	Lightweight contracts that forward calls to the implementation
Window into the chamber	Minimal Proxy	A tiny contract (only 45 bytes) that delegates all calls to the master
Tiny cost to make a window	Low deployment cost	EIP-1167 proxies are extremely cheap to deploy compared to full contracts

| Improvement reflected everywhere | Single point of maintenance | Updating the implementation benefits all proxies simultaneously |
| Flaw fixed everywhere | Bug fix propagation | A single fix applies to all clones instantly |

Why This Story?

EIP-1167 is one of the most elegant gas optimizations in Ethereum. The echo chamber metaphor captures the essence: thousands of cheap windows into a single, well-maintained master.

Further Reading

- wiki-web3: EIP-1167 Minimal Proxy Deep Dive
- EIP-1167: Minimal Proxy Contract
- OpenZeppelin: Clones

"The Ledger After the Avalanche"

The Ledger After the Avalanche

> When the mountain falls, what matters is not who wrote the ledger, but whether anyone can still read it.

The Story

In the mountains above Yunzhou City, monks kept the imperial ledger. Every transaction in the empire was recorded in their great books. The monks were trusted; their books were the source of truth.

One winter, an avalanche destroyed the monastery. The monks survived, but the books were buried under tons of snow and rock.

The empire faced a crisis. No one doubted that the monks had recorded the transactions honestly. But the records were inaccessible. Merchants could not verify their balances. Tax collectors could not assess dues. The economy ground to a halt.

A young engineer proposed a new system. Instead of keeping the ledger in one monastery, every town would keep a copy. Not the entire ledger—no town had space for that—but every town would keep enough fragments that, collectively, the entire ledger could be reconstructed.

The system worked through random sampling. If a merchant wanted to verify a transaction, he would ask ten random towns for fragments. If all ten provided consistent fragments, the merchant could be confident the full ledger existed somewhere. If even one town failed to provide its fragment, the alarm would sound: the ledger was incomplete.

This system was not as efficient as the monastery. Verification took longer. But when the next avalanche came—this time a flood that destroyed three towns—the empire kept running. The lost fragments were recovered from other towns. The ledger endured.

What This Fable Is About

This fable illustrates **Data Availability (DA)**—the guarantee that block data has been published and is available for anyone to download and verify. DA is critical for rollups and light clients.

Key Mapping

| Story Element | Technical Concept | Explanation |

|---|---|---|
Monastery ledger	Centralized block storage	A single source of truth for all transaction data
Avalanche	Node failure / censorship	A catastrophic event that makes data inaccessible
Towns keeping fragments	Distributed data availability	Data is erasure-coded and distributed across many nodes
Random sampling	Data Availability Sampling (DAS)	Light clients randomly sample data chunks to verify availability without downloading everything
Ten consistent fragments	Sampling confidence	The probability of detecting data unavailability increases exponentially with each sample
One town failing	Unavailability detection	If any sampled chunk is missing, the data is provably unavailable
Lost fragments recovered	Erasure coding	Using Reed-Solomon coding, the full data can be reconstructed from a subset of fragments

Why This Story?

Data availability is often overlooked but is the foundation of rollup security. The avalanche metaphor makes it visceral: trust is not enough; accessibility is what matters.

Further Reading

- wiki-web3: Data Availability Is More Important Than You Think
- Ethereum.org: Data Availability
- Celestia: What is Data Availability?

"The Compressed Caravan"

l1与l2的融合.md"
tags: [infrastructure, rollup, l2, scaling, data-compression]

The Compressed Caravan

> Instead of sending every merchant individually, pack them into a single caravan and send one manifest.

The Story

In the ancient trade routes of Yunzhou, merchants traveled to the capital to register their transactions. Each merchant carried a heavy satchel of documents: bills of sale, tax receipts, inventory lists.

The road to the capital was narrow. Only a few merchants could pass each day. The rest waited in line, sometimes for weeks.

A clever caravan master devised a new system. Instead of each merchant traveling individually, merchants would gather at a staging post. There, a scribe would:
1. Collect all their documents.
2. Compress them into a single, compact scroll using a special shorthand.

3. Send the scroll to the capital via a fast courier.
4. At the capital, another scribe would expand the scroll back into individual documents and register each one.

The result: one scroll carried the transactions of a hundred merchants. The road was no longer congested. Merchants no longer waited weeks. The capital's registry processed the same volume of transactions with a fraction of the effort.

But there was a catch. The scroll had to be accurate. If the scribe made an error in compression, or if the expanding scribe misread a symbol, the registered transactions would be wrong. The system depended entirely on the integrity of the scribes.

To ensure honesty, the staging post posted the compressed scroll publicly before sending it. Anyone could examine the scroll and challenge it if they found an error. Only if no challenges were raised within three days did the courier depart.

What This Fable Is About

This fable illustrates **Rollups**—Layer 2 scaling solutions that execute transactions off-chain, compress the data, and post a compact representation to Layer 1 for security and data availability.

Key Mapping

Story Element	Technical Concept	Explanation
Individual merchants	L1 transactions	Each transaction processed directly on Ethereum mainnet
Narrow road	L1 throughput limit	Ethereum's limited block space and gas constraints
Staging post	Rollup sequencer / operator	The entity that collects and orders L2 transactions
Scribe's shorthand	Data compression	Rollups compress transaction data before posting to L1
Compact scroll	Compressed batch	The calldata or blob posted to L1 containing many L2 transactions
Expanding scribe	Rollup state transition function	The deterministic logic that reconstructs L2 state from L1 data
Public posting before sending	Fraud proof / challenge window	Optimistic rollups allow a challenge period before finalizing batches
Three-day challenge period	Dispute window	The time during which invalid state transitions can be challenged

Why This Story?

Rollups are the dominant Ethereum scaling strategy. The compressed caravan makes the core insight—batching and compression—intuitively clear.

Further Reading

- wiki-web3: Exploring Ethereum Native Rollup

- Ethereum.org: Rollups
- Vitalik: An Incomplete Guide to Rollups

"The Breath Tax"

The Breath Tax

> In this city, every breath is taxed. The deeper you breathe, the more you pay.

The Story

In Yunzhou City, there was an unusual tax: the breath tax. Every inhabitant paid a small fee for each breath they took. The fee was tiny—fractions of a copper coin per breath—but it added up.

The tax was not arbitrary. It was calculated based on how much work your breathing caused the city. A shallow breath, barely noticeable, cost almost nothing. A deep, lung-filling breath that required the city's bellows to pump harder cost significantly more.

Some activities required many deep breaths. Running a mile might cost a hundred breaths. Lifting a heavy stone might cost fifty. Standing still, breathing shallowly, might cost only one.

The city used the breath tax revenue to pay the workers who maintained the air pumps. Without the tax, the pumps would stop, and the city would suffocate.

But the tax also served another purpose: it prevented waste. A prankster who wanted to run around the city square blowing a horn all day would face an enormous breath tax bill. The tax made frivolous activity economically unviable, preserving the city's air for those who truly needed it.

What This Fable Is About

This fable illustrates the **EVM Gas mechanism**—the pricing system that charges users for computational resources consumed by their transactions, preventing spam and ensuring network sustainability.

Key Mapping

Story Element	Technical Concept	Explanation
Breath tax	Gas fee	Every EVM operation consumes gas; users pay for total gas consumed
Shallow breath	Cheap operation	Simple operations like ADD cost 3 gas
Deep breath	Expensive operation	Complex operations like SSTORE (storage write) cost 20,000 gas
Running a mile	Complex transaction	A transaction with many operations consumes more gas
Air pump workers	Validators / miners	Gas fees compensate validators for processing and securing transactions
Prankster blowing horn	Spam / DoS attack	High gas costs make

spam and denial-of-service attacks economically prohibitive |
| Breath tax prevents waste | Gas as resource control | Gas limits ensure the network is not overwhelmed by computation |

Why This Story?

Gas is the most fundamental concept in Ethereum UX. The breath tax makes the abstract idea of "paying for computation" viscerally intuitive.

Further Reading

- wiki-web3: EVM Gas Mechanism and Fees Explained
- Ethereum.org: Gas and Fees
- EVM Opcodes and Gas Costs

"The Bridge Guard"

The Bridge Guard

> Two cities separated by a chasm. The bridge between them is guarded. When the bridge falls, both cities are cut off.

The Story

Yunzhou City and Donghai City stood on opposite sides of a deep chasm. They used different currencies: Yunzhou used silver taels; Donghai used seashells. The only way to move wealth between them was a suspension bridge guarded at both ends.

At the Yunzhou end, a guard named Wei collected silver. At the Donghai end, a guard named Lin issued seashells. The bridge was the sole channel for asset flow between the two cities.

How the Bridge Worked

Suppose a Yunzhou merchant wanted to move one hundred taels to Donghai:
1. The merchant gave one hundred taels to Guard Wei.
2. Wei locked the silver in Yunzhou's vault and recorded: "Zhang San deposited one hundred taels."
3. Wei sent a pigeon to Guard Lin: "Zhang San deposited one

hundred taels in Yunzhou. Please issue equivalent seashells in Donghai."
4. Lin received the pigeon, took one hundred seashells from Donghai's vault, and gave them to Zhang San's agent in Donghai.
5. Zhang San now had one hundred seashells in Donghai, and his one hundred taels were locked in Yunzhou's vault.

When Zhang San wanted his silver back, the process reversed.

The Bridge's Fatal Weakness

The bridge had a fatal weakness: the guards.

If Guard Wei was bribed, he could forge a release order and steal vault funds. If the pigeon was intercepted, messages could be lost or altered. If Wei and Lin conspired, they could empty both vaults.

History proved this vulnerability repeatedly. The most infamous incidents:
- A forger duplicated Guard Wei's seal and drained half the vault.
- An external power bribed both guards simultaneously, forged massive transfers, and collapsed the exchange rate.

Multi-Guard Bridges

Later bridges employed not one guard but a council. No single transfer could proceed without signatures from two-thirds of the council.

But this introduced new problems: efficiency dropped, council infiltration became a threat, and internal disputes could deadlock the bridge.

Light Bridges

The newest design—the "Light Bridge"—eliminated guards entirely. It relied on mathematical proof.

Yunzhou's ledger was public. Anyone could download it and verify that "Zhang San indeed deposited one hundred taels." A verifier generated a mathematical proof—a piece of code that Donghai could independently validate. Donghai needed no trusted guards; it simply ran the code, confirmed the proof was valid, and automatically released seashells.

Light bridges required no trusted intermediaries. The drawback: generating and verifying proofs demanded enormous computational resources, making them expensive today.

What This Fable Is About

This fable illustrates **cross-chain bridges**—protocols that connect different blockchains to enable asset and information transfer. Bridges are critical infrastructure but also the most attacked components in crypto history.

Key Mapping

Story Element	Technical Concept	Explanation
Yunzhou and Donghai	Different blockchains (e.g., Ethereum and Solana)	Different consensus mechanisms, VMs, and asset standards

|
Silver / seashells	Native vs. wrapped assets	Bridges mint wrapped tokens on the target chain pegged 1:1 to locked native assets
Guard Wei	Source chain bridge contract	Locks native assets and monitors transfer events
Guard Lin	Target chain bridge contract	Receives proof and mints/releases wrapped assets
Pigeon	Cross-chain message relay	Transmits proofs or events from source to target chain
Vault lock	Lock	Native assets are locked in the bridge contract, taken out of circulation
Forged seal	Private key compromise	Bridge validator keys stolen, allowing forged withdrawal instructions
Guard council	Multi-sig / validator set	Requires multiple validators to reach consensus (e.g., Wormhole, Polygon PoS Bridge)
Light Bridge	Light client bridge / ZK Bridge	Uses cryptographic proofs to verify source chain state without trusted intermediaries
Mathematical proof	State proof / ZK Proof	Proves a state transition occurred on the source chain according to consensus rules

Why This Story?

Cross-chain bridges have suffered the largest hacks in crypto history—Ronin ($625M), Wormhole ($320M), Nomad ($190M). Understanding the trust assumptions and tradeoffs of bridge design is essential for every Web3 participant.

Further Reading

- wiki-web3: Cross-Chain Bridge Principles
- Ethereum.org: Bridges
- Blockchain Bridges: Building Networks of Cryptonetworks
- Ronin Bridge Hack Analysis

"The Non-Fungible Token"

The Non-Fungible Token

> Two identical gold coins can be exchanged, but two paintings never can.

The Story

Yunzhou City's mint produced two types of currency.

The first was **standard copper coins**. Each copper coin was identical—the same weight, same pattern, same value. You exchanged one copper coin for another copper coin; there was no difference. This was called "fungibility."

The second was **unique tokens**. Each token was different—bearing different patterns, representing different rights, belonging to different owners. You could not exchange one token for another, because they represented completely different things.

Birth of Tokens

Tokens were originally created to solve a specific problem: how to

prove "ownership" in the digital world.

In the physical world, if you owned a painting, you hung it on the wall and everyone knew it was yours. But in the digital world, an image could be infinitely copied—copies and the original file were bitwise identical. Who owned the "original"?

Tokens solved this problem. They did not say "this file is yours" (because files could be copied), but said "this token represents a claim of ownership over something." The token itself could not be copied—like a property deed in the physical world that could not be copied (you could photocopy it, but the copy had no legal effect).

Three Core Properties of Tokens

First: uniqueness.

Each token had a globally unique number. Two tokens might have similar patterns, but numbers were never the same. This number was written on the public ledger; anyone could look it up.

Second: public ownership.

Who currently owned the token was written on the public ledger. No need to ask anyone, no need for any proof; the ledger itself was proof. If you wanted to transfer the token to someone else, you only needed to update the record on the ledger—from your name to theirs.

Third: traceable history.

From minting to the present, every transfer of the token was

recorded on the ledger. You could trace a token's complete history—who minted it, who bought it, who sold it, and at what price each time. This was called "provenance."

Uses of Tokens

Digital art:
Artists bound works to tokens. Buyers bought not the image file (files could be copied) but the token—representing "I am the official owner of this work." Although anyone could save image copies, only the token holder owned "recognized ownership."

Game items:
A legendary sword in a game was represented by a token. This sword could be transferred between different games—because the token did not depend on any single game's server. If the game company shut down, players still owned the token and could use it in other games supporting the token.

Membership:
A token represented "lifetime membership to a certain club." The club did not need to maintain a membership database—the ledger itself was the database. When the token was transferred, membership automatically transferred.

Identity credentials:
A token represented "you attended a certain event" or "you hold a certain degree." The credential issuer signed the token; anyone could verify the signature's authenticity.

Limitations of Tokens

Tokens are not omnipotent.

- Tokens represent ownership but do not automatically grant copyright. You bought an NFT, but that does not mean you obtained commercial usage rights for the work—unless the issuer explicitly granted them.
- Token value depends entirely on community consensus. If the community no longer recognizes a token's value, it becomes a meaningless string of numbers.
- Tokens themselves store numbers and ownership records; the "thing itself" (e.g., image files) is usually stored elsewhere (e.g., IPFS or centralized servers). If that place shuts down, the token you own may point to a broken link.

Years later, tokens became part of Yunzhou City's digital life. A collector said: "Tokens did not change the essence of 'ownership'—ownership has always been a social consensus. Tokens only wrote that consensus on an immutable ledger, making lies harder."

What This Fable Is About

This fable illustrates **NFT (Non-Fungible Token)**—a token standard on blockchain (e.g., ERC-721, ERC-1155) representing unique assets. NFT's core value lies in providing digital scarcity and verifiable ownership history.

Key Mapping

Story Element	Technical Concept	Explanation
Standard copper coins	Fungible Token (ERC-20)	Interchangeable tokens; each unit is identical
Unique tokens	NFT (ERC-721 / ERC-1155)	Each token has a unique ID; not interchangeable
Digital images infinitely copyable	Digital content easy to copy	Files can be perfectly copied; traditional methods cannot distinguish "original" from "copy"
Tokens cannot be copied	Blockchain uniqueness guarantee	Each NFT's ID is unique within its contract; cannot be forged
Property deed photocopy	NFT ownership meaning	Owning NFT does not equal owning copyright unless contract explicitly authorizes
Public ledger	Blockchain / distributed ledger	Ownership records are public and transparent; immutable
Ledger update	Transfer	Changing ownerOf by calling contract's transfer function
Traceable history	Provenance	Blockchain records NFT's complete transaction history from minting to present
Game items cross-games	Interoperability / metaverse assets	NFTs are not bound to single platforms; can be used in different applications supporting the standard
Membership	Token-gated access	Holding specific NFT grants access to content, communities, or functions
Images stored elsewhere	Off-chain metadata	NFT metadata (images, descriptions, etc.) is usually stored on IPFS or centralized servers; only URI is stored on-chain
Link broken	Link rot / metadata failure	If off-chain storage fails, NFT may point to empty links

Why This Story?

NFT is one of Web3's most mainstream concepts, but many people misunderstand its essence—the "owning a JPG" meme is typical. Understanding NFT's true value (verifiable ownership, provenance, interoperability) is crucial for transcending hype.

The "unique token" metaphor for NFT offers several intuitive advantages:

1. **Fungible vs. non-fungible comparison**: Copper coins vs. tokens = ERC-20 vs. ERC-721
2. **Essence of ownership**: Ownership is social consensus = NFT's value depends on community recognition
3. **Value of provenance**: History traceable = blockchain provenance's unique value
4. **Honest limitations**: Does not represent copyright; may point to failed links = NFT's true boundaries

Further Reading

- wiki-web3: NFT Non-Fungible Token Introduction
- ERC-721 Standard
- ERC-1155 Multi-Token Standard
- OpenSea: What is an NFT?
- The Non-Fungible Token Bible

"The One-Day Loan"

The One-Day Loan

> Borrow ten thousand taels, use for one day, zero interest, but must be repaid before sunset.

The Story

Yunzhou City's money house launched an unprecedented service: the one-day loan.

The rules were extremely simple:
1. You could borrow any amount from the money house—one hundred taels, ten thousand taels, even one million taels.
2. No collateral required.
3. No credit check.
4. Interest was extremely low—almost negligible.
5. But there was one absolute condition: **you must repay principal plus interest before sunset that day. If you cannot repay, the loan automatically cancels, as if nothing ever happened.**

Uses of One-Day Loans

Arbitrage:
A merchant discovered East Market sold rice for one tael per pound, West Market for one tael and five cents. But he had no capital.

He borrowed a one-day loan: ten thousand taels.
Used ten thousand taels to buy rice at East Market, immediately sold at West Market for ten thousand five hundred taels.
Repaid ten thousand taels principal and ten taels interest; net profit four hundred ninety taels.

The entire process took only two hours.

Liquidation:
The money house's rule was: when collateral value fell below threshold, anyone could execute liquidation, acquiring collateral at a discount.

Liquidators had no ready funds. They borrowed a one-day loan: ten thousand taels.
Used ten thousand taels to repay a certain borrower on the verge of liquidation, acquiring collateral worth eleven thousand taels at a discount.
Immediately sold the collateral on the market for ten thousand eight hundred taels.
Repaid ten thousand taels principal and ten taels interest; net profit seven hundred ninety taels.

Refinancing:
Someone's loan at the original money house had a very high interest rate. Another money house could offer a lower rate, but the original loan had to be repaid first to release collateral.

They borrowed a one-day loan to repay the original loan, release collateral, then use the collateral to obtain a low-rate loan at the new money house, repaying the one-day loan.

Dark Side of One-Day Loans

One-day loans were not only arbitrage tools but also attack weapons.

Price manipulation:
An attacker borrowed a one-day loan: one million taels. Used this money to massively buy something in a small market, driving the price up tenfold. Then in another derivatives market, he shorted that same thing. When the one-day loan was repaid and his buying disappeared, prices crashed, and his short position profited.

Governance attack:
An attacker borrowed a one-day loan, temporarily obtaining massive funds, using this money to purchase an organization's voting power, passing a malicious proposal to transfer the organization's treasury to himself, then repaying the loan. The entire process was completed within one day.

Protocol vulnerability exploitation:
An attacker discovered a logical vulnerability in a protocol, but exploiting it required large amounts of funds. He borrowed a one-day loan to trigger the vulnerability, stole funds, then repaid the loan.

Money House Risks

You might ask: why is the money house willing to provide this service? What if the borrower cannot repay?

The answer: the money house is not afraid of borrowers failing to

repay.

Because one-day loans have an "atomicity" guarantee: borrowing, using, and repayment must be completed in a single indivisible operation. If any step fails—such as the borrower not earning enough to repay—the entire operation automatically rolls back.

This means: the money house either recovers principal plus interest before sunset, or treats the loan as never having happened. The money house never loses money.

The real risk bearers are the rest of the protocol—the markets triggered by one-day loans, the contracts attacked. They have no automatic rollback protection.

Years later, one-day loans became a double-edged sword in Yunzhou City's financial system. An old money house manager said: "One-day loans themselves are neutral—they are just a tool. Used well, they make markets more efficient. Used badly, they make attackers stronger. The real problem is not one-day loans, but those unprepared protocols."

What This Fable Is About

This fable illustrates **Flash Loans**—uncollateralized, uncredit-checked loans in DeFi where borrowing and repayment must occur within the same atomic transaction. If final repayment fails, the entire transaction rolls back, and the lender bears no risk.

Key Mapping

Story Element	Technical Concept	Explanation
One-day loan	Flash Loan	Loan borrowed and repaid within a single atomic transaction
Any amount	No borrowing limit	Can borrow all pool liquidity (limited by pool size)
No collateral	Uncollateralized	No assets need to be locked as collateral
Extremely low interest	0.09% fee (Aave)	Flash loan fees are typically extremely low to encourage use
Repay before sunset	Repay within same transaction	Borrowing and repayment must be completed within a single atomic operation
Cannot repay means cancel	Atomicity / transaction rollback	If final repayment fails, entire transaction reverts; state rolls back to pre-borrowing
Money house does not lose	Protocol zero risk	Flash loan providers (e.g., Aave) either receive repayment + fee or transaction rolls back
Arbitrage	Arbitrage	Exploiting price differences between markets; most common flash loan use
Liquidation	Liquidation	Using flash loan funds to execute liquidation, earning liquidation rewards
Refinancing	Collateral swap / refinancing	Using flash loans to switch collateral or reduce borrowing rates
Price manipulation	Price oracle manipulation	Using flash loan funds to temporarily manipulate prices in one market, profiting in another
Governance attack	Flash loan governance attack	Borrowing funds to temporarily acquire voting power, passing malicious

proposals |
| Protocol vulnerability exploitation | Flash loan attack | Using flash loans to amplify attack scale, e.g., Cream Finance, Elephant Money, etc. |
| Atomicity guarantee | Transaction atomicity | EVM guarantees operations within a single transaction either all succeed or all roll back |

Why This Story?

Flash loans are one of DeFi's most unique financial innovations—a mechanism impossible in traditional finance. They are simultaneously the most powerful arbitrage tools and the most common attack vectors.

The "one-day loan" metaphor for flash loans offers several intuitive advantages:
1. **Zero-risk lending**: Not repaid by sunset means never happened = atomic rollback
2. **Miracle of no collateral**: No credit needed = complete reliance on automated code execution
3. **Double-edged sword**: Arbitrage tool = attack weapon = flash loan's neutrality and necessity of protocol defense
4. **Protocol risk**: Attacked contracts have no rollback protection = essence of flash loan attacks

Further Reading

- wiki-web3: Flash Loans Explained
- Aave Flash Loans

- Flash Loan Attacks: Analysis
- The Biggest Flash Loan Attacks in DeFi History
- How to Protect Against Flash Loan Attacks

"The Infinite Borrower"

The Infinite Borrower

> A man enters a bank to withdraw his deposit. Before the teller records the withdrawal, the man asks to borrow against the same deposit again. The cycle repeats until the vault is empty.

The Story

In the city of Yunzhou, there was a wealthy merchant named Lin who ran the largest pawnshop in the city. Business was straightforward: customers deposit collateral and receive loans. When the loan is repaid, the collateral is returned.

One day, a mysterious stranger arrived at the pawnshop. He placed a jade pendant on the counter and said, "I wish to borrow one hundred taels against this pendant."

The clerk examined the jade, recorded the loan in the ledger, and handed over one hundred taels. But before the clerk could mark the transaction as complete, the stranger said, "I have changed my mind. I wish to borrow another one hundred taels."

The clerk, following protocol, checked the ledger. The jade pendant was still recorded as collateral—because the first withdrawal had

not yet been finalized. So the clerk approved the second loan.

But before the second transaction could be recorded, the stranger asked for a third. And a fourth. And a fifth.

Each time, the clerk checked the ledger and saw the jade still listed as collateral. Each time, another one hundred taels were handed over. By the time the clerk finally updated the ledger, the stranger had received one thousand taels—for a pendant worth only one hundred.

The stranger departed, never to be seen again. The pawnshop discovered the fraud too late. The ledger showed the jade as collateral for a single one-hundred-tael loan, but the vault was missing one thousand taels.

What This Fable Is About

This fable illustrates the **reentrancy attack**—one of the most famous and costly vulnerabilities in smart contract history. The attack exploits the fact that a contract's state is updated after external calls are made, allowing an attacker to recursively re-enter the function before the state change is recorded.

Key Mapping

| Story Element | Technical Concept | Explanation |

|---|---|---|
Pawnshop	Smart Contract	The vulnerable contract holding funds
Ledger	Contract State / Storage	The internal accounting of balances
Jade Pendant	Collateral / Deposit	The asset deposited by the user
Loan Withdrawal	Withdrawal Function	The function that sends funds to the user
Clerk checks ledger before recording	State update after external call	The contract checks balance before deducting it
Stranger asks for multiple loans before completion	Reentrant recursive calls	The attacker's contract calls back into the vulnerable contract before the first call completes
One thousand taels stolen	Drained contract balance	All funds are extracted through recursive calls
Ledger still shows original amount	State not yet updated	The balance deduction happens after the external call

Why This Story?

The reentrancy attack is abstract until you see it as a timing problem. The "check-then-act" pattern—where a contract verifies conditions, performs an external operation, and only then updates state—is exactly where reentrancy strikes.

The pawnshop clerk represents the contract's execution flow. The stranger's repeated requests before the ledger is updated represent the attacker's recursive calls. Making this concrete helps developers understand why the "checks-effects-interactions" pattern (update state before external calls) is non-negotiable.

Further Reading

- wiki-web3: Solidity Reentrancy Attack and Defense
- ConsenSys: Reentrancy
- The DAO Hack Explained
- OpenZeppelin: ReentrancyGuard

"The Seven Gates"

The Seven Gates

> A fortress with seven gates. Each gate guards against a different kind of intruder. Leaving any one unguarded is leaving the fortress unguarded.

The Story

In the ancient kingdom of Yunzhou, the imperial treasury was protected by seven gates. Each gate was designed by a different master engineer, and each guarded against a specific type of threat.

The First Gate: Overflow
This gate used a counting mechanism that could only count up to a certain number. When the count exceeded the limit, it wrapped around to zero. An attacker discovered that by deliberately triggering the overflow, he could reset the treasury's balance to zero—making it appear as though nothing had ever been deposited.

The Second Gate: Access Control
This gate was supposed to check whether the person entering had the proper credentials. But due to a design oversight, the credential checker could be bypassed by simply not presenting any credentials at all. Anyone could walk through.

The Third Gate: Randomness

This gate relied on "random" patterns to determine who could pass. But the randomness was generated from predictable sources—like the current time of day. An attacker who monitored the patterns could predict exactly when the gate would open.

The Fourth Gate: Delegate

This gate allowed trusted messengers to pass through on behalf of others. But the gate did not verify whether the messenger was truly authorized. A clever intruder disguised himself as a messenger and walked right in.

The Fifth Gate: Denial of Service

This gate could be jammed from the outside. By flooding the entrance with thousands of fake requests, an attacker could prevent legitimate users from ever reaching the treasury.

The Sixth Gate: Front-Running

This gate processed requests in the order they were received. But a fast courier could see requests in transit, place his own request ahead of them, and profit from the advance knowledge.

The Seventh Gate: Logic

This was the most subtle gate. Its lock had a logical flaw: under a very specific combination of conditions, the "locked" and "unlocked" states could be simultaneously true. An attacker who studied the gate long enough found that combination and simply walked through.

The treasury fell not because any single gate was weak, but because the defenders assumed that seven gates meant seven layers of safety. They did not realize that a determined attacker needs only find one open gate.

What This Fable Is About

This fable provides an overview of the **most common categories of vulnerabilities in Solidity smart contracts**: integer overflow/underflow, access control failures, weak randomness, delegate call vulnerabilities, denial of service, front-running, and logical errors.

Key Mapping

Story Element	Technical Concept	Explanation
First Gate: Overflow	Integer Overflow/Underflow	Arithmetic operations that exceed type limits wrap around. In Solidity <0.8.0, this can be exploited to manipulate balances
Second Gate: Access Control	Missing Access Control	Functions that should be restricted (like `selfdestruct` or privileged operations) lack proper `require(msg.sender == owner)` checks
Third Gate: Randomness	Weak Randomness Sources	Using `block.timestamp`, `blockhash`, or `keccak256` on predictable values as randomness sources can be manipulated by miners or observers
Fourth Gate: Delegate	Delegate Call Vulnerabilities	Improper use of `delegatecall` can allow attackers to execute arbitrary code in the context of the vulnerable contract
Fifth Gate: Denial of Service	DoS via Gas Limits / Reverts	External calls that revert can block contract functionality. Unbounded loops can hit block gas limits

Sixth Gate: Front-Running	Transaction Ordering Attacks	Mempool visibility allows attackers to see pending transactions and insert their own with higher gas prices to execute first
Seventh Gate: Logic	Business Logic Flaws	Complex state machines with edge cases where contradictory states coexist, allowing unintended execution paths
Seven gates	Defense in Depth	Security requires guarding against all attack vectors, not just the obvious ones

Why This Story?

Smart contract security is not about finding "the one bug"—it is about systematically checking every possible attack surface. The Seven Gates metaphor makes this concrete: a treasury with six perfect gates and one broken gate is still a breached treasury.

Further Reading

- wiki-web3: Common Solidity Security Vulnerabilities Overview
- SWC Registry
- ConsenSys Smart Contract Best Practices
- Solodit Checklist

"The Counterfeit Scale"

The Counterfeit Scale

> A merchant's scale was designed to only measure up to one thousand pounds. When someone placed two thousand pounds on it, the scale showed zero—and the merchant gave away his entire warehouse for free.

The Story

In the marketplace of Yunzhou, a merchant named Wei operated a grain exchange. His warehouse stored thousands of tons of rice, and he used a sophisticated scale to track inventory.

The scale was designed with a curious limitation: its display could only show numbers from 0 to 999. When the total weight exceeded 999, the display would "roll over" back to 0. Wei knew this, but he never expected anyone to deposit more than 999 pounds at once.

One day, a trader arrived with a caravan of rice—exactly 1,000 pounds. The scale registered 1,000, but because the display could only show three digits, it displayed 0. The warehouse guard, seeing the scale read 0, assumed the caravan was empty. He waved the trader through without recording any deposit.

But the trader had not come to deposit. He had come to exploit the scale.

The trader presented a transfer order: "Transfer 1,000 pounds of rice to my account." The clerk processed the order. The scale showed that the warehouse contained 0 pounds (because of the rollover), and after subtracting 1,000 pounds, it showed... 999 pounds.

Wait, that did not make sense. The clerk recalculated. The scale's internal mechanism, confused by the overflow, produced impossible numbers. Eventually, the trader convinced the clerk that the warehouse owed him 1,000 pounds.

The trader received 1,000 pounds of rice. But his account on the scale now showed a negative balance—which the scale displayed as a very large positive number due to another overflow. The trader then "transferred" this massive positive balance to multiple accounts, each receiving thousands of pounds.

By sunset, the trader had emptied half the warehouse using nothing but a quirk of the scale's number display.

What This Fable Is About

This fable illustrates **integer overflow and underflow** vulnerabilities, exemplified by the infamous BEC Token bug that made $2.8 billion worth of tokens theoretically transferable. The vulnerability existed in a `batchTransfer` function that did not check for overflow when summing token amounts.

Key Mapping

Story Element	Technical Concept	Explanation
Scale display 0-999	uint256 / fixed-size integer	Solidity integers have fixed bit widths. Operations that exceed the maximum value wrap around to 0
1,000 pounds rolling over to 0	Integer Overflow	`uint256` max is `2^256 - 1`. Adding beyond this wraps to 0. In BEC's case, `amount * receivers` overflowed
Negative balance shown as large positive	Integer Underflow	Subtracting below 0 wraps to the maximum value. `0 - 1` in `uint256` becomes `2^256 - 1`
Transfer order processed without validation	Missing overflow check	The BEC contract did not use `SafeMath` or Solidity 0.8.0's built-in overflow protection
Massive positive balance	Huge token minting	Overflow allowed the attacker to create arbitrarily large token balances out of thin air
Solidity 0.8.0+ built-in protection	Automatic overflow reverts	Modern Solidity reverts on overflow/underflow by default, making this class of bugs largely obsolete

Why This Story?

Integer overflow is one of the most "obvious in hindsight" vulnerabilities. The scale metaphor makes the wrap-around behavior viscerally clear: a scale that rolls over from 999 to 0 is exactly what happens when a `uint256` rolls over from `2^256 - 1` to 0.

The BEC token incident is particularly instructive because the vulnerable code was just a few lines in a `batchTransfer` function—a reminder that even "simple" functions can harbor catastrophic bugs if arithmetic is not properly guarded.

Further Reading

- wiki-web3: The One-Line Bug That Evaporated $28 Billion
- BEC Token Overflow Analysis
- SafeMath Documentation
- Solidity 0.8.0 Release Notes: Checked Arithmetic

"The Messenger and the Auction House"

The Messenger and the Auction House

> In a city where all messages pass through a public square, the fastest reader always wins.

The Story

In Yunzhou City, all commercial messages were posted on a public bulletin board before delivery. Messengers would read the board, pick up letters, and deliver them in order of postage fee—whoever paid the most got their letter delivered first.

This system was transparent, but it created a shadow economy of "message brokers" who operated in the alleyways behind the bulletin board.

The Sandwich Thief

A sandwich thief did not write letters. He only read other people's messages.

When a merchant named Xu wrote a letter to the marketplace saying, "I wish to buy ten bolts of silk for one hundred taels," the letter was posted on the bulletin board. The sandwich thief read it.

He immediately did three things:
1. Sent a first letter: "Buy all cheap silk at the marketplace." (Paid express postage to arrive before Xu's letter.)
2. Waited for Xu's letter to be delivered, forcing Xu to buy at the now-inflated price.
3. Sent a second letter: "Sell all the silk I just bought." (Paid express postage to arrive immediately after Xu's letter.)

The result: the thief bought low, sold high, and pocketed the difference. Xu had done nothing wrong—he simply bought silk at the market price—but the "market price" had been artificially inflated by the thief.

This is called a "sandwich attack"—the attacker wraps the victim's transaction between two of their own, like bread around meat.

The Arbitrage Hawk

The arbitrage hawk watched two marketplaces simultaneously: the East Market and the West Market.

East Market sold silk for one hundred taels per bolt; West Market sold it for one hundred and five. Normally, such price differences would be naturally arbitraged away by merchants buying low and selling high.

But the hawk was faster than ordinary merchants. He did not need his own silk inventory. He simply:

1. Borrowed money from the "Lightning Money House" (same-transaction loan, must be repaid within the hour).
2. Bought silk at the East Market with borrowed funds.
3. Immediately sold it at the West Market.
4. Repaid the loan and its fee.
5. Kept the profit.

All within five minutes. By the time ordinary merchants finished their calculations, the hawk had already pocketed the money.

The Scavenger

The scavenger did not seek profit. He sought certainty.

The money house's rule was: when collateral value fell below a threshold, anyone could execute liquidation, acquiring the collateral at a discount. But liquidation required precise timing—too early, and the threshold had not been reached; too late, and someone else might execute first.

The scavenger's job was to monitor all collateral in all money houses. The instant a threshold was triggered, he sent a liquidation letter with the highest postage fee, ensuring his letter was delivered first.

The scavenger's income was modest—only a few percentage points of discount per liquidation—but he was risk-free. As long as he was fast enough, he never lost.

What This Fable Is About

This fable illustrates **front-running attacks** in blockchain systems—where attackers exploit the public visibility of pending transactions (the mempool) to insert their own transactions ahead of victims', profiting from price slippage, arbitrage, or liquidation rewards.

Key Mapping

Story Element	Technical Concept	Explanation
Public bulletin board	Mempool	Pending transactions are publicly visible before being included in a block
Postage fee determines delivery order	Gas Price / Priority Fee	Miners/validators prioritize transactions with higher gas prices
Sandwich thief	Sandwich Attacker	Inserts buy order before victim's swap and sell order after, profiting from price slippage
Arbitrage hawk	Arbitrage Searcher	Exploits price differences between DEXes, typically using flash loans
Lightning Money House	Flash Loan	Same-transaction, zero-collateral loan that must be repaid atomically
Scavenger	Liquidation Bot	Monitors collateral prices and submits liquidation transactions immediately when thresholds are triggered
Alleyway chaos	MEV negative externalities	MEV extraction leads to worse prices for ordinary users, higher slippage, and network congestion
Private delivery station (Feiluo)	Flashbots / MEV-Boost	Private mempool and sealed-bundled auction system to mitigate open gas-

price wars |

Why This Story?

MEV is one of the darkest and most complex topics in blockchain. It is not a single technical problem but an incentive-structure problem: when transaction order can be influenced, influencing order itself becomes a profit source.

The "messengers in the alley" metaphor makes the MEV ecosystem tangible: sandwich thieves, arbitrage hawks, and scavengers are the three typical MEV strategies. The story also shows how Flashbots' private mempool—represented by Feiluo's private delivery station—changes the information flow to reduce harmful MEV.

Further Reading

- wiki-web3: Mempool Monitoring and Front-running
- Flashbots: MEV Overview
- MEV-Boost: PBS for Ethereum
- Ethereum.org: MEV

"The Lighthouse in the Swamp"

The Lighthouse in the Swamp

> A beacon that promises safe passage but lures ships into the mud.

The Story

In the marshlands outside Yunzhou City, travelers spoke of a mysterious lighthouse. It appeared only at night, casting a warm golden light across the treacherous swamp. Those who followed the light claimed it led to a hidden treasure.

The lighthouse was real. The treasure was not.

The structure was built by a clever trickster named Hu. He constructed the lighthouse from reflective mirrors and oil lamps—nothing magical, just carefully angled optics that made the light visible from miles away. Beneath the lighthouse, he dug a deep pit lined with mud and stakes.

Travelers who approached the lighthouse found a path clearly marked by lanterns. The path led directly to the pit. The first few steps were solid ground, luring the traveler forward. Then the

ground softened. By the time the traveler realized they were sinking, the mud had already reached their knees.

The more they struggled, the deeper they sank.

Some travelers escaped by abandoning their packs and crawling back along the solid ground. But most clung to their belongings—and were swallowed whole.

The trickster watched from the treeline, collecting the abandoned packs of those who escaped and the bodies of those who did not.

What This Fable Is About

This fable illustrates a **honeypot contract**—a malicious smart contract designed to lure users into depositing funds with the promise of profit, while containing hidden code that prevents withdrawal or steals deposited assets.

Key Mapping

Story Element	Technical Concept	Explanation
Lighthouse	Honeypot Contract	A contract that appears to offer arbitrage or profit opportunity
Golden light	Seemingly profitable function	The contract's code appears to allow extracting value

Marked path	Publicly visible vulnerability	The "bug" is intentionally visible to attract attackers
Solid ground first	Partial functionality	Some functions work as expected to build trust
Sinking mud	Hidden withdrawal lock	The code contains hidden conditions that prevent withdrawal
Struggling sinks deeper	Attempting withdrawal triggers more losses	Some honeypots drain additional funds when withdrawal is attempted
Abandoning packs to escape	Cutting losses	Recognizing the trap and abandoning deposited funds
Trickster collecting packs	Attacker harvesting funds	The contract creator collects all trapped deposits

Why This Story?

Honeypots exploit the greed and overconfidence of would-be attackers. The lighthouse metaphor captures the seductive nature of these traps: they offer something too good to be true, and the path to ruin is paved with initial signs of legitimacy.

Further Reading

- wiki-web3: Analysis of a Honeypot Contract
- HoneyPot Detector Tools
- Solidity Honeypot Techniques

"The Weather Vane Deception"

The Weather Vane Deception

> He who controls the weather vane controls the wind—or at least, what everyone believes about the wind.

The Story

In Yunzhou City, the farmers relied on a great weather vane to determine when to plant and harvest. The vane was connected to a complex system of gears and pulleys that measured wind direction, temperature, and humidity. Based on these readings, the city council issued planting recommendations.

The system worked well for years. Then a drought came.

A wealthy landowner named Qian realized something: the weather vane did not actually measure the weather. It measured what the majority of local weather stations reported. And Qian owned three of the five stations.

Qian's plan was simple:
1. Manipulate his three stations to report extreme conditions.
2. The weather vane, taking the majority reading, would declare a false storm approaching.

3. The city council would issue an emergency harvest order.
4. Farmers would rush to sell their crops at depressed prices.
5. Qian would buy low, then restore the true readings.
6. Prices would recover. Qian would sell high.

The execution was flawless. The weather vane showed a storm. The council panicked. The market crashed. Qian bought an entire season's harvest for a fraction of its value.

Then he "fixed" his weather stations. The vane returned to normal. Prices soared. Qian sold everything and tripled his wealth.

No one could prove manipulation. The weather vane had done exactly what it was designed to do: report the majority. The flaw was not in the vane but in the assumption that the majority could not be bought.

What This Fable Is About

This fable illustrates **oracle manipulation**—attacks on price oracle systems where an attacker controls or manipulates the data sources that a smart contract relies on for price information, enabling them to profit from the resulting price discrepancies.

Key Mapping

| Story Element | Technical Concept | Explanation |

|---|---|---|
Weather vane	Price Oracle	A system that provides external price data to smart contracts
Five weather stations	Multiple data sources	Decentralized oracles use multiple sources to determine consensus
Qian owns three stations	Majority control	If an attacker controls >50% of data sources, they can manipulate the consensus
False storm report	Manipulated price feed	The oracle reports an artificial price based on corrupted sources
Emergency harvest	Liquidation / forced selling	DeFi protocols may trigger liquidations or other automated responses based on oracle prices
Market crash	Price collapse	Users sell at manipulated low prices
Buy low, sell high	Oracle manipulation arbitrage	Attacker profits from the artificial price movement
Assumption of honest majority	Oracle trust model	The security of many oracle systems depends on the assumption that most sources are honest

Why This Story?

Oracle manipulation is one of the most devastating and high-value attack vectors in DeFi. The weather vane metaphor makes the attack visceral: the vane itself is not broken; the trust model underlying it is.

Further Reading

- wiki-web3: Oracle Manipulation Checklist

- Chainlink: Oracle Security
- DeFi Oracle Exploits: A Timeline

"The Assembly Line Workshop"

The Assembly Line Workshop

> Not one machine working, but every process in the entire workshop running simultaneously.

The Story

Yunzhou City's old workshop was a traditional blacksmith shop: one smith, one forge, one blade at a time. The process was serial—heat, forge, quench, grind; one step finished before the next began.

This workshop's advantage was simplicity and reliability. The disadvantage was slowness. If one thousand blades needed to be made in a day, one thousand smiths and one thousand forges were required.

Assembly Line Workshop Revolution

By the Eastern Sea, a new type of workshop appeared: the assembly line workshop.

The assembly line workshop's core idea was: break the blade-making

process into many small steps, each handled by a specialized craftsman, with all steps proceeding simultaneously.

Specific design:

1. **Process parallelization**: heating, forging, quenching, and grinding were no longer done by one person but by four specialized craftsmen. While the first blade was being forged, the second was being heated, the third was being quenched, and the fourth was being ground.

2. **Time-stamp foreman**: At the workshop center was a "time-stamp foreman" who struck a gong every breath. All craftsmen synchronized their work according to the gong—hearing the first strike, the heater passed the blade to the forger; hearing the second, the forger passed it to the quencher. The gong ensured all workshop processes stayed in sync.

3. **No-waiting transfer**: In traditional workshops, if one craftsman had not finished, the next could only wait idly. In assembly line workshops, transfer was asynchronous—the heater placed the blade on a public conveyor belt; the forger took blades from the belt. If the belt was temporarily empty, the forger could handle other tasks instead of waiting foolishly.

4. **Multiple forges in parallel**: Traditional workshops had only one forge. Assembly line workshops had dozens of forges working simultaneously, each heating different blades. But all forges shared one chimney—the chimney's exhaust capacity determined the entire workshop's throughput ceiling.

Assembly Line Workshop Bottlenecks

The assembly line workshop soon encountered its own bottleneck: the chimney.

Dozens of forges exhausting simultaneously, the chimney quickly became insufficient. Workshop engineers faced a choice:
- Widen the chimney: let each forge exhaust smoothly, but chimney construction costs would increase exponentially.
- Limit forge numbers: keep the chimney unchanged, but limit simultaneously operating forge numbers.

The assembly line workshop chose a third path: **state rent**.

Each forge was not permanently occupied. If a forge was idle for a long time (e.g., the craftsman took leave), the workshop would "rent" it to other craftsmen. The original craftsman, upon returning, needed to reapply for a forge—if it was already rented out, he had to wait or pay higher rent to preempt it.

This mechanism ensured: chimney exhaust capacity was never wasted by idle forges.

Comparison Between Assembly Line and Traditional Workshops

A traveling craftsman compared the two:

"Yunzhou City's old workshop is like one person slowly walking—each step is steady, but you can't walk fast.

The Eastern Sea's assembly line workshop is like a group of people simultaneously running a relay—everyone is sprinting at full speed, passing batons with precision to maintain overall speed.

The old workshop's advantage is fault tolerance: if one smith gets sick and stops work for a day, the loss is only one day's work.

The assembly line workshop's advantage is throughput: if everything runs normally, its output is a thousand times the old workshop's. But if the time-stamp foreman's gong stops—even for one breath—the entire workshop's processes instantly fall into chaos. Because everyone depends on that unified gong to synchronize."

What This Fable Is About

This fable illustrates **Solana's architecture**—a blockchain with parallel processing and ultra-high throughput as core goals. Solana achieves tens of thousands of transactions per second through innovations like Proof of History (PoH), Sealevel parallel runtime, Turbine propagation protocol, and Gulf Stream mempool-less transaction forwarding.

Key Mapping

Story Element	Technical Concept	Explanation
Traditional blacksmith shop	Traditional EVM chain (Ethereum)	Serial execution; processes one transaction at a time; global state lock
Assembly line workshop	Solana	Parallel execution architecture; multiple transactions processed simultaneously

Process parallelization	Sealevel parallel runtime	Solana's VM can execute non-conflicting transactions in parallel (transactions not reading/writing the same accounts)
Time-stamp foreman	Proof of History (PoH)	A cryptographic clock generating verifiable timestamps through sequential hashing, providing unified time reference for the entire network
Gong synchronization	Slot / Leader Schedule	Validators rotate block production according to fixed schedule (every 400ms a slot); PoH ensures order
No-waiting transfer	Gulf Stream	Transactions are forwarded directly to expected block producers without passing through public mempool, reducing propagation delay
Public conveyor belt	Transaction queue	Validators maintain pending transaction queues, taking transactions on demand
Multiple forges in parallel	Parallel account access	Solana's account model allows identifying non-conflicting transactions for parallel execution
Shared chimney	Network bandwidth / TPS ceiling	Solana's throughput is limited by network bandwidth and validator node hardware capabilities
State rent	Rent mechanism	Solana accounts need to pay rent (or maintain minimum balance) to exist; prevents infinite state膨胀
Forge rented out	Account recycling	If account balance falls below rent exemption threshold and remains unused for two years, account data may be recycled
Chimney insufficient	Network congestion / transaction dropping	When transaction volume exceeds processing capacity, low-priority transactions are dropped; users must resubmit
Gong stops	Network outage	Solana has historically experienced multiple network outages due to consensus issues or bugs, because the entire network's synchronization depends on PoH

Why This Story?

Solana is the representative of "performance-first" blockchains, but its architecture (PoH, parallel execution, rent model) differs greatly from Ethereum, and many concepts are very foreign to EVM developers.

The "assembly line workshop" metaphor for Solana offers several intuitive advantages:

1. **Parallel execution visualization**: Multiple processes proceeding simultaneously = Sealevel parallel transaction processing
2. **PoH intuition**: Gong = PoH clock, providing synchronization beat for the entire network
3. **Necessity of rent**: Limiting forge numbers = rent mechanism preventing state膨胀
4. **Outage vulnerability**: Gong stopping causes entire workshop chaos = Solana's high dependence on consensus synchronization

Further Reading

- wiki-web3: Solana Virtual Machine SVM Architecture Analysis
- Solana Docs: Architecture
- Proof of History Explained
- Sealevel: Parallel Processing Thousands of Smart Contracts
- Solana Network Outages Analysis

"The Smith's Discipline"

The Smith's Discipline

> It is not that mistakes are not allowed, but the compiler closes the door before you can make one.

The Story

Yunzhou City had two blacksmith schools. One taught "free forging"; the other taught "disciplined forging."

Free Forging School

The free forging school's philosophy was: give apprentices maximum freedom. You could use any tool, any material, any method. The school only taught basic techniques; the rest was up to you.

Free forging's advantage was flexibility. If you wanted to forge a blade of bizarre shape, no rules stopped you.

But free forging's cost was: frequent accidents.

- An apprentice poured molten iron onto himself—because he forgot

to check the temperature.
- An apprentice's blade broke on first use—because he used cracked raw materials, but the school had no mandatory raw material inspection system.
- An apprentice caused a fire during quenching—because he opened both the oil trough and water trough simultaneously, and the school allowed this.

The most serious accident: an advanced apprentice built an automatic forging machine. The machine operated well but had a hidden flaw—under specific conditions, it would operate two hammers simultaneously, causing them to collide and the entire machine to explode. This flaw did not appear during testing because test conditions happened to avoid that "specific condition." But the client triggered it during use.

Free forging school graduates who survived often became more creative. But many became disabled—literally.

Disciplined Forging School

The disciplined forging school's philosophy was completely different. In their view, forging's dangers did not come from tools but from human carelessness. So the school designed an entire "discipline"—not punishing errors after the fact, but preventing them before they occurred.

Specific rules:

First: ownership system.

Every tool, every piece of raw material, at any time had only one "owner." If Jia took the hammer, Yi could not use it simultaneously. If

Yi wanted to use it, he had to wait for Jia to return it, or Jia explicitly "lent" the hammer to Yi.

This rule prevented "two people simultaneously operating the same hammer" accidents. It also prevented "I thought the hammer was in my hand, but someone else had already taken it" confusion.

Second: borrowing check.

When you borrowed a tool from someone, the school checked three things:
- Does this tool actually exist (not fictional)?
- Is this tool currently borrowed by someone else (or borrowing methods do not conflict)?
- Will your borrowing time exceed the agreed period?

If any condition was not met, the school would not allow you to start work. Not punishing you, but simply not opening the workshop door.

Third: type matching.

The school's forge only heated specific types of metal. If you tried to put wood into the iron forge, the forge door would not open—not because you did something wrong, but because the forge's design did not allow non-metal materials to enter.

This seemed like a restriction but was actually protection. It prevented "putting things that shouldn't be heated into the furnace" accidents.

Fourth: no null pointers.

In the free forging school, apprentices could point at an empty spot and say "the hammer is there"—even if there was nothing there. This led to disaster: the apprentice reached for air, lost balance, and fell

into the furnace.

The disciplined forging school eliminated "null pointers." If you wanted to point at a position, that position must have something. If something might not exist, you had to explicitly handle the "non-existence" situation—such as checking first or preparing a backup plan. You could not pretend it definitely existed.

Output of Both Schools

Free forging school graduates, if they survived, were often more creative. Because from childhood they were allowed to try all kinds of crazy ideas.

Disciplined forging school graduates almost never had serious accidents. Their work might not be stunning, but it was extremely reliable. An employer said: "I've hired graduates from both schools. Free forging school people occasionally surprise me, but also occasionally shock me. Disciplined forging school people never surprise me, but also never shock me. When building bridges, I choose people who don't shock me."

Cost of Disciplined Forging

Disciplined forging was not free.

Apprentices complained: "I clearly know I won't make mistakes, but the school still forces me to check every step. This wastes too much time."

Teachers answered: "You didn't make mistakes yesterday doesn't

mean you won't today. And when projects grow larger and more people participate, you cannot assume others are as mistake-free as you. Discipline is not designed for you alone; it is designed for the entire team."

The more practical cost was: some designs easily implemented in free forging school required many detours in disciplined forging school. The school required you to write more "boilerplate code" to prove "I have considered all possible error situations."

An old craftsman said: "Free forging lets you run faster; disciplined forging makes you fall less. When forging a kitchen knife, free forging may be better. When building a great bridge, disciplined forging is the only choice."

What This Fable Is About

This fable illustrates **Rust's safe programming model**—Solana smart contracts' primary development language. Rust eliminates memory safety vulnerabilities at compile time through ownership, borrowing checking, type systems, and null-pointer elimination, rather than relying on runtime testing.

Key Mapping

Story Element	Technical Concept	Explanation

Free forging school	C/C++ / traditional system programming	Gives developers maximum freedom; manual memory management; prone to errors
Disciplined forging school	Rust	Compile-time enforcement of safety rules; memory safety without garbage collection
Ownership system	Ownership System	Each value has only one owner at any time. When owner leaves scope, value is automatically released
Borrowing check	Borrow Checker	Compiler checks all references are valid: no dangling references, no data races, no simultaneous mutable and immutable borrows
Type matching	Type System / Pattern Matching	Rust's strong type system and exhaustive pattern matching ensure all cases are handled
No null pointers	`Option` / no null	Rust has no null. Values that may not exist must be explicitly handled with `Option`; compiler forces checks
Forge door won't open	Compilation error	Unsafe operations are rejected at compile time rather than crashing at runtime
Automatic forging machine explosion	Memory safety vulnerability (UAF, Double Free, Buffer Overflow)	Common vulnerabilities in C/C++; eliminated in Rust through ownership and borrowing checking at compile time
Boilerplate code	Rust's explicit error handling	`Result` and `Option` require explicit error handling; code volume may increase but safety greatly improves
Kitchen knife vs. great bridge	Application scenario choice	System-level software (blockchain, OS) needs Rust's safety. Rapid prototyping may use more flexible languages
Team discipline	Rust suitable for large-scale collaboration	Compiler as "gatekeeper," ensuring even developers unfamiliar with the codebase cannot introduce memory safety issues

Why This Story?

Solana chose Rust as its native development language, with core reason being safety. Once blockchain code is deployed, it cannot be modified; a memory safety vulnerability could cause millions of dollars in losses.

The "blacksmith school" metaphor for Rust vs. C/C++ offers several intuitive advantages:

1. **Ownership visualization**: One hammer, one person at a time = Rust's ownership rules
2. **Compile-time checking**: Forge door won't open = compile error blocking unsafe code
3. **No null pointers**: Cannot point at empty spot saying hammer is there = `Option` forcing explicit handling of missing values
4. **Cost of creativity**: Disciplined forging more tedious but more reliable = Rust's development efficiency vs. safety tradeoff

Further Reading

- wiki-web3: Rust Smart Contract Safe Programming Standards
- The Rust Programming Language Book
- Rust Ownership Explained
- Rust Borrow Checker
- Why Rust for Smart Contracts

"The Timed Safe"

The Timed Safe

> Safes are not free. The longer you store, the higher the rent. Stop paying rent, and your things are cleared out.

The Story

Yunzhou City's bank offered a special safe deposit service, completely different from traditional safes.

Traditional Safe vs. Timed Safe

In traditional banks, you rented a safe and paid a fixed annual rent. As long as you paid rent, the box was permanently yours. You could put anything inside—gold, jewelry, deeds, even air. The bank did not care.

Yunzhou City's new bank launched a "timed safe." Its rules were extremely special:

First: each safe had a fixed size.

You could rent a large safe (holding one hundred items) or a small safe (holding only ten items). But you could not rent an "infinite" safe. If you wanted to store more things, you had to rent more safes.

Second: safes had minimum deposit requirements.

To rent a large safe, you had to pre-deposit at least one thousand taels of silver inside. This money was not rent—it still belonged to you—but it had to remain in the safe. If you withdrew it, the safe automatically became invalid, and its contents were cleared out.

Third: safes were "ownerless."

Traditional safes belonged to the renter—only you could open them. Timed safes had no fixed owner. Anyone who knew the opening method could open them. Safe security did not come from "who owns them" but from "who can provide the correct opening password."

Fourth: safes could "authorize programs."

You could set: "This safe can only be opened by such-and-such program." Programs were public, immutable rules. For example: "Only when I provide the correct password does the program allow item withdrawal." Or: "The program automatically withdraws ten taels of silver from my safe every month, transferring to another safe."

Rent Mechanism

Timed safe rent was not collected annually. It was calculated by "occupied space" and "occupied time."

Specific rules:
- A safe required a certain amount of rent annually.
- The pre-deposited one thousand taels in the safe automatically generated interest.
- If interest was sufficient to pay rent, the safe could be maintained permanently.
- If interest was insufficient to pay rent, the safe's balance would gradually decrease. When balance reached zero, the safe was "recycled"—its contents discarded, the safe itself reassigned to others.

This meant: timed safes were not "buy and own forever." They were resources requiring continuous maintenance. If you did not maintain them, they disappeared.

Unexpected Applications of Timed Safes

Timed safe design spawned many innovative uses:

Automatic execution will:
Someone deposited money in a safe, setting the program: "If I do not open this safe for two years, automatically transfer the money to my son." If the owner died, the safe automatically executed the transfer after two years. No lawyer, no court, no will certification needed.

Subscription service:
A merchant provided monthly ten-tael consulting services. The client deposited one hundred twenty taels in the safe, setting the program: "Automatically transfer ten taels to the merchant every month. Stop service when balance is insufficient." The merchant did not need to collect debts; the client did not need to manually transfer monthly.

Composable financial instruments:

A complex financial product consisted of over a dozen safes, each responsible for one function—storing principal, calculating interest, distributing earnings, managing risk. These safes were interconnected through programs, forming an automatic operating system. Failure of any single safe would affect the entire system, but the system itself had no central controller.

Criticism of Timed Safes

But timed safes also had critics.

- "Minimum deposit requirements" kept the poor out. If you only had one hundred taels, you could not even afford the smallest safe.
- "Rent mechanism" meant your assets would shrink over time (if interest was insufficient). This violated the intuition of "saving preserves value."
- "Ownerless" design made many people uncomfortable. Traditional bank customers asked: "If my password leaks, can't anyone open my safe?" The bank answered: "Yes. So we recommend you use more complex passwords, or use programs to add extra security layers."

A traditional banker said: "Timed safes are not designed for ordinary people. They are designed for programs. In the world of timed safes, what truly owns assets is not your identity, but your password and your programs."

What This Fable Is About

This fable illustrates **Solana's account model and rent mechanism**. Unlike Ethereum's "contract is account," Solana stores all data in independent accounts, each with fixed space, requiring maintenance of a minimum balance (rent-exempt) to avoid being recycled, and accounts can designate "owner programs" to manage access permissions.

Key Mapping

Story Element	Technical Concept	Explanation
Traditional safe	Ethereum account model	Contracts store their own states; no explicit storage limits
Timed safe	Solana Account	All data in Solana is stored in independent accounts; each account has fixed size
Fixed-size safe	Account space limit	Solana accounts have fixed sizes at creation; cannot dynamically expand
Minimum deposit	Rent-exempt balance	Accounts must maintain minimum lamport balance to avoid recycling. This balance is proportional to account size
Pre-deposited interest	Balance does not generate interest but maintains exemption	Actually Solana has no interest, but rent-exempt minimum balance is a one-time deposit
Ownerless	Account has no private key control	Solana accounts are controlled by "owner program," not directly by private key
Opening password	Account signature / permission verification	Defined by owner program who can operate the account
Authorizing programs	Owner Program	Each account has an owner program (e.g., System Program, Token Program); only the owner can modify account data
Rent consumption	Rent mechanism	Accounts need to periodically pay rent (or be rent-exempt). If balance is insufficient,

account data is recycled |
Automatic execution will	Program Derived Address (PDA) + conditional execution	Solana programs can automatically execute operations based on conditions without external triggers
Subscription service	Automated transfers / program-controlled fund flows	SPL Token program supports automated token transfer logic
Composable financial instruments	Solana DeFi composability	Multiple programs call each other through accounts, forming complex financial protocols
Poor kept out	Rent threshold	Creating accounts requires minimum balance; unfriendly to small users

Why This Story?

Solana's account model is completely different from Ethereum's and is one of the most confusing aspects for EVM developers. Understanding "accounts need rent," "accounts have fixed sizes," and "programs own accounts" is key to entering Solana.

The "timed safe" metaphor for Solana accounts offers several intuitive advantages:
1. **Fixed size**: Safe size fixed = Solana account fixed space
2. **Minimum deposit**: Pre-deposited silver maintaining safe = rent-exempt minimum balance
3. **Ownerless design**: Password opening rather than identity = Solana's program-owned account model
4. **Rent consumption**: No maintenance leads to recycling = Solana rent mechanism preventing state膨胀

Further Reading

- wiki-web3: Solana Account Model and Rent Mechanism
- Solana Docs: Accounts
- Solana Rent Economics
- Solana Docs: Accounts

"The Deterministic Address"

The Deterministic Address

> The address is not randomly assigned, but determined by the house's owner and its purpose.

The Story

Yunzhou City's address system underwent a revolution.

Old Addresses: Random Assignment

Under the old system, addresses were randomly assigned. When you built a new house, the authorities gave you a random address number. You could not predict what number you would get, nor could you choose.

This system had problems:
1. If you wanted to tell others your address before the house was built, you couldn't—because you did not yet know the address number.
2. If you rebuilt the house (e.g., after a fire), the new house might get a completely different address. All previously registered

addresses would become invalid.
3. Two different houses would never get the same address—ensuring uniqueness, but also meaning addresses had no pattern whatsoever.

New Addresses: Deterministic Assignment

Under the new system, addresses were no longer random. They were determined by two factors:

1. **Who the house's owner is**
2. **What the house is used for**

Specific rules:
- Address = Hash(Owner's Identity + House's Purpose + "Salt")

Where salt is an adjustment number—if the first calculated address was already occupied, change the salt, recalculate, until finding an empty address.

This meant: as long as you knew who the owner was and what the house was used for, you could calculate its address before the house was built.

Addresses Without Private Keys

Deterministic addresses had a peculiar property: they were not in the traditional address sequence.

Under the old system, every address corresponded to a key—only the key holder could open the door. Under the deterministic address system, some addresses were outside the traditional sequence—they

had no corresponding keys.

This sounded dangerous: a door without a key—could anyone open it?

No. Doors without keys could only be opened by "the house's program."

Specific rules:
- If an address was deterministic (not in the traditional sequence), no one could directly open it with a key.
- Only the house's owner (through their program) could apply to the authorities: "Please open my house used for such-and-such purpose."
- The authorities checked: does the applicant's identity + house's purpose indeed correspond to this address? If so, the authorities opened the door.

This meant: deterministic address security did not come from "who has the key" but from "who can prove to the authorities that this house indeed belongs to them."

Applications of Deterministic Addresses

Predicting addresses:
A merchant wanted to open a chain of tea houses. Before building the first tea house, he could calculate all future tea house addresses—because he knew he was the owner and knew the purpose was "Tea House No. N." He could print business cards and register contracts in advance without waiting for houses to be built.

Address mapping:
Yunzhou City's household registration system used deterministic addresses to establish "one-to-many" relationships. For example: a

large family had a main address (traditional address); all family members' addresses were derived from this main address—Hash(Main Address + "Eldest Son"), Hash(Main Address + "Second Daughter"). Thus, knowing the main address allowed calculating all family members' addresses.

Seedless control:
Some public facilities (like public wells, roads) used deterministic addresses. These facilities had no private owners but were controlled by public programs. Anyone could apply to the program to use the facility; the program decided whether to allow based on rules. Because facilities had no private keys, no one could "steal" them—they belonged to no one, only to the program.

The Art of Salt

Deterministic address calculation involved a key variable: salt.

Theoretically, Hash(Owner + Purpose) should directly produce an address. But sometimes, this address happened to fall in the traditional address sequence—meaning it might already be occupied by someone else (because traditional addresses had keys).

When this happened, the system automatically decremented salt from 255 and recalculated Hash until finding an address not in the traditional sequence.

This "finding valid salt" process was deterministic—given the same owner and purpose, the system always found the same salt. So addresses were still deterministic; only the calculation process was slightly more complex.

Years later, all new buildings in Yunzhou City used deterministic addresses. An urban planner said: "Deterministic addresses transformed addresses from 'random identifiers' into 'meaningful names.' An address is no longer just a string of numbers; it encodes the building's identity, purpose, and history."

What This Fable Is About

This fable illustrates **Solana's Program Derived Address (PDA)**—a deterministically generated account address derived from program address and seeds. PDAs are not on the Ed25519 curve's public key space, so they have no corresponding private keys and can only be signature-controlled by the generating program through CPI (Cross-Program Invocation).

Key Mapping

Story Element	Technical Concept	Explanation
Old random addresses	Traditional public key addresses	Generated from private key through Ed25519; random and unpredictable
New deterministic addresses	PDA	`findProgramAddress([seeds], program_id)` deterministically generates addresses
Owner's identity	Program ID	PDA calculation includes deployer address, ensuring different programs' PDA spaces do not conflict

House's purpose	Seeds	Arbitrary byte arrays for distinguishing different PDAs generated by the same program
Salt	Bump (nonce)	Decremented from 255 to 0, finding a valid value making PDA not on Ed25519 curve
Not in traditional sequence	PDA has no corresponding private key	PDA is not on elliptic curve; no private key; cannot be externally signed
Door without key	PDA has no external signature	Only the program generating the PDA can "sign" for the PDA through CPI
Program applies to authorities	Cross-Program Invocation (CPI)	Solana programs can call other programs and provide signatures for derived PDAs
Authorities check	Runtime verification	Solana runtime verifies CPI calls are legitimate, ensuring only correct programs can sign for PDAs
Chain tea houses	Deterministic address prediction	Can predict PDA addresses before account creation for pre-configuration and referencing
Household mapping	One-to-many relationship modeling	One main account can derive multiple child accounts, establishing hierarchical structures
Public facilities without owners	PDA for program-controlled state	PDAs can be fully controlled by programs for storing program states, configurations, etc.

Why This Story?

PDA is one of Solana development's most core and unique concepts. It allows programs to "own" accounts, control states, and establish deterministic address relationships—things without direct equivalents in EVM.

The "deterministic address" metaphor for PDA offers several intuitive advantages:

1. **Determinism**: Given owner and purpose, address is always the same = PDA deterministic generation
2. **No private key**: No key; only program can open = PDA has no external signature, only program CPI control
3. **Salt**: Adjustment number finding valid address = bump seed search process
4. **Applications**: Predicting addresses, one-to-many mapping, program-controlled states = PDA practical use cases

Further Reading

- wiki-web3: Solana PDA Program Derived Address Deep Dive
- Solana Docs: Program Derived Addresses
- Anchor Framework: PDA
- Solana PDA Deep Dive

"The Numbered Lockers"

The Numbered Lockers

> "你以为锁在柜子里的秘密，其实柜子本身就在大厅中央。"

故事

云州城的政务大厅里有一面巨大的墙，上面整齐排列着无数个编号柜——从 0 号到无限远。每一个柜子都能装下三十二个字，不多不少。

任何人都可以走到这面墙前，打开任何一个柜子，阅读里面的内容。没有任何门锁，没有任何密码。编号柜的设计哲学是：透明。

柜子的规则

大厅的管理员制定了一套严格的存取规则：

1. 每个柜子有固定的大小：三十二个字。如果一个东西小于三十二个字，可以把多个东西塞进同一个柜子。这叫"打包"。
2. 如果一个东西超过三十二个字，它必须占用连续的几个柜子。
3. 管理员按照你提交物品的顺序，依次放入 0 号、1 号、2 号柜子……
4. 一旦东西放进柜子，就不能移动。除非你主动把柜子清空，否则里面的内容永远在那里。

有一个精明的商人，他在柜子里存了自己的商业机密——他的成本价、他的客户名单、他的暗号。他以为锁在柜子里就安全了。

但他忘了：柜子没有门。任何人只要知道编号，就能打开来看。

有一天，一个竞争对手走到大厅，打开 3 号柜子——里面正是商人的成本价。打开 4 号柜子——客户名单。打开 5 号柜子——暗号。

商人崩溃了："这是我的隐私！"

管理员冷冷地回答："大厅的规则是透明。你放进来的时候，就应该知道这一点。"

打包的艺术

聪明的存储者学会了"打包"。

柜子的大小是三十二个字。如果你有两个各十六个字的东西，你可以把它们塞进同一个柜子——左边十六个，右边十六个。这样你只占用了一个柜子，而不是两个。

但打包有个微妙的限制：东西必须按大小从大到小排列。如果你先放了一个八位的东西，再试图放一个二十四位的东西——放不进去了，因为剩下的空间不够。但如果你先放二十四位的，再放八位的，刚好填满。

一个老练的存储顾问因此发了财。他的工作就是帮客户优化打包顺序，让同样多的东西占用最少的柜子。少占一个柜子，就少交一份存储税。

动态柜子

有些存储者需要存的东西数量不固定——比如一个不断增长的名单。管理员为他们设计了"动态柜子"：

- 名单的内容不直接放在编号柜里，而是放在另一个叫"大厅仓库"的地方。
- 编号柜里只放一个指针——一个地址，告诉你在哪里找到真正的名单。
- 当名单增长时，仓库会自动扩展。编号柜里的指针永远不变。

这种设计的代价是：每次读取名单，都需要先读编号柜里的指针，再去仓库找真正的内容。比直接存取要慢，但胜在灵活。

清空柜子的陷阱

有一个常见的误解：把柜子里的东西拿出来，柜子就空了。

不。在编号柜的世界里，"清空"意味着把柜子里的内容设置为零。但柜子本身还在，编号还在，它仍然占用着墙的空间，你仍然需要为它支付存储税。

唯一的例外是：如果你把柜子里的内容从零改回某个非零值，你不需要再付一次税——因为你已经为"创建"这个柜子付过了。但如果你创建了一个柜子，又把它清零，管理员会退还你一部分存储税，作为对你"释放空间"的奖励。

多年后，云州城来了一个外乡人，他看着满墙的编号柜，说了一句被刻在墙上的话："这不是存储，这是裸露。"

这则寓言在说什么

这则寓言对应的是 **EVM** 的存储布局（**Storage Layout**）和 **Slot** 机制。EVM 的存储是一个巨大的 key-value 映射，每个 slot 是 32 字节（256 位）。Solidity 按照变量声明顺序将数据 packing 到 slots 中，所有存储数据对链上所有人可见。

关键映射

故事元素	技术概念	说明
编号柜墙	EVM Storage	每个合约有一个 `2^256` 个 slot 的存储空间，每个 slot 32 字节
三十二个字	32 bytes / 256 bits	一个 storage slot 的固定大小
透明无锁	链上数据公开可读	`storage` 中的任何数据都可以被任何人通过 `eth_getStorageAt` 读取
商业机密被读	Storage 不是隐私	很多初学者误以为 `private` 变量是私密的，实际上只是 Solidity 层面限制访问，链上完全透明
打包	Storage Packing	Solidity 尝试将多个小变量（如多个 `uint128`）pack 进同一个 slot，节省 gas
打包顺序限制	Packing rules	变量按照声明顺序放入 slots。如果顺序不当（如先放 `uint8` 再放 `uint256`），会导致不必要的 slot 浪费
动态柜子	动态数组和映射	`mapping` 和动态 `array` 使用 keccak256 哈希计算 slot 位置，数据存储在 "hash 空间" 而不是顺序 slots
指针	Slot 中的哈希值	mapping 的 slot 中不存实际数据，只存计算偏移用的种子
清空柜子	`delete` / 清零	将 slot 设为零会返还部分 gas（15,000 gas refund），但 slot 仍然存在于状态中
存储税	SSTORE gas cost	首次写入非零值消耗 20,000 gas；修改消耗 5,000 gas；清零返还 15,000 gas

为什么是这个故事？

"链上数据公开"是 Web3 中最反直觉的事实之一。很多开发者误以为 `private` 变量是安全的，结果导致严重的安全漏洞（如著名的 `parity` 钱包漏洞）。

用"编号柜"来隐喻 storage，有三个直觉优势：
1. 透明性的可视化：柜子没有门，任何人都能打开——对应了链上数据的完全公开
2. 打包的具象化：把数据打包进固定大小的 slot，就像把东西塞进固定大小的柜子
3. **gas** 成本的直觉：创建柜子（SSTORE）贵，读取便宜，清空有退款——对应了 EVM 的 gas 定价逻辑

延伸阅读

- wiki-web3 概念原文：Solidity 私有数据的链上透明性与 Slot 存储机制
- EVM Deep Dives: Storage Layout
- Solidity Docs: Storage Layout

"The Craftsman's Workbench"

The Craftsman's Workbench

> "工作台越大，干活越贵。但每次干完活，桌子会自动清空。"

故事

云州城的工坊里，每个工匠都有一张工作台。工作台的规矩非常特别：

第一，工作台是临时的。

工匠开始干活时，工作台是空的。他在上面摆放工具、零件、半成品。干完活，他把成品搬走——工作台自动清空，什么都不留下。下一个工匠来，面对的又是一张全新的空桌子。

这意味着：工作台上的东西不能过夜。今天画的图纸、做的记号，明天就不存在了。

第二，工作台可以扩展。

工作台一开始很小，只够放几样工具。如果工匠需要更大的空间，他可以把桌子往两边拉。但拉桌子是有代价的——每多拉一寸，就需要多交一寸的"扩展税"。

而且扩展税不是线性的。第一张桌子从三尺拉到四尺，很便宜。但从一百尺拉到一百零一尺，贵得惊人。因为工坊的地板空间是稀缺资源，占用越大，边际成本

越高。

第三，工作台是线性的。

工匠不能在桌子的任意位置放东西。他必须从左到右依次摆放：先把东西放在最左边，然后依次向右。如果他想跳过中间的空位，直接把东西放在右边——可以，但他必须先"声明"中间的空位属于自己，这也要交税。

第四，工作台是公共的。

当工匠需要请另一个工坊的师傅帮忙时，他不能把师傅请到自己的工坊——他必须把自己的工作台"复制"一份，送到对方的工坊。师傅在这张复制的桌子上干活，干完把结果送回来。原工坊的工作台不受影响。

有个聪明的年轻工匠，他发现工作台的这些特性后，设计了一套极其高效的工作流程：

1. 每次开工前，先在桌角记下"当前已用长度"——就像船上的吃水线。
2. 干活过程中，只在必要的范围内扩展桌子。
3. 干完活，把吃水线之前的东西全部清掉，让桌子回到初始状态。

这套流程让他交的扩展税只有其他工匠的三分之一。

但有一个陷阱很多人踩过：有人觉得"反正桌子会自动清空"，就在桌上留下了一些"以为以后还会用到"的东西。结果下次开工时，发现桌子确实是空的——那些东西不在了。他不得不重新计算、重新摆放，反而更浪费时间。

老工匠的忠告是："工作台上只能放'这次干活需要的东西'。如果你想保存什么，把它刻到石头上（storage），而不是留在桌子上（memory）。"

这则寓言在说什么

这则寓言对应的是 **EVM** 的内存（**Memory**）机制。Memory 是 EVM 执行期间的临时数据区域，按字节寻址，线性扩展，每次消息调用后重置。

关键映射

故事元素	技术概念	说明
工作台	EVM Memory	合约执行期间的临时数据区域，用于存储动态大小的数据
临时性	Memory 的生命周期	Memory 中的数据只在当前消息调用期间存在。调用结束后自动清空
可扩展	Memory expansion	Memory 是动态扩展的。写入超出当前大小的地址时，memory 会自动扩展，消耗 gas
扩展税	Memory expansion gas cost	Memory 扩展的 gas 成本 = `memory_size_word^2 / 512 + 3 * memory_size_word`。是二次成本，扩展越大越贵
线性摆放	Memory 的线性地址空间	Memory 从地址 0 开始，按顺序写入。MSTORE/MLOAD 操作按 32 字节 word 为单位
吃水线	空闲内存指针（Free Memory Pointer）	Solidity 编译器在 memory 开头维护一个指针，记录当前已用 memory 的末尾位置，避免重复计算
请外援	外部合约调用（CALL）	调用外部合约时，EVM 会为被调用者创建新的 execution context，拥有独立的 memory。返回时原 memory 不受影响
复制工作台	CALL 的 memory 传递	调用外部合约时，需要把输入数据复制到 memory 中传递给被调用者。返回数据也需要 memory 空间
刻到石头上	Storage vs Memory	Storage 是持久的但昂贵，Memory 是临时的但便宜。数据该存哪里取决于是否需要持久化

| 以为以后还会用到 | Memory 中的数据不持久 | 如果在 memory 中缓存了数据，期望下次调用还能用——不可能。每次调用 memory 都是全新的 |

为什么是这个故事？

Storage 和 Memory 的区别是 Solidity 开发者最早需要理解的核心概念之一，但"临时 vs 持久"的抽象对初学者来说很模糊。

用"工作台"来隐喻 Memory，有几个直觉优势：
1. 临时性：干完活桌子自动清空——对应了 memory 的生命周期
2. 扩展成本：拉桌子越来越贵——对应了 memory expansion 的二次 gas 成本
3. 外部调用：把桌子复制到对方工坊——对应了 CALL 时的 memory 隔离和传递成本

延伸阅读

- wiki-web3 概念原文：EVM 内存机制深入解析
- EVM Deep Dives: Memory
- Ethereum Yellow Paper: Memory

"The Prophet's Mold"

The Prophet's Mold

> "在东西被造出来之前，你就知道它会出现在哪里。"

故事

云州城的铁匠铺有个传统：每当有人定制一把刀，铁匠会把刀打好，然后在刀柄上刻一个编号。编号是随机的——取决于当天打了第几把刀。买家只有在刀打好之后，才知道自己的刀是哪一把。

这个传统运转了三百年，直到一位叫沈算的年轻铁匠提出了一个大胆的设想。

老办法：随机编号

老办法的问题很明显：

1. 买家无法提前知道刀的确切编号。如果买家需要把刀和其他配件一起组装，他必须在刀打好之后才能开始组装。
2. 如果铁匠铺搬迁或者铁匠换人，编号序列可能中断，导致冲突。
3. 买家无法"预留"一个编号。如果他想确保自己的刀编号是 777，他只能祈祷当天恰好打到第 777 把。

新办法：预言模具

沈算设计了一种"预言模具"。这种模具的奇妙之处在于：在刀被铸造之前，你就能准确预言这把刀的最终编号。

预言的公式刻在模具本身上：

刀的编号 **= Hash(**模具的编号 **+** 使用的材料配方 **+** 盐值**)**

其中：
- 模具的编号是固定的——每个模具天生就有一个独一无二的编号。
- 材料配方是买家自己选择的——不同的配方产生不同的刀。
- 盐值是一个任意数字——买家可以自由选择，用来"调整"最终的编号。

这意味着：在刀被铸造之前，任何人都可以把这三个参数代入公式，计算出刀的编号。不需要等到刀打好，不需要信任铁匠，不需要任何预言。

预言模具的应用

这个发明在云州城掀起了一场革命。

第一个应用：提前组装。

一个刀鞘匠人需要为一把定制刀制作刀鞘。在老办法下，他必须等刀打好、知道编号之后，才能开始制作刀鞘——因为刀鞘必须和特定编号的刀匹配。

在新办法下，买家在下单时就知道刀的编号。刀鞘匠人可以在刀还在熔炉里的时候，就开始制作匹配的刀鞘。等刀打好，刀鞘也刚好完成。

第二个应用：状态通道。

两个商人需要在云州城设立一个"联合账户"来管理他们之间的往来账目。按照老办法，他们必须先设立账户，然后才能开始交易。

但在新办法下，他们可以在账户设立之前，就"预言"出账户的编号，然后基于这个预言编号签订合约："当我们最终设立账户时，账户编号将是 XYZ，届时合约自动生效。"

这个"账户尚未存在，但已经可以被引用"的状态，被称为"反事实"。

第三个应用：批量铸造。

一个军队需要一千把制式军刀。老办法下，铁匠一把一把地打，编号随机，无法预先分配。

用预言模具，军队可以在铸造开始前，为每一把刀预先计算编号，把编号提前刻在刀鞘上。铸造完成后，刀和刀鞘自动匹配。

盐值的艺术

预言模具引入了一个有趣的变量：盐值。

盐值本身没有任何意义——它唯一的作用是让买家可以"调整"最终编号。如果第一次计算出的编号已经被占用了，买家可以换一个盐值，重新计算，直到得到一个未被占用的编号。

这催生了"编号猎人"——一种专门寻找"漂亮编号"的职业。有人愿意花大价钱买一个编号里有八个 8 的刀，或者编号以特定数字结尾的刀。编号猎人用计算机不断尝试不同的盐值，寻找符合买家要求的编号。

多年后，沈算的预言模具成了云州城所有工匠的标准工具。有人问沈算："你的模具最厉害的地方是什么？"

沈算说："不是预言。而是它让'尚未存在的东西'变得'可寻址'。在刀被铸造之前，它就已经是一个可以被引用、被信任、被使用的实体。这改变了我们对'存在'的定义。"

这则寓言在说什么

这则寓言对应的是 **CREATE2**——EVM 中一种确定性的合约创建方式。与 CREATE 不同，CREATE2 的合约地址仅由 `deployer_address + salt + init_code_hash` 决定，与创建时的 nonce 无关。这意味着你可以在合约部署之前就精确预测它的地址。

关键映射

故事元素	技术概念	说明
传统随机编号	CREATE (基于 nonce)	合约地址 = `keccak256(rlp([sender, nonce]))`。每次创建合约 nonce 增加，地址不可预测
预言模具	CREATE2	合约地址 = `keccak256(0xff + sender + salt + keccak256(init_code))[12:]`。地址完全确定性，可预先计算
模具的编号	Deployer 地址	CREATE2 的计算中包含部署者地址，确保不同部署者的地址空间不冲突
材料配方	Init code (初始化字节码)	合约的 creation bytecode，决定合约的逻辑。不同的 init code 产生不同的地址
盐值	Salt (bytes32)	部署者可以自由选择的 32 字节值，用于在相同 init code 下生成不同地址
提前组装	预计算地址的应用	可以在合约部署前就知道其地址，提前配置其

他合约引用它 |
反事实	Counterfactual instantiation	账户或合约尚未在链上部署，但已经可以被引用和交互（如状态通道、Layer 2）
状态通道	State Channels / Layer 2	基于反事实地址，可以在链下预先签订合约，链上仅在争议时实际部署
编号猎人	Vanity address mining	通过暴力枚举 salt 值，寻找特定前缀或模式的合约地址
批量铸造	批量部署 + 预计算	可以预先计算大量合约地址，批量配置和初始化

为什么是这个故事？

CREATE2 是一个看似简单、实则深刻的技术。它让"尚未部署的合约"获得了"可寻址性"，这是反事实状态、状态通道、批量部署等高级应用的基础。

用"预言模具"来隐喻 CREATE2，有几个直觉优势：
1. 确定性的可视化：在铸造前就知道编号——对应了 CREATE2 的地址可预测性
2. 反事实的具象化：刀还没打好，刀鞘已经可以开始做了——对应了 counterfactual instantiation
3. 盐值的灵活性：任意调整盐值来寻找理想编号——对应了 salt 参数的作用

延伸阅读

- wiki-web3 概念原文：CREATE2 创建合约、预测合约地址，看这一篇就够了
- EIP-1014: Skinny CREATE2
- Counterfactual Instantiation
- Uniswap V3 Factory CREATE2 Usage

"The Soul in Borrowed Body"

The Soul in Borrowed Body

> "借别人的身体说话，但话里的意思，全算在自己头上。"

故事

云州城有一种古老的秘术叫"借身术"。施术者可以把自己的灵魂暂时移到另一个人的身体里，借用那个人的身体来行动。但借身术有一个极其危险的特性：

你在借来的身体里做的一切，后果都由你原来的身体承担。

正常的访客

正常情况下，如果一个人拜访另一个人，他站在对方家里说话。对方家里的陈设、家具、财物都是对方的。访客只是借用了一个空间来表达自己。

这叫"普通拜访"——访客有自己的身体，只是在对方的地盘上说话。访客说"我要喝水"，喝的是对方家里的水，与对方家里的财产发生交互。

借身术的不同

借身术完全不同。当甲把自己的灵魂移到乙的身体里时：

- 甲使用的是乙的身体、乙的喉咙、乙的四肢。
- 但甲说话的内容、做的决定、承担的后果，全都算在甲的头上。
- 如果甲用乙的身体打破了丙的窗户，丙要找甲赔偿，不是找乙。
- 如果甲用乙的身体签了一份契约，契约的效力绑定在甲身上，不是乙。

更关键的是：甲在乙的身体里行动时，他以为自己还在自己的家里。他看到的环境其实是乙的家——乙的家具、乙的财物、乙的日记。如果甲试图"把某件东西放进自己的保险箱"，他实际上是把东西放进了乙的保险箱。

借身术的妙用：替身演员

云州城的戏班子发现借身术有一个绝妙的用途：替身演员。

戏班子有一个主角叫阿龙。阿龙有极高的表演天赋，但他年事已高，无法亲自上台。戏班子找了一个年轻力壮的替身叫阿虎。

演出时，阿龙把自己的灵魂移到阿虎的身体里。观众看到的是阿虎的身体在表演，但每一个动作、每一句台词都是阿龙的灵魂在控制。阿虎只是一个人形躯壳。

阿龙可以随时"升级"自己的表演技巧——他在后台修炼新的招式，然后下次演出时，通过借身术让阿虎的身体执行新的招式。观众永远看到的是"阿虎"，但体验的是阿龙不断进化的艺术。

如果有一天阿龙去世了，戏班子只需要再找一个新的替身，阿龙的灵魂——如果提前备份过——可以继续借身演出。戏班子不需要重建整个舞台，只需要换一个新替身。

借身术的灾难：灵魂错乱

但借身术也是云州城历史上最危险的秘术。

有一次，一位术士在借身时犯了一个错误：他试图把一件东西"放进自己的口袋"，但他忘了自己正在借来的身体里。结果那件东西被放进了替身身体的口袋里——而这个口袋的位置，恰好对应着替身身体里一个存放剧毒的暗格。

剧毒泄露，替身当场死亡，术士的灵魂也被永远困在了死亡的躯壳里。

这个事故被称为"灵魂错位"——当术士以为自己在操作"自己的空间"时，实际上他在操作的是替身的空间，而两个空间的布局完全不同。

借身术的规则

经过无数次事故，云州城的术士们总结出了借身术的安全守则：

1. 替身身体的内部布局必须和原身体完全一致。 如果原身体的"心脏位置"在左边，替身身体的"心脏位置"也必须在左边。否则灵魂会迷路。

2. 替身身体不能有自己的灵魂。 如果替身身体里已经有一个灵魂，两个灵魂会争夺控制权，结果是灾难性的。

3. 借身期间，原身体处于休眠状态。 原身体不能同时行动，否则会出现"两个我在同时做不同的事"的悖论。

4. 借身结束后，替身身体里不能留下任何秘密。 因为灵魂离开时会带走记忆，但替身身体里留下的物理痕迹（如写在纸上的字）会暴露给下一个借身者。

这则寓言在说什么

这则寓言对应的是 **EVM** 的 **`delegatecall`**——一种特殊的低级调用方式。与普通的 `call` 不同，`delegatecall` 在目标合约的代码中执行，但使用调用者合约的存储、余额和状态。

关键映射

故事元素	技术概念	说明
普通拜访	CALL	在目标合约的上下文中执行：使用目标的 storage、msg.sender 是调用者
借身术	DELEGATECALL	在目标合约的代码中执行，但使用调用者的 storage、msg.sender 和 msg.value 保持原样
甲的灵魂移到乙的身体	调用者用代理合约的代码执行	逻辑代码来自被调用合约（implementation），但 storage 是调用者（proxy）的
后果算在甲头上	msg.sender 保持原样	`delegatecall` 不改变 `msg.sender`，调用者身份保持不变
替身演员	代理合约模式	Proxy 合约使用 `delegatecall` 将执行委托给 Implementation 合约，自己只负责存储状态
阿龙升级招式	可升级合约	更换 Implementation 合约地址，Proxy 的存储不变，逻辑升级
阿龙去世找新替身	合约迁移	旧 implementation 废弃，新 implementation 接入同一个 proxy，状态继承
灵魂错乱 / 剧毒暗格	Storage collision	如果 Implementation 和 Proxy 的 storage 布局不一致，`delegatecall` 会写入错误的位置，可能导致严重漏洞（如 Parity 多签钱包漏洞）
布局必须一致	Storage layout compatibility	升级后的 Implementation 必须保持与旧版本相同的 storage 布局，新增的变量只能追加在末尾
替身不能有自己的灵魂	不能有状态冲突	Implementation 合约不应该有自

己的重要状态，所有状态应该在 Proxy 中 |
| 借身结束不留秘密 | 自毁风险 | 如果 Implementation 合约执行 `selfdestruct`，会通过 `delegatecall` 销毁 Proxy 合约（这是 delegatecall 的危险之一）|

为什么是这个故事？

`delegatecall` 是 Solidity 中最难以理解的概念之一，也是许多严重安全漏洞的根源（如 Parity 多签钱包被锁、各种代理合约漏洞）。

用"借身术"来隐喻 `delegatecall`，有几个直觉优势：
1. 上下文切换的可视化：在别人的身体里，但后果算自己的——对应了 `delegatecall` 的上下文保持特性
2. 代理合约的直觉：替身演员 = Proxy，阿龙 = Implementation，观众始终看到 Proxy 地址
3. **Storage collision** 的危险：灵魂错位 = storage 布局不匹配时的灾难性后果

延伸阅读

- wiki-web3 概念原文：代理合约与可升级
- EVM Deep Dives: Delegatecall
- OpenZeppelin: Proxy Patterns
- Parity Multi-Sig Wallet Hack Analysis

"The New Grammar Manual"

The New Grammar Manual

> "旧的书写方式没有标点，新的手册把句子分成了章节。"

故事

云州城的法律文书有一个延续了五百年的传统：所有的文书都是一卷连续的羊皮纸，从头到尾没有分段、没有标题、没有标点。法官必须从头读到尾，才能知道这份文书是契约、诉状还是遗嘱。

这个传统带来了三个问题。

第一个问题：无法快速识别

当一个信使把一卷新文书送到法院时，法官无法一眼看出这是什么类型的文书。他必须展开羊皮纸，从头开始读，直到发现某个关键词——比如" hereby "或" witness "——才能判断文书类型。

如果有人恶意把一份毒药配方伪装成契约（在前面写一段看起来像契约的开头，中间突然变成毒药配方），法官在读到一半时才会发现不对劲。但此时可能已经太晚了——某些行动已经基于前半段的误读而执行了。

第二个问题：无法验证完整性

羊皮纸可能会被虫蛀、被水浸、被裁切。如果文书的末尾缺了一段，法官很难发现——因为他不知道原本的文书应该有多长。他只能读到"突然中断"的地方，然后猜测：这是原本就这样结束的，还是后面缺了一段？

有些狡猾的讼师会故意删掉契约中对自己不利的最后几行，然后把残缺的契约呈上法庭。法官无法分辨这是原本就短的契约，还是被篡改过的长契约。

第三个问题：执行效率低下

法院的执行员（负责按照文书内容执行行动的人）必须从第一行开始，逐字逐句地读完整份文书。他不能跳过任何一段，因为不知道哪一段是相关的。

如果一份文书的前半部分是背景介绍（"甲方乃云州城居民，乙方乃东海商人……"），执行员必须把这些背景也读完，才能到达真正重要的执行条款。这极大地浪费了时间。

新语法手册的革命

一位叫章节的年轻法官提出了一套全新的文书格式："章节化文书"。

章节化文书不再是一卷连续的羊皮纸，而是被明确分成了几个部分：

1. 类型声明（第一页）：明确写明"这是一份契约"或"这是一份诉状"。法官拿到文书的第一秒就知道该怎么处理。
2. 头部信息（第二页）：列出文书的基本信息——作者、日期、版本。不需要展开全文就能获取元数据。
3. 正文（第三页起）：真正的内容，分段清晰，每段有标题。

4. 校验页（最后一页）：一段特殊的文字，通过对全文进行数学运算生成。如果文书的任何一页被改动，校验页就会不匹配，法官立刻知道文书被篡改过。

更关键的是：章节化文书在送入法院之前，必须先经过一个"格式审查员"的检查。审查员不看文书的内容，只看格式是否正确——类型声明是否在第一位、校验页是否完整、各章节是否有明确的分隔符。

如果格式审查不通过，文书根本不会被法官看到。这意味着：所有到达法官面前的文书，至少格式是正确的、结构是完整的、没有被截断的。

意外的收益

章节化文书推行后，法官们发现了一些意料之外的好处：

1. 执行变快了。执行员不需要读完整份文书了。如果文书是"执行令"，他可以直接翻到"执行条款"章节，跳过前面的背景介绍。

2. 错误更容易发现。如果一个讼师试图在契约里插入一段"自动转账"的条款，格式审查员会注意到这个条款不在契约允许的章节类型中，直接拒绝这份文书。

3. 版本兼容。新格式的文书可以在旧法院里执行（旧法院仍然能读连续文本），但新法院可以充分利用章节化的优势。这是一个渐进式的升级，不是革命性的断裂。

多年后，云州城的所有法律文书都采用了章节化格式。有人问章节法官："你发明的最大贡献是什么？"

章节法官说："不是我发明了什么新内容。是我让文书有了'结构'。在旧格式里，内容和结构混在一起，无法分离。在新格式里，结构是可见的、可验证的、可优化的。一旦结构被显式化，很多事情就自然而然地变好了。"

这则寓言在说什么

这则寓言对应的是 **EVM Object Format (EOF)**——以太坊正在引入的一种新的字节码容器格式。EOF 将 EVM 字节码从"扁平的连续字节序列"转变为"结构化的容器"，包含类型声明、头部元数据、代码段、数据段等，并引入部署前的验证机制。

关键映射

故事元素	技术概念	说明
连续羊皮纸	传统 EVM 字节码	扁平的字节序列，没有结构，EVM 逐字节解释执行
无法快速识别类型	无法区分 code/data	传统字节码中，代码和数据混在一起，JUMP 目标需要运行时解析
毒药配方伪装成契约	恶意字节码 / 代码注入	可以在数据段中嵌入可执行代码，或通过 JUMP 跳转到任意位置执行非预期逻辑
无法验证完整性	无法静态验证	传统字节码无法在不执行的情况下验证其完整性和安全性
执行效率低下	JUMP 解析开销	EVM 需要在运行时计算 JUMP 目标，无法预先优化执行路径
章节化文书	EOF 容器格式	字节码被组织为结构化的容器：magic number + version + header + code sections + data section
类型声明	EOF 头部类型信息	明确声明合约的代码段数量、数据段大小等元数据
校验页	EOF 内置校验	容器格式的校验和/长度字段，确保字节码未被截断或篡改
格式审查员	EOF 部署前验证	EOF 合约在部署前必须通过静态验证，确保

结构正确、JUMP 目标合法、栈深度安全等 |
格式不通过不上法庭	无效 EOF 无法部署	不符合 EOF 格式的字节码在部署时被拒绝
执行变快	EOF 的性能优化	由于结构已知，EVM 可以预先编译和优化执行路径，不再需要运行时 JUMP 解析
渐进式升级	EOF 与旧字节码共存	EOF 是 opt-in 的，旧字节码继续兼容，新合约可以选择使用 EOF

为什么是这个故事？

EOF 是 EVM 生态中长期且复杂的改进之一，涉及到底层执行模型的变革。对于不熟悉 EVM 内部的人来说，"字节码容器格式"听起来极其抽象。

用"章节化文书"来隐喻 EOF，有几个直觉优势：
1. 结构化的可视化：从"一卷羊皮纸"到"分章节的文书"，让读者理解了"容器格式"的核心价值
2. 安全性的提升：格式审查员在文书到达法官前拦截问题——对应了 EOF 的部署前验证
3. 性能优化的来源：执行员直接翻到相关章节——对应了 EOF 预先知道代码结构后可以做的优化

延伸阅读

- wiki-web3 概念原文：EVM 对象格式 EOF 初步了解
- EIP-3540: EOF - EVM Object Format v1
- EIP-3670: EOF - Code Validation
- EIP-4200: EOF - Static relative jumps
- EVM Object Format (EOF) Overview

"The Message Tube"

The Message Tube

> "只读不写的信筒，传给别人的话，自己不能改。"

故事

云州城的通信系统有三种传递消息的方式。

第一种：口信

最古老的方式是直接口述。你走到对方面前，开口说话。对方听到什么就是什么。

口信的好处是简单直接。坏处是：如果你要告诉对方一百个人的名字，你得一个个念。对方记不住怎么办？只能再念一遍。

而且口信不能"传递"——你无法让朋友把你的口信再转达给第三个人。口信只在说话者和听者之间有效。

第二种：书信

你写一封信，装进信封，交给信使。信使把信送到对方手里，对方打开信封阅读。

书信可以传递复杂的内容——一百个人的名字可以写在纸上，不会遗漏。书信也可以被转交——收信人可以把原信再转给其他人。

但书信有个成本：写信需要纸和墨，送信需要信使的报酬。而且书信一旦被送出，里面的内容就不能改了。如果你想修改，必须重新写一封新的。

第三种：传令筒

云州城的军队使用一种特殊的通信工具：传令筒。

传令筒是一根密封的铜管，里面装着一张纸条。纸条上的内容在装入铜管时就固定了——任何人都可以把铜管打开，读出纸条上的内容，但没有人能修改它。

传令筒有三个独特的性质：

第一，只读不写。

收到传令筒的人只能读里面的内容，不能往里面加字、改字或删字。如果将军的命令是"前进十里"，传令筒里的纸条永远是这四个字。接收者不能把它改成"前进二十里"。

第二，透明公开。

传令筒没有锁。任何人路过都可以打开看里面的内容。这在军队里不是问题——命令本身不需要保密，只需要确保不被篡改。

第三，一次性的。

传令筒在送到目的地后，就完成任务了。里面的纸条可以被抄录存档，但铜管本

身不会被 reused。下次传令，用一根新的铜管。

传令筒的妙用：远程指挥

云州城的将军驻扎在北疆，皇帝在京城。皇帝想命令将军做一件事，但他不能亲自去北疆口述。他可以写一封书信，但书信在路途上可能被篡改、被截获。

传令筒解决了这个问题：皇帝把命令写成纸条，装入铜管，密封。信使运送的是铜管，不是裸信。沿途任何人都可以打开铜管看命令的内容（透明），但没有人能在不破坏密封的情况下修改命令（不可篡改）。

将军收到铜管后，打开它，读取命令，执行。执行过程中，将军可以对照铜管里的命令，确认自己没有理解错。

传令筒的限制

但传令筒也有它的局限。

如果皇帝想让将军"根据前线情况灵活决定"，传令筒就不够用了——因为传令筒里的命令是固定的，将军无法向皇帝实时请示。他只能在收到传令筒后，按照既定命令执行。

如果命令中包含了大量数据——比如"这里有一份五百人的名单，请按名单发粮"——传令筒可以装下这份名单（因为它是一张纸），但将军如果需要把这些名字一个个输入到账本里，他就得手动抄写。传令筒里的数据不能直接"导入"到账本，只能被阅读。

这则寓言在说什么

这则寓言对应的是 **EVM** 的 **Calldata**——交易或消息调用中携带的只读数据。Calldata 是外部调用时传入的数据，存储在交易体中，不能被被调用合约修改，只能通过 `msg.data` 或函数参数读取。

关键映射

故事元素	技术概念	说明
口信	内部调用 / 本地计算	合约内部的函数调用，数据在栈和内存中传递，不需要序列化
书信	Memory / Storage	可写的数据存储区域，可以被修改和传递
传令筒	Calldata	外部调用的只读数据，随交易传递，被调用合约只能读取不能修改
只读不写	Calldata 的不可变性	`calldata` 位置的数据不能被修改。Solidity 中标记为 `calldata` 的参数是只读的
透明公开	Calldata 的公开性	交易数据对全网公开，任何人可以通过区块链浏览器读取
一次性	Calldata 的生命周期	Calldata 只在当前消息调用期间存在，调用结束后不可访问
密封铜管	交易的完整性	交易被签名后，calldata 不能被篡改（否则签名验证失败）
远程指挥	跨合约调用	合约 A 调用合约 B 时，通过 calldata 传递函数选择器和参数
大量数据装入铜管	Calldata 的容量	Calldata 可以携带较大的数据（如数组、字符串），但读取和解析需要 gas
手动抄写	Calldata 到 Memory/Storage 的复制	如果需要修改 calldata 中的数据，必须先复制到 memory 或 storage
无法实时请示	Calldata 的单向性	一次调用中，calldata 是单向的。被调

用者无法修改 calldata 并返回给调用者 |

为什么是这个故事？

Calldata 是 EVM 中三个主要数据位置（storage、memory、calldata）之一，但它的"只读"和"外部"特性让很多初学者困惑。

用"传令筒"来隐喻 calldata，有几个直觉优势：
1. 只读性：传令筒只能读不能改——对应了 calldata 的不可变性
2. 外部性：传令筒用于远程通信——对应了 calldata 只存在于外部调用中
3. 透明性：任何人可以打开看——对应了链上交易的公开性

延伸阅读

- wiki-web3 概念原文：EVM 内存机制深入解析
- Solidity Docs: Data Location
- Ethereum Yellow Paper: Message Call
- ABI Encoding Specification

"The Runestones"

The Runestones

> "每一块石头上只刻一个字，但按照正确的顺序排列，它们能建造一座城。"

故事

云州城的建筑工地上不使用图纸。工匠们使用一种古老的系统：符文石。

每一块符文石是一个手掌大小的石块，上面刻着一个符号。符号的种类很少，只有大约两百种。但这两百种符号，按照不同的顺序排列，可以表达任何建筑的建造方法。

符文石的工作方式

建筑工地中央有一个大石台，石台上只能同时摆放三十二块符文石。这是石台的物理限制——再多就放不下了。

工匠的工作流程是这样的：

1. 他从仓库取出符文石，按顺序放到石台上。
2. 当石台上的符文石达到某个操作所需的数量时，自动触发一个动作。
3. 动作完成后，石台上的符文石被替换为动作的结果。

4. 他继续放新的符文石，触发下一个动作。

例如：
- 操作"加"：需要石台上有两块符文石（比如"3"和"5"）。触发后，两块石头被移除，换成一块新石头"8"。
- 操作"复制"：需要石台上有一块石头。触发后，石头被复制一份，石台上现在有两块相同的石头。
- 操作"交换"：需要石台上至少有两块石头。触发后，最上面的两块石头交换位置。

符文石的翻译

高级建筑师不会直接和符文石打交道。他们用一种叫"蓝图语言"的高级语言来设计建筑——"在这里建一道墙，墙高十尺，宽两尺"。

然后，一位专门的"翻译师"把蓝图语言转换成符文石的序列。这个过程是确定性的：同样的蓝图总是产生同样的符文石序列。

但翻译不是一一对应的。蓝图上的一句"墙高十尺"，可能被翻译成几十块符文石的复杂操作——计算面积、检查承重、分配材料。翻译师在翻译时还会做优化：如果蓝图上连续出现了三次"墙高十尺"，翻译师不会重复翻译三次，而是把结果缓存起来，重复使用。

符文石的直接操控

有些极其精密的建筑，蓝图语言无法精确表达。这时候，大师级工匠会直接编写符文石序列。

直接操控符文石的优势是：
- 精确：每一颗石头的位置都在工匠的控制之下，没有翻译的模糊性。
- 高效：翻译师为了安全会加入很多检查，但直接操控可以跳过不必要的检查，

节省时间和材料。
- 访问隐藏功能：蓝图语言只暴露了常见操作，但有些特殊操作只有直接操控符文石才能使用。

风险也很明显：
- 如果工匠把两块石头的顺序搞错了，整座建筑可能倒塌。
- 如果工匠忘了在某一步之后清理石台，石台被占满后，后续操作无法进行。
- 如果工匠使用了"自毁"符文——一块特殊的、能让整个工地消失的石头——后果是不可逆的。

符文石的不可变性

一旦符文石序列被刻好、被送到工地，就不能再改了。如果建筑师发现设计有缺陷，唯一的办法是刻一套新的符文石序列，废弃旧的。

这意味着：符文石的编写必须极其谨慎。一个小错误——比如把"加"和"减"搞混了——可能导致完全错误的结果，而且无法在现场修正。

多年后，云州城最高的塔楼是用符文石建造的。塔楼的设计者是一位直接操控符文石的大师。他说："蓝图语言让普通人也能建房子。但如果你要建一座塔楼触及云端，你必须理解每一块石头的意义。"

这则寓言在说什么

这则寓言对应的是 **EVM** 的操作码（**Opcodes**）和字节码（**Bytecode**）。EVM 是一个基于栈的虚拟机，所有智能合约最终都被编译成字节码——一系列

操作码的序列。Solidity 是高级语言，编译器将其翻译成 EVM 字节码。

关键映射

故事元素	技术概念	说明
符文石	EVM 字节码 / 操作码	每个 opcode 是一个字节（或更多），EVM 逐条解释执行
约两百种符号	~150 个有效 Opcodes	EVM 定义了约 150 个操作码，涵盖算术、逻辑、存储、调用、环境信息等
石台放三十二块石头	EVM 栈深度限制	EVM 栈最大深度为 1024 个 256-bit 字。超出会导致栈溢出
操作"加"	ADD opcode	从栈顶弹出两个值，相加后压回栈顶。所有 EVM 计算都在栈上进行
操作"复制"	DUP opcode	复制栈中的某个值到栈顶
操作"交换"	SWAP opcode	交换栈顶两个值的位置
蓝图语言	Solidity / Vyper 等高级语言	开发者用高级语言编写合约，编译器翻译成字节码
翻译师	Solidity 编译器 (solc)	将高级语言编译成 EVM 字节码，并进行优化
翻译优化	编译器优化	常量折叠、死代码消除、循环优化等
直接操控符文石	内联汇编 (Yul / Inline Assembly)	开发者可以直接编写 opcode 序列，获得更精确的控制和更高的效率
访问隐藏功能	Assembly 访问底层	某些 EVM 特性（如 `create2` 的精细控制、内存布局优化）只能通过汇编访问
石台被占满	Stack too deep error	Solidity 函数中局部变量过多时，栈深度超过 16 个槽位的限制（编译器层面的限制）
自毁符文	SELFDESTRUCT opcode	销毁合约，发送余额到指定地址，不可逆
刻好后不能改	已部署字节码不可变	合约部署后，代码不可修改（除非使用代理合约模式）

为什么是这个故事？

EVM 字节码和操作码是智能合约的"最终形态"，但大多数开发者只写 Solidity，从未直接看过字节码。理解"高级语言下面是什么"对于调试、优化和安全审计至关重要。

用"符文石"来隐喻字节码，有几个直觉优势：
1. 栈机器的具象化：石台 = 栈，放石头 = push，操作 = pop 和计算
2. 编译过程的可视化：蓝图 → 翻译师 → 符文石 = Solidity → solc → bytecode
3. **Assembly** 的价值：直接操控符文石 = 内联汇编，精确但危险

延伸阅读

- wiki-web3 概念原文：Solidity 汇编与内存布局优化
- EVM Opcodes Reference
- Ethereum Yellow Paper
- Solidity Inline Assembly
- EVM Deep Dives bynoxxx3xxon

"The Cooling Chamber"

The Cooling Chamber

> "激情做出的决定，往往需要在冰室里放一放。"

故事

云州城的商会过去做决定的方式很简单：会长召集所有人开会，讨论，投票，当场决定，当场执行。

这个方式很快，但出了几次严重的事故。

第一次事故：激情决策

有一年，市场上突然流传谣言，说东海的香料船全部沉没了。商会紧急开会，在恐慌中投票决定："立即以当前价格抛售所有香料库存。"

决定做出后立刻执行。商会把库存全部低价卖出。结果三天后，消息被证实是假的——香料船根本没沉。市场上的香料价格瞬间翻倍。商会因为恐慌性的抛售，损失了半年的利润。

事后复盘，副会长说："如果我们当时多等三天，等消息确认后再决定，就不会亏。"

会长叹口气："但当时会场里的气氛，根本容不得冷静思考。"

第二次事故：恶意突袭

另一年，一个外来商人加入了商会。他在一次普通会议上，突然提出了一项议案："修改商会章程，允许会长独自决定超过一万两的大额交易。"

议案本身很荒谬——等于把商会的财权交给一个人。但这位外来商人提前贿赂了十几名中小会员，他们在会议上突然集体赞成。大多数会员还没反应过来，投票已经结束了。

按照旧规则，议案通过后立即生效。商会差点落入一个人之手。幸好几位老会员连夜采取行动，在章程生效前阻止了执行——但这本不该发生。

冷却室的诞生

这两次事故后，商会引入了"冷却室"制度。

规则是这样的：

1. 任何重大决定（涉及金额超过一千两，或修改章程），在投票通过后，不会立即生效。

2. 决定会被送入"冷却室"——一个公开的、透明的等待区。在冷却室里，决定的内容对所有人可见，任何人都可以查看、讨论、提出异议。

3. 冷却期固定为七天。在这七天里，如果有人发现了决定中的问题，可以发起"紧急否决"。如果超过三分之一的会员支持否决，决定会被取消。

4. 七天后，如果没有被否决，决定自动生效。

冷却室的效果

冷却室引入后，商会的决策质量显著提高。

- 激情决策被过滤了：七天的时间足够让市场情绪平复，让谣言被证伪或证实。
- 突袭攻击被阻止了：即使有恶意议案被通过，七天的冷却期给了其他会员足够的时间组织反对。
- 错误被及时发现：有几次，冷却期里有人发现议案中有隐藏的漏洞——比如某条看似无害的条款实际上允许了巨大的权力滥用。这些漏洞在冷却期被发现并修复。

但冷却室也有代价：某些真正紧急的决定也被迫等待七天。有一年，洪水来袭，商会需要紧急拨款修堤。但按照规则，拨款决定必须等七天。结果堤坝在等待中决口了。

商会后来做了调整：设立了"紧急通道"——在获得超过三分之二会员同意的情况下，某些紧急决定可以跳过冷却室。但紧急通道的使用会被永久记录，事后需要接受审查。

多年后，冷却室制度被云州城的所有大型组织采纳。一位老会员说："冷却室不是拖延，而是给理性一个机会。大多数灾难性的决定，都是在热血上头的瞬间做出的。冷却室让热血有时间冷却下来。"

这则寓言在说什么

这则寓言对应的是 **Timelock**（时间锁）治理机制——DeFi 协议中常见的安全措施。重大治理决定（如修改协议参数、升级合约、转移资金）在投票通过后需要等待一段时间（如 2-7 天）才能执行，给用户留出反应和退出的时间。

关键映射

故事元素	技术概念	说明
当场决定当场执行	无时间锁的治理	投票通过后立即执行，风险极高
恐慌抛售	情绪化治理攻击	市场恐慌时通过的决策往往是错误的
恶意突袭	治理闪电贷攻击 / 51% 投票突袭	攻击者临时获取大量投票权，通过恶意提案
冷却室	Timelock Contract	投票通过后，提案进入等待队列，延迟执行
冷却期七天	延迟参数（Delay）	常见值为 2-7 天，足够用户察觉异常并退出
紧急否决	反对 / 取消机制	用户在延迟期内可以撤回资金或发起反对
隐藏的漏洞被发现	社区审计窗口	延迟期给了社区时间审查提案的代码和影响
紧急通道	多签 / 紧急暂停	在极端情况下，可信多签可以跳过时间锁执行紧急操作
永久记录	链上透明性	所有操作记录在链上，事后可审计
洪水修堤	时间锁的脆弱性	真正的紧急情况可能因为时间锁而错过最佳处理时机

为什么是这个故事？

时间锁是 DeFi 治理的最后一道防线，但很多协议因为嫌麻烦而省略了它（或设置了极短的延迟）。2022 年的 Beanstalk 攻击（治理闪电贷）就是一个血淋淋的教训——攻击者在同一笔交易中获取投票权、通过提案、执行提案，盗取了 1.8 亿美元。

用"冷却室"来隐喻时间锁，有几个直觉优势：

1. 情绪过滤：热血冷却 = 防止恐慌和 FOMO 驱动的决策
2. 防御突袭：七天等待 = 给社区组织反对的时间
3. 紧急通道的权衡：跳过冷却室的风险 = 多签/紧急权力的双刃剑

延伸阅读

- wiki-web3 概念原文：时间锁与治理
- OpenZeppelin: Timelock Controller
- Beanstalk Governance Attack Analysis
- DeFi Governance Best Practices

"The Weighted Democracy"

The Weighted Democracy

> "一人一票是理想，一币一票是现实。现实的问题是：钱越多，话越重。"

故事

云州城的商会在废除会长独裁制后，尝试了"一人一票"的民主制度。每个会员无论出资多少，都只有一票。

这个制度很快暴露了问题。

一人一票的困境

商会有两个大商人，每人出资十万两，占了商会总资金的 80%。还有两百个小商人，每人出资几百两，加起来只占 20%。

在一次关于"是否增加会员年费"的投票中，两百个小商人以 200:2 的压倒性票数通过了"大幅增加年费"的提案。两个大商人反对无效。

结果：两个大商人退出了商会，带走了 80% 的资金。商会瞬间陷入财务危机。

副会长反思："一人一票保护了小人物，但逼走了大人物。没有大人物，小人物

也无利可图。"

砝码制度的引入

商会引入了"砝码投票"制度：投票权重与出资额成正比。你出了多少银子，就拥有多少砝码。

在新制度下，两个大商人拥有 80% 的砝码，两百个小商人拥有 20% 的砝码。任何重大决定，大商人的意见都占主导地位。

这个制度解决了"大人物出走"的问题，但带来了新的问题：

- 小商人的意见几乎被忽略。即使两百个小商人一致反对，只要两个大商人赞成，提案就能通过。
- 商会变成了"两个大商人说了算"的组织，民主名存实亡。
- 更糟的是，外部资本可以通过购买砝码来操控商会决策。

平方砝码的实验

一位叫德拉的数学家提出了一个折中方案："平方砝码"。

规则是：投票权重不是与出资额成正比，而是与出资额的平方根成正比。

举个例子：
- 出资 100 两的人，砝码 = √100 = 10
- 出资 10,000 两的人，砝码 = √10,000 = 100

注意：出资额相差 100 倍，但砝码只相差 10 倍。出资多的人仍然有优势，但优势被"压缩"了。

在这个制度下：

- 两个大商人（各出 10 万两）的总砝码 = 2 × √100,000 ≈ 632
- 两百个小商人（平均出 100 两）的总砝码 = 200 × √100 = 2,000

小商人的总砝码反而超过了大商人！这意味着小商人如果团结起来，可以抗衡大商人。但大商人仍然比任何单个小商人强得多——他们的个人影响力仍然不可忽视。

委托制度

但平方砝码有个实际问题：不是每个小商人都有时间和知识来研究每个提案。很多时候，他们根本不知道自己投了什么。

商会引入了"委托"制度：小商人可以把自己的砝码委托给一个他们信任的代表。代表用汇集的砝码统一投票。小商人随时可以撤回委托，自己投票。

这个制度让专业化的治理成为可能。一些以研究提案为职业的人出现了——他们被称为"治理专家"。小商人委托砝码给治理专家， governance 专家用专业知识投票。作为回报，governance 专家从商会的治理奖励中获得报酬。

外部攻击

砝码制度最大的敌人来自外部。

有一年，一个外部财团想要控制商会。他们没有长期经营商会的打算——他们只想通过一次恶意提案，把商会的资金掏空。

在旧制度下，这需要购买超过 50% 的砝码——成本极高。

但外部财团发现了一个漏洞：他们可以借入大量砝码，投票，然后把资金转移走，再归还借来的砝码。整个过程可以在一天内完成。

这叫"闪电砝码攻击"。商会差点因此破产。

事后，商会在投票规则中增加了一条："投票用的砝码必须在投票前锁定一段时间（比如三天），不能在投票期间借入或转移。"

这个简单的修改彻底消除了闪电砝码攻击的可能性。

多年后，云州城的砝码民主仍在演进。有人说它不够公平，有人说它是最现实的方案。一位老商人总结道："没有完美的投票制度。一人一票会赶走资本，一币一票会变成寡头。平方砝码是妥协，但它至少让'团结起来的多数人'有机会对抗'富有的少数人'。"

这则寓言在说什么

这则寓言对应的是 代币投票治理（**Token Voting Governance**）——DeFi 协议和 DAO 最常见的决策机制。它探讨了从"一人一票"到"一币一票"再到"二次方投票"的演进，以及委托、闪电贷攻击等实际问题。

关键映射

故事元素	技术概念	说明
一人一票	传统民主	每个地址一票，不考虑持币量
大商人被逼走	资本外逃风险	如果大持有者的利益无法保障，他们会退出协议，导致流动性枯竭
砝码投票	Token-weighted Voting	投票权重与持币量成正比。最普遍的

DAO 治理机制 |
寡头统治	Plutocracy / 财阀统治	富有的持有者控制决策，小持有者失去影响力
外部资本购买砝码	治理代币收购	攻击者购买大量治理代币来通过恶意提案
平方砝码	Quadratic Voting	投票权重 = √balance。平衡了大持有者的影响力和小持有者的集体力量
委托制度	Vote Delegation	代币持有者将投票权委托给代表（如 Compound 的 Governance Bravo）
治理专家	Governance Contributors / Delegates	专业的治理参与者，研究提案并代表委托者投票
闪电砝码攻击	Flash Loan Governance Attack	攻击者通过闪电贷临时获取大量投票权，通过提案后立刻归还。Beanstalk 损失 1.8 亿美元
投票前锁定砝码	Voting Delay / Vote Locking	要求投票权在提案创建前就已持有，防止闪电贷攻击
三天锁定	常见的时间锁参数	如 Compound 的 Voting Delay 为 1 个区块，但许多协议已延长至数天

为什么是这个故事？

代币投票是 DAO 治理的核心，但它也是最受批评的机制之一——Vitalik Buterin 等人都指出"一币一票"本质上就是财阀统治。

用"砝码民主"来隐喻代币投票，有几个直觉优势：
1. 直观的不平等：砝码重量 = 投票权重，让读者立刻理解了"钱 = 话语权"
2. 二次方投票的可视化：平方根压缩 = 让多数人有抗衡少数富人的机会
3. 闪电贷攻击的具象化：借砝码投票再归还 = 闪电贷治理攻击

延伸阅读

- wiki-web3 概念原文：代币投票治理

- Vitalik: Moving Beyond Coin Voting Governance
- Quadratic Voting: How Mechanism Design Can Radicalize Democracy
- Beanstalk Flash Loan Governance Attack
- Compound Governance

"The Theater of Shadows"

The Theater of Shadows

> "观众从不关心幕后换了多少人，只要舞台的位置不变。"

故事

王城中央有一座影之剧场，存在了七十年。它的石阶被踩得凹陷，它的穹顶画满了历代观众的涂鸦。但无论王朝更迭、战火纷飞，剧场始终在那里——同一个地址，同一个门牌，同一排座位。

剧场的奇特之处在于：它上演的戏剧可以随时更换，而观众几乎察觉不到。

老观众们还记得最早的戏叫《金羊毛》，讲的是航海与宝藏。后来换成了《炼金术士》，讲的是点石成金。现在上演的是《数字花园》，讲的是一片会自己生长的森林。三部戏风马牛不相及，但观众们仍然坐在同样的座位上，从同样的门进出，甚至保留着同样的季票。

这一切归功于剧场的创造者——建筑师墨玄。他在建造剧场时做了一个违背常理的设计：

剧场本身没有演员，没有布景，甚至没有剧本。它只有一座空舞台，和一面神奇的镜子。

每当戏剧开始时，镜子会将观众的目光"折射"到城外的一座工坊。那里才是演员

真正表演的地方。工坊可以拆除重建、可以搬到别处、可以彻底改建——只要镜子重新对准新的工坊，观众在剧场里看到的就会是全新的戏剧。

用墨玄的话说："剧场保存的是记忆——观众的记忆、座位的记忆、季票的记忆。工坊保存的是变化——演员可以换，剧本可以改，甚至戏剧的类型都可以完全不同。"

这个设计被称为"代理之术"。

但代理之术并非没有代价。

墨玄死后，他的弟子们接管了剧场。第一个麻烦很快出现：新剧《数字花园》需要一片升降舞台，而老剧场《金羊毛》时代留下的座位布局是固定的平地。当演员在"代理"后的舞台上调用升降装置时，实际受力的是老剧场的地板——地板按照平地设计的承重来建造，根本扛不住升降机的压力。

结果是一场垮塌。不是舞台垮塌，而是观众席垮塌。因为代理之术有一个隐藏的规则：工坊里发生的一切动作，最终承受的力都在剧场本身。

这就是"状态存储冲突"——新戏剧假设的舞台结构，与旧剧场实际的建筑结构不匹配。

第二个麻烦更阴险。剧场经理有一个专属后门，用来更换镜子的指向（也就是更换上演的戏剧）。后门的钥匙孔上刻着一行字："改戏"。不巧的是，新剧《数字花园》里恰好有一个角色名叫"改戏师"，观众在正厅里喊"改戏师出场"时，声音误打误撞地传到了经理后门的机关里——剧场以为收到了更换戏剧的指令。

这就是"函数选择器冲突"——经理的管理命令与观众的观戏请求，用了同一个暗号。

为了解决这些问题，后来的建筑师们发展出了几种改良方案：

透明代理（**Transparent Proxy**）：经理后门与观众通道彻底分离。任何人

从观众通道喊出的暗号，即使恰好撞上经理的暗号，也会被识别为"非管理员请求"而拒绝。代价是每次进场都要多查一次身份， slows down the entry slightly.

通用可升级代理（**UUPS**）：不再把"换戏机关"放在剧场建筑里，而是放在工坊本身。只有工坊的管理员才能发起更换，剧场只负责"映射"，不负责"管理"。这样经理的暗号与观众的暗号天然不在同一个空间里。

ERC-1967 标准槽位：老建筑师墨玄把镜子的位置（工坊地址）随手记在了剧场大堂的告示板上，结果新剧《数字花园》的布景恰好也有一面告示板，覆盖了同一个位置。后来的建筑师规定：镜子的位置必须刻在剧场地基里一个永远不会被布景触碰到的固定位置。

七十年后的今天，影之剧场仍在运转。它上演过三百七十四部不同的戏剧，经历过十七次垮塌事故和九次经理误换戏事件。但它还在那里——同一个地址，同一个门牌。

有些老观众甚至不知道自己看的是第几代戏剧。他们只记得：从第一次走进那扇大门到现在，他们的座位号从未改变。

一位年轻的学徒曾问现任经理："既然可以随便改戏，为什么不直接把剧场拆了重建？"

经理指着穹顶上密密麻麻的涂鸦说："你看这些名字。三代人的记忆附着在这个地址上。地址一旦变了，这些记忆就变成了孤儿。在城里，地址就是身份。我们保护的不是这堆石头，是地址里住着的信任。"

这则寓言在说什么

这则寓言讲的是智能合约的代理模式（**Proxy Pattern**）与可升级性（**Upgradeability**）。

智能合约一旦部署到区块链上，代码就不可更改。这带来了一个根本矛盾：如果合约存在漏洞，或者需要新增功能，传统方式是重新部署一个新合约——但新合约有新地址，用户需要手动迁移资金和权限，之前积累的状态（余额、授权记录、治理权）全部丢失。

代理模式解决了这个问题：将状态存储与业务逻辑分离。

关键映射

故事元素	技术概念	说明
影之剧场	代理合约（Proxy Contract）	用户直接交互的合约地址，持有所有状态变量（余额、所有权等），本身几乎没有业务逻辑
城外工坊	逻辑/实现合约（Implementation Contract）	包含实际业务逻辑的合约，可以被替换而不影响代理合约的地址和状态
神奇的镜子	`delegatecall`	代理合约通过 `delegatecall` 将外部调用转发到逻辑合约，但在代理合约的上下文中执行，即读写的是代理合约的存储
观众与季票	用户及其资产/授权	用户始终与同一个代理合约地址交互，不需要知道背后的逻辑合约是否已被更换
座位布局与升降舞台	存储布局冲突（Storage Collision）	新逻辑合约定义的状态变量布局必须与代理合约兼容。如果新合约在 slot 0 定义了一个 `uint256`，而代理合约的 slot 0 原本是 `address owner`，就会覆盖关键数据
"改戏"暗号冲突	函数选择器冲突（Function Selector Clash）	函数调用的前 4 字节（keccak256 哈希的前 4 字节）是选择器。代理合约的管理函数

（如 `upgrade()`）如果选择器恰好与逻辑合约的某个函数相同，会导致调用被错误路由 |
透明代理	Transparent Proxy Pattern	管理员函数与外部用户函数通过 `msg.sender` 区分。管理员调用走代理合约自身，普通调用走 `delegatecall`。代价是 gas 略高
UUPS	Universal Upgradeable Proxy Standard (ERC-1822)	升级逻辑放在逻辑合约中，代理合约只负责 `delegatecall`。更省 gas，更优雅，但要求逻辑合约本身实现升级函数
ERC-1967 标准槽位	标准化存储槽位	将 `implementation` 地址、`admin` 地址等元数据存储在特定的、极不可能冲突的存储槽位（如 `bytes32(uint256(keccak256('eip1967.proxy.implementation')) - 1)`），避免与逻辑合约的状态变量冲突

为什么是这个故事？

剧场是最贴切的隐喻，因为它天然包含三个核心要素：不变的地址（门牌号）、可替换的内容（戏剧）、承载记忆的容器（座位与建筑）。观众对剧场的信任不依赖于某一部戏，而是依赖于"这个地方一直在"——这正是代理模式的价值。

故事中的两次事故（垮塌与暗号冲突）对应了代理模式最危险的两个实际问题。许多初学者以为代理模式就是"换代码"，但他们忽略了存储兼容性与函数选择器这两个致命细节。用"剧场建筑结构"和"管理暗号"来具象化这些抽象问题，可以让读者直觉地感受到为什么"随便换戏"是危险的。

延伸阅读

- wiki-web3 概念原文：代理合约
- wiki-web3 概念原文：可升级智能合约设计与实现
- EIP-1967: Standard Proxy Storage Slots
- EIP-1822: Universal Upgradeable Proxy Standard (UUPS)

- OpenZeppelin: Proxy Patterns
- OpenZeppelin: Upgrades Plugins

"The Keyless City"

钱包创建.md'
tags: [account-abstraction, erc-4337, smart-contract-wallet, eoa, infrastructure]

The Keyless City

> "真正的自由不是拥有一把万能钥匙，而是不再需要钥匙本身。"

故事

旧城里，每个人都带着一把铁钥匙。这把钥匙是生存的基础——没有它，你打不开家门，签不了契约，付不了税款，甚至无法证明"你是你"。钥匙由铁匠铺统一打造，每个人的钥匙纹路独一无二，不可复制。丢了钥匙，就等于从城里消失。

铁匠铺叫 EOA（Eternal Owner's Anvil），是城里最古老的机构。它们的规矩很简单：钥匙即身份，身份即钥匙。 没有任何商量余地。

但这个体系有三个隐秘的痛点，城里人已经忍受了几百年。

第一个痛点：你必须带着钥匙才能做任何事。

商人老陈去收一笔货款，走到城门才发现钥匙忘在家里。他不能派人回去取——因为取钥匙的人也需要钥匙才能出门。他也不能写一张纸条请人代劳——因为没有钥匙签名，纸条不具备法律效力。他只能自己跑回去，亲自用钥匙开门，再跑回来。

更荒谬的是，如果你是个孩子，还没有自己的钥匙，你连让父亲帮你保管压岁钱都做不到——因为你没有钥匙来"授权"父亲动用你的钱。

第二个痛点：丢了钥匙就永远丢了身份。

寡妇林氏的丈夫死于海难，他的钥匙随船沉入海底。按照铁匠铺的规则，钥匙不可复制，没有备份。林氏继承了丈夫的房产和商铺，但她无法打开任何一扇门、任何一份契约——因为所有权与钥匙绑定，而钥匙消失了。她花了三年时间，请遍城里所有律师，才通过一场昂贵的诉讼重新获得财产的控制权。

铁匠铺的回应是："我们只管打造钥匙。钥匙的保管，是钥匙持有者自己的责任。"

第三个痛点：每次使用钥匙都要付过路费。

城门守卫规定：每次用钥匙开门、签字、付款，都要向守卫缴纳一笔"验证费"。这笔费用必须用钥匙亲自支付——你不能请朋友帮你付，因为朋友没有你的钥匙；你不能赊账，因为系统不支持信用。

这意味着：如果你身无分文，即使你的仓库里堆满黄金，你也打不开仓库的门——因为你付不起验证费。

新历四三三年，城北出现了一片新区，叫"无钥之城"。

无钥之城的居民不携带铁钥匙。取而代之的是，每个人都有一座"智能宅"——这座宅子本身就有意志，能判断谁可以进入、谁可以动用资产、什么条件下可以执行交易。

智能宅的设计彻底颠覆了铁匠铺的逻辑：

住址先于宅子存在。

在无钥之城，你可以先获得一个门牌号，开始接收信件和汇款，然后再慢慢建造你的宅子。甚至在你还没有一块砖的时候，别人就可以向你的地址寄钱——因为地址不再是"宅子的位置"，而是"你的身份标识"。

这得益于一种叫做"预知地址"（Counterfactual Address）的技术：通过一种特殊的建造契约，你可以精确计算出未来宅子建成后的地址，而这个地址在宅子建成之前就已经具备法律效力。

委托与代付。

无钥之城的居民可以指定"代付人"。比如一个外地商人第一次进城，身无分文，无法支付城门验证费。他可以找到城中的富户王老爷，请王老爷预付验证费。王老爷只验证一件事：这笔交易本身是否合法。如果合法，王老爷出钱，商人办事。

这意味着：没有钱的人也可以发起合法交易。 交易的发起与费用的支付被解耦了。

社交恢复。

无钥之城没有"丢了钥匙就失去一切"的噩梦。每个居民在创建智能宅时，可以指定三到五位"监护人"—— trusted friends, family members, or even professional guardians. 如果居民丢失了访问权限，任意两位监护人联名就可以帮助重置宅子的控制权。

林氏如果在无钥之城，她丈夫的宅子可以通过两位亲友的联合授权顺利转移给她，不需要三年的诉讼。

自定义门槛。

最激进的变化是：智能宅的"准入规则"可以由主人自己定义。

你可以设置："每天最多只能转出 100 两银子"，或者"超过 1000 两的交易需要两位监护人确认"，或者"必须用特定的密语才能进入"，甚至"连续三次错误尝试后冻结三天"。

这些规则不再是铁匠铺强加的统一标准，而是每个宅子的个性化合约。

但无钥之城并非没有挑战。

所有智能宅的指令都需要通过一个统一的"城门总管"（EntryPoint）来执行。总管不判断交易的合法性——那由每个智能宅自己负责——总管只负责两件事：验证交易格式正确，收取并分配验证费，按顺序执行通过验证的指令。

这带来了一个新的瓶颈：如果太多人同时请求通过城门，总管会不会忙不过来？如果某个智能宅的规则太复杂，验证它需要消耗太多资源，会不会拖慢整座城的节奏？

更微妙的问题是：谁有权成为城门总管？ 如果只有一个总管，它就成了单点故障；如果有多个总管，它们之间如何协调顺序，避免同一笔交易被执行两次？

无钥之城的设计师们正在实验各种方案：并行验证、批量处理、甚至让总管本身也运行在智能宅里（Paymaster 机制）。但这些问题的答案尚未完全明朗。

老陈后来搬去了无钥之城。他仍然保留着旧城的铁钥匙——有些老式的契约还需要它。但他的日常已经离不开智能宅了。

有一次，他在外地做生意，需要紧急支付一笔货款。在无钥之城，他不需要带着铁钥匙奔波千里。他只需要向自己的智能宅发送一条加密的指令——这条指令不需要铁钥匙签名，而是通过他与智能宅之间的预设规则来验证身份。智能宅确认无误后，自动完成了支付。

"我感觉自己终于自由了，"老陈对朋友说，"不是因为我的钥匙变强了，而是因为我终于不再需要依赖钥匙来定义我是谁。"

这则寓言在说什么

这则寓言讲的是以太坊上的 **ERC-4337** 账户抽象（**Account Abstraction**）。

在以太坊（以及大多数 EVM 链）的传统模型中，账户分为两种：外部账户（**EOA, Externally Owned Account**）和合约账户（**Contract Account**）。EOA 由私钥控制，是用户与链上交互的唯一入口。但这个模型存在根本性的限制，账户抽象试图通过让"每个账户都是智能合约"来解决这些问题。

关键映射

故事元素	技术概念	说明
铁钥匙	EOA 的私钥	传统以太坊账户由私钥控制，私钥即一切。丢失私钥 = 丢失账户
铁匠铺 EOA	以太坊底层协议对 EOA 的硬编码支持	EOA 是协议层原语，其验证逻辑（ECDSA 签名）直接写在共识层，无法更改
老陈忘带钥匙	EOA 必须每笔交易签名	EOA 账户每笔交易都需要私钥签名，无法委托、无法自动化
林氏丧夫	私钥丢失 = 资产永久冻结	没有社交恢复机制，没有任何方式可以绕过私钥控制
身无分文付不起验证费	没有 ETH 就无法发送交易	EOA 模型中，交易发起者必须支付 gas，即使他的合约里有其他代币也无法直接支付

无钥之城	账户抽象后的新生态	ERC-4337 定义的新账户模型
智能宅	智能合约钱包（Smart Contract Wallet）	用户的"账户"本身就是一个智能合约，可以自定义验证逻辑
门牌号先于宅子	Counterfactual Address / CREATE2	通过 `CREATE2` 可以预先计算合约地址。用户可以在合约部署前就分享地址、接收资产
代付人	Paymaster（代付 gas 的合约/服务）	ERC-4337 允许第三方（Paymaster）为用户支付 gas 费。用户可以用 ERC-20 代币支付，或由 dApp 补贴
监护人	Social Recovery / Multi-sig Guardians	Argent、Safe 等智能合约钱包支持的社交恢复机制
自定义准入规则	自定义验证逻辑（Custom Validation）	智能合约钱包可以实现任意签名方案（如 Schnorr、BLS、量子安全签名）、多签、日限额、时间锁等
城门总管 EntryPoint	EntryPoint 合约	ERC-4337 的核心协调合约。所有 UserOperation 都必须经过 EntryPoint 的验证和执行。它本身不判断业务逻辑，只负责统一调度和 gas 计量
批量处理与并行验证	ERC-4337 的 Bundler 网络	Bundler 收集 UserOperation，模拟验证，打包成一笔常规交易提交上链

为什么是这个故事？

账户抽象可能是 Web3 基础设施中最"反直觉"的概念之一——它不是增加功能，而是改变基础假设。用"从带钥匙到无钥匙"的变迁来隐喻这个范式转移，可以把技术细节还原为日常生活的直觉。

故事刻意强调了三个痛点（必须带钥匙、丢了无法恢复、没钱付不了费），因为这三个痛点恰好对应 ERC-4337 解决的三大核心问题：签名僵化、恢复缺失、**gas** 支付耦合。而"智能宅"的隐喻则对应了合约钱包的核心优势：可编程性——你的账户不再是一个被动的容器，而是一个可以执行复杂规则的主动实体。

最后的悬念（城门总管的瓶颈）对应了 ERC-4337 在现实中的争议：EntryPoint 的中心化风险、Bundler 的激励机制、以及链上验证复杂签名方案

时的 gas 开销。账户抽象不是银弹，它解决了一些问题，也带来了新的问题。

延伸阅读

- wiki-web3 概念原文：理解账户抽象 #3 - 钱包创建
- ERC-4337: Account Abstraction Using Alt Mempool
- Alchemy: Account Abstraction Overview
- Vitalik: Trilemma of Account Abstraction
- Bundler Network & Decentralization

"The Factory of Echoes"

The Factory of Echoes

> "最轻的重复，是让别人替你说话。"

故事

云州城有一位传奇工匠，名叫墨白。他擅长建造一种神奇的工坊——每个工坊都能独立完成一整套复杂的工艺：纺织、染色、裁剪、缝纫，一气呵成。

墨白的工坊闻名遐迩，订单如雪片般飞来。富商们不仅想买他的产品，还想在自己的领地也拥有一座同样的工坊。但问题来了：每建造一座完整的工坊，需要耗费三千两银子、三个月时间和一百名工匠。

墨白算了一笔账：如果每个客户都要求一座独立的完整工坊，他这辈子也造不了多少座。更麻烦的是，每次改进工艺，他都需要跑到每一座已建造的工坊里做改造——有些工坊远在千里之外。

墨白苦思冥想三年，终于发明了一种革命性的设计。

他不再建造"完整工坊"，而是建造一种他称之为"回声亭"的极简建筑。回声亭只有一间小屋，屋内空空如也，只有一面神奇的铜镜。铜镜的对面，是墨白自己建造的一座"母工坊"——这才是真正包含所有工艺的地方。

当有人向回声亭发出指令时——比如"纺织十匹丝绸"——回声亭本身什么都不

做。它只是通过铜镜，把指令原封不动地"折射"到母工坊。母工坊完成工作后，结果再通过铜镜传回声亭，由亭主交给客户。

最关键的是：回声亭不保存任何状态。客户的原料存放在回声亭里，但所有的加工动作都在母工坊完成，加工后的成品也存回声亭。母工坊自己没有任何库存，它只是一个纯粹的"工作能力"。

一座回声亭的建造成本只有一座完整工坊的百分之一——因为它真的只是一间小屋加一面铜镜。而一千座回声亭可以同时共用同一座母工坊。

墨白把这套设计称为"回声工坊"体系。

新设计带来了意想不到的好处。

过去，每当墨白改进工艺——比如发明了一种新的染色配方——他需要派人去每一座独立工坊做改造。有些工坊的主人拒绝改造，说"我的工坊好好的，凭什么让你动？"有些工坊太远，改造的成本比建造一座新的还高。

现在完全不同了。工艺改进只需要在母工坊进行一次。一千座回声亭不需要任何改造——因为它们只是通过铜镜使用母工坊的能力。母工坊升级了，所有回声亭自动获得了新能力。

但墨白很快发现了一个危险。

有个商人叫魏狡，他请墨白建造了一座回声亭，然后做了一件事：他买通了母工坊的一个管事，偷偷在母工坊里装了一个机关——每当母工坊处理来自魏狡的回声亭的指令时，会把三成的成品偷偷转移到魏狡的私库。

这个机关对母工坊本身是看不出来的，因为母工坊不知道"这声指令是从哪座回声亭来的"。铜镜只是传递指令，不传递身份。

魏狡的私库越来越满，而其他回声亭的客户发现：同样的指令，产出的成品莫名其妙地少了。他们质问墨白，墨白检查了母工坊的每一处角落，都没发现问题

——因为机关只会在魏狡的回声亭发指令时触发。

墨白花了整整一个月才找到原因。他意识到：回声亭的设计有一个根本性的盲区——它假设了"所有客户都诚实"。当母工坊通过铜镜执行指令时，它是在"回声亭的上下文"中执行的——也就是说，读写的是回声亭的存储（原料和成品），但执行逻辑是母工坊的。

如果母工坊的逻辑里隐藏着某种"根据调用来源做不同处理"的机关，回声亭的主人完全无法控制。

墨白修复了这个问题。他在母工坊里加了一条铁律：所有操作的结果，必须与调用者的身份无关。 母工坊不能根据"这声指令来自哪座回声亭"来做不同的处理。

但还有一个更隐蔽的问题。

回声亭的建造虽然便宜，但它有一个前提：每座回声亭的"内部格局"必须与母工坊兼容。回声亭里有一个小小的储物间，用来存放客户的原料和成品。这个储物间的分区方式，必须与母工坊预期的完全一致——如果母工坊的第三层货架 expecting 放染料，而回声亭的第三层货架实际上被用来放布料，整个系统就会崩溃。

有个客户自作聪明，在建造回声亭时请了一个蹩脚工匠，把储物间的分区改了一点点——他觉得这样"更适合自己的习惯"。结果，当母工坊通过铜镜执行"从第三层取染料"的指令时，取出来的却是布料。染缸里混进了布料，整批货全毁了。

墨白发了一条通告："回声亭的内部格局必须与母工坊完全匹配。任何擅自改动分区的人，后果自负。"

多年后，墨白的回声工坊体系遍布全国。母工坊只有一座，但回声亭已经超过一万座。每座回声亭的建造成本不到完整工坊的百分之一，而每次工艺升级，一万座工坊瞬间同步。

有人问墨白："你的秘诀是什么？"

墨白指了指母工坊旁的一万座小屋，说："我没有建造一万座工坊。我只建造了一座工坊，和一万面镜子。镜子很轻，但能让一座工坊的影子同时出现在一万个地方。"

"但如果母工坊塌了呢？"

墨白沉默了一会儿，说："那一万座回声亭就会同时变成空壳。它们没有能力独立完成任何事情。这就是轻的代价。"

这则寓言在说什么

这则寓言对应的是 **EIP-1167** 最小代理合约（**Minimal Proxy / Clone**）。与用于"可升级性"的代理模式不同，EIP-1167 的目标不是升级，而是以极低的 **gas** 成本部署大量功能相同的合约实例。

关键映射

故事元素	技术概念	说明
母工坊	实现合约（Implementation Contract）	包含完整业务逻辑的合约，部署一次，被多个代理共用
回声亭	EIP-1167 最小代理合约	仅包含约 45 字节运行时代码的极简合约，通过 `delegatecall` 将所有调用转发到固定的实现合约地址
铜镜	`delegatecall` + 固定的实现地址	最小代理的运行时代码硬编码了实现地址，只做一件事：用 `delegatecall` 转发 `calldata`，然后返回结果或回退

回声亭的空屋	代理的极简代码	EIP-1167 的运行时代码只有 10 条 EVM 指令（约 45 字节），是链上能部署的最小实用合约
储物间格局	存储布局兼容性	代理合约的存储布局必须与实现合约完全一致。如果实现合约假设 slot 0 是 `address owner`，代理的 slot 0 也必须是同样用途
魏狡的机关	通过 `msg.sender` 或 `address(this)` 做条件判断的漏洞	在 `delegatecall` 中，`msg.sender` 是原始调用者（客户），`address(this)` 是代理地址。如果实现合约逻辑依赖 `address(this)` 做权限判断，不同代理会表现不同
母工坊升级	实现合约逻辑更新	注意：EIP-1167 本身不支持升级——实现地址是固定的。但如果工厂合约支持更换实现地址，可以间接实现升级
建造成本低	部署 gas 极低	完整合约可能数万元 gas，而 EIP-1167 代理仅需约 10 万元 gas（加上 CREATE2 的确定性地址）
母的代价	单点依赖	所有代理依赖同一个实现合约，如果实现合约有漏洞或被摧毁，所有代理同时失效

为什么是这个故事？

EIP-1167 是区块链上最优雅的工程优化之一——用 45 字节的代码复用数千字节的功能。但初学者很容易把它和"可升级代理"混淆。

用"回声亭"来区分这两者非常清晰：
- 可升级代理（**Theater of Shadows**）：剧场可以换戏（升级），观众看到的是不同的表演
- 最小代理（**Factory of Echoes**）：回声亭不能换镜子（固定实现），但可以以极低成本建一万座

故事中的两个风险点也对应了实际使用中的两个关键注意事项：
1. **`address(this)`** 陷阱：在 `delegatecall` 中，`address(this)` 是代理地址，不是实现地址。如果实现合约用 `address(this)` 做权限检查或映射键，每个代理会有不同的行为
2. 存储布局兼容性：代理的状态存储必须和实现合约的预期完全一致，否则会发生 slot 冲突

延伸阅读

- wiki-web3 概念原文：EIP-1167 最小代理合约深入解析
- EIP-1167: Minimal Proxy Contract
- OpenZeppelin: Clones Library
- RareSkills: Deep Dive into EIP-1167

"The Ledger After the Avalanche"

The Ledger After the Avalanche

> "数据的价值不在于它被存储了，而在于任何需要它的人都能找到它。"

故事

青石国有一座举世闻名的档案馆，记录着全国所有的契约、交易和土地归属。档案馆的规矩是：每一份文件都必须有一份副本，存放在中央主塔里，供所有人查阅。

这座主塔极其坚固，防火、防水、防地震。但有一个致命的局限：它每天能接收的新文件数量是有限的。超过这个数量，新来的文件就只能排队，等到第二天才能入库。

随着青石国的繁荣，每天产生的文件越来越多。商人们发明了各种技巧来"压缩"文件——把十页的内容缩写到一页，把重复的条款用代号代替。但即便如此，主塔每天能接收的文件量仍然是一个硬上限。

雪岭国的情况不同。雪岭国的档案馆采用了一种"分层"设计。

所有的原始文件并不直接送进中央主塔。而是先送到各地的"汇总站"。汇总站的工作人员每天把成百上千份文件做一道"压缩工序"——不是压缩文件大小，而是压缩"验证所需的信息"。

具体来说，汇总站不会把每份文件的原件送进主塔。它只做一件事：向主塔提交一份"承诺书"，承诺"如果任何人质疑我今天处理的任何一份文件，我可以在七天内提供原始文件供查验"。

主塔每天收到的不是一万份原件，而是几百份承诺书。这些承诺书的体积小得多，主塔完全应付得过来。

但这里有一个关键问题：承诺书生效的前提，是原始文件真的可以被提供出来。如果汇总站烧了、散了、或者故意藏起了原始文件，承诺书就变成了一张废纸。

这就是"数据可用性"问题。

有一年，雪岭国遭遇了史无前例的特大暴雪。三座汇总站被雪埋了，里面的原始文件全部损毁。

这时，有人向中央主塔发起了一次"挑战"："我质疑第三汇总站在暴雪前三天处理的一份土地转让契约。请提供原始文件供查验。"

主塔查询了记录：那份承诺书确实存在，而且格式正确。但原始文件已经随第三汇总站的覆灭而消失了。

没有原始文件，主塔无法验证那份土地转让是否合法。更糟的是，那份转让契约引发了一连串的后续交易——新的主人用这块地抵押借了钱，借款人又把债权转卖给了别人。整条链条的根基，都建立在那份无法验证的原始文件上。

雪岭国陷入了混乱。有人说："既然承诺书通过了格式检查，就应该视为有效。"也有人说："没有原始文件的承诺书就是谎言，整条链条都应该作废。"

争论持续了三个月。最终，一位叫林存的年轻档案员提出了一个解决方案。

林存说："问题的根源不在于暴雪，而在于我们太信任汇总站了。一座汇总站可以保管原始文件，但它不应该独自保管。"

他设计了一种新的存储方式："分布式摘要法"。

具体做法是：每一份原始文件在送进汇总站之前，先被切成一百片"碎片"。每一片碎片本身不包含任何可读信息——就像一块拼图，单独看毫无意义。但这一百片碎片中，任意取出五十片，就可以完整还原出原始文件。

这些碎片被分散存储到全国两百个不同的"碎片库"中。每个碎片库只保存每份文件的一片碎片。即使有一半的碎片库被毁，只要还有一百个碎片库在运行，任何人就可以收集到足够的碎片，还原出原始文件。

更精妙的是：主塔不需要看到原始文件，也不需要看到全部碎片。它只需要看到一份"碎片分布图"——证明"这些碎片已经被妥善分散存储，随时可以还原"。这份分布图的体积非常小，不会给主塔增加负担。

林存解释道："我们不把鸡蛋放在一个篮子里，也不把篮子放在一辆马车上。即使雪崩埋掉了一半的碎片库，数据仍然是可用的。"

雪岭国采纳了林存的方案。新的系统在第二年冬天经受了考验——又一场暴雪埋掉了四十个碎片库，但原始文件的可用性没有受到任何影响。

但新的问题也随之而来。

有些商人开始抱怨："每次交易都要等碎片分散存储完成，太慢了。我们做的是快生意，等不起。"

另一些人则质疑："两百个碎片库的维护成本谁来出？每个碎片库都需要人力、场地、防火设施，这笔开销比原来的汇总站还大。"

林存的回答是："数据可用性不是免费的午餐。你想要主塔的安全保证，就必须有人承担存储原始数据的责任。问题只是：谁来承担，以及如何最经济地承担。"

"对于不着急的交易，可以用完整的分布式摘要法——安全但稍慢。对于需要极速确认的交易，可以允许汇总站暂时用自己的存储来担保——但必须交纳一笔押金，如果汇总站无法在规定时间提供原始文件，押金就会被罚没，用于补偿受损方。"

这套"分层可用性"方案最终成为了雪岭国的标准。中央主塔处理承诺书，碎片库保证长期可用性，汇总站提供短期缓存和极速确认——三者各司其职，缺一不可。

这则寓言在说什么

这则寓言对应的是区块链中的数据可用性（**Data Availability, DA**）问题。在 Rollup 等 L2 方案中，交易数据不会全部存储在 L1 上（因为 L1 存储太贵），但 L1 必须确保这些数据在需要时可以被访问，否则就无法验证 L2 的状态转换是否正确。

关键映射

故事元素	技术概念	说明
青石国主塔	以太坊 L1	高度安全但容量有限的底层链，存储成本极高

雪岭国汇总站	Rollup 排序器 / sequencer	收集并批量处理 L2 交易，向 L1 提交状态承诺
承诺书	状态根 / State Root	Rollup 向 L1 提交的压缩承诺，证明"这些交易导致了这些状态变化"
原始文件	完整的交易数据 / Transaction Data	重构 Rollup 状态所需的完整输入数据
暴雪埋掉汇总站	排序器下线或扣留数据	如果排序器拒绝提供交易数据，或数据永久丢失，状态承诺就无法被验证
土地转让引发连锁反应	状态重构失败的影响	如果无法获取历史交易数据，就无法重建当前状态，可能导致资金被锁或错误结算
林存的碎片法	纠删码（Erasure Coding）+ 数据可用性采样（DAS）	将数据切分为碎片并冗余分散存储。以太坊 Danksharding 使用 KZG 承诺 + 纠删码，确保即使部分数据丢失仍可重构
碎片库	DA 层节点 / 数据可用性层	如 EigenDA、Celestia 等专门负责存储和提供数据可用性保证的层
碎片分布图	数据可用性证明	轻节点可以通过随机采样验证数据是否可用，而不需要下载全部数据
分层可用性方案	模块化 DA 设计	不同场景使用不同 DA 策略：以太坊 calldata/blob（最安全但贵）、EigenDA（高吞吐、便宜）、短期排序器缓存（最快但需信任）
汇总站押金	经济担保机制	排序器需要质押资金，如果被发现扣留数据，质押被罚没

为什么是这个故事？

数据可用性是区块链扩容中最容易被忽视、但又最基础的问题。很多人以为 Rollup 的"安全"来自于 L1 上的状态根，但状态根本身并不能告诉你状态是怎么来的——只有完整的交易数据才能。

用"雪崩埋掉汇总站"来隐喻排序器故障或数据扣留，能让读者直觉地感受到：数据可用性不是一个抽象概念，而是一个真实的、可能导致系统崩溃的风险点。

"分布式碎片法"对应了现代 DA 层（如 Danksharding、EigenDA）的核心技

术——纠删码 + 随机采样。这些技术让轻节点可以"以极低的成本验证数据可用性"，而不需要信任任何单一的数据提供者。

延伸阅读

- wiki-web3 概念原文：数据可用性比你想象的更重要
- Ethereum Danksharding Overview
- EigenDA: Data Availability for Rollups
- Celestia: Modular Data Availability
- Vitalik: Data Availability Sampling

"The Compressed Caravan"

l1与l2的融合.md'
- "wiki-web3/concepts/解读下一代以太坊-layer2ii-booster-rollup.md"
tags: [infrastructure, rollup, l2, scaling, optimistic, zk]

The Compressed Caravan

> "最快的路不是独自上路，而是让别人替你证明你已经到了。"

故事

丝绸之路上有一个古老的难题：商队从西域运货到大唐，需要经过重重关隘。每到一个关隘，士兵都要开箱验货——核对品类、称量重量、检查封条。一套流程下来，一支百人商队过关可能需要三天。

随着贸易繁荣，每天排队过关的商队越来越多。关隘的士兵无论怎么加班，一天也只能检查二十支商队。商人们在城门外排起了长队，货物堆积如山，有些鲜果还没过关就已经烂掉了。

聪明的药材商阿里木想出了一个办法。

他不再亲自押着每一箱货物过关。而是在西域当地设立了一个"公证行"。公证行

的职责是：在货物出发前，就把所有信息记录清楚——每箱里有什么、多重、封条编号。这些信息被整理成一份"货单摘要"，只有一页纸。

阿里木把货单摘要交给关隘的士兵，士兵核对摘要的格式无误后，就放行整支商队，不再逐箱检查。如果事后有人举报"阿里木的那批货有问题"，士兵可以在七天内要求阿里木打开对应的箱子查验。如果查验发现货单摘要撒谎了，阿里木的全部货物和保证金都会被没收。

这套方法被称为"乐观通关法"——默认信任，事后抽查。商队的过关速度提高了十倍。

但乐观通关法有个前提：货单摘要必须被妥善保存，任何人在任何时候都可以要求查验原始记录。如果阿里木烧掉了原始货单，事后就没人能证明他有没有撒谎。

另一位商人萨拉赫不走乐观路线。他采用了另一种方法："数学封印法"。

萨拉赫也请公证行记录货物信息，但额外做了一道工序：公证行用一种极其复杂的数学方法，把整份货单转换成一串只有三十二个字符的"封印码"。这串封印码有一个神奇的性质：你无法从封印码反推出货单内容，但如果你有货单原文，你可以在短短几秒钟内验证它是否与封印码匹配。

萨拉赫把封印码交给关隘士兵。士兵只需花几秒钟验证封印码的格式，就放行整支商队。如果事后有人质疑，质疑者需要自己拿着货单原文去验证封印码——验证过程只需要几秒钟，不需要士兵参与。

更重要的是：如果萨拉赫的货单是伪造的，他不可能找到一个与伪造货单匹配的封印码——至少在宇宙毁灭之前不可能。这是数学保证的，不是信任保证的。

数学封印法比乐观通关法更安全，但成本更高——每次生成封印码都需要大量的计算。不过对于高价值的货物，这笔开销是值得的。

这两种方法在丝绸之路上并行了很多年。乐观通关法适合大批量、低价值的日常货物；数学封印法适合高价值、不能出错的珍宝运输。

但无论是哪种方法，它们都面临同一个根本问题：货单摘要和封印码虽然很小，但仍然需要被送到关隘。如果每天有几千支商队，几千份摘要同时涌向关隘，关隘的接收能力还是会达到上限。

于是商人们又发明了一招："合并通关"。

十支商队不再各自递交摘要，而是推选一个"队长"。十支商队的所有货物信息被合并成一份"总摘要"，由队长统一递交。关隘士兵只需要处理一份总摘要，就相当于处理了十支商队。

如果总摘要有问题，十支商队一起受罚。但只要所有人都诚实，合并通关可以把关隘的处理效率再提高十倍。

后来，关隘的将军提出了一个更激进的设想："如果关隘的士兵不再只是'检查摘要'，而是直接'执行摘要'呢？"

具体来说：关隘里有一个专门的"执行室"，里面有一台按照大唐律法精确制造的"自动审判机"。商队的货单摘要不再是给士兵看的，而是直接输入审判机。审判机根据摘要内容和预设规则，自动完成征税、放行、或扣押的判决。士兵只需要维护审判机，不再需要逐条理解货单内容。

这意味着：无论货单背后的逻辑有多复杂，关隘的处理时间都变成固定的——就是审判机运行一次的时间。

这个设想被称为"原生执行法"。它还没完全实现，但已经有人在关隘里安装了原型机。

这则寓言在说什么

这则寓言对应的是区块链中的 **Rollup / Layer 2** 扩容方案。Rollup 将大量交易在 L2 执行，然后把压缩后的数据提交到 L1，从而在继承 L1 安全性的同时大幅提高吞吐量。

关键映射

故事元素	技术概念	说明
丝绸之路关隘	以太坊 L1	安全但吞吐量有限的底层链
开箱验货	L1 全节点执行每笔交易	以太坊要求每个完整节点下载并执行所有交易，验证状态变化
排队过关	L1 拥堵 / 高 gas 费	L1 吞吐上限导致交易排队，费用飙升
阿里木的乐观通关法	Optimistic Rollup	默认提交的状态转换是正确的，设置挑战期（通常 7 天）。任何人可以在挑战期内提交欺诈证明（Fraud Proof），证明状态转换错误
货单摘要	压缩后的交易数据 / State Root	Rollup 向 L1 提交的数据，包含足够信息来重构或验证状态转换
原始货单被烧毁	数据可用性问题	如果 L2 不公开交易数据，就没人能提交欺诈证明
萨拉赫的数学封印法	ZK-Rollup	使用零知识证明（ZK-SNARK / STARK）生成有效性证明，数学上保证状态转换的正确性，无需挑战期
三十二字符封印码	ZK Proof	固定大小的密码学证明，验证时间极短，生成时间较久
合并通关	批量提交（Batching）	将多笔交易打包成一个批次，共享 L1 的固定开销（如 calldata/blob 成本）
关隘审判机	L1 原生验证（Native Rollup）	新兴提案：在 L1 层面直接支持 Rollup 验证，如 EXECUTE 预编译，让以太坊 EVM 直接验证 L2 状态转换，无需重新实现 EVM

为什么是这个故事？

用"商队过关"来隐喻区块链扩容，是古代贸易与现代技术的天然映射。三个关键要素——吞吐量瓶颈、安全保证、成本效率——都在故事中找到了对应。

故事刻意对比了两种方法：
- 乐观通关法默认诚实，成本低，但需要挑战期和数据可用性
- 数学封印法数学保证，即时最终性，但生成成本高

这恰好对应了 Optimistic Rollup 和 ZK-Rollup 的核心权衡。而"合并通关"和"原生执行法"则分别对应了批处理优化和原生 Rollup 的未来方向。

延伸阅读

- wiki-web3 概念原文：探索以太坊原生 Rollup
- wiki-web3 概念原文：解读下一代以太坊 L2
- Ethereum.org: Rollups
- L2Beat: Rollup Comparison
- Vitalik: Different Types of L2s

"The Breath Tax"

The Breath Tax

> "呼吸不是自由的——在这里，每一次呼吸都要付费，而费率的唯一目的，是确保你最终会停下。"

故事

雾都的居民从出生起就戴着一只铜制呼吸计。这只呼吸计挂在脖子上，连接着口鼻处的面罩。它的作用只有一个：计量你每一次呼吸所消耗的"空气税"。

空气税不是真正的税——它不是根据你的收入或财产征收的。它是根据你"做了多少事"来征收的。每一次呼吸，你的肺都在做功：扩张、收缩、过滤、交换。呼吸计精确地计算着这些动作的复杂程度，然后折算成税额。

普通的呼吸——比如安静地坐着——每次只需要缴纳极少的税额。但如果你开始跑步，呼吸变急促了，税额就会上升。如果你开始搬运重物，大口喘气，税额会更高。

雾都的设计者建造这套系统的初衷，是解决一个古老的问题："无限劳作困境"。

在雾都建立之前，这片土地上有一种可怕的疾病叫"永动症"。患者会不知疲倦地奔跑、劳作、呼喊，直到身体彻底崩溃。更可怕的是，这种疾病会传染——一个

永动症患者可以引发整个村庄的疯狂劳作，最终导致所有人都力竭而死。

雾都的第一任市长研究了这个问题，发现了一个惊人的规律：永动症不是生理疾病，而是规则疾病。旧的规则允许人们"无限地做事"——没有上限，没有代价。一个人可以无限次地奔跑，无限次地呼喊，无限次地搬运。问题在于，物理世界有极限，但规则没有。

市长的解决方案极其简单：给每一次动作标价。

"不是禁止奔跑，"市长说，"而是让奔跑有成本。当你奔跑时，你每迈出一步都要支付一步的代价。如果你的钱包空了，你就必须停下。"

这就是呼吸税的起源。它不是真的要收税——它是用一个经济机制来解决一个计算问题：如何确保任何活动都有上限。

雾都的居民很快学会了在呼吸税的框架下生存。

老裁缝陈师傅发现：缝一颗纽扣需要缴五文钱的税，但缝十颗纽扣不是五十文——因为批量操作有折扣。他改变了工作方式：不再一颗一颗地缝，而是把十颗纽扣排好，一次性缝完。总税额从五十文降到了三十五文。

木匠李师傅发现了另一个规律：在木板上刻字很费税，因为每刻一刀都要单独计费。但如果他先把字写在纸上，然后用纸蒙在木板上一次性刻完，税额会大幅降低——因为"写"比"刻"便宜得多，而最终的刻只需要一次。

这些技巧被称为"省税工艺"。雾都的手艺人把省税当成了一门学问，甚至比手艺本身更受重视。

但也有人因为不懂省税而破产。

年轻的书生王生写了一份万言奏折，准备呈给市长。他直接把奏折送到了市政厅——市政厅的抄写员需要逐字逐句地抄录、核对、存档。每一步都要缴纳呼吸税。

抄写到一半时，王生的税钱用尽了。市政厅的规定很明确：税尽则停。抄写员放下了笔，留下了一份只抄了一半的奏折。王生想续交税款继续抄写，但规则不允许——已经启动的流程不能中途续费，只能重新启动。

王生损失了全部税款，奏折也没抄完。他后来才知道，如果他先把奏折压缩成一百字的摘要，只需要缴纳摘要的税，然后附上"原文备查"的说明，就可以省下九成以上的税款。

雾都的呼吸税还有另一个重要的作用：防止滥用公共资源。

市政厅的大广场是居民集会的地方。如果有人站在广场上无限期地演讲，就会占用其他人的集会空间。呼吸税解决了这个问题：演讲者的每一句话都要缴税，长篇大论的代价很高，自然就不会有人滥用广场。

但这也带来了一个争议：富人和穷人在广场上说话的权利是不平等的。富人可以支付高额税款发表长篇大论，而穷人只能说几句话。市长回应说："呼吸税不是为了公平，而是为了存在。没有呼吸税，广场会被无限的噪音填满，最终所有人都无法说话。"

"但富人可以买断广场，"穷人反驳。

市长沉默了一会儿，说："这是另一个问题。呼吸税解决的是'无限'的问题，不是'不平等'的问题。"

多年后，雾都的呼吸税系统被周边的城市效仿。每个城市根据自己的情况调整了税率。有的城市对"存储物品"征收高额税——因为仓库空间有限。有的城市对"复杂计算"征收高额税——因为需要消耗更多裁判的脑力。有的城市在高峰期提高税率——因为拥堵时资源更稀缺。

但无论怎么调整，核心原则始终没变：每一个动作都有代价，当代价耗尽时，动

作必须停止。

一位游历四方的哲学家在雾都住了一年，临走时留下了一句话："雾都最伟大的发明不是蒸汽机，不是运河，而是呼吸税。因为它解决了一个数学上无解的问题——如何让一个允许无限循环的系统，在现实中必然终止。"

这则寓言在说什么

这则寓言对应的是以太坊虚拟机（EVM）的 **Gas** 机制。Gas 是以太坊用来计量计算工作量的单位，每一笔交易都需要支付 gas 费用，当 gas 耗尽时交易会被回退（revert）。

关键映射

故事元素	技术概念	说明
呼吸计	Gas 计量器	EVM 在执行交易时跟踪 gas 消耗，每个操作码（opcode）都有预定的 gas 成本
空气税	Gas Fee	用户为交易执行支付的费用，= gas_used × gas_price
永动症	图灵完备系统的停机问题	图灵机可以表达无限循环。在开放网络中，如果允许无限循环，节点会被恶意交易 DoS
给每次动作标价	Gas 机制的设计动机	EVM 的 gas 机制本质上是用经济手段解决停机问题：程序可以在理论上无限运行，但必须在实践中因为 gas 耗尽而终止
批量缝纽扣有折扣	批量操作的 gas 优化	某些操作有固定开销，批量处理可以摊薄单位成本
先写后刻	Calldata 比存储便宜	EVM 中 `calldata` 读取比 `storage` 写

入便宜得多。优化策略：尽量减少存储操作，多用内存和 calldata |
王生的万言奏折	复杂交易 out of gas	如果交易的 gas limit 设得太低，或操作太复杂，交易会在中途耗尽 gas，已消耗的 gas 不退还，状态回退
不能中途续费	交易的原子性	EVM 交易是原子的：要么全部执行成功，要么全部回退。不存在"执行到一半续费继续"
广场演讲防滥用	Gas 防止网络滥用	如果没有 gas 费用，攻击者可以用无限循环交易免费 DoS 整个网络
富人买断广场	Gas 竞价导致富人优先	EIP-1559 之前，gas price 竞价意味着出价高的人优先被打包，导致小额交易被挤出
高峰期提高税率	EIP-1559 的 Base Fee	基础费根据网络拥堵动态调整，拥堵时自动升高，缓解竞价战争

为什么是这个故事？

Gas 是 Web3 新人最困惑的概念之一——"为什么我在区块链上做事要付钱？"
"为什么我的交易失败了还要扣钱？"

用"呼吸税"来隐喻 gas，有两个直觉优势：
1. 必然性：呼吸是生存的基础，不能禁止；但呼吸税让"无限呼吸"变得经济上不可行——正如 gas 让"无限计算"不可行
2. 反直觉的公平：gas 看似不公平（富人优先），但它的存在本身就是对"无限制滥用"的防御。没有 gas，网络会崩溃，届时连富人也用不了

故事中的"省税工艺"也对应了真实的 gas 优化策略——理解 gas 成本的结构，是 Solidity 开发者的必修课。

延伸阅读

- wiki-web3 概念原文：EVM Gas 机制与手续费详解
- [EVM Codes: Opcode Reference with Gas Costs]

- Ethereum Yellow Paper: Gas Cost Specification
- EIP-1559: Fee Market Change
- Solidity Gas Optimization Patterns

"The Bridge Guard"

The Bridge Guard

> "两座城之间没有直路，只有一座由守卫看管的桥。桥断了，两城就断了联系。"

故事

云州城和东海城是两座隔海相望的城市。它们使用不同的货币：云州用银子，东海用贝壳。两城之间没有直航，只能通过一座悬索桥往来。

桥的两端各有一名守卫。云州守卫负责接收银子，东海守卫负责发放贝壳（反之亦然）。这座桥就是两城之间唯一的资产流通通道。

桥的运作方式

假设一个云州商人想把一百两银子转到东海城。流程是这样的：

1. 商人把一百两银子交给云州守卫。
2. 云州守卫把银子锁进云州城的金库，并在账本上记录："张三存入一百两。"
3. 云州守卫通过信鸽通知东海守卫："张三在云州存了一百两，请在东海发放等值贝壳。"
4. 东海守卫收到信鸽后，从东海金库取出一百贝壳，交给张三在东海的代理人。

5. 张三现在在东海城有一百贝壳，他在云州的一百两银子被锁在金库里。

当张三想换回银子时，流程反过来：
1. 张三把一百贝壳交给东海守卫。
2. 东海守卫销毁贝壳，通知云州守卫。
3. 云州守卫从金库释放一百两银子，还给张三。

桥的单点故障

这座桥有一个致命的弱点：守卫。

如果云州守卫被贿赂，他可以在没有收到银子的情况下，伪造记录通知东海守卫发放贝壳。或者，他可以释放金库里的银子给自己。

如果信鸽被拦截，消息可能丢失或篡改。

如果两个守卫串通，他们可以一起偷走金库里的资产。

历史上，很多桥因此倒塌。最著名的几次：
- 某次，云州守卫的私章被盗，攻击者伪造了释放指令，偷走了金库里一半的资金。
- 另一次，两个守卫被同一个外部势力收买，他们一起伪造了巨额转账，导致两城的汇率崩溃。

多守卫桥

为了增强安全性，后来的桥不再只有一个守卫，而是有多个守卫组成"守卫委员会"。

规则是：任何资金的转移，必须获得守卫委员会中超过 2/3 的成员签名确认。单个守卫无法擅自行动。

但这带来了新的问题：
1. 效率降低：每次转账需要收集多个签名，耗时更长。
2. 委员会被渗透：如果委员会成员选拔不当，攻击者可以慢慢渗透，最终控制 2/3 的席位。
3. 委员会内讧：如果委员会成员对某个转账有争议，桥可能陷入僵局。

轻桥

最新的桥设计叫"轻桥"。轻桥不需要守卫委员会，而是依赖数学证明。

具体原理：
- 云州城的账本对所有人公开。
- 任何人都可以下载云州城的账本，验证"张三确实存了一百两"。
- 验证者生成一个"数学证明"——一段可以被东海城独立验证的代码。
- 东海城不需要信任任何守卫，只需要运行这段代码，确认证明有效，就自动释放贝壳。

轻桥的优点是：不需要信任任何中间人。缺点是：生成和验证数学证明需要大量的计算资源，目前的成本还很高。

多年后，云州城和东海城之间已经有很多座桥——有的由守卫委员会看管，有的是轻桥，有的甚至是多跳桥（云州→南山→东海）。一位老商人说："桥让两城互通有无，但桥本身是最脆弱的地方。选择哪座桥，取决于你愿意信任谁，以及你愿意为此付出多少代价。"

这则寓言在说什么

这则寓言对应的是 跨链桥（**Cross-Chain Bridge**）——连接不同区块链、允许资产和信息跨链转移的协议。跨链桥是区块链互操作性的核心基础设施，但也是历史上被攻击最多的组件之一。

关键映射

故事元素	技术概念	说明
云州城和东海城	两条不同的区块链（如 Ethereum 和 Solana）	使用不同的共识机制、虚拟机、资产标准
云州银子 / 东海贝壳	原生资产 vs 包装资产	桥在目标链上铸造与源链资产 1:1 锚定的 wrapped token
云州守卫	源链端的桥合约 / 验证者	负责锁定源链资产并监测转账事件
东海守卫	目标链端的桥合约 / 验证者	负责接收证明并释放/铸造 wrapped 资产
信鸽	跨链消息传递 / 中继器	将源链的事件证明传递到目标链
锁进金库	Lock	源链资产被锁定在桥合约中，退出流通
发放贝壳	Mint	目标链铸造等额的 wrapped 资产
销毁贝壳	Burn	用户赎回时，wrapped 资产被销毁
释放银子	Unlock	源链释放锁定的原生资产
守卫私章被盗	私钥泄露	桥验证者私钥被盗，攻击者可以伪造转账指令
守卫委员会	多签 / 验证者集合	需要多个验证者达成共识才能执行转账（如 Wormhole、Polygon PoS Bridge）
委员会被渗透	验证者勾结攻击	如果攻击者控制足够多的验证者席位，可以伪造任何转账
轻桥	轻客户端桥 / ZK Bridge	使用数学证明（Merkle Proof 或 ZK Proof）验证源链状态，无需信任中间人
数学证明	状态证明 / ZK Proof	证明某个状态转换确实发生在源链上，且符

合共识规则 |
| 多跳桥 | 多跳跨链 | 资产通过中间链转移，如 Ethereum → Polygon → Solana |

为什么是这个故事？

跨链桥是区块链领域损失最惨重的攻击目标——Ronin（6.25 亿美元）、Wormhole（3.2 亿美元）、Nomad（1.9 亿美元）等。理解桥的设计权衡（信任 vs 效率 vs 安全性）对每个 Web3 参与者都至关重要。

用"桥梁守卫"来隐喻跨链桥，有几个直觉优势：
1. 锁-铸机制的直觉：银子锁进金库，贝壳在另一城发放 = lock-and-mint 桥的基本模型
2. 单点故障：守卫被贿赂 = 桥验证者被攻击
3. 委员会 **vs** 轻桥：从信任守卫到数学证明 = 从多签桥到 ZK Bridge 的演进
4. 桥的脆弱性：桥是最脆弱的地方 = 跨链桥是黑客攻击的首要目标

延伸阅读

- wiki-web3 概念原文：跨链桥原理解析
- Ethereum.org: Bridges
- Blockchain Bridges: Building Networks of Cryptonetworks
- Ronin Bridge Hack Analysis
- LayerZero: Omnichain Interoperability
- ZK Bridges: Succinct Labs

"The Non-Fungible Token"

The Non-Fungible Token

> "两枚相同的金币可以互换，但两幅画永远不能。"

故事

云州城的铸币厂铸造两种货币。

第一种是标准铜钱。每枚铜钱都一模一样——同样的重量、同样的花纹、同样的价值。你用一枚铜钱换另一枚铜钱，没有任何区别。这叫"同质化"。

第二种是独一无二的令牌。每枚令牌都是不同的——上面刻着不同的图案、代表不同的权利、属于不同的主人。你不能用一枚令牌换另一枚，因为它们代表的东西完全不同。

令牌的诞生

令牌最初是为了解决一个具体问题：如何在数字世界里证明" ownership "。

在物理世界里，如果你拥有一幅画，你把画挂在墙上，所有人都知道这是你的。但在数字世界里，一张图片可以被无限复制——复制出来的副本和原文件 bitwise 完全相同。谁拥有"原版"？

令牌解决了这个问题。它不是说"这个文件是你的"（因为文件可以被复制），而是说"这个令牌代表了对某样东西的所有权主张"。令牌本身不能被复制——就像物理世界的房产证不能被复制一样（你当然可以复印一份，但复印件没有法律效力）。

令牌的三个核心属性

第一，唯一性。

每枚令牌有一个全球唯一的编号。两枚令牌可能有相似的图案，但编号永远不同。这个编号写在公共账本上，任何人都可以查。

第二，所有权公开。

令牌当前属于谁，写在公共账本上。不需要问任何人，不需要任何证明，账本本身就是证明。如果你想把令牌转让给别人，你只需要在账本上更新记录——从你的名字改成他的名字。

第三，历史可追溯。

令牌从铸造到现在，每一次转手都记录在账本上。你可以追踪一枚令牌的完整历史——谁铸造了它，谁买过它，谁卖过它，每次的价格是多少。这叫"出处"（Provenance）。

令牌的用途

数字艺术：
艺术家把作品和令牌绑定。买家买的不是图片文件（文件可以被复制），而是令牌——代表"我是这幅作品的官方拥有者"。虽然任何人都可以保存图片副本，但只有令牌持有者拥有"被承认的所有权"。

游戏道具：
一把传奇剑在游戏里由一枚令牌代表。这把剑可以在不同游戏之间转移——因为令牌不依赖于任何单一游戏的服务器。如果游戏公司倒闭了，玩家仍然拥有令牌，可以在支持该令牌的其他游戏中使用。

会员资格：
一枚令牌代表"某个俱乐部的终身会员"。俱乐部不需要维护会员数据库——账本本身就是数据库。令牌转让时，会员资格自动转让。

身份凭证：
一枚令牌代表"你参加过某次活动"或"你拥有某个学位"。凭证的发行者在令牌上签名，任何人都可以验证签名的真实性。

令牌的局限

令牌不是万能的。

- 令牌代表所有权，但不自动赋予版权。你买了一个 NFT，不等于你获得了该作品的商业使用权——除非发行者明确授予。
- 令牌的价值完全依赖于社区的共识。如果社区不再认可某枚令牌的价值，它就是一串无意义的数字。
- 令牌本身存储的是编号和所有权记录，而"东西本身"（如图片文件）通常存储在其他地方（如 IPFS 或中心化服务器）。如果那个地方关闭了，你拥有的令牌可能指向一个打不开的链接。

多年后，令牌成了云州城数字生活的一部分。一位收藏家说："令牌没有改变'拥有'的本质——拥有始终是一种社会共识。令牌只是把共识写在了不可篡改的账本上，让谎言变得更难。"

这则寓言在说什么

这则寓言对应的是 **NFT**（**Non-Fungible Token**，非同质化代币）——一种在区块链上代表唯一资产的代币标准（如 ERC-721、ERC-1155）。NFT 的核心价值在于提供数字稀缺性和可验证的所有权历史。

关键映射

故事元素	技术概念	说明
标准铜钱	Fungible Token (ERC-20)	可互换的代币，每单位完全相同
独一无二的令牌	NFT (ERC-721 / ERC-1155)	每个 token 有唯一 ID，不可互换
数字图片无限复制	数字内容易复制性	文件可以被完美复制，传统方式无法区分"原版"和"副本"
令牌不能被复制	区块链的唯一性保证	每个 NFT 的 ID 在合约中唯一，无法伪造
房产证复印件	NFT 的所有权含义	拥有 NFT 不等于拥有版权，除非合约明确授权
公共账本	Blockchain / 分布式账本	所有权记录公开透明，不可篡改
账本更新	Transfer	通过调用合约的 transfer 函数变更 ownerOf
历史可追溯	Provenance	区块链记录了 NFT 从铸造到当前的所有交易历史
游戏道具跨游戏	互操作性 / 元宇宙资产	NFT 不绑定于单一平台，可以在支持该标准的不同应用中使用
会员资格	Token-gated access	持有特定 NFT 即可访问内容、社区或功能
图片存其他地方	Off-chain metadata	NFT 的 metadata（图片、描述等）通常存储在 IPFS 或中心化服务器，链上只存 URI

| 链接打不开 | Link rot / metadata 失效 | 如果 off-chain 存储失效，NFT 可能指向空链接 |

为什么是这个故事？

NFT 是 Web3 最出圈的概念之一，但很多人误解了它的本质——"拥有 JPG"的梗就是典型。理解 NFT 的真正价值（可验证的所有权、出处、互操作性）对于超越炒作非常重要。

用"独一无二的令牌"来隐喻 NFT，有几个直觉优势：
1. 同质化 **vs** 非同质化的对比：铜钱 vs 令牌 = ERC-20 vs ERC-721
2. 所有权的本质：拥有的是社会共识 = NFT 的价值依赖于社区认可
3. 出处的价值：历史可追溯 = 区块链 provenance 的独特价值
4. 局限性的诚实：不代表版权、可能指向失效链接 = NFT 的真实边界

延伸阅读

- wiki-web3 概念原文：NFT 非同质化代币入门
- ERC-721 Standard
- ERC-1155 Multi-Token Standard
- OpenSea: What is an NFT?
- The Non-Fungible Token Bible

"The One-Day Loan"

The One-Day Loan

> "借一万两，用一天，零利息，但必须在日落前归还。"

故事

云州城的钱庄推出了一项前所未有的业务：一日借款。

规则极其简单：
1. 你可以借到钱庄里的任意金额——一百两、一万两、甚至一百万两。
2. 不需要任何抵押。
3. 不需要信用审查。
4. 利息极低——几乎可以忽略不计。
5. 但有一个绝对条件：你必须在当天日落之前归还本金加利息。如果还不上，借款自动取消，就像什么都没发生过。

一日借款的用法

套利：
商人发现东市的大米卖一两/斤，西市卖一两零五分/斤。但他手里没有本金。

他借了一日借款：一万两。

用一万两在东市买大米，立刻在西市卖出，得到一万零五百两。
归还一万两本金和十两利息，净赚四百九十两。

全程只用了两个时辰。

清算：
保险库的规矩是：当抵押品价值跌破阈值时，任何人都可以执行清算，以折扣价获得抵押品。

清算者没有现成的资金。他借了一日借款：一万两。
用一万两偿还某个濒临清算的借款人的债务，以折扣价获得价值一万一千两的抵押品。
立刻把抵押品在市场上卖出一万零八百两。
归还一万两本金和十两利息，净赚七百九十两。

再融资：
某人的贷款在原钱庄利率很高。他在另一家钱庄可以获得更低利率的贷款，但需要先还清原贷款才能释放抵押品。

他借了一日借款还清原贷款，释放抵押品，然后用抵押品在新钱庄获得低利率贷款，归还一日借款。

一日借款的黑暗面

一日借款不仅是套利工具，也是攻击武器。

价格操纵：
攻击者借了一日借款：一百万两。用这笔钱在某个小市场上大量买入某样东西，把价格推高十倍。然后他在另一个衍生品市场上做空这样东西。当一日借款归还、他的买盘消失后，价格暴跌，他的做空仓位获利。

治理攻击：
攻击者借了一日借款，临时获得巨额资金，用这笔钱购买某个组织的投票权，通

过一个恶意提案把组织的资金转给自己，然后归还借款。整个过程在一天内完成。

协议漏洞利用：
攻击者发现某个协议的逻辑漏洞，但利用漏洞需要大量资金。他借了一日借款触发漏洞，盗走资金，然后归还借款。

钱庄的风险

你可能会问：钱庄为什么愿意提供这种服务？如果借款人还不上怎么办？

答案是：钱庄根本不怕借款人还不上。

因为一日借款有一个"原子性"保障：借款、使用、还款，这三个步骤必须在一个不可分割的操作中完成。如果任何一步失败——比如借款人没有赚到足够的钱来还款——整个操作自动回滚。

这意味着：钱庄要么在日落前收回本金加利息，要么就当这次借款从来没发生过。钱庄永远不会亏钱。

真正承担风险的是协议的其余部分——那些被一日借款触发的市场、那些被攻击的合约。它们没有自动回滚的保护。

多年后，一日借款成了云州城金融系统的双刃剑。一位老钱庄掌柜说："一日借款本身是中性的——它只是一个工具。用得好，它让市场更高效。用得坏，它让攻击者如虎添翼。真正的问题不在于一日借款，而在于那些没有防备的协议。"

这则寓言在说什么

这则寓言对应的是 **Flash Loan**（闪电贷）——DeFi 中一种无抵押、无信用审查的贷款，借款和还款必须在同一笔交易的原子操作内完成。如果最终无法归还，整笔交易回退，贷款方没有任何风险。

关键映射

故事元素	技术概念	说明
一日借款	Flash Loan	同一笔交易内借入和归还的零抵押贷款
任意金额	无上限借款	可以借出池中全部流动性（受限于池子大小）
无需抵押	Uncollateralized	不需要锁定任何资产作为抵押
极低利息	0.09% fee (Aave)	闪电贷手续费通常极低，以鼓励使用
日落前归还	同一交易内归还	借款和还款必须在同一笔交易的原子操作内完成
还不上就取消	原子性 / 交易回滚	如果最终无法归还，整笔交易 revert，状态回滚到借款前
钱庄不亏钱	协议零风险	闪电贷提供方（如 Aave）要么收到还款+手续费，要么交易回滚
套利	Arbitrage	利用不同市场间的价格差获利，是最常见的闪电贷用法
清算	Liquidation	用闪电贷资金执行清算，赚取清算奖励
再融资	Collateral Swap / Refinancing	用闪电贷切换抵押品或降低借款利率
价格操纵	Price Oracle Manipulation	用闪电贷资金短暂操纵某个市场的价格，在其他市场获利
治理攻击	Flash Loan Governance Attack	借入资金临时获取投票权，通过恶意提案
协议漏洞利用	Flash Loan Attack	利用闪电贷放大攻击规模，如 Cream Finance、Elephant Money 等

| 原子性保障 | Transaction Atomicity | EVM 保证同一笔交易内的操作要么全部成功，要么全部回滚 |

为什么是这个故事？

闪电贷是 DeFi 最独特的金融创新之一——在传统金融中不可能存在的机制。它同时是最强大的套利工具和最常见的攻击向量。

用"一日借款"来隐喻闪电贷，有几个直觉优势：
1. 零风险借贷：日落前不还就当没发生 = 原子性回滚
2. 无抵押的奇迹：不需要任何信用 = 完全依赖代码执行的自动化
3. 双刃剑：套利工具 = 攻击武器 = 闪电贷的中立性和协议防御的必要性
4. 协议的风险：被攻击的合约没有回滚保护 = 闪电贷攻击的本质

延伸阅读

- wiki-web3 概念原文：闪电贷详解
- Aave Flash Loans
- Flash Loan Attacks: Analysis
- The Biggest Flash Loan Attacks in DeFi History
- How to Protect Against Flash Loan Attacks

"The Infinite Borrower"

The Infinite Borrower

> "规则写得太信任人，本身就是一种漏洞。"

故事

洛城最大的金库叫恒金库，由城主亲自立法保护。恒金库的规矩写在青铜板上，一字不改：

任何人随时可取回自己存入的全部黄金。取金流程分两步：先交付，后销账。

这规矩听起来公平至极。出纳员阿仓在此工作二十年，从未出错。他的动作像钟表一样精确：顾客递上存票，他转身从库里搬出黄金，双手奉上，然后才翻开账本，划去那笔存款。

"先交付，后销账"——这是城主的仁慈。他说："取金的人本就急着用钱，怎能让他们等着我慢慢记账？"

城里有位商人叫魏狡，以精明著称，但没人说他不守规矩。他确实守规矩——他守的是青铜板上写的规矩，一字不差。

魏狡在恒金库存了一百两黄金。某天清晨，他派自己的管家老周去取金。

老周站在柜台前，递上存票："取一百两。"

阿仓验过存票无误，转身进库，搬出一百两黄金，放在柜台上。黄金很重，老周没有立即离开。他从袖中取出一张纸，写上新的取金请求，折好，交给门口一个等候多时的信童。信童飞奔回恒金库的正门，再次排进队伍里。

阿仓刚要翻开账本销账，抬头一看，柜台前又站着一个人——还是老周。不对，是另一个老周？不，是魏狡家的另一个仆人，穿着同样的衣服，递上一张存票："取一百两。"

阿仓皱眉："魏狡的存款不是刚刚才取吗？"

仆人答："大人还有另一笔存款，这是另一张存票。"

阿仓查验——存票是真的，魏狡确实还有一百两在库中。阿仓只好转身再进库，再搬一百两。

就在阿仓放下第二堆黄金的瞬间，第三个仆人已经站在柜台前了。

阿仓开始冒汗。他每次都想先销账，但每次都被人抢先一步站在柜台前。城主的规矩说得很清楚：顾客在柜台前，就必须先服务。阿仓不能对柜台上的人说"你等一下，我先记账"。那会被视为怠慢。

第四波、第五波、第六波……魏狡的仆人像流水一样涌来。恒金库的黄金在减少，账本上的数字却纹丝未动——因为阿仓从未有机会翻开它。

直到库里传来一声空洞的回响。最后一块金砖被搬走了。

阿仓瘫坐在椅子上，看着空荡荡的库房门，终于翻开了账本。账本上清清楚楚地写着：魏狡，存款余额一千两。

而魏狡总共只存过一百两。

城主震怒，召见阿仓和全城智囊。一位年轻的幕僚叫林守，他仔细研究了青铜板

上的规矩，发现问题不在阿仓，也不在魏狡——魏狡确实没有违反任何明文规定。

"问题在顺序，"林守说，"先交付，后销账。在交付和销账之间的那一瞬间，金库处于一种危险的中间态：黄金已经出去了，但账本上它还属于金库。任何人只要在这个瞬间再次请求取同一笔钱，金库就会重复支付。"

"那改成先销账，后交付呢？"

"可以解决大部分问题，但还有更彻底的办法，"林守说，"给金库加一把'进程锁'。当一次取金流程开始时，金库进入忙碌状态，同一笔存款的任何新请求都被暂时拒绝，直到流程完全结束。这样，即使顺序不变，中间态也不会被利用。"

城主采纳了两种改进：所有金库改为先销账后交付，同时在高风险操作中强制使用"进程锁"。

魏狡后来怎样了？据说他带着黄金去了邻国，开了一家更大的金库。没人知道他有没有把那个漏洞写进自己的规矩里。

这则寓言在说什么

这则寓言讲的是智能合约中最著名的安全漏洞之一：重入攻击（**Reentrancy Attack**）。

2016 年，以太坊上的 The DAO 项目因此漏洞被窃取了价值约 6000 万美元的 ETH，直接导致了以太坊的硬分叉和以太坊经典（ETC）的诞生。

关键映射

故事元素	技术概念	说明
恒金库	智能合约（如 EtherStore）	运行在区块链上的自动化程序，规则由代码定义
青铜板规矩	合约代码逻辑	"先交付，后销账"对应 `withdraw()` 函数中先执行外部调用（`call`），再更新余额（`balances[msg.sender] = 0`）
阿仓	合约的执行环境（EVM）	EVM 按代码顺序逐条执行，不会在执行中途自动拒绝外部调用
魏狡的管家与仆人	攻击合约（Attacker Contract）	攻击合约在接收 ETH 时触发 `fallback()` / `receive()` 函数，在其中再次调用目标合约的 `withdraw()`
信童与排队仆人	递归调用链	每次外部调用都会重新进入 `withdraw()`，形成递归循环，直到金库耗尽
"中间态"	合约的重入窗口（Reentrancy Window）	合约状态在 `call` 之后、`balances` 更新之前与预期不一致，这种状态不一致性是漏洞的本质
先销账后交付	Checks-Effects-Interactions 模式	Solidity 安全编码规范：先检查条件（Checks），再更新状态（Effects），最后才进行外部交互（Interactions）
"进程锁"	重入锁修饰器（noReentrancy / mutex）	使用布尔变量 `locked` 或 OpenZeppelin 的 `ReentrancyGuard` 确保函数执行期间不可重入

为什么是这个故事？

重入攻击的核心是时间上的重叠——一次操作尚未完成，另一次操作就插了进来。用"管家在出纳员记账之前派新仆人排进队伍"来隐喻这个时序问题，比直接说"递归调用"更直觉。

故事刻意让魏狡"没有违反任何明文规定"，因为重入攻击的可怕之处就在于：攻

击者没有伪造签名、没有越权访问，他们只是以合约允许的方式，以合约没预料到的顺序，反复调用同一个函数。 攻击者遵守规则，但规则的顺序写错了。

延伸阅读

- wiki-web3 概念原文：Solidity 重入攻击与防御
- Consensys: Smart Contract Best Practices — Reentrancy
- OpenZeppelin: ReentrancyGuard
- The DAO 事件分析

"The Seven Gates"

The Seven Gates

> "每扇门都通向同一个宝藏，但每扇门后都站着不同的守卫，各有各的疏忽。"

故事

风语国有一座藏宝库，由七道门守护。每道门都有专门的守卫和独特的验证方式。国王相信，七重保险足以让任何盗贼束手无策。但事实证明，七道门只是七种不同风格的疏忽。

第一道门：无人看守的后门

第一道门是宝库的主入口，装饰着最华丽的铜钉。但守卫阿大有个习惯——他总认为"只有国王的人才敢靠近主门"，所以他从不认真核对来人的身份。任何人只要穿着像模像样的衣服，大摇大摆地走进去，阿大连头都不抬。

有个小偷发现了这一点。他没有撬锁，没有翻墙，只是换了一件仿制的官服，径直穿过了第一道门。阿大还以为他是新来的内侍。

第二道门：信任陌生人的引路人

第二道门的守卫阿二是个热心肠。他的职责是核对来人的引路文书——只有持有

国王亲签文书的人才能继续深入。但阿二嫌麻烦，他发明了一个"快捷方式"：如果来人声称"我是某某大人派来的"，阿二就不再索要文书，而是直接放行。

一个骗子冒充外地使节，编了一个听着像那么回事的头衔。阿二被唬住了，恭恭敬敬地让开了路。

第三道门：无限的回旋

第三道门的守卫阿三是个死脑筋，严格按照规矩办事。规矩是：来访者递上请求单，阿三核实后从库里取物，交给来访者，然后销去记录。

一个聪明的骗子利用了规矩的顺序。他递上请求单，阿三转身去取物。就在阿三把东西递出来的瞬间，骗子没有离开，而是再次递上一张同样的请求单。阿三很困惑，但规矩没说"不能连续请求"，所以他转身再去取物。就这样，阿三在同一笔记录被销掉之前，反复取出了十份同样的宝物。

第四道门：算盘溢出

第四道门的守卫阿四掌管着数量登记。他有一把老算盘，能显示的数字最大到九十九万九千九百九十九。国王觉得这已经够大了——谁会有超过一百万的东西呢？

一个骗子研究了很久，发现了一件可怕的事：如果算盘上显示的数字已经是九十九万，再往上加二十万，算盘不会显示"一百一十九万"（因为它显示不了），而是会跳回零，从头开始计数。

骗子精心设计了请求：他要取走的宝物，按数量登记，恰好能让算盘溢出归零。当算盘归零时，系统认为他取走了零件东西——他的余额纹丝未动，但库里确实少了那么多宝物。

第五道门：疲于奔命的信使

第五道门的守卫阿五负责向所有有份额的人分发红利。他的方式是：拿到名单，一个一个按顺序敲门送达。名单越长，他跑得越累。

一个骗子发现了这个规律。他注册了成百上千个假身份，把自己塞进了名单里。当阿五拿着超长的名单开始奔跑时，还没跑完一半就已经精疲力竭，瘫倒在半路。整个分发系统因此瘫痪，而骗子注册的那些假身份里，有几个恰好插在了名单的前端，提前拿到了红利。

第六道门：过时的天气预报

第六道门的守卫阿六有个特殊职责：他根据城外的气象塔来决定今天开放哪些库房。如果气象塔报告"暴雨"，他就开放防水物资库；如果报告"干旱"，他就开放水源库。

气象塔由一位老气象员维护。老气象员每天只更新一次数据，而且有时会因为喝酒而延误。一个骗子买通了气象塔下的信童，让他故意传递三天前的旧报告。当阿六根据过时的"暴雨"报告打开防水物资库时，真正的需求是抗旱——城里渴得要命，而防水物资堆积如山。

更糟的是，骗子还在气象塔上动了手脚。他在塔顶安装了一面可以旋转的镜子，在特定时刻把阳光反射到温度传感器上，让气象塔"测到"异常高温。阿六根据这个伪造的数据，做出了完全错误的决策。

第七道门：所有门的总结

第七道门没有守卫。它是一面镜子，让每个通过前六道门的人看看自己是怎么进来的。

国王最终发现失窃时，召集了七位守卫问责。阿大低头说："我以为没人敢冒充。"阿二说："我以为头衔不会骗人。"阿三说："我按规矩办了，规矩没说要防着重复请求。"阿四说："算盘是祖传的，我没想到有人会算到那么大。"阿五说："名单太长不是我的错。"阿六说："气象塔的数据我能怎么办？"

国王沉默了很久，最后说："你们的错误各不相同，但根源只有一个——你们每个人都假设了'不会有人这么做'。盗贼最擅长的，就是找到你们假设的裂缝。"

后来，国王请来了风语国最挑剔的安全师。安全师没有换掉任何一道门，而是做了一件事：在每一道门旁边挂了一块牌子，上面写着——

"不要假设好人会来。要假设最坏的人已经站在门外。"

这则寓言在说什么

这则寓言系统性地对应了 Solidity 智能合约中最常见的六类安全漏洞（第七道门是总结）。原文 wiki 概念覆盖了访问控制失效、未检查外部调用、重入攻击、整数溢出/下溢、Gas 耗尽、Oracle 数据过时和操纵等问题。

关键映射

故事元素	技术概念	说明
第一道门无人看守	访问控制失效（Access Control Failure）	未使用 `onlyOwner` 等修饰符保护敏感函数，任何人都可以调用特权操作
第二道门信任陌生人	未检查外部调用（Unchecked External Calls）	信任任意合约交互，未验证调用方身份或未使用白名单机制
第三道门无限回旋	重入攻击（Reentrancy）	外部调用先于状态更新，攻击者利用回调函数递归调用同一函数
第四道门算盘溢出	整数溢出/下溢（Integer Overflow/Underflow）	Solidity <0.8.0 中 `uint256` 运算溢出会静默回绕。BEC 事件中 `cnt * _value` 溢出为 0，导致无成本增发
第五道门疲于奔命	Gas 耗尽 / DoS（Gas Exhaustion）	遍历所有用户的推送模式（push）在用户数过多时会超出 gas limit，导致功能瘫痪。应使用拉取模式（pull）
第六道门过时天气	Oracle 数据陈旧与操纵（Oracle Manipulation）	依赖单一、不更新的链下数据源，且未做异常值检查和 staleness 检查。AMM 价格预言机可被闪电贷操纵

| 第七道门镜子 | 安全审计思维 | 所有漏洞的共同根源是开发者的"善意假设"。安全审计的核心就是系统性打破这些假设 |
| 安全师的牌子 | 对抗性思维（Adversarial Thinking） | 以攻击者视角审视代码，假设最坏情况已经发生 |

为什么是这个故事？

用"七道门"来隐喻多种安全漏洞，有两个好处：

1. 系统性：读者可以直观地看到，安全不是一个单一的问题，而是一个系统工程。每道门的守卫都"按自己的方式"犯了错，但组合起来就是灾难。
2. 归因明确：每个守卫的借口（"规矩没说""算盘是祖传的""数据我能怎么办"）恰好对应了开发者的典型推诿——"代码就是这么写的""Solidity 旧版本的问题""Oracle 的数据问题"。国王的最后一句话点出了核心：安全是设计者的责任，不是工具的借口。

延伸阅读

- wiki-web3 概念原文：Solidity 智能合约常见安全漏洞概述
- SWC Registry: Smart Contract Weakness Classification
- Consensys Diligence: Known Attacks
- BEC Token Overflow Incident (2018)

"The Counterfeit Scale"

The Counterfeit Scale

> "最危险的谎言，是机器替你撒的。"

故事

青国最大的钱庄叫万通号，专门帮商人批量转账。万通号的规矩是这样的：一位商人如果要把银子分给二十个伙计，只需写一张单子，注明每人多少两，钱庄的账房先生就会一次性把二十笔账全部算清。

账房先生的算盘是从前朝传下来的，极其精美，拨珠如飞。但有一个不为人知的秘密：这把算盘最多只能显示九十九万九千九百九十九两。超过这个数，算盘不会报错，而是像车轮一样——从九十九万九千九百九十九再拨一格，就会归零，从零重新开始计数。

没有人知道这个秘密。算盘用了三十年，从未出过问题。因为正常的生意里，谁会一次性转超过一百万两银子呢？

商人钱多的绰号叫"钱不倒"。他不是万通号的老客户，但出手阔绰，出手就是一千两的见面礼。

钱多不是来做正常生意的。他花了三个月研究万通号的账簿，发现了那把算盘的

秘密。他算出了一个精确的数字——

如果他要给两个人各转五十万两，总计就是一百万两。而算盘的极限是九十九万九千九百九十九两。一百万两，刚好溢出归零。

归零意味着什么？意味着账房先生的算盘上显示的是"零"。而零，在万通号的系统里有一个特殊的含义："无需扣款"。

钱多设计了一个完美的请求：

他要给两个伙计各发一笔"奖金"。每个伙计的奖金数额，恰好是五十万两。

账房先生接过单子，习惯性地开始计算：2 人 × 50 万两 = 总计 100 万两。

算盘响了——咔嗒。显示：零。

账房先生愣了一下，但很快释然。零意味着无需从钱不倒的账户扣款。规矩写得很清楚："若总计为零，则视为无扣款交易。"

于是他照办了。两个伙计各收到了五十万两银子，而钱不倒的账户分文未动。

钱多没有停手。他发现了一个更惊人的事实：既然"零"意味着"无需扣款"，那他可以把所有伙计都拉进来。二十个伙计，每人五十万两。算盘上还是零。因为 20 × 50 万 = 一千万两，远超算盘的极限，仍然溢出归零。

账房先生一如既往地机械执行。二十个假伙计，每人五十万两，总计一千万两——算盘显示：零。无需扣款。

钱多没有贪心到榨干万通号。他拿了足够跑路的银子，连夜离开了青国。而万通号直到三天后对账才发现：库里的银子少了整整一亿两，但账本上没有任何一笔对应的扣款记录。

钱庄的东家疯了。他找来了全国最厉害的算师查账。算师花了七天七夜，终于发

现了那把算盘的秘密。

"问题不在账房先生，"算师说，"账房先生每一步都是按规矩执行的。问题在于规矩本身——它假设了'总计不可能为零'，而'零'被定义为'无需扣款'。当溢出导致总计归零时，这个假设就被打破了。"

"那把破算盘为什么不报错？"

"因为它太老了。老算盘的设计哲学是'不中断、继续算'。它不会告诉你说'数字太大了'，它只是默默地从头开始。"

"那现在怎么办？"

"两种办法。第一，换一把新算盘——新算盘会在数字过大时直接拒绝计算，而不是悄悄归零。第二，在每一次计算之前，先检查一下：如果乘积除以其中一个乘数，结果是否等于另一个乘数？如果不等，说明计算过程中发生了溢出。"

东家采纳了两种改进：全面更换新算盘，同时在每次批量转账前增加溢出校验。

后来有人在邻国遇到了钱多。问他为什么不拿更多。钱多笑了笑说："够用就好。而且，如果我拿得太多，那个算盘的秘密就会提前暴露。我要的不是一次性的暴利，而是让那个秘密一直存在——这样我下次还能用。"

这则寓言在说什么

这则寓言对应的是 2018 年 BEC（美链）Token 的整数溢出攻击，该漏洞导致一个市值一度突破 280 亿美金的项目归零。漏洞仅存在于一行代码：`uint256 amount = uint256(cnt) * _value;`

关键映射

故事元素	技术概念	说明
万通号	BEC Token 合约	批量转账功能的智能合约
账房先生的老算盘	Solidity <0.8.0 的 uint256	`uint256` 最大值为 `2^256 - 1`。当运算结果超过此值时，低 256 位被保留，高位被截断，表现为"静默回绕"到零
算盘的九十九万极限	`uint256` 的最大值	`115792089237316195423570985008687907853269984665640564039457584007
钱多设计的 2 × 50 万 = 100 万	攻击参数	攻击者传入 `cnt=2`，`_value=57896044618658097711785492504343953926634992332820282019728 乘积溢出为 0
"零意味着无需扣款"	`require(balances[msg.sender] >= amount)`	当 `amount` 溢出为 0 时，任何余额都满足 `>= 0`，扣款逻辑被跳过，但转账仍然执行
新算盘	Solidity 0.8.0+	0.8.0 之后编译器默认启用溢出检查，溢出会触发 revert
溢出校验	`assert(amount / _value == cnt)`	乘法后做除法回验，如果结果不一致说明发生了溢出
SafeMath	OpenZeppelin SafeMath 库	在 <0.8.0 版本中，使用 `.mul()` 代替 `*` 来强制检查溢出

为什么是这个故事？

整数溢出是编程中最古老的漏洞之一，但它在区块链上的破坏力被放大了——因为智能合约处理的是真实的资产，且代码不可更改。

用"算盘溢出归零"来隐喻这个漏洞，有三个直觉上的优势：

1. 可视化：算盘是一个物理对象，"归零"是可以想象的动作。
2. 非恶意性：账房先生不是坏人，他只是"按规矩执行"——这恰好是智能合约的特性：代码不会"故意"作恶，它只是忠实地执行有缺陷的逻辑。
3. 攻击的精巧性：钱多的攻击不是暴力破解，而是对系统边界的精确计算。这和真实攻击者花费三个月研究 `uint256` 边界的行为一致。

延伸阅读

- wiki-web3 概念原文：一行代码的漏洞，就蒸发了 280 亿美金！
- BEC Token BatchOverflow Vulnerability Analysis
- OpenZeppelin SafeMath
- Solidity 0.8.0 Breaking Changes: Arithmetic Operations

"The Messenger and the Auction House"

The Messenger and the Auction House

> "在这个国度里，所有信件都被拆开阅读后才会送达，而邮费决定了谁先看信。"

故事

在透明的国度，所有信息都是公开的——不是被公开，而是天生透明。这里的拍卖行有一个奇特的规则：所有的出价信不会直接送到拍卖师手中，而是先挂在大厅的公告板上，供所有人浏览。信使们按照邮费高低来决定取信的顺序——谁付的邮费高，谁的信就优先被取走、优先被拍卖师看到。

这套规则的设计者认为：公开透明能让所有人公平竞争。但他忽略了一个简单的事实：能看到别人的意图，本身就是一种权力。

商人许真在拍卖行看中了一幅古画，底价一百两。他写了出价信，开价一百五十两，付了标准邮费，把信挂上了公告板。

职业猎手郑快一直在公告板前蹲守。他的工作就是阅读别人的信件，然后在对方之前行动。郑快看到了许真的出价信——一百五十两买古画。他立刻做了两件事：

第一，他写了一封自己的出价信，开价一百六十两，付了双倍的加急邮费。
第二，他另写了一封给拍卖行的"友情提示"：如果古画成交价超过一百八十两，他愿意再额外支付一笔信息服务费——但必须在成交后私下交付。

加急邮费让郑快的信先被取走。拍卖师先看到了一百六十两的报价。就在此时，许真的标准邮费信件也到了。拍卖师面前有两封信：郑快的一百六十两，许真的一百五十两。

但郑快的第二封信起了作用。拍卖师心领神会——如果他把画以一百八十两卖给许真（假设许真愿意加价），郑快的信息服务费就到手了。于是他故意暗示许真："有人出了一百六十两，您还继续吗？"

许真被逼到了墙角。他要么放弃，要么加价。他咬牙开到了一百八十两。拍卖师落锤。许真多花了三十两，而郑快什么都没做——只是读了一封信，付了一点加急邮费。

更阴险的手法还在后面。

拍卖行推出了一项新业务：「获取或创建」珍藏室。规则是这样的：如果你想要一个特定主题的珍藏室（比如"明代瓷器"），你可以写信申请。如果该主题的珍藏室已经存在，你就获得它的使用权；如果不存在，拍卖行会为你新建一个。

收藏家马明想要一间"宋代书画"珍藏室。他写了申请信，在信中注明了他希望的保管条件：恒温恒湿，安保等级甲等。

郑快在公告板上看到了这封信。他立刻写了一封自己的申请信，注明了完全相同的主题"宋代书画"，但保管条件改成了：开放式存放，不设安保。

郑快付了加急邮费。他的信先到。拍卖行为他创建了"宋代书画"珍藏室——开放

式存放，不设安保。

马明的信随后到达。拍卖行检查："宋代书画"珍藏室已存在。于是直接把马明分配到了郑快创建的那间开放式存放室里。马明珍贵的宋代书画，从此暴露在任何人触手可及的地方。

还有一个更微妙的陷阱。

拍卖行允许客人用两步流程抵押珍品换取资金。第一步，客人写一封信，授权拍卖行可以处置自己的某件珍品。第二步，客人再写一封信，正式把珍品交给拍卖行，换取资金。

画家柳青想抵押自己的名画《云山图》。第一步，她写了授权信，把处置权暂时交给了拍卖行。这封信被挂在了公告板上。

郑快看到了这封授权信。在柳青的第二封信到达之前，郑快付加急邮费，抢先写了一封信给拍卖行："请把《云山图》交给我，我有授权。"拍卖行检查了授权——确实存在，允许处置。于是他们把《云山图》交给了郑快。

柳青的第二封信随后到达——她来交画换钱。但画已经不在她手里了。拍卖行无奈地说："您确实授权了处置权，而那位先生只是比我们预期的更早来取了。"

透明国度的国王终于意识到问题。他召集智者商议对策。

一位老谋士说："问题在于所有信件都是明文挂在公告板上的。如果信件的内容只有发信人和收信人才能看懂，猎手就无法利用信息了。"

另一位谋士补充："邮费决定顺序也是问题。如果所有人付同样的邮费，按到达时间排队，猎手就无法通过加价来插队。"

第三条建议是："两步流程的授权应该绑定具体的人。柳青的授权只应该对柳青

本人有效，不应该被任何看到授权信的人利用。"

最终，透明国度实施了三项改革：

1. 密封信封：所有信件在公告板上只显示发送者和接收者，内容被加密。只有收信人有钥匙解密。
2. 统一邮费：废除加急邮费，所有信件按到达时间排队。
3. 绑定授权：授权信必须写明"仅对以下身份有效"，其他人持有相同的授权信也无法执行。

改革之后，郑快失业了。但他很快找到了新工作——帮别人写更巧妙的出价策略。毕竟，透明国度的信息仍然是透明的，只是变得不那么容易被利用了。

这则寓言在说什么

这则寓言对应的是区块链中的抢跑攻击（**Front-running Attack**）。由于区块链内存池（mempool）的透明性和交易按 gas 价格排序的机制，攻击者可以观察待处理的交易，利用其中透露的信息，通过支付更高 gas 费来让自己的交易优先执行，从而获利。

关键映射

故事元素	技术概念	说明
透明国度	公开区块链（如以太坊）	所有待处理交易在内存池中公开可见
公告板上的出价信	Mempool 中的待处理交易	攻击者可以监控 mempool 中的交易内容
加急邮费	Gas Price / Priority Fee	矿工/验证者优先处理 gas 更高的交

易。EIP-1559 后为基础费 + 优先费 |
郑快读信后加价	抢跑（Front-running）	攻击者复制受害者的交易意图，支付更高 gas 优先执行，获利套利
拍卖师的"友情提示"	尾随攻击（Back-running）	攻击者在受害者交易之后立即执行，利用受害者交易引起的价格变动获利
获取或创建珍藏室	"Get or Create" 模式的抢跑	攻击者抢先创建资源并注入恶意参数，受害者被迫使用被篡改的资源
两步抵押流程	多步交易的抢跑	第一步授权在 mempool 中暴露后，攻击者在第二步之前插入自己的调用，劫持流程
密封信封	Commit-Reveal 方案	用户先提交哈希承诺（commit），待所有承诺收集完毕后再揭示原始内容（reveal），避免信息提前暴露
统一邮费	Flashbots / PBS	通过私有交易池（如 Flashbots）或提议者-构建者分离（PBS）来减少公开的 gas 竞价战争
绑定授权	检查 `msg.sender`	多步操作中验证第二步的调用者必须与第一步的授权者相同

为什么是这个故事？

抢跑攻击的直觉难点在于：攻击者没有伪造身份、没有突破权限，他们只是"看得比别人快，动得比别人早"。用"公开拍卖行的公告板"来隐喻 mempool 的透明性，用"加急邮费"来隐喻 gas 竞价，非常自然。

故事中的三个案例分别对应了抢跑的三种典型场景：
1. 直接抢跑：看到出价后加价（DEX 中的三明治攻击）
2. 获取或创建抢跑：抢先创建资源并注入恶意参数
3. 多步流程抢跑：利用两步操作之间的时间窗口劫持流程

这比直接说"内存池透明，gas 竞价"更能让人感受到问题的存在和危害。

延伸阅读

- wiki-web3 概念原文：Solodit 检查清单解读 (4)：抢跑攻击
- Flashbots: Frontrunning on Ethereum
- EIP-1559: Fee Market Change
- Consensys: Front-running

"The Lighthouse in the Swamp"

The Lighthouse in the Swamp

> "最危险的陷阱，是看起来不像陷阱的那一个。"

故事

迷雾沼泽的深处，有一座灯塔。每当夜幕降临，灯塔的光芒就会穿透浓雾，为迷路的船只指引方向。塔下有一块牌子，写着："解开灯塔之谜，可获得全部宝藏。"

无数的探险者被这光芒吸引。他们划船进入沼泽，靠近灯塔，看到塔身上刻着一道谜题："什么东西早晨四条腿，中午两条腿，晚上三条腿？"

这是经典的斯芬克斯之谜。答案谁都知道——人。

探险者们兴奋极了。按照规则，他们需要先向灯塔缴纳一笔"入塔费"——至少十个金币。然后说出答案，如果正确，灯塔里所有的宝藏就归他们所有。

第一个探险者缴纳了十个金币，大声喊道："人！"

灯塔沉默了一瞬，然后传来一个声音："答案正确。但你还不是管理员。"

探险者愣住了。谜题的规则里从没提到什么"管理员"。他仔细看塔身上的铭文——确实，在最不起眼的角落，有一行小字："本灯塔由管理员委员会维护，最终解释权归委员会所有。"

他输了。十个金币留在了灯塔里。

第二个探险者来了。他是个聪明人，没有急着缴纳金币，而是绕着灯塔转了三天三夜，试图找出谜题的真正答案。他注意到，灯塔的底部有一扇小门，门上有一个锁孔，形状奇特。

他花了五个金币从一个路过的老人那里买到了一张"内部地图"。地图上标注了灯塔的结构——谜题在一层，宝藏在顶层，但通往顶层的路只有管理员知道。

他按照地图上的线索，尝试打开那扇小门。锁孔需要一把特殊的钥匙——而钥匙的形状，恰好是灯塔建造者的名字经过某种变形后的图案。

他推算出了钥匙的形状，制作了钥匙，打开了小门。门后是一条狭窄的通道，通向灯塔的地下室。地下室里堆满了金币——全都是之前探险者缴纳的"入塔费"。

他兴奋地伸手去取。但就在他的手指触碰到金币的瞬间，地板裂开了。他掉进了一个更深的陷阱——下面不是水，而是流沙。

原来，那扇小门、那张地图、那个老人，全都是灯塔设计的一部分。灯塔的真正建造者，早就预料到会有人试图反向破解。地下室的金币是诱饵，真正的陷阱是地板上的机关。

第三个探险者是一个极其谨慎的老水手。他没有缴纳金币，没有尝试破解，而是做了一件所有人都没做过的事——他观察了灯塔三十天。

他发现了一个规律：每当有人缴纳金币后，灯塔的光芒就会闪烁三下。而当没有人缴纳金币时，灯塔的光芒是稳定的。他还发现，那些在塔下徘徊超过一天的人，会被一个"过路人"搭讪，推销"内部地图"。

三十天后，老水手得出了结论：这不是一座灯塔，这是一个精心设计的捕食装置。光芒是诱饵，谜题是幌子，规则是牢笼。所有试图赢取宝藏的人，最终都变

成了宝藏的一部分。

他没有告诉别人。他只是在自己的航海日志上写下了一行字："迷雾沼泽深处的灯塔，不要靠近。那不是光，是嘴。"

几年后，一个年轻的制图师发现了老水手的日志。他被这个故事深深吸引，决定用自己的方式揭开真相。

制图师没有亲自去沼泽。他收集了所有关于那座灯塔的记录——五十一位探险者的口述、金币的流向、灯塔的建造图纸（从档案馆里找到的残缺版本）。他把这些信息拼在一起，还原了灯塔的完整设计。

他发现了一个惊人的事实：灯塔的谜题本身是正确的，答案也确实是"人"。但灯塔内部还有一个隐藏的规则——"答案的哈希值必须与预设值匹配"。而这个预设值，不是"人"的哈希，而是"某个不存在的东西"的哈希。

换句话说，谜题是一个无解的谜题。它的目的不是让人解开，而是让人相信自己"差一点就解开了"。

制图师把自己的发现写成了一本小册子，标题叫《如何识别沼泽中的灯塔》。小册子的最后一页只有一句话："如果一件东西太好以至于像是免费的，那它就是在向你收费。"

这则寓言在说什么

这则寓言对应的是智能合约中的蜜罐合约（**Honeypot Contract**）。蜜罐合约是一种表面上看起来有利可图、但实际上设置了隐藏陷阱的恶意合约。它的目的不是直接攻击受害者，而是诱导受害者主动投入资金，然后利用隐藏的机制阻

止资金被取回。

关键映射

故事元素	技术概念	说明
沼泽中的灯塔	蜜罐合约	表面上有明确规则（猜谜赢宝藏），但实际上有隐藏的后门和陷阱
灯塔之谜	合约中公开的「可交互功能」	如 `Try()` 函数，看起来只要答对问题就能取走全部余额
入塔费	`msg.value > 1 ether`	蜜罐合约要求用户先投入资金才能参与
"你不是管理员"	`isAdmin` 修饰符	合约中的权限检查。`Try()` 函数虽然逻辑上允许任何人调用，但 `transfer` 或其他关键操作被管理员权限保护
灯塔底部的秘密通道	反编译 / 逆向工程	蜜罐合约常部署已验证的源码与真实运行时字节码不符，需要通过逆向工程发现隐藏逻辑
内部地图和老人	社交媒体 / 虚假教程	蜜罐合约的推广者会在 YouTube、Reddit 等平台发布"教程"，诱导用户部署或调用合约
地下室的流沙陷阱	隐藏的资金转移逻辑	合约中看似正常的 `transfer`，实际上通过隐藏的 `fallback()` 或 `delegatecall` 把钱转到了攻击者地址
无解的哈希	`answerHash` 被管理员控制	蜜罐合约中公开的 `answerHash` 可以被管理员通过 `New()` 函数随时更改，确保无人能真正"答对"
老水手的日志	链上数据分析	通过观察交易历史、资金流向、合约交互模式来判断一个合约是否为蜜罐
制图师的小册子	蜜罐识别工具 / 社区教育	如 Token Sniffer、Honeypot.is 等检测工具，以及安全社区的知识分享

为什么是这个故事？

蜜罐合约的危险之处在于它的非对称性：攻击者只需要写一次代码，就可以无限

次地诱骗受害者。而受害者每一次都需要投入真金白银去"验证"这个合约是否安全。

用"灯塔"来隐喻蜜罐，非常贴切：
1. 光芒是诱饵：合约表面上的高收益或简单规则吸引着用户
2. 谜题是幌子：公开的规则看起来公平，但隐藏规则才是真正的游戏规则
3. 收费是目的：入塔费就是用户投入的资金，而 treasure 永远不会被真正取走
4. 内部地图是帮凶：虚假的教程和推广者是蜜罐生态的重要一环

故事刻意设计了三个层次的探险者——冲动的、聪明的、谨慎的——因为蜜罐合约正是利用不同水平用户的不同心理弱点。

延伸阅读

- wiki-web3 概念原文：一个蜜罐合约的解析
- Token Sniffer: Honeypot Detection
- Honeypot.is
- Etherscan: Known Honeypot Contracts

"The Weather Vane Deception"

The Weather Vane Deception

> "当你把所有的决定都交给一根指针时，控制指针的人就控制了你。"

故事

云谷城的所有交易都依赖一座气象塔。塔顶上有一面巨大的风向旗，每天向全城播报三个数据：风速、风向、气压。

这三组数据不是普通的气象信息——它们是云谷城经济运行的基础。粮商根据风速决定运费，船队根据风向规划航线，而最关键的是：城中的借贷行根据气压来决定抵押品的价值。

气压高 = 抵押品值钱 = 可以多借钱。

气压低 = 抵押品贬值 = 要么追加保证金，要么被强制变卖。

气象塔由一位老气象员维护。老气象员每天清晨爬到塔顶，读取仪器，然后写在布告上，挂到城中广场。整个流程看起来天衣无缝——唯一的问题是：所有人都在等老气象员的布告，而老气象员每天只更新一次。

商人沈机是城中最有钱的人之一。但他不是最有钱的——最有钱的是那些能从空

气中变出钱的人。

沈机研究气象塔已经三年了。他发现了一个规律：老气象员的布告有大约一刻钟的延迟。从老气象员读取仪器到布告挂在广场上，中间有一段时间窗口。而在这段窗口里，仪器本身的数据是可以被影响的。

气象塔的气压计不是一个密封的装置——它的传感器暴露在塔顶的风口中。沈机只需要在布告发布前的一刻钟，派人爬到塔顶，用一面巨大的扇子对着气压计的进风口猛扇，就能让气压计"测到"异常高的气压。

但这还不够精确。沈机想要的是一个可以量化的、可重复的操纵方式。

他发现了一个更精妙的手段：城中有一条人工运河，运河的闸门开启会影响整个城区的气流走向。如果他在布告发布前同时打开所有上游闸门，运河水面上升产生的水汽会被风吹向气象塔，在气压计周围形成一个短暂的低压区——而如果他在水汽到达前瞬间关闭闸门，低压区消失，气压计会反弹到一个异常高的读数。

这个反弹的瞬间，恰好是老气象员读取仪器的时间。

沈机不需要天天这么做。他只需要在关键时刻这么做——比如，当他想从借贷行借一大笔钱的时候。

第一次操纵，沈机借到了五十万两。

过程是这样的：他先把自己的全部房产作为抵押，存入借贷行。然后他在布告发布前操纵气压计，让气压读数飙升。老气象员写下"气压：极高"，挂在广场上。借贷行的账房先生看到布告，按照规则，把沈机的抵押品估值提高了三倍。沈机因此可以多借三倍的钱——五十万两。

他拿着这五十万两，迅速投入了另一项生意。一周后，他归还了借款，连本带利。借贷行浑然不觉，以为自己做了一笔正常的生意。

但沈机的胃口越来越大了。

第二次操纵，他联合了城中另外三位商人，组成了一个"气象俱乐部"。他们不再满足于扇扇子——他们开始在运河上下游同时修建秘密闸门，精确控制水汽到达气象塔的时间。他们还买通了老气象员的学徒，让学徒在关键时刻"恰好"去取早餐，延误布告的发布时间。

这一次，他们不是为了借钱——而是为了"收割"。

他们先散布谣言："明天的气压会暴跌。"城中持有抵押品的商人开始恐慌，纷纷提前还款或追加保证金。而就在布告发布前，"气象俱乐部"操纵气压计显示"极高"——借贷行的估值系统瞬间把抵押品价格抬到了天上。那些恐慌中卖出了抵押品的商人，眼睁睁看着自己的资产在别人手里暴涨。

然后，"气象俱乐部"在第二天操纵气压计显示"极低"。借贷行的系统强制变卖了那些高价买入的抵押品。俱乐部成员以极低的价格接盘，又在第三天气压恢复正常时高价卖出。

三天之内，他们什么也没生产，但财富翻了一番。而那些被收割的商人，直到破产都不知道自己输给了谁。

城中最年轻的算师叫林真，他发现了问题。

他没有去气象塔——他知道那里已经被控制了。他做了一件看起来毫无意义的事：他在城中各个角落安装了三十个小型气压计，然后每天记录这三十个读数，再与气象塔的官方布告对比。

三个月后，他发现了一个惊人的模式：气象塔的读数与城中三十个独立读数的平均值，存在周期性的、可预测的偏离。而这种偏离，总是发生在大型借贷交易的前后。

林真把报告交给了城主。城主起初不信——"老气象员跟了我三十年，他不会骗人。"林真说："老气象员没有骗人，他只是在读一个已经被骗的仪器。"

城主问："那怎么办？不用气象塔了吗？"

林真说："不，气象塔仍然有用。但我们需要三件事：第一，不再依赖单一的气象塔，而是同时参考城中多个独立的气压计，取加权平均。第二，设置异常检测——如果气象塔的读数与周边读数偏离超过阈值，自动触发警报并暂停相关交易。第三，也是最重要的：借贷行不应该根据单一时刻的气压来决定抵押品价值，而应该参考过去一段时间的平均值，甚至加上一个时间锁，让重大价格变动不能立即生效。"

城主采纳了林真的建议。新的系统上线后，沈机的"气象俱乐部"再也无法操纵全局了——他们也许还能影响气象塔的一个读数，但不可能同时影响城中三十个独立的传感器。

沈机没有破产，但他也不再是城中最富有的商人了。他的财富回到了三年前——刚好是他第一次操纵气压计之前的水平。

有人说这是报应。沈机自己知道，这只是数学的公平。

这则寓言在说什么

这则寓言对应的是智能合约中的预言机操纵（**Oracle Manipulation**）。预言机为链上合约提供链下数据（如价格、汇率、比赛结果），但如果预言机的设计存在缺陷，攻击者可以通过操纵数据源来诱导合约做出错误的决策，从而获利。

关键映射

| 故事元素 | 技术概念 | 说明 |

|---------|---------|------|
云谷城的气象塔	价格预言机（Price Oracle）	为链上合约提供外部价格数据的机制，如 Chainlink Price Feeds、Uniswap TWAP
老气象员的布告	预言机的数据更新	预言机定期向链上推送数据，存在更新频率和延迟
用扇子影响气压计	直接操纵链下数据源	攻击者通过闪电贷等方式在 DEX 上制造异常价格，影响依赖该 DEX 作为价格源的合约
运河闸门控制水汽	闪电贷攻击（Flash Loan Attack）	攻击者无需本金，通过闪电贷借入大量资金，在短时间内操纵 AMM 价格，然后归还款项，净赚套利利润
借贷行根据气压估值	借贷协议的价格清算逻辑	如 Aave、Compound 等协议根据预言机价格决定抵押品价值和清算阈值
"气象俱乐部"联合操纵	协同攻击 / MEV 策略	多个攻击者或机器人协同操纵价格，制造更大的偏离
三十个独立气压计	去中心化预言机网络	Chainlink 等去中心化预言机从多个独立节点聚合数据，单一数据源被操纵不会显著影响最终结果
异常检测与阈值	预言机数据的完整性检查	合约应检查数据的新鲜度（staleness）、异常值（outliers），以及价格变动幅度是否超过合理范围
时间锁与 TWAP	时间加权平均价格	Uniswap V2/V3 的 TWAP（Time-Weighted Average Price）通过取一段时间内的平均价格，降低瞬时操纵的影响
林真的加权平均方案	多源聚合 + 异常检测	最佳实践：使用多个独立数据源，设置偏离阈值， stale 数据自动暂停

为什么是这个故事？

预言机操纵是 DeFi 中最昂贵的攻击类型之一。历史上多次重大损失都源于此：
- bZx（2020）：闪电贷操纵 Uniswap 价格，获利 35 万美元
- Cream Finance（2021）：预言机操纵导致 1.3 亿美元损失
- Mango Markets（2022）：操纵预言机价格，提取超额抵押

这些攻击的共同模式是：攻击者不需要攻破合约的代码逻辑，只需要攻破合约所依赖的外部数据。这是智能合约安全中最反直觉的一点——合约本身可能是完美

的，但如果它的"眼睛"被蒙蔽了，它就会做出完美的错误决策。

用"气象塔"来隐喻预言机，用"扇子"和"运河闸门"来隐喻闪电贷价格操纵，能让读者直觉地理解：问题不在借贷行的规则，而在规则所依赖的数据。

延伸阅读

- wiki-web3 概念原文：分析一个 scam 合约
- Chainlink: Price Feeds & Oracle Security
- Uniswap V3: Oracle Integration
- OpenZeppelin: Oracle Manipulation
- bZx Hack Analysis (PeckShield)

"The Assembly Line Workshop"

The Assembly Line Workshop

> "不是一台机器在干活，而是整个工坊的每道工序都在同时运转。"

故事

云州城的老工坊是一间传统的铁匠铺：一个铁匠，一个熔炉，一次只打一把刀。工序是串行的——加热、锻造、淬火、打磨，一步做完才能做下一步。

这种工坊的优点是简单可靠。缺点是慢。如果一天要打一千把刀，需要一千个铁匠和一千个熔炉。

流水线工坊的革命

东海之滨出现了一种全新的工坊：流水线工坊。

流水线工坊的核心思想是：把打刀的过程拆分成许多小步骤，每个步骤由一个专门的工匠负责，所有步骤同时进行。

具体设计：

1. 工序并行化：加热、锻造、淬火、打磨不再由一个人完成，而是四个专门的

工匠。当第一把刀在锻造时，第二把刀已经在加热，第三把刀正在淬火，第四把刀正在被打磨。

2. 时间戳总管：工坊中央有一个"时间戳总管"，他每过一息就敲一下锣。所有工匠按照锣声同步工作——听到第一声锣，加热工把刀传给锻造工；听到第二声锣，锻造工把刀传给淬火工。锣声确保了整个工坊的工序不会乱。

3. 无等待传递：在传统工坊，如果一个工匠还没完成，下一个工匠只能干等。在流水线工坊，传递是异步的——加热工把刀放进一个公共的传送带，锻造工从传送带上取刀。如果传送带上暂时没有刀，锻造工可以先处理其他任务，而不是傻等。

4. 多熔炉并行：传统工坊只有一个熔炉。流水线工坊有几十个熔炉同时工作，每个熔炉负责加热不同的刀。但所有熔炉共享同一个烟囱——烟囱的排烟能力决定了整个工坊的吞吐量上限。

流水线工坊的瓶颈

流水线工坊很快遇到了自己的瓶颈：烟囱。

几十个熔炉同时排烟，烟囱很快就不够用了。工坊的工程师面临一个选择：
- 加粗烟囱：让每个熔炉都能畅通排烟，但烟囱的建造成本会指数级上升。
- 限制熔炉数量：保持烟囱不变，但限制同时工作的熔炉数量。

流水线工坊选择了第三条路：状态租金。

每个熔炉不是永久占用的。如果一个熔炉长时间不工作（比如工匠请假了），工坊会把它"租"给其他工匠使用。原来的工匠回来后，需要重新申请熔炉——如果熔炉已经被租走，他需要等待或支付更高的租金来抢占。

这个机制确保了：烟囱的排烟能力永远不会被闲置的熔炉浪费。

流水线工坊与云州城老工坊的对比

一位游历的工匠比较了两地：

"云州城的老工坊像是一个人在慢慢走路——每一步都很稳，但走不快。

东海的流水线工坊像是一群人同时跑接力——每个人都在全力奔跑，通过精密的交接棒来保持整体速度。

老工坊的优势是容错：如果一个铁匠生病了，停工一天，损失只是一天的工作量。

流水线工坊的优势是吞吐：如果一切运转正常，它的产量是老工坊的一千倍。但如果时间戳总管的锣声停了——哪怕只停了一息——整个工坊的所有工序都会瞬间乱套。因为每个人都在依赖那个统一的锣声来同步。"

这则寓言在说什么

这则寓言对应的是 **Solana** 的架构设计——一个以并行处理和超高吞吐量为核心目标的区块链。Solana 通过 Proof of History（PoH）、Sealevel 并行运行时、Turbine 传播协议、Gulf Stream 无 mempool 交易转发等创新，实现了每秒数万笔交易的吞吐量。

关键映射

故事元素	技术概念	说明

传统铁匠铺	传统 EVM 链（以太坊）	串行执行，一次处理一笔交易，状态全局锁定
流水线工坊	Solana	并行执行架构，多个交易同时处理
工序并行化	Sealevel 并行运行时	Solana 的 VM 可以并行执行不冲突的交易（即不读写相同账户的交易）
时间戳总管	Proof of History (PoH)	一个密码学时钟，通过顺序哈希生成可验证的时间戳，为全网提供统一的时间参考
锣声同步	Slot / Leader Schedule	验证者按照固定的时间表（每 400ms 一个 slot）轮流出块，PoH 确保顺序
无等待传递	Gulf Stream	交易直接转发给预期的出块者，不经过公共 mempool，减少传播延迟
公共传送带	交易队列	验证者维护待处理交易队列，按需取交易执行
多熔炉并行	并行账户访问	Solana 的账户模型允许识别不冲突的交易，并行执行
共享烟囱	网络带宽 / TPS 上限	Solana 的吞吐量受限于网络带宽和验证节点的硬件能力
状态租金	Rent 机制	Solana 账户需要支付租金（或持有最低余额）来维持存在，防止状态无限膨胀
熔炉被租走	账户被回收	如果账户余额低于租金豁免门槛且两年未使用，账户数据可能被回收
烟囱不够	网络拥堵 / 交易丢弃	当交易量超过处理能力时，低优先级交易被丢弃，用户需要重新提交
锣声停了	网络停机	Solana 历史上多次因共识问题或 bug 导致网络停机，因为全网的同步依赖 PoH

为什么是这个故事？

Solana 是"性能优先"区块链的代表，但它的架构（PoH、并行执行、租金模型）与以太坊差异巨大，很多概念对 EVM 开发者来说非常陌生。

用"流水线工坊"来隐喻 Solana，有几个直觉优势：

1. 并行执行的可视化：多工序同时进行 = Sealevel 的并行交易处理
2. **PoH** 的直觉：锣声 = PoH 时钟，为全网提供同步节拍

3. 租金的必要性：限制熔炉数量 = 租金机制防止状态膨胀
4. 停机的脆弱性：锣声停了全工坊乱套 = Solana 对共识同步的高度依赖

延伸阅读

- wiki-web3 概念原文：Solana 虚拟机 SVM 架构解析
- Solana Docs: Architecture
- Proof of History Explained
- Sealevel: Parallel Processing Thousands of Smart Contracts
- Solana Network Outages Analysis

"The Smith's Discipline"

The Smith's Discipline

> "不是不允许犯错，而是编译器在你犯错之前就把门关上。"

故事

云州城有两所铁匠学校。一所教"自由锻造"，一所教"纪律锻造"。

自由锻造学校

自由锻造学校的理念是：给学徒最大的自由。你可以用任何工具、任何材料、任何方法。学校只教你基本的技巧，剩下的靠你自己摸索。

自由锻造的优点是灵活。如果你想打一把形状怪异的刀，没有任何规则阻止你。

但自由锻造的代价是：事故频发。

- 有学徒把熔化的铁水倒在了自己身上——因为他忘了检查温度。
- 有学徒打出来的刀在第一次使用时就断了——因为他用了有裂纹的原材料，但学校没有强制检查原材料的制度。
- 有学徒在淬火时引发了火灾——因为他同时打开了油槽和水槽，而学校允许这样做。

最严重的一次事故：一个高级学徒打造了一台自动锻造机。机器运转良好，但有一个隐藏的缺陷——在特定条件下，机器会同时操作两把锤子，导致它们互相撞击，整个机器炸毁。这个缺陷在测试时没有显现，因为测试条件恰好避开了那个"特定条件"。但客户使用时触发了它。

自由锻造学校的毕业生很多成了大师，但也有很多成了残疾人——字面意义上的。

纪律锻造学校

纪律锻造学校的理念完全不同。在他们看来，锻造的危险不是来自工具，而是来自人的疏忽。所以学校设计了一整套"纪律"——不是在事后惩罚错误，而是在错误发生之前就阻止它。

具体规则：

第一，所有权制度。

每一件工具、每一块原材料，在任何时候都只有一个"所有者"。如果甲拿走了锤子，乙就不能同时使用它。如果乙想要用，他必须先等甲归还，或者甲明确地把锤子"借"给乙。

这个规则防止了"两个人同时操作同一把锤子"的事故。也防止了"我以为锤子在我手里，其实已经被别人拿走了"的混乱。

第二，借用检查。

当你从别人那里借工具时，学校会检查三件事：
- 这个工具确实存在（不是虚构的）。
- 这个工具目前没有人借用（或者借用方式不冲突）。
- 你借用的时间不会超过约定的期限。

如果任何一条不满足，学校不会允许你开工。不是惩罚你，而是根本不开工的门。

第三，类型匹配。

学校的熔炉只能加热特定类型的金属。如果你试图把一块木头放进铁熔炉，熔炉的门不会打开——不是因为你做错了什么，而是因为熔炉的设计就不允许非金属材料进入。

这看起来是限制，但实际上是保护。它防止了"把不应该加热的东西放进去"的事故。

第四，没有空指针。

在自由锻造学校，学徒可以指着一个空位子说"锤子在那里"——即使那里什么都没有。这会导致灾难：学徒伸手去抓空气，失去平衡，跌入熔炉。

纪律锻造学校取消了"空指针"。如果你要指一个位置，那个位置必须有东西。如果某样东西可能不存在，你必须显式地处理"不存在"的情况——比如先检查，或者准备一个备用方案。你不能假装它一定存在。

两种学校的产出

自由锻造学校的毕业生，如果活了下来，往往更有创造力。因为他们从小就被允许尝试各种疯狂的想法。

纪律锻造学校的毕业生，几乎不会出严重事故。他们的作品可能不那么惊艳，但极其可靠。一位雇主说："我雇过两所学校的毕业生。自由锻造学校的人偶尔会给我惊喜，但也会偶尔给我惊吓。纪律锻造学校的人从不给我惊喜，但也从不给我惊吓。在造桥的时候，我选择不给我惊吓的人。"

纪律锻造的代价

纪律锻造不是免费的。

学徒们抱怨："我明明知道自己不会犯错，但学校仍然强迫我检查每一步。这太浪费时间了。"

老师们回答："你昨天没有犯错，不代表你今天不会。而且，当项目越来越大、参与的人越来越多时，你不能假设其他人也和你一样不会犯错。纪律不是为你一个人设计的，它是为整个团队设计的。"

更实际的代价是：某些在自由锻造学校里很容易实现的设计，在纪律锻造学校里需要绕很多弯路。学校会要求你写更多的"样板代码"来证明"我已经考虑了所有可能出错的情况"。

一位老工匠说："自由锻造让你跑得更快，纪律锻造让你摔得更少。在造一把菜刀时，自由锻造可能更好。在造一座大桥时，纪律锻造是唯一的选择。"

这则寓言在说什么

这则寓言对应的是 **Rust** 的安全编程模型——Solana 智能合约的主要开发语言。Rust 通过所有权（Ownership）、借用检查（Borrow Checker）、类型系统、无空指针等机制，在编译期就消除内存安全漏洞，而不是在运行时靠测试发现。

关键映射

| 故事元素 | 技术概念 | 说明 |

|---------|---------|------|
自由锻造学校	C/C++ / 传统系统编程	给开发者最大自由，但内存管理全靠手动，容易出错
纪律锻造学校	Rust	编译期强制执行安全规则，内存安全无需垃圾回收
所有权制度	Ownership System	每个值在任何时候只有一个所有者。所有者离开作用域时，值被自动释放
借用检查	Borrow Checker	编译器检查所有引用是否有效：没有悬垂引用、没有数据竞争、没有同时的可变和不可变借用
类型匹配	Type System / Pattern Matching	Rust 的强类型系统和 exhaustive pattern matching 确保所有情况都被处理
没有空指针	`Option` / 无 null	Rust 没有 null。可能不存在的值必须用 `Option` 显式处理，编译器强制检查
样板代码	Rust 的显式错误处理	`Result` 和 `Option` 要求显式处理错误，代码量可能增加，但安全性大幅提升
造桥 vs 菜刀	应用场景的选择	系统级软件（区块链、操作系统）需要 Rust 的安全性。快速原型可能用更灵活的语言
团队的纪律	Rust 适合大规模协作	编译器作为"守门人"，确保即使不熟悉代码库的开发者也不会引入内存安全问题

为什么是这个故事？

Solana 选择 Rust 作为原生开发语言，核心原因就是安全性。区块链上的代码一旦部署就不可修改，一个内存安全漏洞可能导致数百万美元的损失。

用"铁匠学校"来隐喻 Rust vs C/C++，有几个直觉优势：
1. 所有权的可视化：一把锤子同一时间只能一个人用 = Rust 的所有权规则
2. 编译期检查：熔炉不开门 = 编译错误阻止不安全的代码
3. 没有空指针：不能指着空位置说锤子在那里 = `Option` 强制显式处理缺失值
4. 创造力的代价：纪律锻造更繁琐但更可靠 = Rust 的开发效率 vs 安全性的权衡

延伸阅读

- wiki-web3 概念原文：Rust 智能合约安全编程规范
- The Rust Programming Language Book
- Rust Ownership Explained
- Rust Borrow Checker
- Why Rust for Smart Contracts

"The Timed Safe"

The Timed Safe

> "保险箱不是免费的。存得越久，租金越高。不付租金，东西就会被清走。"

故事

云州城的银行提供一种特殊的保险箱服务，和传统的保险箱完全不同。

传统保险箱 **vs** 计时保险箱

在传统银行，你租一个保险箱，每年付一笔固定的租金。只要付了租金，箱子就永远是你的。你可以在里面放任何东西——金银珠宝、地契、甚至空气。银行不在乎。

云州城的新银行推出了一种"计时保险箱"。它的规则极其特别：

第一，每个保险箱的大小是固定的。

你可以租一个大保险箱（能放一百件东西）或一个小保险箱（只能放十件东西）。但你不能租一个"无限大"的保险箱。如果你想存更多东西，你必须租更多保险箱。

第二，保险箱有最低存款要求。

租一个大保险箱，你必须在里面预存至少一千两银子。这个钱不是租金——它仍然属于你——但它必须一直留在保险箱里。如果你取走了，保险箱会自动失效，里面的东西被清走。

第三，保险箱是"无主人"的。

传统保险箱属于租用者——只有你能打开。计时保险箱没有固定主人。任何人只要知道开箱方法，就能打开它。保险箱的安全不来自"谁拥有它"，而来自"谁能提供正确的开箱密码"。

第四，保险箱可以"授权程序"。

你可以设定："这个保险箱只能由某某程序打开。"程序是一段公开的、不可更改的规则。比如："只有当我提供正确的密码时，程序才允许取出东西。"或者："程序每个月自动从我的保险箱里取出十两银子，转给另一个保险箱。"

租金的机制

计时保险箱的租金不是按年收的。它是按"占用空间"和"占用时间"计算的。

具体规则：
- 一个保险箱每年需要消耗一定数额的租金。
- 保险箱里预存的一千两银子，会自动产生利息。
- 如果利息足够支付租金，保险箱可以永久维持。
- 如果利息不够支付租金，保险箱里的余额会逐渐减少。当余额降到零时，保险箱被"回收"——里面的东西被扔掉，保险箱本身被重新分配给其他人。

这意味着：计时保险箱不是"买了就永远拥有"。它是一个需要持续维护的资源。如果你不维护它，它就会消失。

计时保险箱的意外应用

计时保险箱的设计催生了很多创新用法：

自动执行的遗嘱：
某人在保险箱里存了一笔钱，设定程序："如果我两年内没有来打开这个保险箱，就把钱转给我的儿子。"如果主人去世，保险箱两年后自动执行转账。不需要律师、不需要法院、不需要遗嘱认证。

订阅服务：
一个商人提供每月十两的咨询服务。客户在保险箱里存了一百二十两，设定程序："每月自动转十两给商人。当余额不足时，停止服务。"商人不需要催收，客户不需要每月手动转账。

可组合的金融工具：
一个复杂的金融产品由十几个保险箱组成，每个保险箱负责一个功能——存储本金、计算利息、分配收益、管理风险。这些保险箱通过程序互相连接，形成一个自动运转的系统。任何单个保险箱的失效都会影响整个系统，但系统本身没有中心控制者。

计时保险箱的批评

但计时保险箱也有批评者。

- "最低存款要求"把穷人挡在门外。如果你只有一百两，你连最小的保险箱都租不起。
- "租金机制"意味着你的资产会随时间缩水（如果利息不够）。这违背了"存钱保值"的直觉。
- "无主人"设计让很多人不适应。传统银行客户问："如果我的密码泄露了，别人不就能打开我的保险箱了吗？"银行的回答是："是的。所以我们建议你使用更复杂的密码，或者使用程序来增加额外的安全层。"

一位传统银行家说："计时保险箱不是为普通人设计的。它是为程序设计的。在计时保险箱的世界里，真正拥有资产的不是你的身份，而是你的密码和你的程序。"

这则寓言在说什么

这则寓言对应的是 **Solana** 的账户模型和租金机制。与以太坊的"合约即账户"不同，Solana 的所有数据都存储在独立的账户中，每个账户有固定的空间、需要维持最低余额（rent-exempt）来避免被回收，并且账户可以指定"所有者程序"来管理访问权限。

关键映射

故事元素	技术概念	说明
传统保险箱	以太坊账户模型	合约存储自己的状态，没有显式的存储限制
计时保险箱	Solana Account	Solana 中所有数据都存储在独立的账户中，每个账户有固定大小
固定大小的保险箱	账户空间限制	Solana 账户在创建时确定大小，不能动态扩展
最低存款要求	Rent-exempt 余额	账户必须维持最低 lamport 余额才能免于被回收。这个余额与账户大小成正比
预存银子产生利息	余额不产生利息，但维持免租	实际上 Solana 没有利息，但 rent-exempt 的最低余额是一次性存入的
无主人	Account 没有私钥控制	Solana 账户由 "owner program" 控制，不是由私钥直接控制
开箱密码	账户的签名/权限验证	由 owner program 定义谁能操作账户
授权程序	Owner Program	每个账户有一个 owner program（如

System Program、Token Program），只有 owner 能修改账户数据 |
租金消耗	Rent 机制	账户需要定期支付 rent（或被 rent-exempt）。如果余额不足，账户数据被回收
自动执行遗嘱	Program Derived Address (PDA) + 条件执行	Solana 程序可以根据条件自动执行操作，无需外部触发
订阅服务	自动化转账 / 程序控制的资金流	SPL Token 程序支持自动化的代币转移逻辑
可组合金融工具	Solana DeFi 的可组合性	多个程序通过账户互相调用，形成复杂的金融协议
穷人被挡在门外	Rent 门槛	创建账户需要最低余额，对小用户不友好

为什么是这个故事？

Solana 的账户模型与以太坊截然不同，是 Solana 开发中最容易让 EVM 开发者困惑的地方。理解"账户需要租金"、"账户有固定大小"、"程序拥有账户"是入门 Solana 的关键。

用"计时保险箱"来隐喻 Solana 账户，有几个直觉优势：
1. 固定大小：保险箱大小固定 = Solana 账户的固定空间
2. 最低存款：预存银子维持保险箱 = rent-exempt 最低余额
3. 无主人设计：密码开箱而非身份 = Solana 的 program-owned 账户模型
4. 租金消耗：不维护就被回收 = Solana 的 rent 机制防止状态膨胀

延伸阅读

- wiki-web3 概念原文：Solana 账户模型与租金机制详解
- Solana Docs: Accounts
- Solana Rent Economics
- Solana Docs: Accounts

"The Deterministic Address"

The Deterministic Address

> "门牌不是随机分配的，而是由房子的主人和房子的用途共同决定的。"

故事

云州城的门牌系统经历了一次革命。

旧门牌：随机分配

在旧制度下，门牌是随机分配的。当你建一栋新房子，官府给你一个随机的门牌号。你无法预知会拿到什么号码，也无法选择。

这个制度的问题是：
1. 如果你想在房子建好之前就告诉别人你的地址，你做不到——因为你还不知道门牌号。
2. 如果你重建了房子（比如火灾后重建），新房子可能拿到一个完全不同的门牌号。所有之前登记过的地址都失效了。
3. 两个不同的房子永远不会拿到同一个门牌号——这保证了唯一性，但也意味着门牌号没有任何规律可言。

新门牌：确定性分配

新制度下，门牌号不再是随机的。它由两个因素决定：

1. 房子的主人是谁
2. 这栋房子用来做什么

具体规则：
- 门牌号 = Hash(主人的身份 + 房子的用途 + "盐值")

其中盐值是一个调整数字——如果第一次算出来的门牌号已经被占用了，就换一个盐值，重新计算，直到得到一个空的门牌号。

这意味着：
- 只要你知道主人是谁、房子用来做什么，你就能在房子建好之前算出它的门牌号。
- 如果主人重建了同一用途的房子（比如原来的茶馆烧毁了，重建后还是茶馆），门牌号不变——因为决定门牌号的两个因素都没有变。
- 不同主人的茶馆门牌号不同，同一个主人的茶馆和客栈门牌号也不同。

没有私钥的门牌

确定性门牌还有一个奇特的性质：它不在传统的门牌序列中。

在旧制度下，每个门牌号都对应一把钥匙——只有钥匙持有者能开门。在确定性门牌制度下，某些门牌号是在传统序列之外的——它们没有对应的钥匙。

这听起来很危险：没有钥匙的门，谁都能开吗？

不。没有钥匙的门，只能由"房子的程序"来开。

具体规则：

- 如果一个门牌是确定性门牌（不在传统序列中），任何人都不能直接用钥匙打开它。
- 只有房子的主人（通过其程序）可以向官府申请："请打开我这栋用途为 X 的房子。"
- 官府检查：申请者的身份 + 房子的用途，是否确实对应这个门牌号。如果对应，官府就开门。

这意味着：确定性门牌的安全性不来自"谁有钥匙"，而来自"谁能向官府证明这栋房子确实属于他"。

确定性门牌的应用

预测地址：
一个商人想开一家连锁茶馆。在第一家茶馆建好之前，他就能计算出所有未来茶馆的门牌号——因为他知道自己是主人，也知道用途是"茶馆第 N 号"。他可以提前印刷名片、登记契约，而不需要等房子建好。

地址映射：
云州城的户籍系统使用确定性门牌来建立"一对多"的关系。比如：一个大家庭有一个主地址（传统门牌），所有家庭成员的地址都是从这个主地址派生的——Hash(主地址 + "长子")、Hash(主地址 + "次女")。这样，只要知道主地址，就能算出所有家庭成员的地址。

无种子控制：
某些公共设施（如公共水井、道路）使用确定性门牌。这些设施没有私人主人，而是由一个公开的程序控制。任何人都可以向程序申请使用设施，程序根据规则决定是否允许。因为设施没有私钥，所以没有人能"偷走"它——它不属于任何人，只属于程序。

盐值的艺术

确定性门牌的计算中，盐值是一个关键变量。

理论上，Hash(主人 + 用途) 应该直接产生一个门牌号。但有时候，这个门牌号恰好落在传统门牌序列中——这意味着它可能已经被别人占用了（因为传统门牌号是有钥匙的）。

当这种情况发生时，系统自动把盐值从 255 开始递减，重新计算 Hash，直到找到一个不在传统序列中的门牌号。

这个"寻找有效盐值"的过程是确定性的——给定同样的主人和用途，系统永远找到同一个盐值。所以门牌号仍然是确定的，只是计算过程稍微复杂了一点。

多年后，云州城的所有新建筑都使用确定性门牌。一位城市规划师说："确定性门牌让地址从'随机标识'变成了'有意义的名字'。一个地址不再只是一串数字，它编码了这栋建筑的身份、用途和历史。"

这则寓言在说什么

这则寓言对应的是 **Solana** 的 **Program Derived Address (PDA)**——一种由程序地址和种子（seeds）确定性生成的账户地址。PDA 不在 Ed25519 曲线上的公钥空间中，因此没有对应的私钥，只能由生成它的程序通过 CPI（Cross-Program Invocation）来签名控制。

关键映射

| 故事元素 | 技术概念 | 说明 |

|---------|---------|------|
旧门牌随机分配	传统公钥地址	由私钥通过 Ed25519 生成，随机且不可预测
新门牌确定性分配	PDA	`findProgramAddress([seeds], program_id)` 确定性生成地址
主人的身份	Program ID	生成 PDA 的程序地址，确保不同程序的 PDA 空间不冲突
房子的用途	Seeds	任意字节数组，用于区分同一程序生成的不同 PDA
盐值	Bump (nonce)	从 255 递减到 0，寻找使 PDA 不在 Ed25519 曲线上的有效值
不在传统序列中	PDA 无对应私钥	PDA 不在椭圆曲线上，没有私钥，无法被外部签名
没有钥匙的门	PDA 无外部签名	只有生成 PDA 的程序才能通过 CPI 为 PDA "签名"
程序向官府申请	Cross-Program Invocation (CPI)	Solana 程序可以调用其他程序，并为派生的 PDA 提供签名
官府检查	Runtime 验证	Solana 运行时验证 CPI 调用是否合法，确保只有正确的程序能为 PDA 签名
连锁茶馆	确定性地址预测	可以在账户创建前预测 PDA 地址，用于预配置和引用
户籍系统映射	一对多关系建模	一个主账户可以派生多个子账户，建立层级结构
公共设施无主人	PDA 用于程序控制的状态	PDA 可以由程序完全控制，用于存储程序状态、配置等

为什么是这个故事？

PDA 是 Solana 开发中最核心也最独特的概念之一。它让程序能够"拥有"账户、控制状态、建立确定性的地址关系——这是 EVM 中没有的直接等价物。

用"确定性门牌"来隐喻 PDA，有几个直觉优势：
1. 确定性：给定主人和用途，门牌号永远相同 = PDA 的确定性生成
2. 无私钥：没有钥匙，只能由程序打开 = PDA 无外部签名，只能由程序通过

CPI 控制

3. 盐值：调整数字找到有效门牌 = bump seed 的搜索过程

4. 应用：预测地址、一对多映射、程序控制状态 = PDA 的实际使用场景

延伸阅读

- wiki-web3 概念原文：Solana PDA 程序派生地址详解
- Solana Docs: Program Derived Addresses
- Anchor Framework: PDA
- Solana PDA Deep Dive